

**ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO MÔN HỌC
THỰC TẬP NGOÀI TRƯỜNG
(Mã số môn học: CO3335)**

HỌC KỲ 253 NĂM HỌC 2025–2026

- NGÀNH: Computer Science**
- CHƯƠNG TRÌNH ĐÀO TẠO: High-quality**
- ĐƠN VỊ/ DOANH NGHIỆP NHẬN THỰC TẬP:**
Công ty TNHH Amazon Web Services Việt Nam
- CÁN BỘ HƯỚNG DẪN CHUYÊN MÔN TRỰC TIẾP
CỦA ĐƠN VỊ/ DOANH NGHIỆP:**
Lữ Hoàn Thiện
- CÁN BỘ HỖ TRỢ HƯỚNG DẪN/ GIÁM SÁT/ CHẤM
ĐIỂM BÁO CÁO CỦA KHOA (CBHT HD/GS/CĐBC):**
Vũ Ngọc Tú
- SINH VIÊN THỰC HIỆN (SVTH):**
HỌ VÀ TÊN: Hoàng Đức Hiếu Anh **MSSV: 2352030**

TP. HỒ CHÍ MINH, THÁNG 08/2026

BÁO CÁO THỰC TẬP

Mục lục

1	Nhật ký công việc	1
1.1	Tuần 1	1
1.2	Tuần 2	3
1.3	Tuần 3	6
1.4	Tuần 4	8
1.5	Tuần 5	10
1.6	Tuần 6	13
1.7	Tuần 7	15
1.8	Tuần 8	17
2	Đề xuất	29
3	Các bài viết đã đăng	42
3.1	Bài viết 1	42
3.2	Bài viết 2	43
3.3	Bài viết 3	49
3.4	Bài viết 4	51
4	Các sự kiện đã tham gia	57
4.1	Sự kiện 1: Ngày hội cộng đồng FCAJ – tháng 6 năm 2026	57
4.2	Sự kiện 2: Kiến trúc Cloud doanh nghiệp và ứng dụng trong thực tế	59
4.3	Sự kiện 3: FCAJ x Agentic AI Build Week 2026	60
5	Hội thảo thực hành	62
5.1	Tổng quan	64
5.2	Kiến trúc	67
5.3	Điều kiện tiên quyết	70
5.4	Chuẩn bị mã nguồn	74
5.5	Hạ tầng AWS	79
5.6	Triển khai cơ sở dữ liệu	84
5.7	Triển khai backend	88
5.8	Triển khai frontend	93
5.9	Giám sát và nhật ký	96
5.10	Kiểm thử đầu cuối	101
5.11	Bảo mật và Chi phí	106
5.12	Xử lý sự cố	113
5.13	Dọn dẹp tài nguyên	118
6	Tự đánh giá	124
7	Chia sẻ, đóng góp ý kiến	126
8	Tổng kết	128



Nhật ký công việc

Nhật ký thực tập

Dự án AI-Powered Internship Application Tracker được thực hiện bởi nhóm 5 thành viên. Nhật ký này chỉ ghi nhận phần việc được phân công cho một thành viên trong 8 tuần, từ 08/06/2026 đến 30/07/2026, không xem toàn bộ hệ thống là sản phẩm của cá nhân. Lịch báo cáo được chia theo từng tuần bắt đầu vào thứ hai; riêng tuần cuối được rút ngắn theo ngày kết thúc kỳ thực tập.

Vai trò cá nhân trong nhóm

Tôi phụ trách chính mảng Cloud/DevOps và tích hợp hệ thống: chuẩn bị quy trình container và Kubernetes, hỗ trợ kiến trúc AWS và CI/CD, kiểm tra môi trường production, thu thập minh chứng kỹ thuật và duy trì tài liệu triển khai. Các chức năng backend, frontend, chat và AI chỉ được đề cập khi tôi có tham gia review kiến trúc, tích hợp vào quy trình triển khai hoặc phối hợp kiểm thử với thành viên phụ trách.

Tuần	Thời gian	Trọng tâm cá nhân
1	08/06/2026 - 14/06/2026	Khảo sát repository và lập bản đồ phụ thuộc giữa các service phục vụ triển khai
2	15/06/2026 - 21/06/2026	Review yêu cầu concurrency và idempotency cho môi trường nhiều replica
3	22/06/2026 - 28/06/2026	Đóng góp yêu cầu triển khai cho outbox, SQS và worker xử lý bất đồng bộ
4	29/06/2026 - 05/07/2026	Chuẩn hóa container, Docker Compose, CI quality gates và smoke test
5	06/07/2026 - 12/07/2026	Chuẩn bị Kubernetes local, scaling controls và tài nguyên observability
6	13/07/2026 - 19/07/2026	Chuẩn bị IAM/OIDC, ECR và quy trình CI đẩy image lên AWS
7	20/07/2026 - 26/07/2026	Tích hợp workload EKS, dịch vụ AWS managed, ALB và rollout automation
8	27/07/2026 - 30/07/2026	Kiểm tra runtime AWS, phân tích lỗi và chi phí, thu thập minh chứng, hoàn thiện tài liệu

Kiến trúc và kết quả chung của project vẫn được giữ làm bối cảnh; từng trang theo tuần phân biệt rõ phần đóng góp của tôi với phần việc do bốn thành viên còn lại thực hiện.

1.1 Tuần 1

Tuần 1 - Phân tích project và kiến trúc cơ sở

Mục tiêu

Trong nhóm 5 thành viên, mục tiêu cá nhân của tôi trong tuần đầu là tìm hiểu các phần mã nguồn liên quan đến triển khai, xác định ranh giới và phụ thuộc giữa các service, đồng thời lập tài liệu kiến trúc cơ sở cho công việc Cloud/DevOps ở các tuần sau.

Phần việc cá nhân

Trạng thái	Phần việc được phân công	Cơ sở minh chứng
Hoàn thành	Khảo sát cấu trúc monorepo và xác định các service chạy chính.	Cây mã nguồn, README.md, docker-compose.yml và commit 4e939cf.
Hoàn thành	Lập bản đồ phụ thuộc giữa frontend, FastAPI backend, chat service, AI service, PostgreSQL, Redis và DynamoDB.	Các thư mục service, Compose và cấu hình hệ thống.
Hoàn thành	Review luồng request ban đầu giữa browser, API, chat và các lớp lưu trữ.	API client, backend router, Socket.IO server và storage service.



Trạng thái	Phần việc được phân công	Cơ sở minh chứng
Hoàn thành	Kiểm tra mô hình khởi chạy local và tài liệu hóa cách chạy riêng từng service.	Commit d3c1168 và e43cc04.

Triển khai kỹ thuật

Kho mã nguồn là một monorepo, trong đó mỗi dịch vụ đảm nhiệm một trách nhiệm riêng:

- `frontend/`: ứng dụng React và Vite dành cho ứng viên và nhân sự.
- `backend/`: API FastAPI, mô hình SQLAlchemy, migration Alembic, xác thực, công việc, hồ sơ ứng tuyển, tài liệu và các tuyến điều phối AI.
- `chat-service/`: Node.js, Express, Socket.IO, bộ điều hợp Redis và cơ chế lưu hội thoại trên DynamoDB.
- `ai_service/`: phân tích AI, chuẩn hóa CV/công việc, xếp hạng lại và mã bộ điều hợp SageMaker được bổ sung sau đó.
- `docker-compose.yml`: các thành phần phụ thuộc cục bộ và quy trình khởi chạy nhiều dịch vụ.

Kiến trúc cơ sở ban đầu:

Mô tả sơ bộ. Trình duyệt sử dụng frontend React/Vite; frontend gọi backend FastAPI cho các chức năng ứng tuyển và dịch vụ Socket.IO cho trò chuyện thời gian thực. Backend dùng PostgreSQL, kho tài liệu và dịch vụ AI, còn chat dùng Redis pub/sub và các bảng DynamoDB.

Kiến trúc cơ sở cũng xác định hướng chuyển đổi lên đám mây ban đầu: đóng gói backend và dịch vụ chat chạy dài hạn trong container, chuyển dữ liệu quan hệ bền vững sang PostgreSQL/RDS, dùng DynamoDB cho bản ghi hội thoại, dùng Redis để phân phối sự kiện thời gian thực, đồng thời chuẩn bị ứng dụng cho container và CI/CD.

Vấn đề và giải pháp

Vấn đề	Nguyên nhân gốc	Giải pháp	Trạng thái
Ảnh chụp mã nguồn đầu tiên chứa phạm vi mã ứng dụng và thành phần phụ thuộc được sinh tự động quá rộng.	Commit ban đầu đưa toàn bộ nền tảng vào một commit lớn.	Các công việc sau đó tách tài liệu, tập lệnh và trách nhiệm thời gian chạy thành các đường dẫn rõ ràng hơn.	Hoàn thành
Dự án ban đầu còn thành phần cơ sở dữ liệu Node liên quan đến Prisma trong khi backend dùng SQLAlchemy/PostgreSQL.	Chiến lược cơ sở dữ liệu chuyển sang FastAPI, SQLAlchemy, Alembic và PostgreSQL.	Commit ab60f7d loại bỏ thành phần Prisma và ghi lại hướng triển khai EC2/RDS.	Hoàn thành
Cách khởi chạy cục bộ cần hỗ trợ nhiều quy trình phát triển.	Một lệnh duy nhất thuận tiện khi trình diễn, còn các lệnh terminal riêng để gỡ lỗi hơn.	Các commit d3c1168 và e43cc04 ghi lại cả hai cách khởi chạy.	Hoàn thành

Kết quả kiểm thử, build và triển khai

Hạng mục	Kết quả	Minh chứng
Rà soát mã nguồn	Hoàn thành	<code>git show --stat 4e939cf</code> cho thấy backend, frontend, Docker, migration, kiểm thử và tài liệu được thêm trong commit đầu tiên.
Khởi chạy cục bộ	Hoàn thành một phần	Lịch sử có tập lệnh khởi chạy và hướng dẫn README nhưng không tìm thấy nhật ký terminal đã lưu.
Kiểm thử	Hoàn thành một phần	Có các tệp kiểm thử trong <code>backend/tests</code> ; chưa tìm thấy kết quả kiểm thử của tuần 1.
Build	Hoàn thành một phần	Có Dockerfile và các tệp dự án Vite; chưa tìm thấy nhật ký build của tuần 1.



Hạng mục	Kết quả	Minh chứng
Triển khai	Đã lập kế hoạch	Tuần 1 chỉ xác định hướng triển khai, chưa đặt mục tiêu triển khai AWS.

Minh chứng

Commit và yêu cầu kéo mã

Commit	Mô tả	Minh chứng	Yêu cầu kéo mã
4e939cf	Tạo nền tảng ứng dụng ban đầu gồm backend, frontend, Docker Compose, migration và kiểm thử.	Xem commit	Minh chứng đang chờ
5ab2924	Bổ sung đối sánh CV bằng AI và cấu hình AWS RDS.	Xem commit	Minh chứng đang chờ
ab60f7d	Loại bỏ thành phần Prisma và ghi lại hướng triển khai EC2/RDS.	Xem commit	Minh chứng đang chờ
d3c1168	Bổ sung cách khởi chạy toàn bộ hệ thống bằng một lệnh.	Xem commit	Minh chứng đang chờ
e43cc04	Ghi lại các lệnh khởi chạy riêng cho từng dịch vụ.	Xem commit	Minh chứng đang chờ

Nhật ký kiểm thử

Minh chứng đang chờ: không tìm thấy nhật ký kiểm thử cục bộ hoặc CI của tuần 1 trong kho mã nguồn.

Nhật ký build

Minh chứng đang chờ: không tìm thấy nhật ký build Docker hoặc frontend của tuần 1 trong kho mã nguồn.

Nhật ký triển khai

Không áp dụng cho tuần 1. Giai đoạn này bắt đầu lập kế hoạch triển khai; việc triển khai lên đám mây được dành cho các tuần sau.

Kết quả trong tuần

Cuối tuần 1, tôi đã hoàn thành bản đồ service/phụ thuộc phục vụ phạm vi triển khai và xác định hướng container hóa, triển khai AWS ban đầu. Mã nguồn ứng dụng và các chức năng nền tảng là kết quả chung của nhóm 5 thành viên.

Bài học rút ra

Bài học chính là kế hoạch triển khai phụ thuộc vào ranh giới thời gian chạy rõ ràng. Việc sớm tách API, chat, AI, dữ liệu quan hệ, cơ chế phát/nhận thời gian thực và lưu trữ tài liệu giúp công việc Docker, Kubernetes và AWS ở các tuần sau dễ phân tích hơn.

Kế hoạch tuần tiếp theo

Tuần tiếp theo tập trung vào tính toàn vẹn dữ liệu và xử lý đồng thời: ràng buộc cơ sở dữ liệu, xử lý yêu cầu trùng, khóa chống lặp, lệnh quy trình có phiên bản và các kiểm thử chứng minh thao tác ghi đồng thời trả về xung đột có kiểm soát thay vì dữ liệu không nhất quán.

1.2 Tuần 2

Tuần 2 - Kiểm soát concurrency và data integrity

Mục tiêu

Trong nhóm 5 thành viên, mục tiêu tuần 2 của tôi là review các yêu cầu concurrency và data integrity ảnh hưởng đến môi trường nhiều replica. Tôi hỗ trợ thành viên backend và chat kiểm tra idempotency, version check và database constraint trước khi đưa hệ thống lên Kubernetes.

Phản việc cá nhân



Trạng thái	Phần việc được phân công	Cơ sở minh chứng
Hoàn thành	Phân tích concurrent write khi nhiều backend pod xử lý đồng thời.	docs/architecture/concurrency-policy.md.
Hoàn thành	Đóng góp góc nhìn triển khai vào chính sách optimistic/pessimistic UI.	docs/architecture/concurrency-policy.md và docs/frontend/CMD_MUTATION_POLICY.md.
Hoàn thành	Review cơ chế idempotency do thành viên backend triển khai cho thao tác ứng tuyển.	Service, router và migration 0004_idempotency_records.py.
Hoàn thành	Review workflow command dùng expectedVersion trong môi trường nhiều pod.	Optimistic concurrency service, workflow command và các router liên quan.
Hoàn thành	Phối hợp với thành viên chat kiểm tra hành vi retry và chống ghi trùng.	Chat repository và concurrency test.

Triển khai kỹ thuật

Kiểm soát đồng thời được triển khai ở tầng cơ sở dữ liệu và hợp đồng API:

- Các ràng buộc duy nhất của PostgreSQL ngăn bản ghi người dùng, công ty, hồ sơ ứng tuyển và chống lặp bị trùng.
- Idempotency-Key được chấp nhận khi ứng viên nộp hồ sơ và khi gọi một số lệnh quy trình.
- Dùng lại cùng khóa chống lặp với cùng dữ liệu đầu vào sẽ phát lại phản hồi đã lưu.
- Dùng lại cùng khóa với dữ liệu đầu vào khác sẽ trả về mã 409 và lỗi idempotency_key_reused.
- Các lệnh sửa công việc và hồ sơ ứng tuyển của nhân sự yêu cầu expectedVersion.
- Cập nhật dựa trên phiên bản cũ trả về phản hồi 409 Conflict có kiểm soát.
- Khi thử gửi lại tin nhắn, dịch vụ chat dùng mã tin nhắn phía máy khách có tính xác định và phép ghi có điều kiện của DynamoDB để tránh trùng tin nhắn logic.

Luồng xử lý xung đột:

Mô tả sơ bộ. Client gửi yêu cầu ứng tuyển kèm khóa idempotency. Backend giữ chỗ khóa trong PostgreSQL, tạo hồ sơ đối với yêu cầu mới, trả lại kết quả đã lưu khi yêu cầu được gửi lại y hệt, và trả HTTP 409 khi payload hoặc trạng thái nghiệp vụ xung đột.

Vấn đề và giải pháp

Vấn đề	Nguyên nhân gốc	Giải pháp	Trạng thái
Đăng ký hoặc tạo công ty trùng có thể làm lộ lỗi cơ sở dữ liệu chưa kiểm soát.	Đã có hoặc cần bổ sung ràng buộc duy nhất, đồng thời phải ánh xạ lỗi tường minh.	Lỗi toàn vẹn được hoàn tác và chuyển thành 409 Conflict.	Hoàn thành
Trình duyệt hoặc mạng có thể khiến yêu cầu ứng tuyển được gửi lại.	Yêu cầu thứ hai có thể đến pod backend khác trước khi phản hồi đầu tiên quay về.	Bản ghi chống lặp theo phạm vi lưu hàm băm yêu cầu và dữ liệu phản hồi.	Hoàn thành
Cập nhật quy trình nhân sự có thể dùng dữ liệu cũ khi hai người cùng thao tác.	Cập nhật thông thường không biết máy khách đã nhìn thấy phiên bản mới nhất hay chưa.	Lệnh yêu cầu expectedVersion và dùng cập nhật có điều kiện.	Hoàn thành
Thử gửi lại có thể làm trùng tin nhắn thời gian thực.	Cơ chế thử lại qua Socket và REST có thể gửi lại cùng một tin nhắn logic.	Kho chat dùng định danh tin nhắn có tính xác định và bỏ qua phát lại khi nhận bản trùng.	Hoàn thành

Kết quả kiểm thử, build và triển khai



Hạng mục	Kết quả	Minh chứng
Kiểm thử đồng thời của backend	Hoàn thành một phần	Có các tệp kiểm thử: backend/tests/test_apply_idempotency.py, backend/tests/test_apply_idempotency_postgres.py, backend/tests/test_optimistic_concurrency.py và backend/tests/test_optimistic_concurrency_postgres.py. Kết quả chạy hiện tại đang chờ.
Kiểm thử đồng thời của chat	Hoàn thành một phần	Có tệp chat-service/tests/chatRepository.concurrency.test.js. Kết quả chạy hiện tại đang chờ.
Hành vi xung đột của API	Đã triển khai	Mã nguồn xử lý tường minh mã 409 cho phiên bản cũ, ứng tuyển trùng và dùng lại khóa chống lặp.
Triển khai	Đã lập kế hoạch	Tuần này tập trung bảo đảm tính đúng đắn trước khi triển khai.

Minh chứng

Commit và yêu cầu kéo mã

Commit	Mô tả	Minh chứng	Yêu cầu kéo mã
0f9e7f2	Bổ sung outbox quy trình backend, bản ghi chống lặp, công cụ đồng thời lạc quan, nền tảng xử lý bất đồng bộ và kiểm thử.	Xem commit	Minh chứng đang chờ
5326cb0	Củng cố xử lý đồng thời của kho chat và bổ sung kiểm thử.	Xem commit	Minh chứng đang chờ
77faab4	Tích hợp hợp đồng quy trình và xử lý vào công cụ API cùng luồng giao diện frontend.	Xem commit	Minh chứng đang chờ
5041f84	Sửa các khẳng định bị lỗi trong kiểm thử đồng thời lạc quan.	Xem commit	Minh chứng đang chờ

Nhật ký kiểm thử

Minh chứng đang chờ: đính kèm kết quả thực tế từ các lệnh như:

```
cd backend
python -m pytest tests/test_apply_idempotency.py tests/test_optimistic_concurrency.py
python -m pytest -m postgres tests/test_apply_idempotency_postgres.py
→ tests/test_optimistic_concurrency_postgres.py
```

Nhật ký build

Không áp dụng cho nhiệm vụ chính của tuần 2. Không tìm thấy sản phẩm build riêng cho giai đoạn này.

Nhật ký triển khai

Không áp dụng cho tuần 2. Tính năng được thiết kế an toàn trước khi mở rộng backend trên Kubernetes.

Kết quả trong tuần

Kết quả cá nhân tuần 2 là bản review theo góc nhìn triển khai đối với chính sách concurrency của nhóm và hành vi khi chạy nhiều replica. Phần triển khai backend và chat thuộc trách nhiệm của các thành viên phụ trách hai component này.

Bài học rút ra

Cân bằng tải dịch vụ không tuần tự hóa thao tác ghi. Tính đúng đắn phải được bảo đảm tại ranh giới lưu trữ và giao dịch, sau đó thể hiện rõ qua mã phản hồi API và hành vi giao diện.

Kế hoạch tuần tiếp theo

Xây dựng outbox giao dịch và các tầng xử lý bất đồng bộ để thay đổi nghiệp vụ đã commit có thể kích hoạt công việc phía sau một cách tin cậy mà không giữ yêu cầu API chính chờ lâu.

1.3 Tuần 3

Tuần 3 - Transactional outbox và xử lý bất đồng bộ

Mục tiêu

Trong nhóm 5 thành viên, mục tiêu tuần 3 của tôi là xác định yêu cầu triển khai và tích hợp cho transactional outbox cùng các worker xử lý bất đồng bộ. Tôi tập trung vào production transport, vòng đời worker, retry, cấu hình và observability; thành viên backend phụ trách business implementation.

Phân việc cá nhân

Trạng thái	Phần việc được phân công	Cơ sở minh chứng
Hoàn thành	Review schema outbox_events theo góc nhìn triển khai và phục hồi.	backend/alembic/versions/0007_outbox_events.py.
Hoàn thành	Kiểm tra các trạng thái vòng đời và retry model phục vụ vận hành.	Migration constraint và backend/app/services/outbox.py.
Hoàn thành	Review deduplication_key cùng các index claim/retry cho nhiều worker.	Migration 0007_outbox_events.py.
Hoàn thành	Xác định yêu cầu runtime cho outbox dispatcher và admin operation.	Các worker outbox_dispatcher.py và outbox_admin.py.
Hoàn thành	Tài liệu hóa SQS Standard làm production event transport cho nhóm.	ADR về production event transport.
Hoàn thành	Phối hợp xác định yêu cầu tích hợp cho async_processing_jobs và worker document/AI.	Migration, processing job service và processing worker.
Hoàn thành một phần	Đánh giá mức sẵn sàng của downstream consumer ngoài contract.	Tài liệu nguồn cho biết notification/search/analytics consumer chưa được triển khai ở giai đoạn này.

Triển khai kỹ thuật

The outbox design keeps domain mutation and event creation in the same PostgreSQL transaction. If the business write fails, no event is inserted. If the API pod dies after commit, the outbox row remains PENDING and can be claimed later by the dispatcher.

Mô tả sơ bộ. Backend ghi thay đổi nghiệp vụ và bản ghi outbox trong cùng PostgreSQL. Dispatcher lấy các sự kiện đang chờ và gửi sang SQS; sự kiện thành công được đánh dấu đã gửi, lỗi tạm thời được thử lại có backoff, còn lỗi cuối cùng được đánh dấu DEAD. Consumer khử trùng lặp theo mã sự kiện.

Asynchronous processing uses a separate database-backed queue because document parsing and AI matching are user-visible jobs with status, retry behavior, source-version guards, and result payloads. The worker claims jobs, applies lease-based retry logic, and persists results back to PostgreSQL.

Vấn đề và giải pháp

Problem	Root cause	Resolution	Status
A committed workflow change could lose its follow-up event if publishing failed after the database commit.	Direct publish inside request handling is not transactional with PostgreSQL.	Insert a canonical outbox event in the same DB transaction, then dispatch asynchronously.	Completed



Problem	Root cause	Resolution	Status
SQS Standard can redeliver or reorder messages.	Standard queues provide at-least-once delivery, not exactly-once delivery.	The event envelope includes <code>eventId</code> ; consumers must deduplicate by <code>eventId</code> .	Completed
Dispatcher failure can leave events half-processed.	A worker can die after claiming or publishing an event.	Lease expiry, retry counters, DEAD status, and admin <code>requeue/cleanup</code> commands were added.	Completed
AI and document work can take too long for API request latency.	Parsing CVs, extracting text, and reranking candidates are long-running operations.	Added <code>async_processing_jobs</code> and <code>backend-processing-worker</code> .	Completed
Real downstream consumers were not part of this implementation slice.	I focused the week on producer reliability and the consumer contract.	I kept downstream consumers as future work until runtime evidence is added.	Partially completed

Kết quả kiểm thử, build và triển khai

Area	Result	Evidence
Outbox unit/integration test coverage	Partially completed	Test sources exist: <code>test_outbox_events.py</code> , <code>test_outbox_postgres.py</code> , <code>test_outbox_publishers.py</code> , and <code>test_outbox_admin.py</code> . Current-run output is pending.
Async processing tests	Partially completed	Test sources exist: <code>test_processing_jobs.py</code> and <code>test_processing_jobs_postgres.py</code> . Current-run output is pending.
Kubernetes deployment readiness	Implemented	<code>k8s/app/backend-outbox-dispatcher.yaml</code> and <code>k8s/app/backend-processing-worker.yaml</code> exist.
SQS provisioning	Implemented	<code>scripts/aws/provision-outbox-sqs.ps1</code> and <code>.sh</code> exist. Runtime <code>queue-name</code> evidence still needs a screenshot or CLI log.

Minh chứng

Commit và yêu cầu kéo mã

Commit	Description	Evidence	Pull Request
0f9e7f2	Added backend workflow outbox, async processing jobs, dispatcher, worker, and tests.	View commit	Evidence pending
be84e4e	Added Kubernetes deployments, SQS provisioning scripts, validation gates, and alerts for async workers.	View commit	Evidence pending
4970eba	Documented phase 2 workflow operations, event transport ADR, and consumer contract.	View commit	Evidence pending



Commit	Description	Evidence	Pull Request
14173c6	Merged workflow, outbox, async processing, and infrastructure changes.	View commit	Evidence pending

Nhật ký kiểm thử

Evidence pending: attach actual output from commands such as:

```
cd backend
python -m pytest tests/test_outbox_events.py tests/test_outbox_publishers.py
→ tests/test_processing_jobs.py
python -m pytest -m postgres tests/test_outbox_postgres.py
→ tests/test_processing_jobs_postgres.py
```

Nhật ký build

Evidence pending: no dedicated Week 3 image build log was found.

Nhật ký triển khai

Evidence pending: add dispatcher or worker rollout logs after deployment, for example `kubectl rollout status deployment/backend-outbox-dispatcher -n internship`.

Kết quả trong tuần

Kết quả cá nhân tuần 3 là quyết định dùng SQS và bộ yêu cầu triển khai cho outbox dispatcher, processing worker. Schema outbox, backend service và business logic document/AI vẫn là sản phẩm chung của nhóm.

Bài học rút ra

Idempotency exists at multiple layers. HTTP idempotency prevents duplicate command execution, the transactional outbox prevents lost domain events, and consumers still need event-level deduplication.

Kế hoạch tuần tiếp theo

Standardize container builds, CI quality gates, smoke tests, security scans, and infrastructure validation so the system can be deployed repeatedly with less manual checking.

1.4 Tuần 4

Tuần 4 - Container hóa và CI Quality Gates

Mục tiêu

Tuần 4 là giai đoạn bắt đầu tập trung vào nhiệm vụ Cloud/DevOps chính của tôi trong nhóm 5 thành viên. Mục tiêu là đóng gói các component backend, frontend, chat và AI của các thành viên khác bằng container, Docker Compose, CI quality gates, smoke test và security check.

Phân việc cá nhân

Trạng thái	Phần việc được phân công	Cơ sở minh chứng
Hoàn thành	Chuẩn hóa đường dẫn Docker build cho backend, chat, frontend và AI service.	Các Dockerfile của bốn component.
Hoàn thành	Duy trì Compose làm môi trường parity cho dependency và smoke test local.	Compose files và <code>scripts/ci/smoke-test.sh</code> .
Hoàn thành	Hợp nhất GitHub Actions vào workflow CI/CD chính.	<code>.github/workflows/cicd.yml</code> và commit <code>e09e84e</code> .
Hoàn thành	Bổ sung các quality/security gate cần thiết.	Các script kiểm tra repository, infrastructure, security và cấu hình liên quan.
Hoàn thành	Sửa lỗi ShellCheck SC2155 trong smoke-test script.	Commit <code>98f3420</code> .

Triển khai kỹ thuật

The final CI workflow defines validation jobs for backend, chat service, frontend, Docker images, infrastructure, smoke checks, and security scanning. The workflow uses Python 3.12, Node.js 22, shell validation, Docker image builds, Trivy/Gitleaks images, and actionlint/kubeconform support through the infrastructure script.

Relevant workflow checks include:

```
python -m ruff format --check .
python -m ruff check .
python -m mypy app tests
python -m pytest -m "not postgres"
npm run check --prefix chat-service
npm run lint --prefix chat-service
npm run format:check --prefix chat-service
npm run test:ci --prefix chat-service
npm run test --prefix frontend
npm run build --prefix frontend
docker build -t internship-backend:ci ./backend
docker build -t internship-chat:ci ./chat-service
docker build -f Dockerfile.ai-service -t internship-ai:ci .
```

Vấn đề và giải pháp

Problem	Root cause	Resolution	Status
Multiple workflows made the deployment path harder to control.	Separate CI/deploy workflows split validation and deployment behavior.	Consolidated workflows into <code>.github/workflows/cicd.yml</code> .	Completed
ShellCheck flagged SC2155 in the smoke test script.	Assignment and command substitution were combined in a way ShellCheck warns about.	Commit 98f3420 split the logic to satisfy ShellCheck.	Completed
<code>.env</code> files can contain UTF-8 BOM and break shell parsing.	Windows editors may add BOM at the beginning of files.	<code>scripts/ci/smoke-test.sh</code> strips the BOM when preparing smoke-test env input.	Completed
Docker-based infra validation may hang or fail when Docker is unavailable.	Local Windows/Docker setups vary by machine.	Infrastructure scripts support local CLI binaries and bounded Docker fallback behavior.	Completed

Kết quả kiểm thử, build và triển khai

Area	Result	Evidence
Backend CI commands	Implemented	Commands are defined in <code>.github/workflows/cicd.yml</code> . Current-run output is pending.
Chat CI commands	Implemented	Commands are defined in <code>.github/workflows/cicd.yml</code> and <code>scripts/chat-service.sh</code> . Current-run output is pending.
Frontend CI commands	Implemented	Commands are defined in <code>.github/workflows/cicd.yml</code> and <code>scripts/frontend.sh</code> . Current-run output is pending.
Infrastructure/security checks	Implemented	<code>scripts/ci/infrastructure.py</code> , <code>scripts/check-infra.ps1</code> , <code>scripts/check-infra.sh</code> , and <code>scripts/ci/security_scan.py</code> exist.



Area	Result	Evidence
Compose smoke test	Implemented	<code>scripts/ci/smoke-test.sh</code> exists and ends with <code>Compose smoke test passed</code> on success. Current-run output is pending.

Mình chứng

Commit và yêu cầu kéo mã

Commit	Description	Evidence	Pull Request
b4ef947	Added required pipeline gates, security scanning, pre-commit config, and infrastructure validation scripts.	View commit	Evidence pending
e09e84e	Consolidated workflows into a single <code>cicd.yml</code> pipeline.	View commit	Evidence pending
29de145	Resolved CI failures for pytest, frontend build, and infrastructure checks.	View commit	Evidence pending
98f3420	Fixed ShellCheck SC2155 in the smoke test script.	View commit	Evidence pending
c2ad7cc	Consolidated and optimized the GitHub Actions workflow.	View commit	Evidence pending

Nhật ký kiểm thử

Evidence pending: attach CI or local output for backend, chat, frontend, and smoke-test jobs.

Nhật ký build

Evidence pending: attach Docker build and Vite build output from GitHub Actions or a local run.

Nhật ký triển khai

Not applicable for Week 4. This phase prepared deployable artifacts and gates, while AWS/EKS deployment continued in later weeks.

Kết quả trong tuần

Kết quả cá nhân tuần 4 là CI/CD backbone và quy trình kiểm tra container dùng chung cho cả 5 thành viên. Feature code vẫn thuộc thành viên phụ trách từng component; phạm vi của tôi là đóng gói, tự động hóa và kiểm tra tích hợp.

Bài học rút ra

CI should fail early on repeatable checks: formatting, typing, tests, build, security, shell syntax, Dockerfile quality, and infrastructure manifest validation. Making these checks explicit reduced the amount of manual inspection needed before deployment.

Kế hoạch tuần tiếp theo

Move from Docker Compose to local Kubernetes with kind, Kubernetes manifests, migration/init jobs, probes, HPA/PDB configuration, and observability resources.

1.5 Tuần 5

Tuần 5 - Triển khai Kubernetes local

Mục tiêu

Trong nhóm 5 thành viên, mục tiêu tuần 5 của tôi là chuyển ứng dụng đã tích hợp từ Compose sang Kubernetes local bằng kind. Tôi phụ trách manifest và automation; các thành viên còn lại cung cấp và kiểm tra cấu hình runtime cho component của mình.



Phần việc cá nhân

Trạng thái	Phần việc được phân công	Cơ sở minh chứng
Hoàn thành	Bổ sung cấu hình kind cluster.	k8s/cluster/kind-config.yaml và commit 8dae02c.
Hoàn thành	Tạo Kubernetes resource cho service, dependency, migration job và chat initialization job.	k8s/app/*.yaml.
Hoàn thành	Tự động hóa triển khai local cho PowerShell và shell.	Hai script deploy-local.
Hoàn thành	Bổ sung HPA và PDB cho application service.	k8s/app/autoscaling.yaml.
Hoàn thành	Bổ sung tài nguyên observability cho Prometheus, Grafana, Loki, Tempo, Alloy và OTEL.	k8s/observability/* và observability/grafana/*.

Triển khai kỹ thuật

The local Kubernetes path uses kind for the cluster, in-cluster PostgreSQL/Redis/DynamoDB Local for dependencies, and Kubernetes manifests under k8s/app for the workload layer.

Mô tả sơ bộ. Cụm kind cục bộ chứa các workload ứng dụng và dịch vụ dữ liệu trong namespace internship. Namespace monitoring riêng chạy Prometheus, Grafana, Loki và Tempo; Prometheus thu thập metric từ backend, chat service và outbox dispatcher.

The deployment scripts build local images, load them into kind, apply app manifests, run migration/init jobs, wait for rollouts, and print port-forward commands for backend, chat, and optional AI service checks.

Vấn đề và giải pháp

Problem	Root cause	Resolution	Status
Compose did not prove Kubernetes readiness.	Services could work locally while still missing probes, Services, Jobs, or ConfigMaps.	Added Kubernetes manifests and local deployment scripts.	Completed
Fresh local images might not be used by kind.	Building an image is not enough; kind nodes need the image loaded.	Scripts run <code>kind load docker-image</code> and restart deployments.	Completed
Migrations and chat table creation need ordered startup.	API/chat pods should not assume databases are initialized.	Added <code>backend-migrate</code> and <code>chat-init</code> Jobs.	Completed
Local observability required multiple supporting components.	Metrics, logs, and traces need separate platform services.	Added Helm-backed Prometheus/Grafana/Loki/Tempo/Alloy resources and local runbook guidance.	Completed

Kết quả kiểm thử, build và triển khai

Area	Result	Evidence
Kubernetes manifests	Implemented	k8s/app includes Deployments, Services, Jobs, ConfigMaps, Secret examples, ingress, HPA, and PDB.
Local deployment scripts	Implemented	scripts/k8s/deploy-local.ps1 and .sh automate cluster setup, image load, jobs, rollouts, and resource display.



Area	Result	Evidence
Observability resources	Implemented	k8s/observability and observability/grafana include dashboards, datasources, rules, and collectors.
Runtime validation logs	Partially completed	Commands are scripted, but current report does not include saved kubectl output.

Mình chứng

Commit và yêu cầu kéo mã

Commit	Description	Evidence	Pull Request
8dae02c	Added the first Kubernetes configuration for backend, chat, dependencies, jobs, namespace, and kind config.	View commit	Evidence pending
7dbe7c6	Added Kubernetes/EKS deployment platform, autoscaling, ingress, service accounts, observability, and deploy scripts.	View commit	Evidence pending
4f37d4d	Added Kubernetes deployment workflows.	View commit	Evidence pending
7407fda	Added tracing and chat initialization support.	View commit	Evidence pending
3b6f346	Added local terminal runbook and environment examples.	View commit	Evidence pending

Nhật ký kiểm thử

Evidence pending: attach actual local Kubernetes output from commands such as:

```
kubectl cluster-info
kubectl get nodes
kubectl get pods -n internship
kubectl get services -n internship
kubectl rollout status deployment/backend -n internship
kubectl rollout status deployment/chat-service -n internship
```

Nhật ký build

Evidence pending: attach output from local image builds and kind load docker-image commands.

Nhật ký triển khai

Evidence pending: attach output from scripts/k8s/deploy-local.ps1 or scripts/k8s/deploy-local.sh.

Kết quả trong tuần

Kết quả cá nhân tuần 5 là quy trình Kubernetes local gồm manifest, automation, scaling control và observability. Kiểm thử chức năng backend, frontend, chat và AI được phối hợp với bốn thành viên phụ trách các component tương ứng.

Bài học rút ra

Local Kubernetes validation is more than checking that pods start. Jobs, readiness probes, image freshness, DNS, service ports, ingress, HPA metrics, and observability scraping all need explicit checks.

Kế hoạch tuần tiếp theo

Prepare the AWS foundation: AWS CLI validation, region selection, GitHub OIDC, IAM deployment role, ECR repositories, and image build/push workflow.

1.6 Tuần 6

Tuần 6 - Nền tảng AWS, IAM, OIDC và ECR

Mục tiêu

Trong nhóm 5 thành viên, mục tiêu tuần 6 của tôi là chuẩn bị nền tảng triển khai AWS dùng chung: region, xác thực GitHub OIDC, quyền IAM, ECR repository và CI job đẩy image của các service mà không sử dụng access key dài hạn.

Phân việc cá nhân

Trạng thái	Phần việc được phân công	Cơ sở minh chứng
Hoàn thành	Chuẩn hóa production region là <code>ap-southeast-1</code> .	Project context, CI/CD workflow và deployment script.
Hoàn thành	Bổ sung GitHub OIDC smoke workflow.	Commit 2777b56.
Hoàn thành	Bổ sung workflow build và push application image lên ECR.	Commit 9f9fcf3.
Hoàn thành	Đồng bộ AWS deployment workflow với production architecture.	Commit 7eb1a76.
Hoàn thành	Bổ sung EKS access smoke-test workflow.	Commit bfacd3a.

Triển khai kỹ thuật

The CI/CD path uses GitHub Actions OIDC to assume the AWS deployment role instead of storing long-lived AWS access keys. ECR image URIs are based on AWS account, region, repository name, and immutable Git SHA tags.

Mô tả sơ bộ. GitHub Actions nhận token OIDC và đảm nhận deployment role có phạm vi giới hạn. Role này cho phép đẩy image backend và chat gắn thẻ SHA lên ECR, đồng thời cho phép script triển khai cập nhật workload EKS mà không cần lưu AWS credential dài hạn.

The source repository includes deployment scripts for both EKS application deployment and frontend deployment. I keep sensitive runtime values such as `DATABASE_URL`, `REDIS_URL`, and `SECRET_KEY` in environment variables or Kubernetes Secrets, not in published documentation.

Vấn đề và giải pháp

Problem	Root cause	Resolution	Status
AWS access must not depend on static access keys.	Static credentials are high-risk for CI/CD.	GitHub OIDC was introduced for role assumption.	Completed
ECR push requires authorization permissions.	The GitHub deployment role needs ECR token and repository permissions.	ECR build/push workflow and later permission verification commits were added.	Partially completed
Deployment needs deterministic image names.	Manually passing image URIs is error-prone.	Workflow builds/pushes SHA-tagged images and deployment scripts consume those values.	Completed
EKS access can be blocked by limited permissions.	Deployment role may lack cluster or Kubernetes RBAC permissions.	EKS access smoke workflow and later rollout tolerance were added.	Partially completed

Kết quả kiểm thử, build và triển khai



Area	Result	Evidence
OIDC workflow	Implemented	.github/workflows/aws-oidc-smoke-test.yml existed in commit 2777b56; later consolidated into CI/CD.
ECR build/push workflow	Implemented	Commit 9f9fcf3 added AWS ECR build/push workflow.
Production deploy workflow	Implemented	Commit 7eb1a76 added scripts/ci/deploy-eks-pipeline.sh and scripts/ci/deploy-frontend.sh.
AWS CLI/runtime logs	Partially completed	I did not find local aws sts, aws ecr, or policy simulation output in the local evidence archive.

Mình chứng

Commit và yêu cầu kéo mã

Commit	Description	Evidence	Pull Request
2777b56	Added AWS OIDC smoke test workflow.	View commit	Evidence pending
9f9fcf3	Added ECR build and push workflow.	View commit	Evidence pending
7eb1a76	Aligned AWS deployment with production architecture.	View commit	Evidence pending
bfacd3a	Added EKS access smoke test workflow.	View commit	Evidence pending
404ba6c	Reran deployment path with ECR verify permission.	View commit	Evidence pending

Nhật ký kiểm thử

Evidence pending: attach actual output from:

```
aws sts get-caller-identity
aws ecr describe-repositories --repository-names internship-backend internship-chat
aws ecr describe-images --repository-name internship-backend
aws ecr describe-images --repository-name internship-chat
```

Nhật ký build

Evidence pending: attach GitHub Actions or terminal logs showing Docker image build, tag, push, and digest output.

Nhật ký triển khai

Evidence pending: attach OIDC, ECR, or EKS smoke workflow logs. Do not include secrets or database URLs.

Kết quả trong tuần

Kết quả cá nhân tuần 6 là nền tảng định danh triển khai cloud và phân phối image dùng chung. Các thành viên chịu trách nhiệm về source code và tính đúng đắn của image; tôi phụ trách tích hợp OIDC, IAM, ECR và deployment workflow.

Bài học rút ra

AWS CI/CD work depends on both IAM and runtime evidence. A workflow file can be correct structurally, but I only mark authentication, image push, or cluster access as verified when the corresponding AWS/Actions log is available.

Kế hoạch tuần tiếp theo

Deploy and validate the managed AWS runtime: EKS cluster, namespace, RDS PostgreSQL, ElastiCache Redis, DynamoDB tables, SQS queue/DLQ, IRSA runtime role, and ALB ingress.

1.7 Tuần 7

Tuần 7 - Amazon EKS và các dịch vụ AWS managed

Mục tiêu

Trong nhóm 5 thành viên, mục tiêu tuần 7 của tôi là tích hợp production runtime trên AWS: EKS, managed database, runtime IAM, SQS transport, public ALB routing và automation triển khai an toàn. Các thành viên khác hỗ trợ cấu hình, kiểm tra chức năng cho service mình phụ trách.

Phân việc cá nhân

Trạng thái	Phần việc được phân công	Cơ sở minh chứng
Hoàn thành	Bổ sung logic production pipeline cho EKS, ALB và CloudFront.	Commit 899ff3b.
Hoàn thành	Bổ sung ALB ingress không cần domain cho /api, /chat và /socket.io.	k8s/eks/ingress-alb-no-domain.yaml.
Hoàn thành	Hoàn thiện EKS deployment script cho secret, config, job, rollout, health check và AI service tùy chọn.	scripts/k8s/deploy-eks.sh.
Hoàn thành	Bổ sung chế độ workflow chỉ rollout workload hiện có.	Commit 51bceee và f81e086.
Hoàn thành	Củng cố public ingress deployment dựa trên controller readiness và ALB health check.	Commit 036a516 và 8272c4a.

Triển khai kỹ thuật

The target production environment is centered on EKS namespace `internship`. Backend and chat are long-running Kubernetes Deployments. PostgreSQL, Redis, DynamoDB, SQS, S3, CloudFront, and SageMaker are managed AWS services outside the cluster.

Mô tả sơ bộ. CloudFront chuyển lưu lượng ứng dụng qua ALB công khai đến backend và chat service. Backend dùng RDS, S3 và hàng đợi outbox SQS; chat dùng ElastiCache và DynamoDB; processing worker gọi AI service và SageMaker, còn dispatcher gửi sự kiện vào SQS.

Runtime variables such as `DATABASE_URL`, `REDIS_URL`, `OUTBOX_QUEUE_URL`, and `AWS_REGION` are treated as deployment inputs or Kubernetes secret/config values. The report does not include their secret values.

Vấn đề và giải pháp

Problem	Root cause	Resolution	Status
Public ingress needed to work before a custom domain was available.	Domain and certificate readiness can lag behind backend deployment.	Added <code>ingress-alb-no-domain.yaml</code> with path-based ALB routing.	Completed
AWS Load Balancer Controller may not be ready when ingress is applied.	Webhook endpoints can be unavailable during controller rollout.	Public ingress script waits and retries before applying ingress.	Completed
Health checks can fail briefly while target groups converge.	ALB registration and pod readiness are eventually consistent.	Public deploy script retries ALB health checks.	Completed
EKS rollout may run with limited permissions.	IAM/RBAC permissions may not allow every optional inspection command.	Rollout script tolerates selected limited-permission cases without hiding actual failures.	Completed

Kết quả kiểm thử, build và triển khai



Area	Result	Evidence
EKS deployment script	Implemented	<code>scripts/k8s/deploy-eks.sh</code> validates required variables, applies resources, waits for jobs, rolls out deployments, and performs health smoke checks through port-forwarding.
Public ingress script	Implemented	<code>scripts/aws/deploy-public-ingress.sh</code> waits for controller readiness and polls <code>/api/health/ready</code> and <code>/chat/health/ready</code> .
Rollout mode	Implemented	<code>scripts/aws/rollout-eks-workloads.sh</code> supports restart/scale behavior for backend, chat, dispatcher, and processing worker.
Managed AWS resource proof	Partially completed	<code>PROJECT_CONTEXT.md</code> lists current resource names, but screenshots/CLI logs should be attached as evidence.

Minh chứng

Commit và yêu cầu kéo mã

Commit	Description	Evidence	Pull Request
899ff3b	Added production AWS deployment pipeline, EKS scripts, ALB no-domain ingress, and CloudFront helper.	View commit	Evidence pending
51bceee	Added EKS rollout workflow mode.	View commit	Evidence pending
f81e086	Tolerated limited EKS permissions during rollout.	View commit	Evidence pending
036a516	Waited for ALB webhook before public ingress.	View commit	Evidence pending
8272c4a	Retried ALB health checks during public deploy.	View commit	Evidence pending

Nhật ký kiểm thử

Evidence pending: attach actual output from:

```
aws eks describe-cluster --name internship-prod --region ap-southeast-1
aws rds describe-db-instances --db-instance-identifier internship-prod-postgres --region
→ ap-southeast-1
aws elasticache describe-replication-groups --replication-group-id internship-prod-redis
→ --region ap-southeast-1
aws sqs get-queue-attributes --queue-url <OUTBOX_QUEUE_URL> --attribute-names All
kubectl get serviceaccount -n internship
kubectl describe serviceaccount internship-runtime -n internship
```

Nhật ký build

Evidence pending: attach EKS deployment workflow logs that show the image tags selected for backend, chat, and optional AI service.

Nhật ký triển khai

Evidence pending: attach `kubectl rollout status`, `kubectl get pods -n internship`, and ALB health-check output.

Kết quả trong tuần

Kết quả cá nhân tuần 7 là quy trình AWS tích hợp cho các service của nhóm: rollout EKS workload, ALB routing, cấu hình managed service và kiểm soát vận hành. Điều này không đồng nghĩa với việc cá nhân sở hữu các chức năng ứng dụng chạy trên hạ tầng đó.

Bài học rút ra

Production Kubernetes deployment depends on more than manifests. IAM, controller readiness, target group convergence, network routing, health endpoints, and secret handling must all be validated separately.

Kế hoạch tuần tiếp theo

Run the full build, push, deploy, frontend, SageMaker, and operational-validation path; then document final completed items, partial items, and cost-control actions.

1.8 Tuần 8

Tuần 8 - Triển khai end-to-end và kiểm tra vận hành

Mục tiêu

Trong tuần 8, tôi hoàn thiện phần đóng góp Cloud/DevOps của mình trong project nhóm 5 thành viên. Tôi tích hợp và kiểm tra mô hình production cho các service của nhóm trên Amazon EKS, Application Load Balancer, S3/CloudFront, các dịch vụ dữ liệu AWS managed, event-driven processing và cost tracking.

Tôi sử dụng thư mục minh chứng AWS tuần 08 làm nguồn dữ liệu cho phần này. Số liệu chi phí lấy từ tổng hợp ngày 01-28/07/2026, screenshot Billing credits và Cost Explorer. Hóa đơn tháng đã chốt và AWS Pricing Calculator vẫn chưa có.

Phần việc cá nhân

Trạng thái	Phần việc được phân công	Cơ sở minh chứng
Hoàn thành	Tích hợp AI service theo hướng SageMaker do thành viên AI cung cấp vào deployment path.	Source implementation và sagemaker-health.png.
Hoàn thành	Sửa image propagation của deploy-app và cách chọn ECR image xác định.	CI/CD workflow implementation.
Hoàn thành	Loại frontend khỏi Kubernetes runtime để phân phối qua S3 và CloudFront.	Deployment script và architecture note.
Hoàn thành	Kiểm tra trạng thái Amazon RDS PostgreSQL và ElastiCache Valkey từ screenshot AWS CLI.	RDS-health.png, REDIS-health.png.
Hoàn thành	Kiểm tra trạng thái SageMaker endpoint.	sagemaker-health.png.
Hoàn thành một phần	Kiểm tra EKS workload và ALB runtime.	EKS workload chạy nhưng screenshot ALB còn target không healthy.
Hoàn thành	Bổ sung minh chứng runtime CloudFront, S3, DynamoDB và SQS.	Các screenshot đã được xử lý thông tin nhạy cảm.
Hoàn thành một phần	Bổ sung minh chứng chi phí AWS ngày 01-28/07/2026.	Cost summary và screenshot; Pricing Calculator còn bị chặn.

Triển khai kỹ thuật

The final deployment model separates static frontend hosting from long-running service workloads. Backend, chat, worker, and AI services run on EKS. Static browser assets are intended to be served through S3 and CloudFront, while API, chat, and Socket.IO paths route through the ALB.

Mô tả sơ bộ. Điểm vào production là CloudFront: yêu cầu frontend tĩnh đi đến S3, còn API và chat đi qua ALB đến EKS. Dữ liệu ứng dụng được lưu tại RDS, Redis và DynamoDB; xử lý CV bất đồng bộ dùng worker, AI service và SageMaker; sự kiện outbox đi đến SQS và Lambda thông báo.

The CI/CD workflow supports deployment modes including validate, deploy-app, deploy-public, deploy-frontend, rollout, and full. Backend, chat, and AI images are tagged with the GitHub commit SHA. The frontend deployment path builds the Vite app, publishes static files to S3, and invalidates CloudFront when frontend deployment is enabled.

Minh chứng vận hành AWS

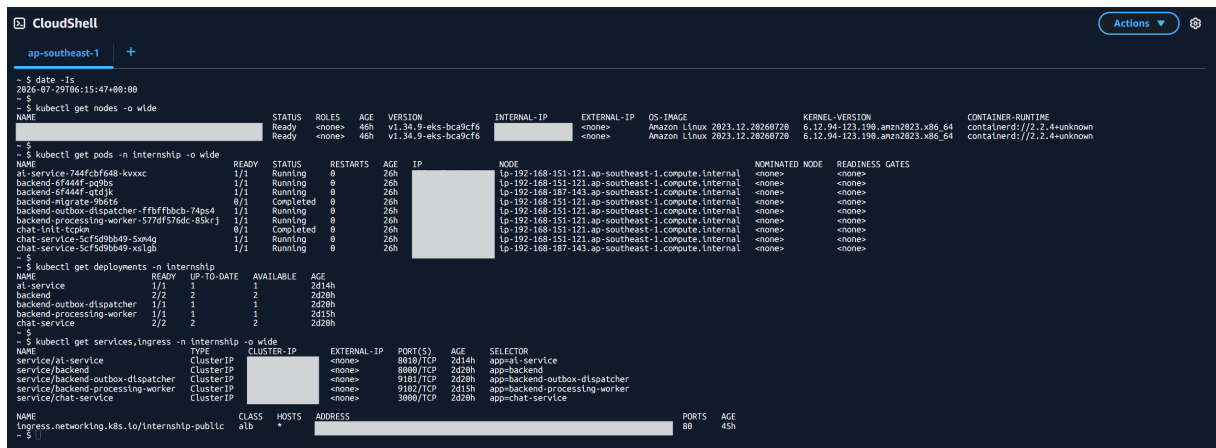
The runtime evidence below uses only files that I placed in the Week 08 AWS evidence directory. I reviewed the following files:

- ALB-target-health.png
- ALB-target.png
- Cloudfront-evidence.png
- Cost-evidence.png
- DynamoDB-evidenc.png
- eks-deployment-rollout-success.png
- eks-runtime-pods-2026-07-28.png
- lambda.png
- Lambda-evidence.png
- lambda-cloudwatch-evidence.png
- RDS-health.png
- REDIS-health.png
- S3-evidenc.png
- sagemaker-health.png
- SQS-evidence.png
- total-cost-july.png

I did not find .txt, .log, .json, .yaml, or .yml evidence files in this directory, so I did not include long CLI log excerpts.

Tổng hợp minh chứng vận hành

AWS service	Evidence status	Runtime status	Evidence
Amazon EKS	Available	Partially verified: nodes, pods, jobs, and deployments were healthy; cluster ACTIVE status was not directly captured	eks-runtime-pods-2026-07-28.png, eks-deployment-rollout-success.png
Application Load Balancer	Available	Partially operational: load balancer was active and target groups existed, but target health was mixed and included unhealthy targets	ALB-target-health.png, ALB-target.png
Amazon CloudFront	Available	Partially verified: I captured monitoring metrics for requests, data transfer, and distribution error rates	Cloudfront-evidence.png
Amazon S3	Available	Verified frontend bucket objects after account-suffix redaction	S3-evidenc.png
Amazon RDS	Available	Available	RDS-health.png
Amazon ElastiCache	Available	Available	REDIS-health.png
Amazon DynamoDB	Available	Verified: chat tables and Lambda dedupe table are Active and on-demand	DynamoDB-evidenc.png
Amazon SQS	Available	Verified: main queue and DLQ exist, show zero visible/in-flight messages, and use SSE-SQS	SQS-evidence.png
AWS Lambda	Available	Partially verified: function configuration, source view, and CloudWatch log group exist; invocation result still not verified	lambda.png, Lambda-evidence.png, lambda-cloudwatch-evidence.png
Amazon SageMaker	Available	InService	sagemaker-health.png



```
CloudShell
ap-southeast-1 +
~ $ date -Is
2026-07-29T06:15:47+08:00
~ $
~ $ kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                                     KERNEL-VERSION   CONTAINER-RUNTIME
v1-34-9-eks-bca9cfe                 Ready    <none>   46h   v1.34.9-eks-bca9cfe   <none>         <none>         Amazon Linux 2023.12.20260720             6.12.94-123.198.amzn2023.x86_64   containerd://2.2.4-unknown

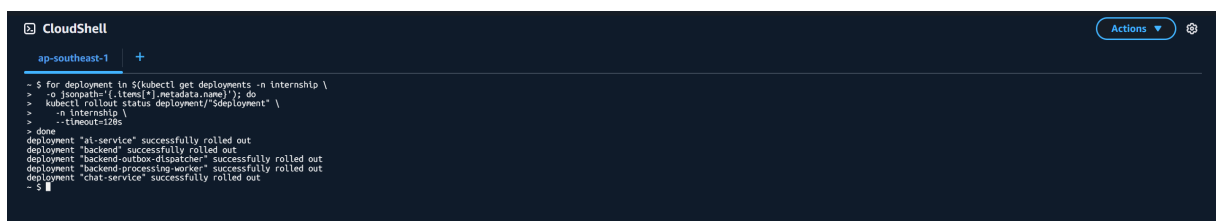
~ $ kubectl get pods -n internship -o wide
NAME                                READY    STATUS    RESTARTS   AGE   IP              NODE                                     NOMINATED NODE   READINESS GATES
ai-service-744fcf648-kvxxc           1/1      Running   0           26h   ip-192-168-151-121.ap-southeast-1.compute.internal   <none>           <none>
backend-6f44ef-pq8us                 1/1      Running   0           26h   ip-192-168-151-121.ap-southeast-1.compute.internal   <none>           <none>
backend-6f44ef-qtdjk                 1/1      Running   0           26h   ip-192-168-107-143.ap-southeast-1.compute.internal   <none>           <none>
backend-migrate-9b6t6                0/1      Completed 0           26h   ip-192-168-151-121.ap-southeast-1.compute.internal   <none>           <none>
backend-outbox-dispatcher-fbf9fbcb-74ps4 1/1      Running   0           26h   ip-192-168-151-121.ap-southeast-1.compute.internal   <none>           <none>
backend-processing-worker-577df56dc-8skrj 1/1      Running   0           26h   ip-192-168-151-121.ap-southeast-1.compute.internal   <none>           <none>
chat-init-icpss                      0/1      Completed 0           26h   ip-192-168-151-121.ap-southeast-1.compute.internal   <none>           <none>
chat-service-5cf5d9b49-5umtg         1/1      Running   0           26h   ip-192-168-151-121.ap-southeast-1.compute.internal   <none>           <none>
chat-service-5cf5d9b49-xslgb         1/1      Running   0           26h   ip-192-168-107-143.ap-southeast-1.compute.internal   <none>           <none>

~ $ kubectl get deployments -n internship
NAME          READY    UP-TO-DATE   AVAILABLE   AGE
ai-service    1/1      1             1            2d14h
backend       2/2      2             2            2d20h
backend-outbox-dispatcher 1/1      1             1            2d20h
backend-processing-worker 1/1      1             1            2d15h
chat-service  2/2      2             2            2d20h

~ $ kubectl get services,ingress -n internship -o wide
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE   SELECTOR
service/ai-service  ClusterIP   <none>        <none>         8080/TCP   2d14h   app=ai-service
service/backend     ClusterIP   <none>        <none>         8080/TCP   2d20h   app=backend
service/backend-outbox-dispatcher ClusterIP   <none>        <none>         9181/TCP   2d20h   app=backend-outbox-dispatcher
service/backend-processing-worker ClusterIP   <none>        <none>         9182/TCP   2d15h   app=backend-processing-worker
service/chat-service ClusterIP   <none>        <none>         3000/TCP   2d20h   app=chat-service

NAME          CLASS   HOSTS   ADDRESS   PORTS   AGE
ingress.networking.k8s.io/internship-public  alb     *       <none>     80      45h
```

Hình 1: Amazon EKS runtime pods and services



```
CloudShell
ap-southeast-1 +
~ $ for deployment in $(kubectl get deployments -n internship \
> -o jsonpath='{range [?(@.metadata.name)]} { \
> kubectl rollout status deployment/"$deployment" \
> -n internship \
> --timeout=120s \
> done \
> deployment "$deployment" successfully rolled out \
> deployment "backend" successfully rolled out \
> deployment "backend-outbox-dispatcher" successfully rolled out \
> deployment "backend-processing-worker" successfully rolled out \
> deployment "chat-service" successfully rolled out \
~ $
```

Hình 2: Amazon EKS deployment rollout success

Amazon EKS

Evidence status: Available **Runtime status:** Partially verified

I verified two Ready worker nodes, running application pods, completed migration/init jobs, and ready deployments in the internship namespace. The visible workloads include ai-service, backend, backend-outbox-dispatcher, backend-processing-worker, and chat-service.

The same evidence confirms deployment readiness:

- ai-service: 1/1 ready
- backend: 2/2 ready
- backend-outbox-dispatcher: 1/1 ready
- backend-processing-worker: 1/1 ready
- chat-service: 2/2 ready

I did not capture an `aws eks describe-cluster` output showing the EKS cluster status as ACTIVE. For that reason, I classify EKS as partially verified instead of claiming that full cluster-level evidence was captured.

Application Load Balancer

Evidence status: Available **Runtime status:** Partially operational

I captured an internet-facing Application Load Balancer in active state. However, the target health evidence still shows mixed target status:

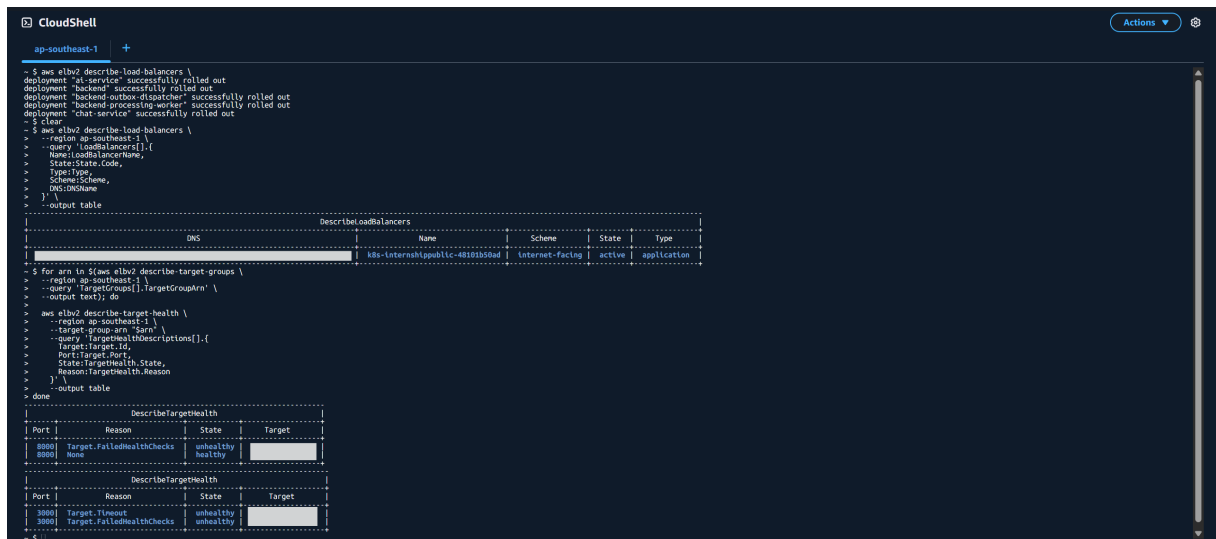
- One backend target on port 8000 was healthy.
- Another backend target on port 8000 was unhealthy with failed health checks.
- Chat targets on port 3000 were unhealthy with timeout or failed health-check reasons.

Because of the unhealthy targets, I do not classify the ALB as fully healthy. The load balancer was deployed, active, and associated with target groups, but the target groups still require follow-up validation and remediation.

Amazon CloudFront và S3

Evidence status: Available **Runtime status:** Partially verified

I captured CloudFront monitoring metrics for distribution EQIGYNECXDYL8, including requests, data transfer, and error-rate graphs over the selected three-day window. This confirms that the distribution was receiving teleme-

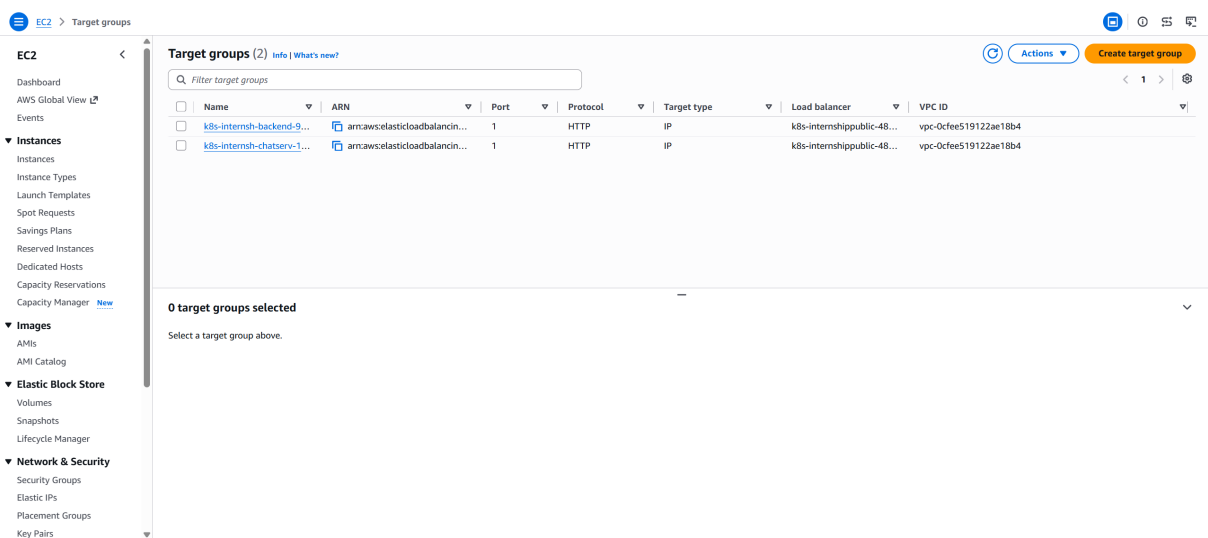


```
> $ aws elbv2 describe-load-balancers \
  deployment "it-service" successfully rolled out
  deployment "backend" successfully rolled out
  deployment "backend-outbox-dispatcher" successfully rolled out
  deployment "backend-processing-worker" successfully rolled out
  deployment "chat-service" successfully rolled out
  $ clear
  > $ aws elbv2 describe-load-balancers \
  --region ap-southeast-1 \
  --query 'LoadBalancers[0].{
  >   Name:LoadBalancerName,
  >   State:State.Code,
  >   Type:Type,
  >   Scheme:Scheme,
  >   DNS:DNSName
  > }' \
  --output table
  +-----+-----+-----+-----+-----+
  |                                     DescribeLoadBalancers                                     |
  +-----+-----+-----+-----+-----+
  | DNS | Name | Scheme | State | Type |
  +-----+-----+-----+-----+-----+
  | k8s-internshpublic-48181358ad | k8s-internshpublic-48181358ad | internet-facing | active | application |
  +-----+-----+-----+-----+-----+

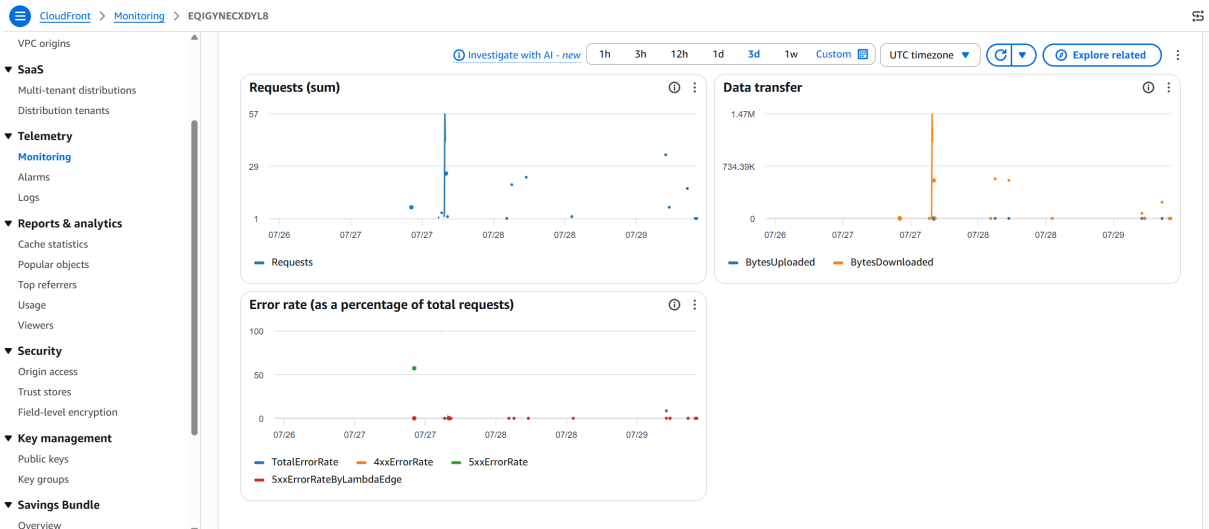
  > $ for arn in $(aws elbv2 describe-target-groups \
  --region ap-southeast-1 \
  --query 'TargetGroups[0].TargetGroupArn' \
  --output text); do
  >   aws elbv2 describe-target-health \
  --region ap-southeast-1 \
  --target-group-arn "$arn" \
  --query 'TargetHealthDescriptions[0].{
  >   Target:TargetId,
  >   Port:Target.Port,
  >   State:TargetHealth.State,
  >   Reason:TargetHealth.Reason
  > }' \
  --output table
  > done
  +-----+-----+-----+-----+
  |                                     DescribeTargetHealth                                     |
  +-----+-----+-----+-----+
  | Port | Reason | State | Target |
  +-----+-----+-----+-----+
  | 8080 | Target.FailedHealthChecks | unhealthy | [REDACTED] |
  | 8080 | None | healthy | [REDACTED] |
  +-----+-----+-----+-----+

  +-----+-----+-----+-----+
  |                                     DescribeTargetHealth                                     |
  +-----+-----+-----+-----+
  | Port | Reason | State | Target |
  +-----+-----+-----+-----+
  | 3000 | Target.Timeout | unhealthy | [REDACTED] |
  | 3000 | Target.FailedHealthChecks | unhealthy | [REDACTED] |
  +-----+-----+-----+-----+
  > $
```

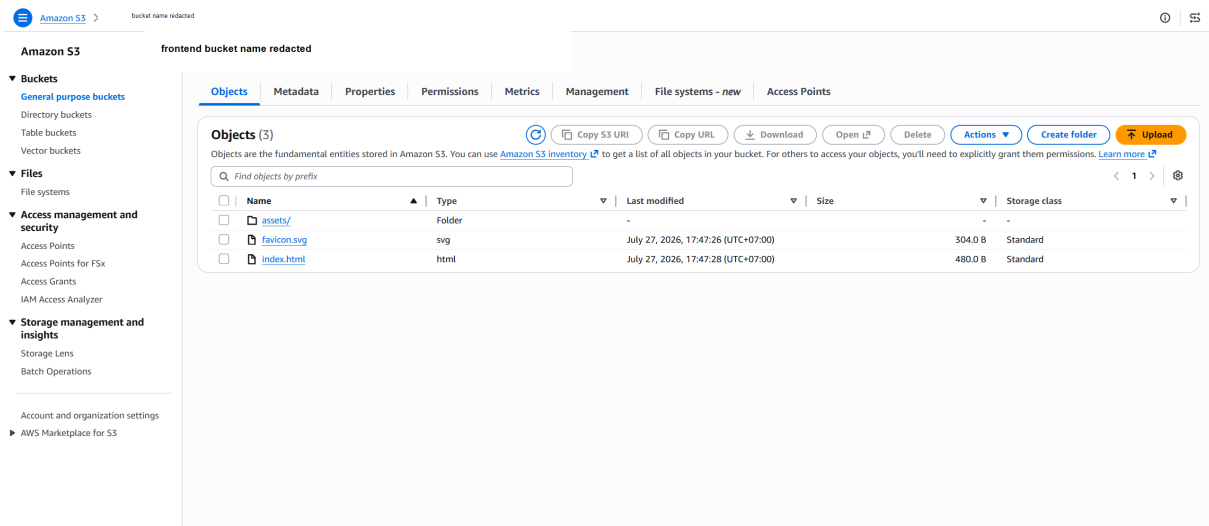
Hình 3: Application Load Balancer target health



Hình 4: Application Load Balancer target groups



Hình 5: CloudFront monitoring evidence



Hình 6: S3 frontend bucket object evidence

I captured the frontend S3 bucket object listing after redacting the bucket name. The visible objects include `assets/`, `favicon.svg`, and `index.html`, which verifies that static frontend assets were present in S3.

These screenshots support frontend hosting evidence, but they do not replace a full CloudFront behavior/origin configuration export or a browser smoke test through the CloudFront URL.

Amazon RDS

Evidence status: Available **Runtime status:** Available

I verified the PostgreSQL database instance `internship-prod-postgres` in status `available`. The captured output also includes PostgreSQL engine metadata, encrypted storage, non-public accessibility, and allocated storage.

Amazon ElastiCache

Evidence status: Available **Runtime status:** Available

I verified the ElastiCache replication group `internship-prod-redis` with Valkey engine status `available`. The captured evidence also confirms encryption at rest and in transit.

Amazon DynamoDB

Evidence status: Available **Runtime status:** Verified Active

I verified four DynamoDB tables in `Active` status:

```
CloudShell
ap-southeast-1
$ aws elbv2 describe-load-balancers \
  --deployment "ai-service" --successfully-rolled-out
$ aws rds describe-db-instances \
  --db-instance-identifier internship-prod-postgres \
  --region ap-southeast-1 \
  --query 'DBInstances[0]' \
  --output table
DescribeDBInstances
+-----+-----+-----+-----+-----+-----+-----+-----+
| Class | Encrypted | Engine | Identifier | Public | Status | Storage | Version |
+-----+-----+-----+-----+-----+-----+-----+-----+
| db.t4g.micro | True | postgres | internship-prod-postgres | False | available | 20 | 18.3 |
+-----+-----+-----+-----+-----+-----+-----+-----+
$
```

Hình 7: Amazon RDS PostgreSQL health

```
CloudShell
ap-southeast-1
$ aws elbv2 describe-load-balancers \
  --deployment "ai-service" --successfully-rolled-out
$ aws elasticache describe-replication-groups \
  --replication-group-id internship-prod-redis \
  --region ap-southeast-1 \
  --query 'ReplicationGroups[0]' \
  --output table
DescribeReplicationGroups
+-----+-----+-----+-----+-----+
| EncryptionAtRest | EncryptionInTransit | Engine | Id | Status |
+-----+-----+-----+-----+-----+
| True | True | valkey | internship-prod-redis | available |
+-----+-----+-----+-----+-----+
$
```

Hình 8: Amazon ElastiCache Valkey health

- ChatGroups
- ChatMessages
- ChatUsers
- InternshipLambdaEventDedupe

The captured table list also shows on-demand read and write capacity mode. This verifies that the AWS DynamoDB tables existed and were active at the time of evidence collection.

Amazon SQS

Evidence status: Available **Runtime status:** Verified available

I verified two Amazon SQS standard queues:

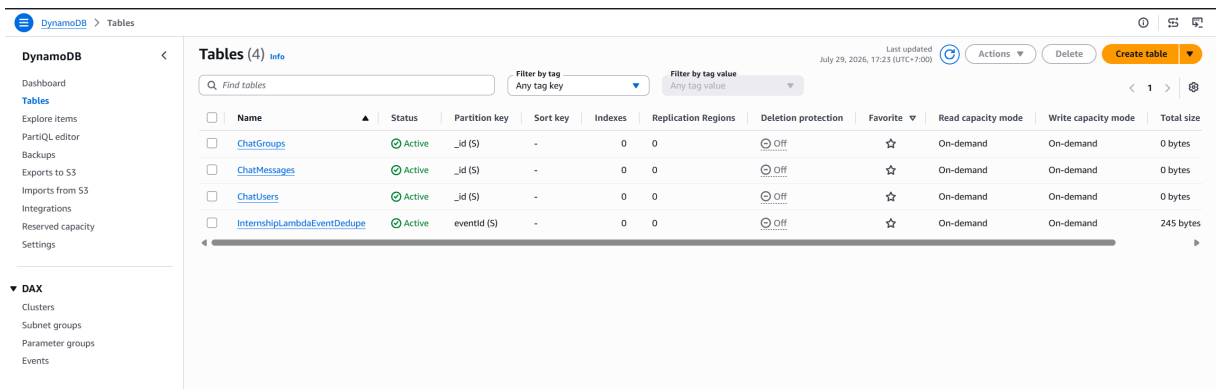
- internship-prod-outbox
- internship-prod-outbox-dlq

Both queues had zero visible messages and zero in-flight messages in the captured evidence. The queues use Amazon SQS managed server-side encryption.

AWS Lambda

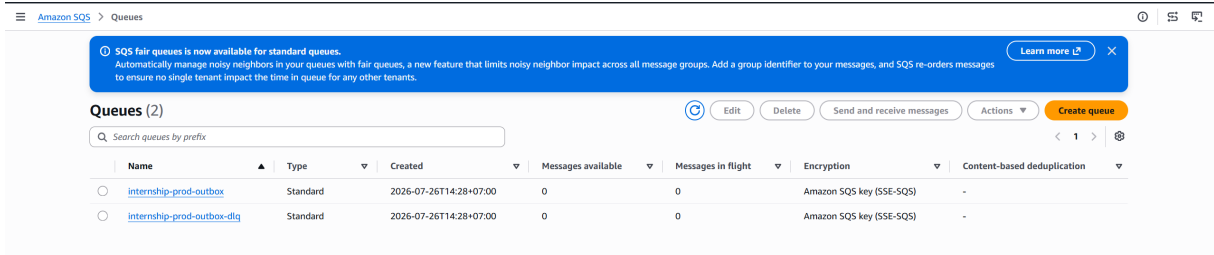
Evidence status: Available **Runtime status:** Partially verified

I verified a Lambda function named `internship-outbox-handler` with a Python runtime and environment configuration. This confirms that the function existed in the selected AWS region.

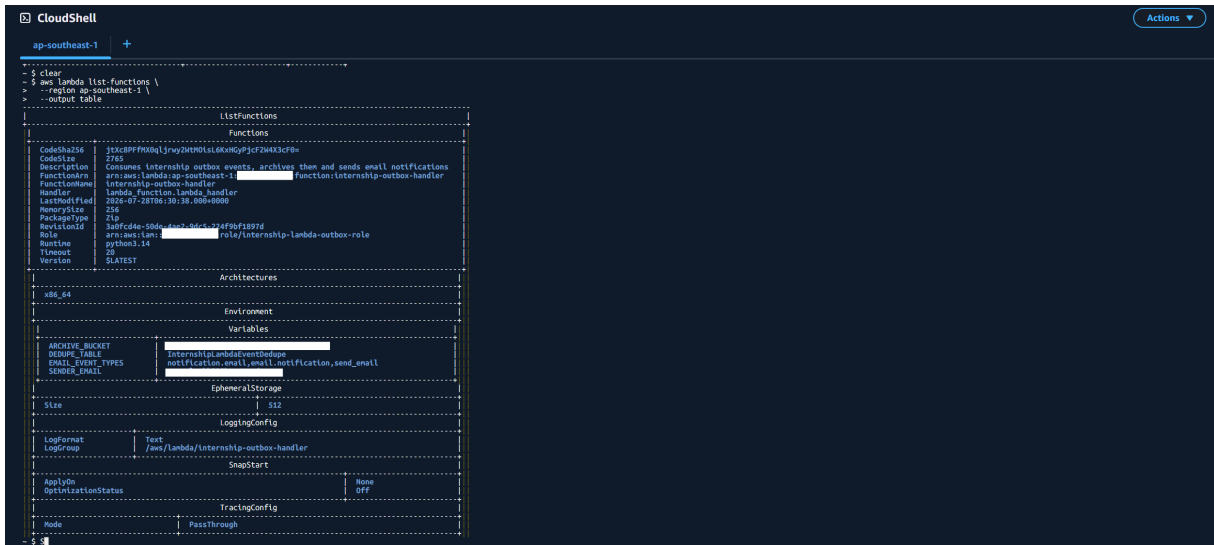


Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode	Total size
ChatGroups	Active	_id (S)	-	0	0	Off	☆	On-demand	On-demand	0 bytes
ChatMessages	Active	_id (S)	-	0	0	Off	☆	On-demand	On-demand	0 bytes
ChatUsers	Active	_id (S)	-	0	0	Off	☆	On-demand	On-demand	0 bytes
InternshipLambdaEventDedupe	Active	eventid (S)	-	0	0	Off	☆	On-demand	On-demand	245 bytes

Hình 9: DynamoDB table status evidence



Hình 10: SQS queue evidence



Hình 11: AWS Lambda function configuration

The screenshot was sanitized before publication by redacting account-scoped ARNs, the IAM role ARN account segment, the bucket value containing an account identifier, and the sender email address.

Additional Lambda evidence captures the function source view and a CloudWatch log group named `/aws/lambda/internship-outbox-handler`. These items confirm that the function code view and log group existed. They do not prove a successful invocation, SQS trigger execution, or SES delivery result, so I keep Lambda runtime execution as partially verified.

Amazon SageMaker

Evidence status: Available **Runtime status:** InService

I verified the SageMaker endpoint `internship-qwen3-4b` in status `InService`. This is sufficient to show that the endpoint existed and was serving at the time of evidence collection.

Mình chứng chỉ phí AWS

Status: Partially available

The July 1-28, 2026 AWS cost summary reported a month-to-date spend of **\$94.92**. I also included a Billing and Cost Management credits screenshot stored as `total-cost-july.png` and a Cost Explorer overview screenshot stored as `Cost-evidence.png`.

The credits screenshot records AWS credits rather than a grouped Cost Explorer service chart. The Cost Explorer screenshot is useful as a billing-console overview, but it uses a different report configuration and is not a Pricing Calculator estimate. Therefore, I keep the service-level cost breakdown tied to the July 1-28 cost summary.

Tổng hợp chi phí từ ngày 1 đến 28 tháng 7

Metric	Reported value
Total spend, July 1-28	\$94.92



Metric	Reported value
Highest daily spend	\$31.83 on July 28
AWS credits total amount used	\$27.90
AWS credits total estimated amount used	\$140.65
AWS credits total amount remaining	\$172.10
AWS credits total estimated amount remaining	\$59.35

Xu hướng chi phí hằng ngày

Period	Daily spend
July 1-8	Approximately \$2.20/day
July 9-25	Approximately \$1.60/day
July 26	\$5.61
July 27	\$11.60
July 28	\$31.83

The July 28 daily cost was the largest reported daily spend in the July 1-28 period and was much higher than the earlier baseline.

Các dịch vụ phát sinh chi phí cao nhất

Rank	Service	Month-to-date cost	Share of total
1	Amazon RDS	\$29.69	31.3%
2	Amazon SageMaker	\$23.45	24.7%
3	Amazon VPC	\$14.11	14.9%
4	Amazon EC2 - Compute	\$12.42	13.1%
5	EC2 - Other	\$7.68	8.1%
6	Amazon EKS	\$5.64	5.9%

These six services account for approximately 98% of the July 1-28 reported spend.

Các bất thường về chi phí

Five active anomalies were reported around July 26-28 in ap-southeast-1:

Service	Anomaly impact	Reported context
Amazon EBS	+\$6.02, +15,050% versus expected	NAT Gateway activity in ap-southeast-1
Amazon EC2 - Compute	+\$2.56, +400% versus expected	New t3.medium instances in ap-southeast-1
Amazon RDS	+\$1.12, +138% versus expected	New db.t4g.micro instance activity in ap-southeast-1
Amazon VPC	+\$0.47, +98% versus expected	VPC/NAT Gateway infrastructure in ap-southeast-1
AWS Secrets Manager	+\$0.01, +50% versus expected	Minor secret-management cost tied to new infrastructure

Cost evidence	Status	Reported value
July 1-28 cost summary	Available	\$94.92
AWS Billing credits screenshot	Available	\$27.90 total amount used; \$140.65 total estimated amount used



Cost evidence	Status	Reported value
Cost Explorer overview screenshot	Available	Included as console evidence; not used as the final service-level breakdown
AWS Bills finalized monthly invoice	Pending	Not available
AWS Pricing Calculator	Blocked	Not available
Actual month-to-date cost	Reported for July 1-28	\$94.92
Estimated monthly cost	Not calculated	N/A

Kết quả kiểm thử, build và triển khai

Area	Result	Evidence
EKS workload rollout	Partially verified	I captured successful rollout output for all listed deployments.
EKS runtime pods and services	Partially verified	I captured Ready nodes, Running pods, Completed jobs, ready deployments, and services/ingress.
ALB deployment	Partially operational	I captured ALB state <code>active</code> , but target health still includes unhealthy targets.
RDS PostgreSQL	Verified available	I captured RDS status <code>available</code> .
ElastiCache Valkey	Verified available	I captured ElastiCache status <code>available</code> .
SageMaker endpoint	Verified InService	I captured SageMaker endpoint status <code>InService</code> .
Lambda function listing	Partially verified	Screenshots confirm function configuration, source view, and CloudWatch log group; invocation and trigger health are not verified.
CloudFront and S3 frontend delivery	Partially verified	CloudFront monitoring and S3 object listing screenshots are available; full behavior/origin export and browser smoke proof remain pending.
DynamoDB and SQS	Verified available	DynamoDB tables are Active; SQS main queue and DLQ are listed with zero messages and SSE-SQS.
Cost evidence	Partially available	July 1-28 cost summary, credits screenshot, and Cost Explorer overview are available; finalized bill and Pricing Calculator remain pending/blocked.

Vấn đề và giải pháp

Các nhóm đích ALB chưa hoàn toàn khỏe mạnh

Problem: I captured the load balancer in `active` state, but target health still included unhealthy backend and chat targets.

Impact: API or chat traffic through the public ALB could fail intermittently or return gateway errors until every required target group becomes healthy.

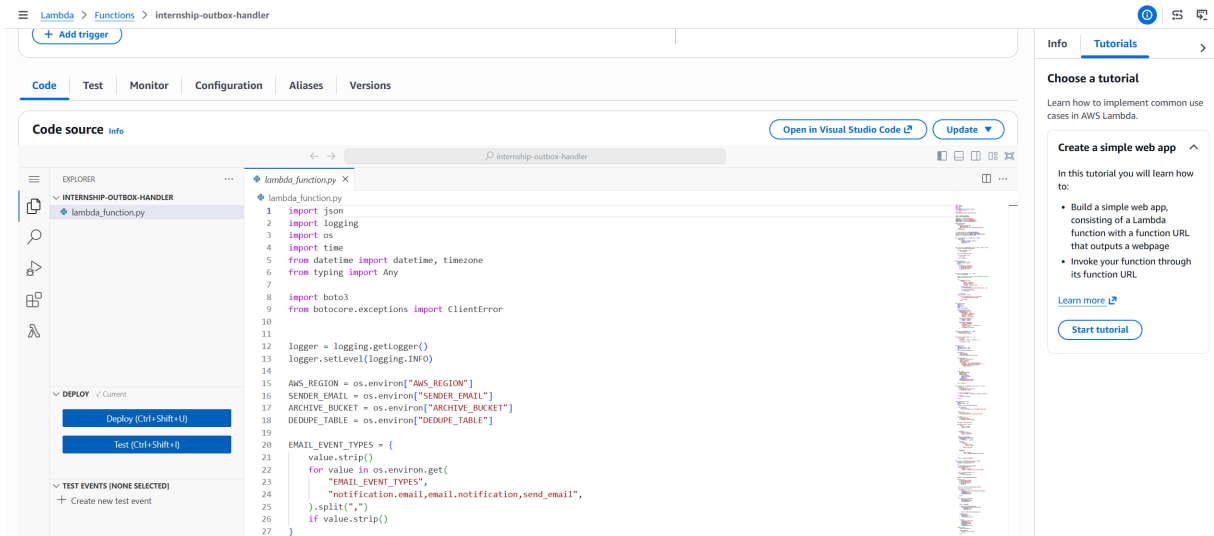
Current resolution: I classify the ALB as partially operational rather than fully healthy.

Next action: Collect updated target health evidence after fixing health-check failures or deployment readiness issues.

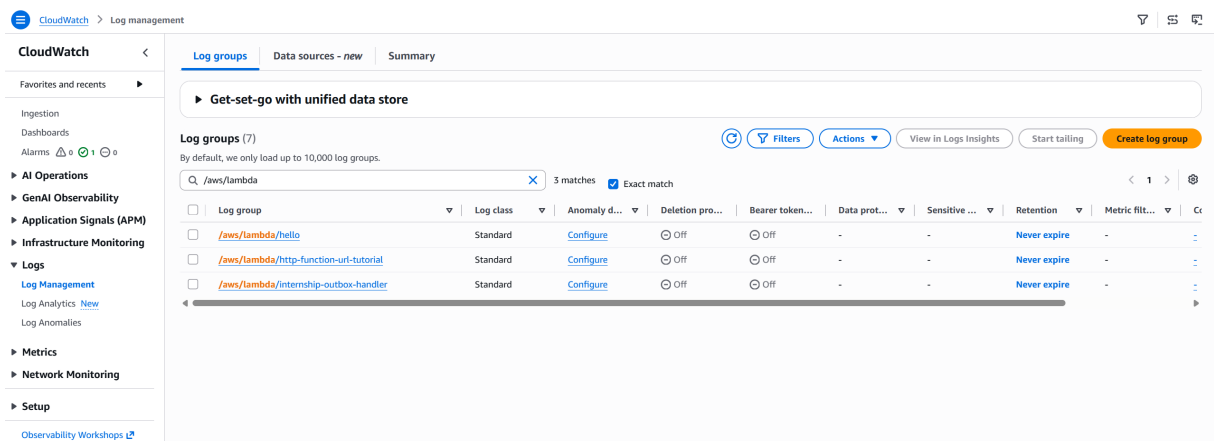
Mình chứng chi phí AWS đã có nhưng chưa đầy đủ

Problem: A July 1-28 cost summary and credits screenshot are available, but the evidence set still does not include a finalized monthly bill or AWS Pricing Calculator estimate.

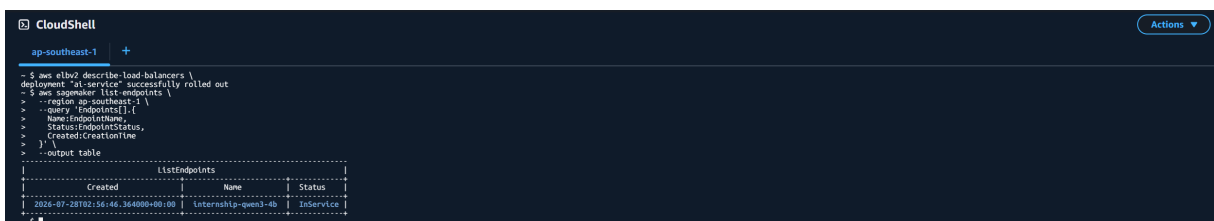
Impact: I can show the July 1-28 month-to-date spend and major cost drivers, but I cannot conclude the final monthly bill or steady-state monthly cost yet.



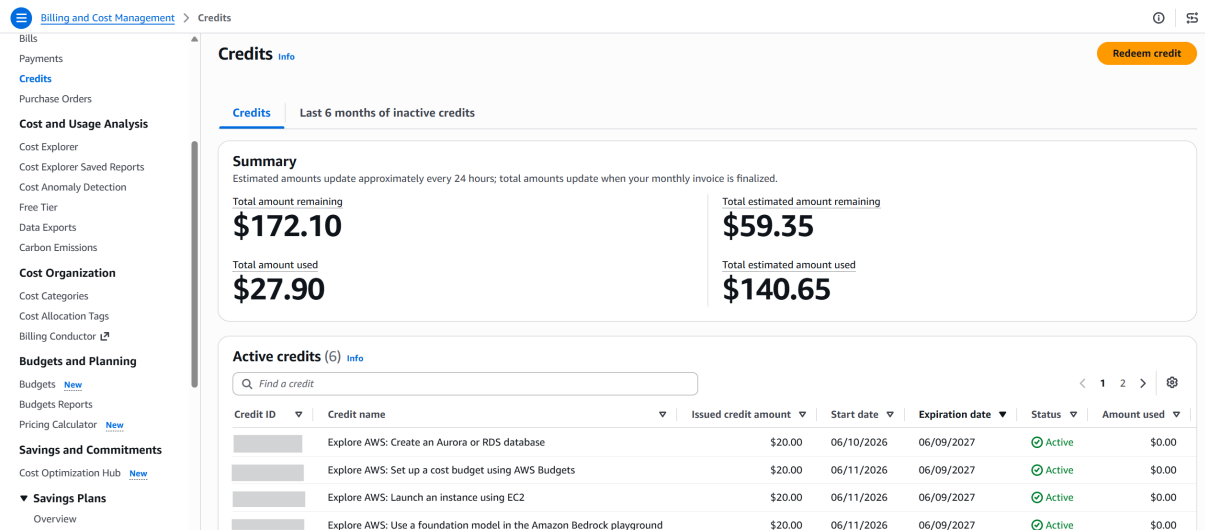
Hình 12: AWS Lambda source evidence



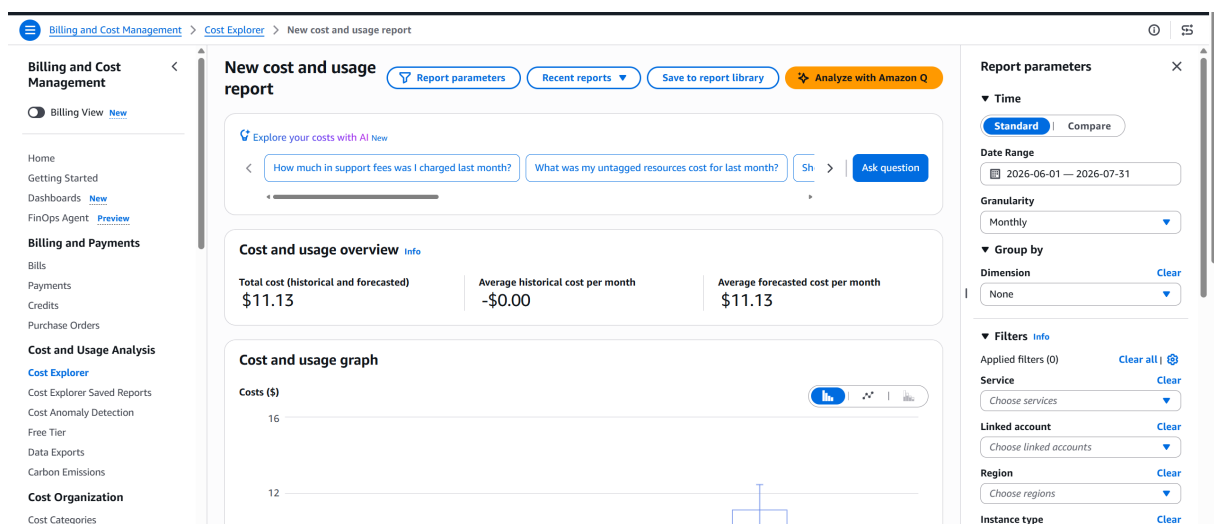
Hình 13: AWS Lambda CloudWatch log group evidence



Hình 14: Amazon SageMaker endpoint status



Hình 15: AWS Billing credits summary for July 2026



Hình 16: AWS Cost Explorer overview evidence

Current resolution: I classify cost evidence as partially available, report only the July 1-28 figures, and keep monthly estimate fields pending.

Next action: Collect the finalized AWS Bills view and build an AWS Pricing Calculator estimate for the production architecture.

Đã bổ sung minh chứng vận hành cho một số dịch vụ AWS

Problem: The first Week 8 evidence set did not include CloudFront, S3, DynamoDB, or SQS screenshots.

Impact: Without screenshots or sanitized CLI output, I could not classify those services as runtime evidenced.

Current resolution: Sanitized screenshots were added for CloudFront monitoring, the S3 frontend bucket object list, DynamoDB table status, and SQS queue status.

Next action: For final production evidence, add CloudFront behavior/origin configuration export and a browser smoke test through the CloudFront URL.

Ảnh chụp Lambda cần được xử lý thông tin nhạy cảm

Problem: The Lambda screenshot contained account-scoped ARNs, an IAM role ARN, a bucket name containing an account identifier, and an email address.

Impact: Embedding the screenshot directly would publish sensitive or semi-sensitive operational values.

Current resolution: I sanitized and embedded the screenshot as Lambda deployment evidence. It confirms the function listing/configuration, but not invocation success.

Next action: Collect a separate invocation, trigger, or CloudWatch event-detail screenshot if Lambda runtime execution must be marked fully verified.

Kết quả trong tuần

Phần đóng góp cá nhân tuần 8 tạo được bộ minh chứng AWS runtime cho compute và data path chính:

- EKS workloads were running and deployments had rolled out.
- I verified RDS PostgreSQL as available.
- I verified ElastiCache Valkey as available.
- CloudFront monitoring and S3 frontend object evidence were added.
- DynamoDB tables and SQS queues were verified from AWS console screenshots.
- SageMaker endpoint evidence showed InService.
- Sanitized Lambda evidence confirms the function, source view, and CloudWatch log group, while invocation proof remains pending.

Ứng dụng end-to-end vẫn là kết quả chung của nhóm. Trong phạm vi của tôi, các khoảng trống còn lại được ghi rõ: ALB target chưa hoàn toàn healthy, còn thiếu CloudFront behavior/origin export và browser smoke test, đồng thời số liệu chi phí cần hóa đơn đã chốt và Pricing Calculator.

Công việc còn lại

- ☒ Add the July 1-28 AWS cost summary.
- ☒ Add the AWS Billing credits screenshot.
- ☒ Add a Cost Explorer overview screenshot.
- ☐ Capture the finalized AWS Billing summary.
- ☐ Build an AWS Pricing Calculator estimate. This is currently blocked.
- ☒ Capture CloudFront monitoring evidence.
- ☒ Capture S3 frontend bucket object evidence.
- ☒ Capture Amazon DynamoDB table status evidence.
- ☒ Capture Amazon SQS queue and message-count evidence.
- ☐ Capture updated ALB target health after every required target is healthy.
- ☐ Capture CloudFront behavior/origin configuration or browser smoke proof.
- ☒ Add a sanitized Lambda screenshot.

Bài học rút ra

Implementation evidence and runtime evidence answer different questions. A deployment script proves that a path exists in source control, but screenshots, AWS CLI output, and health checks prove what actually ran in AWS.

Cost reporting needs the same discipline. The July 1-28 summary is useful month-to-date evidence, but the final monthly bill and steady-state monthly estimate still require separate billing and Pricing Calculator evidence.

2 Đề xuất

Internship Application Platform

Nền tảng quản lý ứng tuyển thực tập tích hợp AI, được triển khai trên AWS

Tóm tắt dự án

Internship Application Platform là một nền tảng tập trung giúp sinh viên, ứng viên và các doanh nghiệp quản lý toàn bộ quá trình tuyển dụng thực tập trên một hệ thống duy nhất.

Đối với Ứng viên, hệ thống hỗ trợ tạo hồ sơ, tìm kiếm cơ hội thực tập, theo dõi trạng thái ứng tuyển, quản lý CV và tài liệu, liên lạc trực tiếp với HR, đồng thời sử dụng AI để đánh giá độ tương thích giữa CV và Mô tả công việc (Job Description).

Đối với HR và các doanh nghiệp, hệ thống hỗ trợ quản lý hồ sơ công ty, đăng tải thông tin tuyển dụng, xem xét ứng viên, cập nhật trạng thái tuyển dụng, liên lạc trực tiếp với Ứng viên, và áp dụng AI để phân tích, so sánh, xếp hạng CV.

Dự án được thiết kế theo kiến trúc đa dịch vụ (multi-service architecture), bao gồm:

- Frontend phát triển bằng React và Vite.
- Backend REST API sử dụng FastAPI.
- Dịch vụ chat sử dụng Node.js, Socket.IO, Redis và DynamoDB.
- Dịch vụ AI để phân tích CV, phân tích Mô tả công việc và xếp hạng ứng viên.
- PostgreSQL để lưu trữ dữ liệu nghiệp vụ.
- Amazon S3 để lưu trữ CV và tài liệu.
- Docker và Kubernetes để tiêu chuẩn hóa môi trường triển khai.
- Hệ thống giám sát (observability) để thu thập metrics, nhật ký (logs) và truy vết phân tán (distributed traces).
- GitHub Actions để chạy CI/CD và kiểm tra bảo mật.

Kiến trúc thực tế (production) được quy hoạch triển khai trên AWS sử dụng Amazon EKS, Amazon ECR, Amazon RDS for PostgreSQL, Amazon DynamoDB, Amazon ElastiCache, Amazon S3, Amazon CloudFront, Application Load Balancer và các dịch vụ giám sát phù hợp.

Bài toán đặt ra

Các vấn đề hiện tại

Trong quá trình tìm kiếm chỗ thực tập, sinh viên thường quản lý thông tin ứng tuyển qua bảng tính (spreadsheets), ghi chú cá nhân, email hoặc qua nhiều nền tảng riêng lẻ. Cách tiếp cận này phát sinh nhiều bất cập:

- Thông tin về công việc và công ty bị phân tán.
- Khó theo dõi chính xác trạng thái của từng đơn ứng tuyển.
- CV, bảng điểm, chứng chỉ và các tài liệu liên quan bị lưu trữ ở nhiều nơi khác nhau.
- Ứng viên có rất ít công cụ hỗ trợ để đánh giá xem CV của mình có phù hợp với Mô tả công việc hay không.
- Quá trình trao đổi với HR không được gắn kết trực tiếp với hồ sơ ứng tuyển.
- Ứng viên dễ bỏ sót hạn nộp, lịch phỏng vấn hoặc các yêu cầu bổ sung tài liệu.

Đối với bộ phận HR và nhà tuyển dụng, quy trình xử lý thủ công cũng tồn tại nhiều hạn chế:

- Khâu đọc và phân loại CV tiêu tốn một lượng thời gian lớn.
- Khó so sánh nhiều ứng viên một cách khách quan dựa trên tiêu chí đồng nhất.
- Trạng thái ứng tuyển có thể không được cập nhật kịp thời và đồng nhất.
- Dữ liệu ứng viên có thể bị lưu trữ thiếu bảo mật.
- Giao tiếp giữa HR và Ứng viên bị phân mảnh trên nhiều kênh khác nhau.
- Thiếu công cụ tập trung để theo dõi các hoạt động và hiệu quả tuyển dụng.

Giải pháp đề xuất

Internship Application Platform cung cấp một hệ thống tập trung cho cả Ứng viên và HR.

Ứng viên có thể:

- Đăng ký và quản lý tài khoản.
- Cập nhật hồ sơ cá nhân.
- Tìm kiếm và xem thông tin tuyển dụng.

- Nộp hồ sơ ứng tuyển.
- Theo dõi trạng thái tuyển dụng.
- Tải lên và quản lý CV, chứng chỉ, bảng điểm.
- Trò chuyện trực tiếp với HR.
- Sử dụng AI để phân tích độ tương thích của CV với công việc.

Bộ phận HR và các doanh nghiệp có thể:

- Tạo và quản lý hồ sơ công ty.
- Đăng tải và chỉnh sửa thông tin tuyển dụng.
- Xem danh sách các hồ sơ đã nộp.
- Cập nhật trạng thái tuyển dụng của ứng viên.
- Trò chuyện trực tiếp với Ứng viên.
- Sử dụng AI để trích xuất thông tin, tính điểm tương thích và hỗ trợ sắp xếp, xếp hạng ứng viên.

Các tác vụ tốn thời gian như đọc tài liệu, phân tích CV và tính điểm xếp hạng lại (reranking) được xử lý bất đồng bộ thông qua các background workers để không làm trễ (block) luồng phản hồi của API chính.

Lợi ích và giá trị

Đối với Ứng viên, nền tảng giúp:

- Quản lý toàn bộ quy trình ứng tuyển tại một nơi duy nhất.
- Giảm rủi ro thất lạc tài liệu và thông tin tuyển dụng.
- Theo dõi rõ ràng trạng thái của từng đơn ứng tuyển.
- Nhận diện được các kỹ năng phù hợp và các kỹ năng còn thiếu.
- Giao tiếp với nhà tuyển dụng thuận tiện hơn.

Đối với HR, nền tảng giúp:

- Giảm bớt thời gian xử lý CV thủ công.
- Tập trung quản lý thông tin ứng viên và trạng thái tuyển dụng.
- Tiêu chuẩn hóa quy trình đánh giá hồ sơ.
- Hỗ trợ ra quyết định dựa trên dữ liệu.
- Cải thiện công tác theo dõi và điều phối tuyển dụng.

Đối với nhóm phát triển, dự án là cơ hội để vận dụng trọn vẹn kiến thức về phát triển full-stack, kiến trúc Cloud, tích hợp AI, liên lạc thời gian thực, Kubernetes, CI/CD, hệ thống giám sát và bảo mật hệ thống.

Kiến trúc giải pháp

Đề xuất ban đầu định hướng xây dựng một kiến trúc đa dịch vụ trên AWS kết hợp Kubernetes cho các tải xử lý container, cơ sở dữ liệu được quản lý (managed databases), lưu trữ đối tượng (object storage), chat thời gian thực, hệ thống giám sát và CI/CD. Trong triển khai thực tế, tôi giữ vững định hướng này và tinh chỉnh lại hai vùng trọng yếu sau khi hoàn thành công việc triển khai:

- Frontend React không còn hoạt động như một workload bên trong Kubernetes. Code được build tĩnh bằng Vite, lưu trữ trên một private S3 bucket và phân phối trực tiếp qua CloudFront.
- Backend, chat, worker, outbox dispatcher và dịch vụ bộ đệm SageMaker (adapter) tiếp tục chạy trên EKS vì đây là các tiến trình hoạt động dài hạn hoặc đòi hỏi các tính năng theo dõi sức khỏe, kiểm soát cuộn bản phát hành (rollout) và tự động dẫn dắt của Kubernetes.

Kiến trúc triển khai thực tế

Mô tả sơ bộ. CloudFront phục vụ frontend từ S3 riêng tư và chuyển lưu lượng API, chat, Socket.IO đến ALB. EKS chạy backend, chat, dispatcher, worker và AI service, kết nối với RDS, Redis, DynamoDB, S3, SQS, Lambda, SES và SageMaker. GitHub Actions triển khai image, workload, frontend và cấu hình CloudFront thông qua quyền AWS dựa trên OIDC.

Giải thích kiến trúc

CloudFront đóng vai trò là điểm truy cập công cộng duy nhất cho người dùng. Các tài nguyên tĩnh sử dụng behavior mặc định của CloudFront và được đọc trực tiếp từ S3 bucket frontend ở chế độ riêng tư (private) thông qua Origin Access Control của CloudFront. Các đường dẫn API động và liên lạc thời gian thực được định hướng qua Application Load Balancer công cộng:

- /api/* được ghi đè (rewrite) bởi ALB Ingress và chuyển tiếp đến backend FastAPI.
- /chat/* được ghi đè và chuyển tiếp đến dịch vụ chat.
- /socket.io/* được chuyển tiếp đến dịch vụ chat để xử lý giao thức liên lạc Socket.IO.

Cụm EKS chỉ lưu trữ và vận hành các dịch vụ cần thời gian chạy liên tục lâu dài: backend, chat, outbox dispatcher, processing worker và bộ đệm AI adapter. PostgreSQL đóng vai trò là cơ sở dữ liệu giao dịch chính lưu trữ thông tin người dùng, công việc, đơn ứng tuyển, trạng thái quy trình, các công việc xử lý bất đồng bộ, bản ghi outbox và hồ sơ tính bất biến (idempotency records). DynamoDB lưu trữ lâu dài các thực thể tin nhắn chat và trạng thái khử lặp (deduplication) sự kiện cho Lambda. Redis chỉ sử dụng cho giao thức pub/sub của Socket.IO giữa các pod chat. SQS tách biệt luồng sự kiện nghiệp vụ ra khỏi việc xử lý gửi thông báo, và Lambda phụ trách thực hiện các tác vụ theo sự kiện (event-driven) ngăn như khử lặp, lưu trữ dữ liệu cũ ra S3 và gửi email qua SES.

Bảng phân công trách nhiệm tài nguyên

Thành phần	Trách nhiệm cuối cùng	Bảng chứng triển khai
Frontend React/Vite	Giao diện người dùng cho Ứng viên và HR, được build thành các tệp tĩnh	frontend/package.json, scripts/ci/deploy-frontend.sh
CloudFront	Điểm truy cập HTTPS công cộng và phân chia ranh giới định hướng	scripts/aws/ensure-cloudfront.sh, distribution Tuần 8 EQIGYNECXDL8
S3 bucket cho frontend	Nơi lưu trữ an toàn các tài nguyên tĩnh từ frontend/dist	S3 bucket riêng tư trong bảng chứng triển khai Tuần 8; đã ẩn số tài khoản
Application Load Balancer	Bộ cân bằng tải định tuyến công cộng vào các dịch vụ trên EKS	k8s/eks/ingress-alb-no-domain.yaml
Backend FastAPI	Dịch vụ API xử lý xác thực, việc làm, ứng tuyển, tải tệp, bảng điều khiển, hàng đợi processing job	backend/app/main.py, các file router của backend
Dịch vụ chat	Cung cấp REST API cho chat và kênh thông tin realtime qua Socket.IO	chat-service/server.js
Processing worker	Tiến trình tiếp nhận các công việc xử lý bất đồng bộ và ghi nhận kết quả	backend/app/workers/processing_worker.py, k8s/app/backend-processing-worker.yaml
Outbox dispatcher	Tiến trình trích xuất sự kiện outbox đã commit từ PostgreSQL và phát tán lên SQS	backend/app/workers/outbox_dispatcher.py, ADR-001
Dịch vụ AI	Dịch vụ trung gian ổn định để worker giao tiếp với SageMaker	ai_service/app.py, k8s/app/ai-service.yaml
RDS PostgreSQL	Lưu trữ dữ liệu giao dịch cốt lõi và các hàng đợi xử lý bền vững của worker	Các file migration Alembic đến 0008_async_processing_jobs.py
DynamoDB	Lưu trữ bền vững dữ liệu chat và bản ghi khử lặp sự kiện cho Lambda	Tên các bảng chat trong cấu hình Kubernetes và bảng chứng chạy thực thi Tuần 8
Redis	Điều phối tin nhắn Socket.IO pub/sub giữa các pod chat	chat-service/lib/redis.js, cấu hình Kubernetes
SQS và DLQ	Đảm bảo chuyển giao thông báo sự kiện ít-nhất-một-lần và cô lập tin nhắn lỗi	ADR-001 và bảng chứng hàng đợi trong Tuần 8
Lambda và SES	Ghi nhận sự kiện, sao lưu văn thư (archive) và xuất chuyển email thông báo	Cấu hình Lambda và bảng chứng nhật ký CloudWatch trong Tuần 8
GitHub Actions OIDC	Dây chuyền CI/CD an toàn không cần khóa truy cập AWS dài hạn (long-lived access keys)	.github/workflows/cicd.yml

Lý do thiết kế

Frontend được gỡ khỏi EKS để chuyển sang lưu trữ trên S3 và phân phối bằng CloudFront vì đây là tài nguyên hoàn toàn tĩnh sau chu trình build. Cách làm này giúp giảm tải số lượng workload trên Kubernetes, loại bỏ nhu cầu cấu hình Deployment, Service, HPA và PDB riêng cho frontend, đồng thời tận dụng triệt để năng lực tối ưu bộ nhớ đệm (caching) và hỗ trợ định hướng SPA của CloudFront.

Backend và dịch vụ chat tiếp tục được giữ lại trên EKS vì chúng là các REST API/Realtime Server hoạt động dài hạn đòi hỏi các tính năng theo dõi sức khỏe (probe), nhân bản nhiều replica, dẫn dắt tự động qua HPA/PDB và

quy trình cuộn bản phát hành (rolling deployment). Tiến trình processing worker cũng nằm trong EKS do tác vụ đọc tách CV, phân tích công việc và đối chiếu ứng viên thường kéo dài hơn khung thời gian ngắn cho phép của cấu trúc Lambda và cần có cơ chế thuê hàng đợi (leases), tự động thử lại và không chế tiến trình song song an toàn.

Dịch vụ AI được tách riêng nhằm gom toàn bộ logic suy luận cụ thể của SageMaker vào một cụm ranh giới biệt lập với quy chuẩn giao tiếp của worker. Worker chỉ cần gửi request đến các route nội bộ ổn định như `/parse-job`, `/parse-cv`, và `/match-applications`, trong khi bộ trung gian AI adapter chịu trách nhiệm thực thi lời gọi tới SageMaker endpoint, quản lý trễ hạn (timeout), thử lại (retry) và chuyển đổi kết quả đầu ra về định dạng chuẩn.

Kiến trúc transactional outbox được triển khai vì việc chốt commit dữ liệu hồ sơ xin việc và việc phát đi một sự kiện sang hàng đợi SQS thuộc hai vùng có thể xảy ra rủi ro thất bại khác nhau. Để chống mất tin nhắn, sự kiện được gộp ghi thẳng vào PostgreSQL trong cùng một giao dịch (transaction) với biến đổi dữ liệu nghiệp vụ. Tiến trình dispatcher sau đó trích xuất ra và xuất giao cho SQS. Thiết kế này bảo đảm sự kiện luôn được giao nhận ít-nhất-một-lần, sau đó Lambda sử dụng cơ chế ghi có điều kiện (conditional writes) trên DynamoDB để loại bỏ lặp đặt đối với eventId.

Đề xuất so với thực tế triển khai

Lĩnh vực	Đề xuất ban đầu	Kiến trúc triển khai thực tế
Lưu trữ frontend	Phân phối thông qua Kubernetes hoặc dịch vụ lưu trữ trang tĩnh của AWS	Triển khai trên private S3 kết hợp CloudFront
Điểm truy cập chung	Dự kiến dùng ALB hoặc CloudFront	CloudFront là điểm tiếp nhận duy nhất cho người dùng; ALB đóng vai trò origin cho dải đường dẫn API, chat và socket
Môi trường chạy backend	Kubernetes trên EKS	EKS Deployment backend, duy trì 2 replica, HPA thiết lập từ 2-5
Môi trường chạy chat	Node.js và Socket.IO phối hợp cùng Redis/DynamoDB	EKS Deployment chat-service, 2 replica, áp dụng ALB stickiness và Redis pub/sub
Môi trường chạy AI	Dự kiến có một dịch vụ AI riêng	Pod adapter ai-service trên EKS gánh việc gọi ra SageMaker endpoint internship-qwen3-4b
Luồng xử lý sự kiện	Dự kiến có hệ thống thông báo bất đồng bộ	Hệ thống transactional outbox với PostgreSQL, SQS, Lambda, DynamoDB dedupe, lưu giữ vào S3 và gửi mail bằng SES
Quy trình triển khai	Thiết lập quy trình CI/CD	GitHub Actions chạy qua lệnh workflow dispatch, tích hợp OIDC và thẩm tra nhân tag của Amazon ECR image
Minh chứng hoạt động	Giả định và mô hình hóa trên bản vẽ	Đã thu thập và lưu giữ đầy đủ bằng chứng chạy thực thi của CloudFront, các pod trong EKS, RDS, Redis, DynamoDB, SQS, cấu hình Lambda và SageMaker endpoint

Các dịch vụ AWS được sử dụng

Dịch vụ AWS	Vai trò trong hệ thống	Lý do lựa chọn
Amazon EKS	Quản lý và chạy backend, worker, dịch vụ chat và các workload dạng container	Hỗ trợ Kubernetes, dẫn dèo tài nguyên, cập nhật không ngắt quãng và quản lý vi dịch vụ
Amazon ECR	Lưu trữ container images	Tích hợp sâu sát với EKS và dây chuyền GitHub Actions
Application Load Balancer	Tiếp nhận và định hướng các request public sang dịch vụ tương ứng	Có khả năng kiểm soát luồng điều hướng, health check và kết nối trực tiếp với Kubernetes Ingress



Dịch vụ AWS	Vai trò trong hệ thống	Lý do lựa chọn
Amazon RDS for PostgreSQL	Lưu trữ người dùng, công ty, tin tuyển dụng, hồ sơ ứng cử, siêu dữ liệu tệp và thông tin processing jobs	Rất tin cậy cho nhu cầu lưu trữ dữ liệu quan hệ và xử lý giao dịch
Amazon DynamoDB	Lưu giữ danh sách người dùng trong khung chat, nhóm trò chuyện và nội dung tin nhắn	Phù hợp tối đa với tải trôi chao đảo tốc độ cao và số lần đọc/ghi lớn của dịch vụ liên lạc
Amazon ElastiCache	Cung cấp cụm Redis phục vụ pub/sub cho Socket.IO và dữ liệu đệm	Giúp đồng bộ tin nhắn realtime mượt mà giữa các replica của dịch vụ chat
Amazon S3	Lưu hồ sơ CV, chứng chỉ, bảng điểm và toàn bộ file tĩnh của frontend	Đem lại độ ổn định cao, khả năng dẫn dắt không giới hạn và hỗ trợ liên kết tạm thời (presigned URL)
Amazon CloudFront	Phân tán nội dung tĩnh cho frontend và bảo vệ các luồng điều hướng	Tăng tốc độ truy xuất, tích hợp bảo mật HTTPS mượt mà
Amazon SQS	Nhận các thông báo sự kiện bất đồng bộ từ outbox dispatcher	Tách biệt các dịch vụ nghiệp vụ ra khỏi ràng buộc gay gắt trong thực thi
Amazon SageMaker	Đóng vai trò máy chủ xử lý suy luận (inference) trong môi trường production	Cung cấp môi trường quản trị model AI mạnh mẽ và dễ mở rộng tải
Amazon CloudWatch	Đón nhận các tập tin nhật ký AWS, thông số giám sát metrics và các cảnh báo (alarms)	Trụ cột không thể thiếu trong khâu vận hành và điều tra rò rỉ nguyên nhân sự cố
AWS IAM	Kiểm soát chi tiết các ràng buộc quyền lực giữa cụm container và tài nguyên AWS	Giữ trọn tùy chọn an toàn và cam kết theo phương châm đặc quyền tối thiểu (least privilege)

Thiết kế các thành phần

Frontend

Frontend được xây dựng bởi bộ khung React kết hợp công cụ Vite đem đến hệ thống giao diện cho cả Ứng viên lẫn HR. Sau khâu build tĩnh, sản phẩm bao gồm các tài nguyên file web được chuyển gửi lưu vào trong Amazon S3 và khuếch tán qua mạng CDN của Amazon CloudFront. Frontend liên lạc với backend cùng máy chủ chat thông qua đường dẫn được công khai bằng Application Load Balancer.

Backend API

Backend xây dựng từ ngôn ngữ Python và framework FastAPI, quán xuyến mọi tiến trình xác thực danh tính, quản lý thông tin Ứng viên/HR, tạo mới tài khoản công ty và danh mục tuyển dụng, điều phối thông số siêu dữ liệu tài liệu ứng tuyển, khởi tạo đường dẫn bảo mật presigned URL, sinh tạo nhiệm vụ processing jobs cho worker, chiết xuất dữ liệu thống kê bằng điều khiển cùng các hệ API nghiệp vụ.

Processing Worker

Processing worker chuyên gánh vác các tác vụ chạy nền hao tốn thời gian bao gồm: trích xuất ký tự văn bản CV, phân tích trường yêu cầu Mô tả công việc, thẩm định sâu sắc CV, đo lường điểm số tương đương và chạy vòng chốt cho sắp xếp lại (reranking) danh sách ứng viên. Các rào gài an toàn cơ cấu thuê (lease), tự hồi phục (retry) và trần số lần thử giúp cách ly rủi ro hỏng lỗi trong tình trạng worker bỗng dưng mất điện giữa chừng.

Dịch vụ chat (Chat Service)

Dịch vụ chat thiết lập bằng Node.js, Express và giao thức Socket.IO nhắm tới phục vụ các mạch kết nối thời gian thực. Amazon DynamoDB lưu giữ triệt để thực thể người dùng, hội thoại và thư tín riêng lẻ; bộ giao diện adapter với Redis giúp lan tỏa liên kết sự kiện đi khắp các cụm pod; tính năng bám dính (sticky sessions) được bật để gìn giữ luồng handshake liên đới bền bỉ qua Socket.IO khi chạy song song trên nhiều replicas.

Dịch vụ AI (AI Service)

Dịch vụ AI đón nhận nguyên liệu văn tự từ CV cùng JD, chuyển dịch toàn bộ từ khối ký tự thô thành hồ sơ trường cấu trúc chuẩn xác và phụ trách khâu tính toán điểm số phù hợp. Dịch vụ AI tích hợp khâu soát lỗi cú pháp (schema validation), khóa ngắt thời hạn (timeout), thử lại hạn ngạch (limited retry), kiểm chứng an toàn sức khỏe (health checks), che chắn thông tin bí mật trong nhật ký ghi chú, đồng thời linh hoạt tự lùi trượt về phương án dự phòng (fallback) nếu model trung tâm quá tải hoặc mất kết nối.

Lưu trữ dữ liệu (Data Storage)

PostgreSQL lưu trữ vững chắc các bảng dữ liệu quan hệ đi kèm các nghiệp vụ giao dịch nghiêm ngặt. DynamoDB chuyên quản dữ liệu tin nhắn tức thời. Amazon S3 đảm nhận trọng trách lưu giữ tệp tin lượng tồn lớn. Redis chi phối tốc độ dẫn tỏa của luồng tin realtime. Dùng chiến thuật đa kho công nghệ giúp từng vùng dữ liệu



riêng lẻ được sắp xếp theo một giải pháp lưu trữ tối ưu hóa trùng theo mô hình tần suất truy xuất của chính nó.

Hệ thống giám sát (Observability)

Hệ thống tiến hành gặt hái liên tục thông số đo lường (metrics) nhằm giám sát hiệu năng và sức khỏe cụm máy, ghi nhận nhật ký (logs) chuyên trách hỗ trợ theo dõi sự cố và điều tra cội nguồn rách nhĩ lỗi lầm, đồng thời truy vết luồng phân tán (distributed traces) giúp phân giải ranh giới từng nhíp request luân chuyển qua lại giữa các máy chủ vi dịch vụ. Bộ công cụ Prometheus, Grafana, Loki, OpenTelemetry và Tempo làm trụ cột trong cụm Kubernetes; Amazon CloudWatch được tích hợp để điều tra hiệu năng hạ tầng AWS và tổng hợp các tập tin log hệ thống.

Kế hoạch triển khai kỹ thuật

Các giai đoạn triển khai

Giai đoạn	Hạng mục hoạt động chính
1. Phân tích và thiết kế	Phân tích trọn bài toán phối kết giữa Ứng viên - HR, xác lập đầy đủ tiêu chí chức năng lẫn phi chức năng, lập bản vẽ cơ sở dữ liệu, quy hoạch kiến trúc đa vi dịch vụ và điểm tên nhóm dịch vụ AWS thiết yếu
2. Nền tảng nghiệp vụ cốt lõi	Khởi tạo hệ thống xác thực người dùng, hồ sơ cá nhân Ứng viên/HR, bảng quản lý công ty, luồng tạo việc làm, chu trình theo dõi đơn ứng tuyển và bộ điều phối hồ sơ tài liệu
3. Tích hợp liên lạc thời gian thực và AI	Thiết lập máy chủ chat, triển khai giao thức Socket.IO, lưu trữ vĩnh viễn trên DynamoDB, bầu chốt adapter kết nối Redis, dựng máy chủ AI, vận hành chức năng đọc phân tích CV/JD, chấm điểm tương ứng và sắp xếp danh sách (reranking)
4. Gia cố độ tin cậy hệ thống	Hoàn chỉnh khối background processing workers, bổ sung tính năng thử lại (retry), thuê tác vụ (lease), khóa tính bất biến (idempotency), kiểm soát cạnh tranh dữ liệu lạc quan (optimistic concurrency), trê ranh giới Transactional Outbox, thăm rà tham số cùng khối catch rà lỗi
5. Đóng gói container và Kubernetes	Biên soạn chuỗi file Dockerfile, tạo kịch bản Docker Compose, khởi dệt cụm Kubernetes cơ sở tại chỗ bằng kind, ban bố các manifest Deployment, Service, Ingress, HPA, PDB, health probes và cụm job chạy migration
6. Giám sát hệ thống và CI/CD	Vận hành hệ quy củ trích rút metrics, logs và traces, tạo dựng bảng theo dõi Grafana, lập các báo nguy cảnh báo (alerts), thi công luồng tác nghiệp GitHub Actions rà kiểm thử động, quét test nhanh hồ và soát an ninh bộ mã
7. Triển khai AWS và hoàn tất hồ sơ	Đóng gói image, đẩy lên hòm lưu trữ Amazon ECR, khai triển các dịch vụ ra cụm Amazon EKS, bắt tuyển liên kết sang RDS, DynamoDB, ElastiCache và S3, đăng tải frontend tĩnh lên cụm S3/CloudFront, thực thi chu trình kiểm tra nghiệm thu đầu cuối và hoàn thiện toàn bộ bộ tài liệu dự án

Yêu cầu kỹ thuật

Thành phần	Công nghệ hoặc tiêu chuẩn kỹ thuật
Frontend	React, Vite, Tailwind CSS
Backend	Python, FastAPI, SQLAlchemy, Alembic
Dịch vụ chat	Node.js, Express, Socket.IO
Dịch vụ AI	Python, FastAPI, model tương thích Qwen hoặc SageMaker endpoint
Cơ sở dữ liệu quan hệ	PostgreSQL
Cơ sở dữ liệu chat	DynamoDB
Bộ nhớ đệm / pub-sub	Redis hoặc Amazon ElastiCache
Lưu trữ đối tượng (Object storage)	Amazon S3
Công cụ Container	Docker



Thành phần	Công nghệ hoặc tiêu chuẩn kỹ thuật
Công cụ quản trị cụm Container	Kubernetes, kind, và Amazon EKS
CI/CD	GitHub Actions
Giám sát hiệu suất	Prometheus, Grafana, và CloudWatch
Nhật ký hệ thống	Loki và CloudWatch Logs
Truy vết phân tán	OpenTelemetry và Tempo
Bảo mật và An ninh	JWT, IAM Role, OIDC, đặc quyền tối thiểu (least privilege), và quản trị bí mật an toàn

Lịch trình và các mốc thời gian

Tôi đã điều chỉnh lịch trình triển khai thành 8 tuần, kể từ ngày 08/06/2026 đến ngày 30/07/2026, khớp chính xác với mốc thời gian trong nhật ký làm việc (worklog).

Tuần	Thời gian	Mốc quan trọng (Milestone)	Kết quả đầu ra dự kiến
1	08/06/2026 - 14/06/2026	Phân tích yêu cầu, kiến trúc hệ thống, dựng nền tảng backend và xác thực	Phạm vi dự án, sơ đồ kiến trúc tổng quan, chức năng đăng ký, đăng nhập và tập file migration đầu tiên
2	15/06/2026 - 21/06/2026	Nền tảng Ứng viên - HR, quản lý đơn ứng tuyển, tài liệu và kết nối S3	Trang quản lý công ty, tin việc làm, luồng nộp hồ sơ ứng cử, cơ chế tải tệp bảo mật và tải hồ sơ xuống
3	22/06/2026 - 28/06/2026	Dựng Frontend và kết nối trọn gói với REST API	Hệ thống giao diện Ứng viên/HR, các bộ bảo vệ phân tuyến route và luồng giao tiếp API mượt mà
4	29/06/2026 - 05/07/2026	Kênh liên lạc trực tuyến thời gian thực	Trao đổi thời gian thực mượt mà giữa Ứng viên và HR bằng Socket.IO, Redis và DynamoDB
5	06/07/2026 - 12/07/2026	Tích hợp AI và củng cố độ tin cậy trong xử lý	Chức năng đọc hiểu CV/JD, chốt điểm phù hợp, background worker, tính bất biến, chống xung đột cạnh tranh và cơ cấu outbox
6	13/07/2026 - 19/07/2026	Đóng gói Docker Compose	Toàn diện bộ hệ thống vận hành trôi chảy ngay trên máy tại chỗ kèm chu trình chạy thử nghiệm khói (smoke test)
7	20/07/2026 - 26/07/2026	Triển khai Kubernetes và hệ thống giám sát	Hệ thống khởi chạy trọn vẹn trên môi trường kind đi cùng bộ theo dõi thông lượng metrics, logs, traces và cảnh báo lỗi
8	27/07/2026 - 30/07/2026	Tự động hóa CI/CD, đưa lên AWS và viết báo cáo	Thực thi kiểm thử trọn luồng đầu cuối, hoàn chỉnh công tác đưa hệ thống lên hạ tầng AWS và xuất bản trọn vẹn báo cáo chi tiết

Dự toán ngân sách

Chi phí tiêu tốn hàng tháng hợp lý nên được dự trù rõ ràng thông qua bộ công cụ AWS Pricing Calculator và tiến hành đối soát song song cùng các báo cáo hóa đơn thực tính trên tài khoản AWS vùng ap-southeast-1. Trong khuôn khổ báo cáo này, thư viện chứng cứ hạ tầng AWS ở Tuần 8 được coi là nguồn dữ liệu chuẩn xác về hiện trạng hao tổn ngân sách hiện tại.

Bảng chứng chi phí AWS Tuần 8

Bảng chứng Tuần 8 ghi nhận tổng hao phí AWS tính trong tháng từ ngày 1-28 tháng 7 năm 2026 cùng một ảnh chụp màn hình ghi nợ tín dụng từ trang Billing and Cost Management. Hóa đơn chốt tháng và bản dự trừ chi tiết từ AWS Pricing Calculator hiện đang chờ bổ sung.

Chứng cứ chi phí	Giá trị ghi nhận	Giải thích chi tiết
Tổng chi phí từ 1-28 tháng 07/2026	\$94.92	Tổng hao phí phát sinh trong tháng rút từ bảng tổng kết Tuần 8
Ngày hao chi cao nhất trong kỳ	\$31.83 vào ngày 28/07	Mức tiêu tốn đỉnh điểm cao nhất trong chuỗi ngày 1-28 tháng 7
Tổng hạn ngạch tín dụng đã tiêu dùng	\$27.90	Trích xuất từ minh chứng ảnh chụp trang Billing credits
Tổng chi phí ước tính tín dụng đã dùng	\$140.65	Trích xuất từ minh chứng ảnh chụp trang Billing credits
Số tín dụng khả dụng còn lại	\$172.10	Trích xuất từ minh chứng ảnh chụp trang Billing credits
Số tín dụng ước tính khả dụng còn lại	\$59.35	Trích xuất từ minh chứng ảnh chụp trang Billing credits

Các khu vực dẫn đầu về tiêu chi AWS trong Tuần 8 bao gồm Amazon RDS (\$29.69), Amazon SageMaker (\$23.45), Amazon VPC (\$14.11), Amazon EC2 - Compute (\$12.42), EC2 - Other (\$7.68), và Amazon EKS (\$5.64). Nhóm sáu dịch vụ chủ lực này chiếm lĩnh xấp xỉ 98% tổng mức chi tiêu được báo cáo trong khoảng thời gian từ ngày 1-28 tháng 7.

Các giả định tính phí

Lĩnh vực	Giả định tính phí dựa trên bằng chứng	Trạng thái ước tính
Môi trường	Triển khai production chính thức ở vùng ap-southeast-1	Đã xác minh
Lưu lượng truy cập public	CloudFront tiếp đón truy cập trình duyệt và luân lạc đường dẫn API/chat qua ALB	Đã xác minh
Frontend	Các tài nguyên tĩnh được nạp ở S3 bucket riêng tư và lan tỏa qua CDN CloudFront	Đã xác minh
Kubernetes	Một cụm EKS độc lập điều phối backend, chat, worker, dispatcher và dịch vụ adapter AI	Đã xác minh
Các node xử lý (Worker nodes)	Hai máy trạm EC2 ở trạng thái Ready được xác thực trong hồ sơ minh chứng	Cần thêm thông số loại máy instance và kích thước đĩa EBS
RDS PostgreSQL	Hai máy chủ cơ sở dữ liệu db.t4g.micro mã hóa nằm trong mạng riêng với dung lượng 20 GiB mỗi con	Đã xác minh
ElastiCache / Valkey	Một cụm Redis replication group ở trạng thái available có mã hóa bảo mật thời gian tính lẫn luân chuyển	Cần thêm số lượng và loại máy con
DynamoDB	Các bảng lưu trữ chat và khứ lặp Lambda vận hành theo cơ chế chi tiêu theo nhu cầu (on-demand) và gần như trắng tải ở mốc ghi nhận	Đã xác minh
SQS	Hàng đợi sự kiện outbox chủ lực cùng DLQ đều có 0 tin nhắn ứ tắc ở mốc kiểm tra	Đã xác minh
Lambda	Hàm outbox handler có trạng thái Active, sở hữu 256 MB bộ nhớ cùng mốc thời hạn (timeout) 20 giây	Đã xác minh



Lĩnh vực	Giả định tính phí dựa trên bằng chứng	Trạng thái ước tính
SageMaker	Máy chủ real-time endpoint mang trạng thái InService cùng một production variant gắn kèm	Cần bổ sung loại máy instance và dự kiến chu kỳ duy trì mở cửa
CloudWatch	Log cùng dữ liệu giám sát đã sinh ra đầy đủ từ cụm dịch vụ AWS và pod bên trong Kubernetes	Cần bổ sung thời gian bảo lưu (retention) và dung lượng log tiếp thu

Phương pháp ước tính chi phí

1. Mở cụm công cụ AWS Pricing Calculator tại ranh giới khu vực ap-southeast-1.
2. Khai báo danh mục tài nguyên triển khai thực tế được tổng hợp từ thư viện bằng chứng.
3. Nhập thêm các thông số runtime còn khuyết thiếu: phân loại máy chủ con node instance, kích thước đĩa EBS, quy chuẩn và số lượng node Redis, dòng máy chủ SageMaker instance, tổng số giờ duy trì online trong tháng, lưu lượng truyền dẫn ra ngoại vi, quy mô request truy xuất, cùng hạn nấc lưu giữ tệp nhật ký CloudWatch log.
4. trích xuất bảng tổng hợp ước tính từ Pricing Calculator ra làm bản dự trù chi phí.
5. Kiểm chứng chéo bản dự trù với bảng sao kê chi tiết AWS Cost Explorer sau khi hệ thống đã qua vận hành đầy đủ một khoảng thời gian đại diện hợp lý.

Dịch vụ	Phương pháp định ước tính theo tháng	Trạng thái bằng chứng hiện tại
Amazon EKS	Giá duy trì cụm cluster theo giờ nhân với tổng số giờ bật cụm	Đã triển khai; bảng tổng hợp chi phí Tuần 8 báo cáo \$5.64
EC2 worker nodes	Giá thuê bao instance theo giờ nhân cho 2 máy trạm cộng thêm dung lượng EBS bám kèm	Đã kiểm chứng 2 nodes; EC2 Compute ghi nhận \$12.42, EC2 - Other ghi nhận \$7.68
Amazon RDS PostgreSQL	Giá thuê db.t4g.micro theo giờ, cộng chi phí 20 GiB bộ nhớ mỗi instance, lưu trữ backup và chỉ số I/O	Đã kiểm chứng 2 máy chủ riêng tư có mã hóa; chi phí Tuần 8 ghi nhận \$29.69
Amazon ElastiCache / Valkey	Giá node theo giờ nhân số lượng cụm máy con cộng chi phí luân chuyển băng thông	Đã kiểm chứng replication group; đang đợi nạp bổ sung kiểu node cùng số con
Application Load Balancer	Phí cơ sở duy trì ALB theo giờ cộng hao tổn tải theo chỉ số LCU	Đã kiểm chứng ALB hoạt động; được cộng chung vào nhóm kiểm toán mạng và chu kỳ thời gian chạy
Amazon CloudFront	Tác quyền lượt request cộng dung lượng truyền xuất ngoại và lưu lượng invalidation cache	Có sẵn ảnh minh chứng giám sát; chờ bổ sung file chiết xuất danh tính origin và behavior
Amazon S3	Số dung lượng GB lưu trữ, lượt gọi lệnh PUT/GET cùng phí thao tác vòng đời lifecycle	Có sẵn minh chứng danh tính và nội dung object trên frontend bucket
Amazon DynamoDB	Đơn giá lượt truy xuất thao tác đọc/ghi theo chế độ on-demand cùng quy mô bộ nhớ chiếm dụng	Có sẵn ảnh minh chứng thông số trạng thái bảng
Amazon SQS	Lượt gửi thông điệp SQS standard queue kết hợp cùng tổng trọng lượng tệp dữ liệu mang theo	Có sẵn ảnh minh chứng theo dõi hàng đợi cùng DLQ
AWS Lambda	Tổng lượng lần nhảy hàm kết hợp hao tổn tính theo GB-giây	Hàm đã hiện diện; thông số tổn chi khi kích hoạt cùng sức khỏe trigger đang được đợi
Amazon SES	Tổng lượng email gửi vãng đi cùng dung lượng tài liệu đính kèm (nếu có)	Sử dụng qua luồng điều phối gửi thông báo
Amazon ECR	Trữ lượng tệp tin image (GB) cùng cước phí thông chuyển vùng ngoại biên (nếu có)	Chưa bẻ rạch phân định thành cột riêng trong bảng thông qua 6 khuếch chi lớn ở Tuần 8



Dịch vụ	Phương pháp định ước tính theo tháng	Trạng thái bằng chứng hiện tại
Amazon SageMaker	Giá thuê máy chủ endpoint theo giờ cộng với chi phí các lượt gõ lời gọi và dung lượng suy đoán	Đã xác minh endpoint mang trạng thái InService; báo cáo chi phí Tuần 8 ghi nhận \$23.45
Amazon CloudWatch	Quy mô dung lượng nhật ký tiếp nạp, mốc ngày lưu trữ (retention), thông lượng metrics và các kênh báo nguy	Đang nín đợi xác lập chỉ tiêu thể tích nạp log cùng giới hạn ngày giữ dữ liệu
Amazon VPC và data transfer	Trôi chuyển ra CloudFront, trao đổi ALB, xử lý qua cổng NAT cùng lưu lượng qua lại giữa các AZ	Báo cáo chi phí Tuần 8 ghi nhận Amazon VPC ở mức \$14.11; cần chiết tách bằng biểu luân chuyển cụ thể
Tổng	Chạy ra bảng dự báo từ Pricing Calculator, tiếp đó đối soát với minh chứng sao kê chốt nợ	Mức hao tiêu tính từ ngày 1-28 tháng 7 ghi nhận \$94.92; hóa đơn chốt kỳ và dự tính dài hạn đang chờ cập nhật

Các đầu trực hao tổn tài chính chủ lực hiện nay và trong tương lai bao gồm: RDS, số giờ mở liên tục của SageMaker endpoint, lưu lượng trao đổi hạ tầng VPC/NAT, chi phí EC2 worker nodes và đĩa EBS, giá duy trì EKS control plane, cụm Redis, ALB, phí băng thông CloudFront và chi phí gom giữ tệp nhật ký CloudWatch logs. Nếu máy chủ SageMaker endpoint bị lãng quên duy trì trực tuyến liên miên bất kể lúc trống tải, hao tổn dịch vụ suy luận AI sẽ vụt sáng biến thành một trong những đợt gánh phí cao ngất đường trong bảng sao kê hàng tháng.

Kỷ luật an toàn cho hồ sơ minh chứng chi phí

Tuyệt đối cấm mọi bức ảnh chụp màn hình hay tập nhật ký có bộc lộ thông số ID tài khoản AWS, access keys, secret keys, mật mã truy nhập cơ sở dữ liệu, chuỗi kết nối (connection strings), thông số giải mật Kubernetes Secret, token GitHub, liên kết temporary presigned URL hay chuỗi ghi nhớ đăng nhập nhạy cảm. Nội dung minh chứng log từ các pod phải được cất lược hoặc làm bớt sạch trước khi tung lên báo cáo vì file log gốc 02-eks/pod-logs-tail.txt hiện mang vết hằn lưu chuỗi kết nối Redis đi kèm thông tin mật mã truy nhập (auth credential).

Tối ưu hóa chi phí

- Chỉ cấm bật môi trường production những dịp thao diễn demo hoặc thực hành chạy kiểm thử.
- Chọn lựa kỹ các phiên bản instance tương đương trùng pháo với tải nhu cầu thu gặt thực tế.
- Không chừa con số tối đa cho đàn máy EC2 worker nodes.
- Dừng ngay (stop) hoặc ra lệnh chém bỏ (delete) các SageMaker endpoints nếu xong công đoạn thực tập.
- Thiết lập chu kỳ tự tiêu diệt nhật ký log (retention) phù hợp cho CloudWatch.
- Định kỳ tháo rửa các container image cũ kỹ bám bẩn trong hòm Amazon ECR.
- Dựng quy luật chuyển trượt vòng đời (lifecycle policies) hợp thời cho kho S3.
- Áp dụng cấu trúc thanh toán theo nhu cầu (on-demand) cho DynamoDB trong thời gian lượng người dùng còn thăng giều.
- Tự g cài chốt khóa báo nguy hóa đơn cùng ngân sách thông qua bộ đôi AWS Budget và Billing Alarm.
- Trệt quét sạch bãi các đối tượng công phí không thọ tiện như Load Balancers mỗi trợ, địa chỉ công cộng public IPv4, snapshots cũ và đĩa EBS phế truất.
- Thẩm định cho kỳ rõ ràng xem nhu cầu mạng có khẩn khoản bắt buộc cổng NAT Gateway trước lúc phán phán quyết thả cho nó chạy tiếp miên man rả rích theo giờ hay không.

Đánh giá rủi ro

Ma trận rủi ro

Rủi ro	Khả năng xảy ra	Mức độ ảnh hưởng	Mức độ nguy hiểm
Lộ bí mật hệ thống hoặc khóa AWS credential	Thấp	Rất cao	Cao
RDS hoặc các dịch vụ nhạy cảm vô tình bị mở rộng truy cập ra public	Trung bình	Cao	Cao
Dữ liệu CV và thông tin cá nhân bị truy xuất trái phép	Thấp	Rất cao	Cao



Rủi ro	Khả năng xảy ra	Mức độ ảnh hưởng	Mức độ nguy hiểm
AI trả lại các phân tích thiếu chính xác	Trung bình	Trung bình	Trung bình
Dịch vụ AI kiệt quệ bộ nhớ tài nguyên hoặc lâm vào cảnh time out	Trung bình	Cao	Cao
Mạch tin nhắn chat realtime ngắt ngứ lúc cụm pod được nhân lên dần dể	Trung bình	Trung bình	Trung bình
Tiến trình worker vờ trối xử lý lập một đơn xin việc 2 lần	Trung bình	Cao	Cao
Thao tác ghi cộ cơ sở dữ liệu dính rào hãm dọa race condition	Trung bình	Cao	Cao
Các kubernetes pod hoặc node kẹt ồm ngả chết đột ngột	Trung bình	Trung bình	Trung bình
Hóa đơn chi cước AWS vượt bổng vượt ranh giới kỳ vọng	Trung bình	Cao	Cao
Dây chuyền triển khai CI/CD đổ bể	Trung bình	Trung bình	Trung bình
Thư viện mã nguồn từ đối tác ngoại kiều trôi rách mỗi thăm kỳ lọt bề lỗi an ninh	Trung bình	Cao	Cao

Biện pháp giảm thiểu

Bảo mật

- Sử dụng IAM Roles tuân theo nguyên tắc đặc quyền tối thiểu (least privilege).
- Cấm để lại khóa AWS Access Keys trên thư mục mã nguồn repository.
- Ưu tiên triển khai GitHub Actions OIDC.
- Thu xếp cất giấu xa xôi các tệp .env, khóa cá nhân private keys và thông số bí mật ra xa vạch chỉ tay repo.
- Gài chặt ranh giới S3 buckets ở thể cấm đoán hoàn toàn (private).
- Ứng dụng dải liên kết hạn thời presigned URLs cho khâu lấy tài liệu.
- cấm bừa giăng thông tin văn bản CV hoặc token ra các tệp log nhật ký trình trượt.
- Thẩm tra gắt gao danh tính quyền hạn và tính sở hữu chủ đối tượng từ trong lòng lớp backend.

Độ tin cậy

- Ứng dụng tường tận bộ hai cơ chế đo đặc sức khỏe readiness và liveness probes.
- Dùng HPA (Horizontal Pod Autoscaler) điều tiết sức gánh dần dể tải.
- Sử dụng PDB (Pod Disruption Budget) hạn chế chao đảo trong những kỳ gở giảng bảo trì.
- Trang bị cơ cấu xử lý thử lại (retry) với chỉ tiêu gò hẹp cho phép.
- Gắn thêm chốt cam đoan quyền sở hữu (lease) cho các nghiệp vụ worker.
- Luôn kê thêm thông số khóa tính bất biến (idempotency keys) cho mỗi request giao dịch.
- Triển khai thuật toán chống đụng độ cơ sở dữ liệu lạc quan (optimistic concurrency).
- Sử dụng cấu trúc transactional outbox để gìn vững các lệnh bài phát tin nhắn sự kiện.

AI

- Kiểm định tính hợp pháp của kết quả AI trả ra nhờ cấu trúc schema.
- Thiết lập giới hạn thời gian thực thi tối đa (timeout).
- Đóng trần lượt cố thi thử lại khi thất bại.
- Luôn giữ không gian cho con người (HR) trực tiếp soi xét trước mọi quyết định chọn hay loại ứng viên.
- Cấm sử dụng độc lập điểm số do AI chấm để ra quyết định tuyển dụng hợp chém cuối cùng.
- Lưu lại cụ thể cội rễ lý luận hoặc thông số căn dặn đi kèm nếu model cho phép thăng chi tiết.

Chi phí

- Kích hoạt hệ thống cảnh báo cước phí AWS Budget.
- Thăm viếng quan trắc bảng báo cáo Cost Explorer định kỳ.
- Ra tay dọn dẹp sạch sẽ tài nguyên Cloud lúc chấm dứt chuỗi Workshop thực nghiệm.
- Hạn chế số giờ trực tuyến tối thiểu cho cụm endpoint máy chủ AI.
- Áp chế khung ngày hợp lý tự thiêu log nhật ký CloudWatch.
- Kiểm toán gắt gao truy diệt trót lọt mọi đĩa EBS, Elastic IPs, Load Balancers cùng snapshots bóc mỗi bỏ phẩy.

Kế hoạch dự phòng (Contingency Plan)

- Nếu EKS trục trặc hoặc chưa sẵn sàng, mau chóng xoay lùi sang mô phỏng trên Docker Compose hay Kubernetes tại chỗ để phục vụ thao diễn demo.
- Nếu model AI gặp vướng mắc ngắt tuyến, áp dụng phương thức phản hồi giả định (mock responses) hoặc logic chấm điểm theo biểu thức tính để duy trì mạch kiểm định nghiệp vụ.
- Nếu hệ thống Redis trục trặc, cho chuyển ngay về mô hình độc nhất 1 pod chat-service chạy lẻ loi trong môi trường thao diễn.
- Nếu vạch đường tới kho S3 bế tắc, quay về tận dụng kho lưu trữ cứng ngay trên máy địa phương lúc lập trình phát triển.
- Nếu một tiến trình worker gục sập lác đường, các thẻ bài nhiệm vụ sẽ được thu gặt cho thi hành lại tự động tuân theo ranh giới cam gặt thuê lease và giới hạn lần thử.
- Nếu đợt cuộn bản phát hành mới đổ nhào, lập tức xuất phát thủ tục rút chao (roll back) ngược trở lại phiên bản container image hoặc số kubernetes revision cũ nhẽ.
- Nếu ranh giới ngân sách cước ngắc xô trần, cho ngưng gác lại lập tức mọi workload không thuộc mạn sinh tồn rồi kết liễu tráo hạ các cụm tài nguyên xa hoa gánh phí.

Kết quả dự kiến

Sau khi hoàn thiện, hệ thống dự kiến đạt được các thành quả đầu ra như sau:

- Cung cấp nền tảng quản lý tập trung toàn cầu cho Ứng viên và HR.
- Quán xuyên trôi chảy doanh nghiệp, vị trí việc làm, đơn từ ứng tuyển và toàn bộ tệp tài liệu.
- Gìn giữ an ninh an toàn cho các tập CV và chứng từ.
- Phối kết liên lạc tức thời thông qua khung chat realtime.
- Có khả năng nhờ cậy sức mạnh AI chiết tách và đối sánh CV với Mô tả công việc.
- Quản lý các tiến trình tốn kém thời gian nhờ chuỗi worker sau cánh gà.
- Vững chí kháng lại những chuỗi request lặp lại vô ý cũng như thao tác sửa đề song song.
- Chạy thông dãi trên nền tảng Docker Compose và Kubernetes.
- Xây lắp chín chu ranh giới tích hợp liên tục CI/CD.
- Thu gặt đầy đủ 3 chân kiềng quan trắc gồm thông số metrics, tệp log nhật ký, dấu nhíp truy vết traces và mạng cảnh báo.
- Quy hoạch bài bản kiến trúc vận hành cho cụm thực hành trên hạ tầng AWS.
- Tổ chức hồ sơ cẩm nang vô cùng cặn kẽ hướng dẫn từng nấc từ vạch dựng máy, thi kiểm đến chặng thu rửa tài nguyên thọ mãn.

Giá trị dài hạn

Dự án này sở hữu trọn bộ nền móng vững chắc để có thể vươn vai bành trướng thành một giải pháp phần mềm khổng lồ với các tính năng:

- Gửi email hoặc push notification ngay khoảnh khắc đơn xin việc được thao tác đổi trạng thái.
- Tự động đặt và chốt lịch phỏng vấn giữa đôi bên.
- Tích hợp AI chấp bút viết hộ thư giới thiệu (cover letters).
- Phỏng đoán và lập danh sách câu hỏi phỏng vấn sát nấc theo dữ liệu trích ra từ CV và JD.
- Đóng vai trò hệ khuyến nghị gợi ý việc làm hợp sở trường dựa trên hồ sơ cá nhân của Ứng viên.
- Trang bị bảng điều khiển BI (Business Intelligence) tối cao soi bối cảnh tuyển dụng cho doanh nghiệp.
- Móc nối đồng bộ sang lịch công tác và thư thông tin công đoàn (Calendar & Email Integration).
- Nâng cấp độ tường tận khả dĩ mang lại cội nguồn diễn giải cho logic suy đoán chấm điểm AI.
- Đóng gói ra mắt các phiên bản trên thiết bị thông minh di động (Mobile Application).

- Liên kết và xâm nhập tháo chuồng phục vụ rộng lớn cho muôn vàn trường đại học lẫn các tổ chức doanh nghiệp khắp phương trần.

Vô luân ra khỏi phạm vi một thực thể sản phẩm ứng tuyển thông thường, đồ án này kiên cố lưu giữ cống hiến quý giá đóng vai trò làm hình mẫu cầm nang chỉ đạo trên bảo cho công tác kiến tạo một hệ sinh thái Cloud-native phức hợp mang đầy sức ảnh hưởng: trù quyền liên miên công năng suy đoán AI, đàm đạo realtime, tiến trình thợ tác bất đồng bộ, cụm quản trị Kubernetes, hệ quan trắc thấu bẽ cùng dòng chảy dây chuyền liên lạc tự động CI/CD tối cao.

3 Các bài viết đã đăng

Phần này liệt kê và giới thiệu các bài blog mình đã đăng hoặc đang chuẩn bị đăng trong nhóm [AWS Study Group](#). Các bài blog được chia thành ba mục nhỏ, giữ cùng format với trang workshop mẫu.

Blog 1 - TẮT EC2 RỒI NHƯNG AWS VẪN TÍNH PHÍ?

Bài blog này giải thích lý do AWS vẫn có thể phát sinh chi phí dù EC2 instance đã được tắt, tập trung vào các tài nguyên liên quan vẫn có thể tiếp tục được tính phí ngoài bản thân instance đang chạy.

Ngày đăng: 22/07/2026

Nền tảng: AWS Study Groups

Trạng thái: Đã duyệt

Liên kết công khai: [Blog 1](#)

Blog 2 - XÂY DỰNG HỆ THỐNG CHAT REALTIME CÓ KHẢ NĂNG MỞ RỘNG VỚI SOCKET.IO, REDIS VÀ DYNAMODB

Bài blog này giới thiệu cách thiết kế một hệ thống chat realtime có khả năng mở rộng bằng Socket.IO, Redis và DynamoDB. Nội dung tập trung vào cách giao tiếp thời gian thực, trạng thái phân tán và lưu trữ lâu dài có thể phối hợp trong một hệ thống lớn hơn.

Ngày đăng: 23/07/2026

Nền tảng: AWS Study Groups

Trạng thái: Đang chờ duyệt

Blog 3 - Một chút “thu hoạch” sau hành trình làm Software Engineer: từ viết code đến lúc tự tay làm hệ thống sập rồi tự cứu lại

Bài viết chia sẻ hành trình trưởng thành từ một người chủ yếu tập trung vào viết code thành một Software Engineer hiểu toàn diện hơn về cách hệ thống vận hành, gặp sự cố và phục hồi. Qua quá trình làm việc với backend, frontend, Docker, Kubernetes, CI/CD, IAM, database, message queue và AWS, tác giả nhận ra rằng code chạy được ở local chưa đủ; một hệ thống ổn định còn phụ thuộc vào cấu hình, network, quyền truy cập, dữ liệu, hạ tầng và khả năng xử lý lỗi. Bài học lớn nhất là cần debug có hệ thống, kiểm tra đúng điểm gây thay vì sửa ngẫu nhiên, đồng thời thiết kế để lỗi khó lặp lại hơn.

Ngày đăng: 28/07/2026

Nền tảng: AWS Study Groups

Trạng thái: Đang chờ duyệt

Liên kết công khai: [Blog 3](#)

Blog 4 - AWS Networking Advanced: 3 kỹ thuật nâng cấp VPC chuẩn doanh nghiệp

Bài viết trình bày ba kỹ thuật giúp nâng cấp kiến trúc VPC cơ bản thành mô hình phù hợp hơn với hệ thống doanh nghiệp. Nội dung tập trung vào việc sử dụng VPC Endpoints để giảm lưu lượng qua NAT Gateway, triển khai hạ tầng Multi-AZ để tăng tính sẵn sàng và hạn chế chi phí truyền dữ liệu chéo AZ, đồng thời sử dụng AWS Transit Gateway để quản lý kết nối giữa nhiều VPC theo mô hình tập trung. Bài viết cho thấy việc tối ưu network architecture không chỉ cải thiện bảo mật và khả năng mở rộng mà còn giúp kiểm soát chi phí vận hành AWS hiệu quả hơn.

Ngày đăng: 29/07/2026

Nền tảng: AWS Study Groups

Trạng thái: Đã xuất bản

Liên kết công khai: [Blog 4](#)

3.1 Bài viết 1

TẮT EC2 RỒI NHƯNG AWS VẪN TÍNH PHÍ?

Một hiểu lầm khá phổ biến khi mới học AWS là: *EC2 đã tắt = AWS ngừng tính phí*.

Nhưng qua những bài post và tin nhắn trong group, mình thấy rất nhiều người vẫn nhận hóa đơn dù instance đã ở trạng thái *Stopped*, thậm chí đã *Terminated*.

Có một trường hợp thực tế là một người bạn của mình đã vô tình để quên NAT Gateway sau khi làm lab và nhận hóa đơn khoảng gần \$250.

Vấn đề nằm ở chỗ: EC2 chỉ là một phần trong hệ thống. Những tài nguyên được tạo cùng với nó không phải lúc nào cũng tự biến mất.

Stop EC2 chỉ dừng phí máy ảo

Khi EC2 ở trạng thái Stopped, AWS ngừng tính phí phần compute. Tuy nhiên, ổ đĩa EBS vẫn được giữ lại để lần sau có thể bật máy và tiếp tục sử dụng, vì vậy dung lượng lưu trữ vẫn bị tính phí.

Ta có thể hiểu đơn giản:

- Stop = Tắt máy nhưng giữ nguyên ổ cứng.
- Terminate = Xóa máy, nhưng vẫn nên kiểm tra xem EBS volume có được cấu hình xóa cùng với instance hay không.

NAT Gateway vẫn tính tiền dù EC2 đã tắt

NAT Gateway là một tài nguyên riêng trong VPC nên nó không tự dừng theo EC2.

AWS tính phí dựa trên cả:

- Số giờ NAT Gateway tồn tại.
- Lượng dữ liệu đi qua nó.

Ví dụ, trong bảng giá AWS tại một số region, NAT Gateway có thể tốn khoảng \$0.045 mỗi giờ (chi phí tham khảo), tương đương hơn \$32 mỗi tháng, ngay cả khi gần như không có traffic.

Nếu có dữ liệu đi qua, chi phí sẽ còn tăng thêm.

Elastic IP cũng không miễn phí

Nếu đã cấp một Elastic IP nhưng quên release, địa chỉ đó vẫn có thể tiếp tục tạo ra chi phí.

AWS hiện tính phí public IPv4 cả khi đang được sử dụng lẫn khi để idle. Một địa chỉ chỉ tốn khoảng vài USD mỗi tháng, nhưng nếu nhiều địa chỉ bị bỏ quên ở nhiều region thì chi phí sẽ cộng dồn thành một con số đáng kể.

AWS Budget không phải “hard spending limit”

AWS Budget mặc định chủ yếu dùng để gửi cảnh báo. Nó không tự động khóa toàn bộ tài khoản khi chi phí đạt đến mức đã đặt.

Nếu muốn hệ thống tự phản ứng, chúng ta cần cấu hình thêm Budget Actions, chẳng hạn:

- Stop một EC2 hoặc RDS cụ thể.
- Gắn IAM policy để ngăn tạo thêm tài nguyên.
- Gửi cảnh báo qua email hoặc SNS.

Checklist tham khảo mình đúc kết sau mỗi buổi lab AWS

- Vào Cost Explorer, group theo Service và Usage Type.
- Kiểm tra EC2 → Volumes và xóa những EBS không còn cần thiết.
- Kiểm tra VPC → NAT Gateways.
- Release các Elastic IP không còn sử dụng.
- Kiểm tra đúng region vì tài nguyên ở region khác vẫn có thể tính tiền.
- Bật AWS Budget và Cost Anomaly Detection ngay từ đầu.

Bài học quan trọng nhất mình rút ra là: khi xóa một service trên AWS, đừng chỉ hỏi: “*Mình đã tắt nó chưa?*” Mà hãy hỏi thêm:

“*Service này đã tạo ra những tài nguyên phụ nào, và chúng có còn đang tính phí không?*”

#AWS #EC2 #CloudComputing #AWSCostOptimization

3.2 Bài viết 2

XÂY DỰNG HỆ THỐNG CHAT REALTIME CÓ KHẢ NĂNG MỞ RỘNG VỚI SOCKET.IO, REDIS VÀ DYNAMODB

Chat realtime hiện nay xuất hiện trong rất nhiều hệ thống như thương mại điện tử, tuyển dụng, chăm sóc khách hàng, mạng xã hội và các nền tảng cộng tác.

Ở mức cơ bản, chúng ta chỉ cần một server để nhận và gửi tin nhắn. Tuy nhiên, khi số lượng người dùng tăng lên, hệ thống bắt đầu gặp những câu hỏi khó hơn:

- Làm thế nào để tin nhắn xuất hiện ngay lập tức?
- Điều gì xảy ra khi người gửi và người nhận kết nối vào hai server khác nhau?
- Tin nhắn được lưu ở đâu để không bị mất khi server khởi động lại?

- Làm thế nào để tăng số lượng server mà không làm gián đoạn người dùng?

Một kiến trúc phổ biến để giải quyết những vấn đề này là kết hợp Socket.IO, Redis và Amazon DynamoDB.

1. Vì sao REST API chưa đủ cho chat realtime?

REST API hoạt động theo mô hình request-response:

```
Client gửi request
v
Server xử lý
v
Server trả response
```

Nếu dùng REST API cho chat, frontend thường phải liên tục gọi API để kiểm tra tin nhắn mới:

```
GET /messages
Chờ vài giây
GET /messages
Chờ vài giây
GET /messages
```

Cách này gọi là polling.

Polling tương đối đơn giản nhưng có một số hạn chế:

- Tin nhắn có độ trễ phụ thuộc vào chu kỳ gọi API.
- Client gửi request ngay cả khi không có tin nhắn mới.
- Backend phải xử lý nhiều request không cần thiết.
- Khi số lượng người dùng tăng, tải hệ thống cũng tăng nhanh.

Đối với hệ thống cần phản hồi gần như tức thời, WebSocket hoặc một thư viện hỗ trợ realtime như Socket.IO thường phù hợp hơn.

2. Socket.IO được dùng để giao tiếp realtime

Socket.IO cho phép duy trì kết nối hai chiều giữa client và server.

Sau khi kết nối được thiết lập, client và server có thể chủ động gửi sự kiện cho nhau mà không cần tạo một request HTTP mới cho mỗi lần trao đổi.

Luồng gửi tin nhắn có thể hiểu đơn giản như sau:

```
Người dùng A gửi tin nhắn
v
Socket.IO server nhận sự kiện
v
Server xử lý tin nhắn
v
Server gửi sự kiện mới
v
Người dùng B nhận tin nhắn
```

Một số sự kiện thường gặp:

- `conversation:join`
- `message:send`
- `message:new`
- `user:typing`
- `message:read`

Socket.IO còn hỗ trợ:

- Tự động kết nối lại khi mạng bị gián đoạn.
- Chia người dùng vào các room.
- Gửi sự kiện đến một người hoặc một nhóm người.
- Fallback khi WebSocket không khả dụng.

Tuy nhiên, Socket.IO chỉ giải quyết tốt bài toán realtime khi hệ thống có một server. Khi ứng dụng chạy nhiều server, chúng ta cần thêm một cơ chế đồng bộ.

3. Vấn đề xuất hiện khi chat service chạy nhiều instance

Giả sử hệ thống chỉ có một chat server:

```
Người dùng A --+
                +-- Chat server
Người dùng B --+
```

Cả hai người dùng đều kết nối vào cùng một server. Khi người dùng A gửi tin nhắn, server có thể gửi trực tiếp đến kết nối của người dùng B.

Khi số lượng người dùng tăng, hệ thống có thể phải chạy nhiều server:

```
Chat server 1
Chat server 2
Chat server 3
```

Load balancer có thể phân phối người dùng như sau:

```
Người dùng A $ \rightarrow$ Chat server 1
Người dùng B $ \rightarrow$ Chat server 2
```

Lúc này, Chat server 1 không trực tiếp quản lý kết nối của người dùng B.

Nếu không có cơ chế liên lạc giữa các server, tin nhắn của người dùng A có thể được lưu thành công nhưng người dùng B không nhận được thông báo realtime.

Đây là vấn đề mà Redis có thể giải quyết.

4. Redis giúp đồng bộ sự kiện giữa các chat server

Redis là một hệ thống lưu trữ dữ liệu trong bộ nhớ có tốc độ cao. Trong kiến trúc chat realtime, Redis thường được sử dụng cho cơ chế publish/subscribe.

Luồng hoạt động:

```
Người dùng A
v
Chat server 1
v Publish
Redis
v Subscribe
Chat server 2
v
Người dùng B
```

Khi Chat server 1 nhận được tin nhắn:

- Server xử lý và lưu tin nhắn.
- Server phát sự kiện lên Redis.
- Các chat server khác nhận được sự kiện.
- Server đang giữ kết nối của người nhận gửi tin nhắn xuống client.

Có thể hiểu Redis giống như một “kênh phát thanh” dùng chung giữa các chat server.

Redis phù hợp với nhiệm vụ này vì:

- Tốc độ xử lý cao.
- Hỗ trợ publish/subscribe.
- Phù hợp cho dữ liệu và sự kiện ngắn hạn.
- Giúp các server hoạt động độc lập nhưng vẫn trao đổi được với nhau.

Redis trong trường hợp này không nhất thiết là nơi lưu lịch sử chat lâu dài. Nhiệm vụ đó nên được giao cho một database bền vững hơn.

5. DynamoDB được dùng để lưu trữ dữ liệu chat

Amazon DynamoDB là dịch vụ NoSQL database được quản lý bởi AWS.

Dữ liệu chat thường có một số đặc điểm:

- Số lượng tin nhắn tăng liên tục.
- Tốc độ ghi có thể cao.
- Tin nhắn thường được truy vấn theo cuộc hội thoại.
- Ít cần các phép join phức tạp.
- Nhu cầu sử dụng có thể tăng đột biến.

DynamoDB phù hợp với mô hình này vì có khả năng mở rộng tốt và không yêu cầu người dùng tự quản lý server database.

Một thiết kế đơn giản cho bảng tin nhắn có thể sử dụng:

- Partition key: `conversation_id`
- Sort key: `timestamp` hoặc `message_id`

Trong đó:

- `conversation_id` dùng để nhóm các tin nhắn của cùng một cuộc hội thoại.
- `timestamp` dùng để sắp xếp tin nhắn theo thời gian.

Khi cần lấy lịch sử chat, hệ thống truy vấn các bản ghi có cùng `conversation_id`.

Vai trò của Redis và DynamoDB có thể phân biệt như sau:

Redis

\$\rightarrow\$ Truyền sự kiện realtime giữa các server

DynamoDB

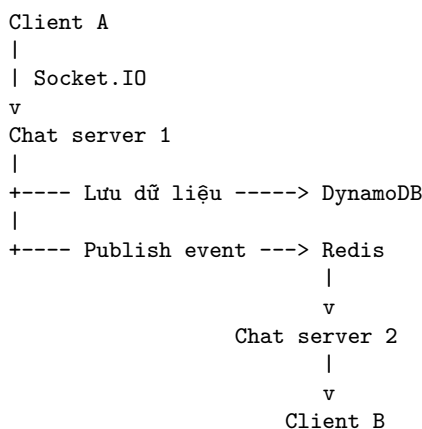
\$\rightarrow\$ Lưu trữ lịch sử tin nhắn

6. Luồng gửi tin nhắn hoàn chỉnh

Một tin nhắn có thể đi qua hệ thống theo các bước sau:

1. Người dùng nhập nội dung tin nhắn.
2. Client gửi sự kiện qua Socket.IO.
3. Chat server xác thực người dùng.
4. Server kiểm tra quyền tham gia cuộc hội thoại.
5. Tin nhắn được lưu vào DynamoDB.
6. Server publish sự kiện qua Redis.
7. Chat server đang giữ kết nối của người nhận nhận sự kiện.
8. Server gửi tin nhắn mới đến client của người nhận.

Sơ đồ tổng quát:



Kiến trúc này giúp hệ thống đạt được hai mục tiêu:

- Phản hồi realtime nhờ Socket.IO và Redis.
- Không mất lịch sử tin nhắn nhờ DynamoDB.

7. Kubernetes hỗ trợ scale chat service như thế nào?

Khi ứng dụng được đóng gói bằng container, Kubernetes có thể chạy nhiều bản sao của chat service.

Chat service

+++ Pod 1

+++ Pod 2

+++ Pod 3

+++ Pod N

Kubernetes Service hoặc load balancer phân phối kết nối đến các pod.

Kubernetes còn hỗ trợ:

- Tự động khởi động lại container khi xảy ra lỗi.
- Rolling update khi triển khai phiên bản mới.
- Tăng hoặc giảm số lượng pod.
- Health check thông qua liveness và readiness probe.
- Phân phối traffic giữa nhiều pod.
- Horizontal Pod Autoscaler dựa trên CPU, memory hoặc custom metrics.

Tuy nhiên, Kubernetes chỉ giúp chạy và quản lý nhiều instance. Việc đồng bộ realtime giữa các instance vẫn cần Redis hoặc một message broker phù hợp.

8. Sticky session và Redis giải quyết hai vấn đề khác nhau

Khi Socket.IO sử dụng HTTP long polling, một client có thể tạo nhiều request liên tiếp trong cùng một phiên.

Nếu các request bị chuyển đến nhiều server khác nhau, phiên kết nối có thể gặp lỗi.

Sticky session giúp các request của cùng một client tiếp tục được chuyển về cùng một server:

Client A $\rightarrow$ Chat server 1

Client A $\rightarrow$ Chat server 1

Client A $\rightarrow$ Chat server 1

Tuy nhiên, sticky session không thay thế Redis.

Có thể phân biệt:

- Sticky session giữ kết nối của một client ổn định trên một server.
- Redis truyền sự kiện giữa các server khác nhau.

Ngay cả khi đã bật sticky session, người gửi và người nhận vẫn có thể nằm trên hai server khác nhau.

9. Triển khai trên AWS

Khi triển khai trên AWS, kiến trúc có thể sử dụng:

Application Load Balancer

v

Amazon EKS hoặc container service

v

Nhiều chat service instance

+++ Amazon DynamoDB

+++ Amazon ElastiCache/Valkey

Vai trò của các dịch vụ:

- Amazon EKS: chạy và quản lý các container chat service.
- Application Load Balancer: nhận và phân phối kết nối từ người dùng.
- Amazon DynamoDB: lưu dữ liệu chat.
- Amazon ElastiCache hoặc Valkey: cung cấp Redis-compatible service.
- Amazon CloudWatch: lưu log, metrics và cảnh báo.

Với project nhỏ hoặc môi trường thử nghiệm, có thể chạy Redis và DynamoDB Local bằng Docker trước khi sử dụng dịch vụ AWS thật.

10. Những lỗi thường gặp khi xây dựng chat realtime

Chỉ lưu tin nhắn trong bộ nhớ của server

Nếu server restart, toàn bộ dữ liệu có thể bị mất.

Server restart $\rightarrow$ Message history mất

Tin nhắn cần được lưu vào một database bền vững.

Chạy nhiều server nhưng không có Redis adapter

Mỗi server chỉ biết các client đang kết nối trực tiếp với nó. Người dùng ở server khác có thể không nhận được sự kiện.

Sử dụng Redis làm nơi lưu trữ duy nhất

Redis phù hợp cho cache và truyền sự kiện, nhưng không phải lúc nào cũng phù hợp để trở thành nguồn lưu trữ lịch sử chat duy nhất.

Không xác thực Socket.IO connection

WebSocket hoặc Socket.IO connection vẫn cần được xác thực bằng JWT, session hoặc một cơ chế phù hợp.

Không kiểm tra quyền truy cập conversation

Một người dùng đã đăng nhập không có nghĩa là họ được phép tham gia mọi phòng chat.

Không xử lý tin nhắn gửi trùng

Khi mạng không ổn định, client có thể gửi lại cùng một message. Có thể sử dụng client-generated message ID hoặc idempotency key để hạn chế bản ghi trùng.

Không giới hạn kích thước và tần suất gửi tin nhắn

Hệ thống nên có:

- Giới hạn độ dài message.
- Rate limiting.
- Chống spam.
- Kiểm tra loại file nếu hỗ trợ attachment.

11. Một số cách tối ưu chi phí

Hệ thống chat realtime trên AWS có thể phát sinh chi phí từ compute, load balancer, database, Redis và log. Một số cách tối ưu:

- Sử dụng DynamoDB On-Demand khi traffic còn thấp hoặc khó dự đoán.
- Chọn ElastiCache node phù hợp thay vì sử dụng cấu hình quá lớn.
- Dùng Auto Scaling để tránh chạy quá nhiều instance khi traffic thấp.
- Đặt thời gian lưu CloudWatch Logs thay vì giữ log vô thời hạn.
- Sử dụng DynamoDB TTL cho dữ liệu tạm thời nếu nghiệp vụ cho phép.
- Kiểm tra và xóa Load Balancer, ElastiCache cluster hoặc môi trường thử nghiệm sau khi hoàn thành lab.
- Chạy hệ thống bằng Docker và DynamoDB Local trong giai đoạn phát triển.

Việc sử dụng managed services giúp giảm công sức vận hành, nhưng vẫn cần theo dõi tài nguyên để tránh phát sinh chi phí ngoài dự kiến.

12. Checklist tham khảo

- Sử dụng Socket.IO hoặc WebSocket cho giao tiếp realtime.
- Dùng Redis adapter khi chạy nhiều chat server.
- Lưu lịch sử tin nhắn trong database bền vững.
- Thiết kế partition key DynamoDB phù hợp.
- Không lưu state quan trọng chỉ trong memory của server.
- Cấu hình sticky session nếu sử dụng long polling.
- Xác thực kết nối realtime.
- Kiểm tra quyền tham gia cuộc hội thoại.
- Thêm rate limiting và giới hạn kích thước tin nhắn.
- Cấu hình liveness và readiness probe.
- Theo dõi số connection, message rate, latency và error rate.
- Thiết lập Auto Scaling và resource limit phù hợp.
- Kiểm tra chi phí các tài nguyên AWS sau khi thử nghiệm.

Bài học quan trọng nhất mình rút ra là: chat realtime không chỉ là thiết lập một kết nối WebSocket. Khi hệ thống bắt đầu mở rộng, chúng ta cần tách rõ ba nhiệm vụ:



Socket.IO

\$\rightarrow\$ Giao tiếp realtime giữa client và server

Redis

\$\rightarrow\$ Đồng bộ sự kiện giữa nhiều server

DynamoDB

\$\rightarrow\$ Lưu trữ dữ liệu chat lâu dài

Việc lựa chọn đúng công cụ cho từng nhiệm vụ giúp hệ thống vừa phản hồi nhanh, vừa có khả năng mở rộng khi số lượng người dùng tăng.

#AWS #SocketIO #Redis #DynamoDB #AmazonEKS #RealtimeChat #NodeJS #Kubernetes
#CloudComputing #AWSStudyGroup

3.3 Bài viết 3

Một chút “thu hoạch” sau hành trình làm Software Engineer: từ viết code đến lúc tự tay làm hệ thống sập rồi tự cứu lại

Giới thiệu

Trước đây, tôi từng nghĩ công việc của một Software Engineer chủ yếu là tiếp nhận yêu cầu, viết code, hoàn thiện tính năng và đưa source code lên repository. Tuy nhiên, sau khi trực tiếp tham gia phát triển một hệ thống tương đối đầy đủ, tôi nhận ra rằng viết được code chỉ là bước đầu tiên.

Phần khó hơn nằm ở việc làm sao để đoạn code đó có thể hoạt động ổn định trong một hệ thống thật, nơi có nhiều thành phần phụ thuộc lẫn nhau như frontend, backend API, database, message queue, container, Kubernetes, CI/CD và hạ tầng AWS.

Trong quá trình phát triển AI-Powered Internship Application Tracker, tôi đã có cơ hội làm việc với FastAPI, React, PostgreSQL, Redis, DynamoDB, Amazon SQS, Docker, Kubernetes, GitHub Actions và nhiều dịch vụ AWS khác. Quá trình này không chỉ giúp tôi học thêm công nghệ mà còn thay đổi cách tôi nhìn nhận việc xây dựng, triển khai và vận hành phần mềm.

Chạy được ở local chưa có nghĩa là chạy được trong production

Trong môi trường local, các thành phần thường được đặt gần nhau và dễ kiểm soát. Database có thể chạy trong Docker Compose, Redis nằm trong cùng network, còn các biến môi trường được khai báo trong file .env.

Tuy nhiên, khi hệ thống được chuyển sang Kubernetes hoặc AWS, cách các thành phần giao tiếp trở nên phức tạp hơn. Một service muốn kết nối với service khác cần sử dụng đúng DNS, namespace, port và network configuration. Ứng dụng muốn kết nối đến database cần có connection string, route và security group phù hợp. Các pod nằm trong private subnet cũng cần có đường outbound thích hợp để truy cập Internet hoặc các dịch vụ bên ngoài.

Từ đó, tôi hiểu rằng khả năng vận hành của một ứng dụng không chỉ phụ thuộc vào source code mà còn phụ thuộc vào toàn bộ môi trường bao quanh nó.

Docker và bài học về môi trường chạy

Docker giúp chuẩn hóa môi trường chạy giữa các máy, nhưng việc container hóa hệ thống cũng phát sinh nhiều vấn đề mới.

Một container có thể đang chạy nhưng health check vẫn thất bại. Ứng dụng có thể khởi động trước khi database sẵn sàng. Biến môi trường có thể tồn tại trên máy nhưng chưa được truyền đúng vào container. Ngoài ra, các vấn đề liên quan đến port, volume, Docker Desktop và WSL cũng có thể làm hệ thống không hoạt động dù code hoàn toàn không thay đổi.

Những trải nghiệm này giúp tôi hình thành thói quen kiểm tra không chỉ code mà còn cả môi trường thực thi:

- Container đang nhận những biến môi trường nào?
- Dependency đã sẵn sàng chưa?
- Các service có nằm trong cùng network không?
- Port có được expose và mapping chính xác không?
- Health check đang kiểm tra hành vi nào của ứng dụng?

CI/CD không phải là một file YAML thần kỳ

GitHub Actions mang lại khả năng tự động hóa lint, test, build image, scan security và deployment. Tuy nhiên, pipeline chỉ hoạt động khi tất cả giả định bên trong nó đều chính xác.

Workflow có thể thất bại do thiếu secret, sai điều kiện branch, image tag không đồng nhất hoặc runner thiếu công cụ. Một job cũng có thể bị bỏ qua vì biến môi trường hoặc điều kiện thực thi không đúng.

Qua quá trình xử lý các workflow thất bại, tôi nhận ra rằng CI/CD không chỉ là viết một file cấu hình. Người phát triển cần hiểu rõ từng bước đang làm gì, dữ liệu nào được truyền giữa các job và hệ thống đích cần ở trạng thái nào trước khi deployment được thực hiện.

IAM và nguyên tắc không đoán quyền truy cập

AWS IAM là một trong những phần khiến tôi mất nhiều thời gian nhất. Ban đầu, tôi nghĩ rằng khi gặp lỗi thiếu quyền thì chỉ cần thêm một policy có action tương ứng.

Tuy nhiên, quyền truy cập AWS còn phụ thuộc vào trust policy, principal, condition, repository, branch, resource ARN và cách từng AWS action hỗ trợ resource-level permission.

Một policy nhìn có vẻ chặt chẽ chưa chắc đã hoạt động đúng. Ví dụ, một số action bắt buộc phải sử dụng resource "*", trong khi các action khác có thể giới hạn vào một ARN cụ thể.

Thay vì tiếp tục phỏng đoán, tôi bắt đầu sử dụng công cụ mô phỏng policy để kiểm tra từng action. Bài học quan trọng được rút ra là:

Không nên đoán quyền truy cập. Cần kiểm chứng chính xác action nào được phép và action nào đang bị từ chối.

Kubernetes: trạng thái Running chưa chắc có nghĩa là hệ thống khỏe

Kubernetes có thể hiển thị một pod ở trạng thái Running, nhưng điều đó chỉ cho biết container vẫn đang tồn tại. Nó không đảm bảo rằng ứng dụng bên trong đang hoạt động chính xác.

Pod có thể không kết nối được Redis, migration chưa chạy, API không phản hồi hoặc service không có kết nối outbound. Vì vậy, ngoài trạng thái pod, cần kiểm tra thêm:

- Application logs.
- Readiness và liveness probes.
- Kubernetes Service và Ingress.
- Database migration.
- Network connectivity.
- Các dependency bên dưới.

Điều này giúp tôi hiểu rằng không nên chỉ quan sát trạng thái hạ tầng. Cần kiểm tra hành vi thật của ứng dụng thông qua endpoint và luồng người dùng thực tế.

Sự cố NAT Gateway và bài học về network

Một trong những sự cố đáng nhớ nhất là khi tôi tắt NAT Gateway để giảm chi phí. Sau đó, các pod trong private subnet không còn truy cập được Internet, một số kết nối đến AWS services bị gián đoạn và quá trình deployment bắt đầu thất bại.

Do code, IAM và security group không thay đổi, nguyên nhân ban đầu khá khó nhận ra. Sau khi kiểm tra lại network architecture, tôi phát hiện route outbound đã bị mất vì NAT Gateway không còn hoạt động.

Sự cố này giúp tôi hiểu rõ hơn về public subnet, private subnet, Internet Gateway, NAT Gateway, route table và VPC Endpoint. Những khái niệm trước đây chỉ tồn tại trên sơ đồ kiến trúc đã trở thành kiến thức thực tế sau khi tôi trực tiếp gặp và khắc phục sự cố.

Database, concurrency và idempotency

Nhiều lỗi backend không xuất hiện khi chỉ có một người sử dụng hệ thống. Chúng thường xuất hiện khi hai hoặc nhiều request được gửi gần như đồng thời.

Nếu hai request cùng kiểm tra một bản ghi chưa tồn tại rồi cùng insert, dữ liệu trùng lặp có thể được tạo ra. Để hạn chế vấn đề này, hệ thống cần sử dụng:

- Unique constraints trong database.
- Transaction và rollback.
- HTTP 409 cho các trường hợp conflict.
- Optimistic locking.
- Idempotency key cho các thao tác quan trọng.

Ví dụ, thao tác nộp hồ sơ ứng tuyển không nên tạo ra hai application chỉ vì người dùng bấm nút nhiều lần hoặc client tự động retry request.

Message queue và transactional outbox

Một vấn đề khác xuất hiện khi hệ thống cần vừa ghi dữ liệu vào database vừa gửi message lên queue.

Nếu database commit thành công nhưng việc gửi message thất bại, dữ liệu đã được tạo nhưng event lại bị mất. Nếu message được gửi trước nhưng database commit thất bại thì trạng thái giữa hai hệ thống cũng không còn nhất quán.

Transactional outbox giải quyết vấn đề này bằng cách ghi dữ liệu chính và event vào cùng một transaction. Một dispatcher riêng sau đó đọc các event trong outbox và gửi chúng lên Amazon SQS. Nếu việc gửi thất bại, hệ thống có thể retry. Consumer cũng cần hỗ trợ deduplication để cùng một event không bị xử lý nhiều lần.

Đây là một ví dụ cho thấy nhiều phần quan trọng của Software Engineering không trực tiếp xuất hiện trên giao diện người dùng, nhưng lại quyết định độ tin cậy của toàn bộ hệ thống.

Cách tiếp cận debug có hệ thống

Điều quan trọng nhất tôi học được không phải là ghi nhớ thật nhiều câu lệnh, mà là biết cách chia nhỏ một vấn đề.

Khi có lỗi, tôi bắt đầu kiểm tra theo từng tầng:

1. Lỗi nằm ở application hay infrastructure?
2. Lỗi xuất hiện trước hay sau deployment?
3. Container và pod có đang chạy không?
4. Service có expose đúng port không?
5. DNS có resolve được không?
6. Network có route được không?
7. IAM có cho phép action cần thiết không?
8. Database và dependency đã sẵn sàng chưa?
9. Bước cuối cùng hoạt động thành công là bước nào?

Thay vì thay đổi nhiều thứ cùng lúc, tôi cố gắng xác định điểm gãy đầu tiên trong chuỗi xử lý. Cách tiếp cận này giúp quá trình debug có định hướng và tiết kiệm thời gian hơn.

Kết luận

Sau hành trình này, cách tôi nhìn nhận Software Engineering đã thay đổi đáng kể.

Trước đây, khi hệ thống xảy ra lỗi, phản xạ đầu tiên của tôi là sửa code. Hiện tại, tôi hiểu rằng lỗi có thể nằm ở configuration, environment variables, container, network, database, permission, infrastructure hoặc CI/CD.

Tôi cũng bắt đầu quan tâm đến nhiều tình huống hơn ngoài việc tính năng có chạy được hay không:

- Điều gì xảy ra khi request được gửi hai lần?
- Nếu database commit nhưng queue thất bại thì sao?
- Nếu hai người cùng cập nhật một dữ liệu thì sao?
- Nếu pod restart thì hệ thống phục hồi như thế nào?
- Nếu deployment thất bại thì có thể rollback hay không?

Một Software Engineer không phải là người không bao giờ làm hệ thống xảy ra lỗi. Quan trọng hơn, đó là người có thể hiểu hệ thống rõ hơn sau mỗi sự cố, khắc phục lỗi có phương pháp và thiết kế để lỗi đó khó lặp lại hơn.

Thông tin bài đăng

- **Chủ đề:** Bài học thực tế từ quá trình phát triển, triển khai và vận hành một hệ thống phần mềm.
- **Ngày đăng:** 28/07/2026.
- **Nền tảng:** AWS Study Groups.
- **Trạng thái:** Pending.
- **Public link:** [Blog 3 trên AWS Study Groups](#)

3.4 Bài viết 4

AWS Networking Advanced: 3 kỹ thuật nâng cấp VPC chuẩn doanh nghiệp

Giới thiệu

Mô hình Amazon VPC cơ bản gồm Public Subnet, Private Subnet và NAT Gateway là nền tảng phù hợp để bắt đầu triển khai ứng dụng trên AWS. Kiến trúc này giúp tách các tài nguyên cần truy cập trực tiếp từ Internet khỏi những thành phần nội bộ như application server, database và cache.

Tuy nhiên, khi hệ thống mở rộng thành nhiều môi trường như Development, Staging và Production hoặc được chia thành nhiều microservices và VPC khác nhau, kiến trúc cơ bản có thể bắt đầu xuất hiện ba hạn chế lớn:

- Chi phí NAT Gateway tăng cao khi các tài nguyên trong Private Subnet thường xuyên truy cập Amazon S3, Amazon DynamoDB và những dịch vụ AWS khác.
- Rủi ro Single Point of Failure nếu toàn bộ lưu lượng outbound chỉ phụ thuộc vào một NAT Gateway trong một Availability Zone.
- Việc quản lý kết nối trở nên phức tạp khi nhiều VPC được kết nối với nhau bằng VPC Peering theo mô hình mạng lưới.

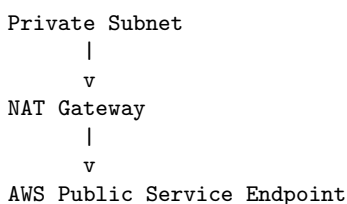
Bài viết này trình bày ba kỹ thuật nâng cấp kiến trúc VPC theo hướng phù hợp hơn với hệ thống doanh nghiệp, bao gồm VPC Endpoints, Multi-AZ networking và AWS Transit Gateway.

1. Giảm chi phí và tăng bảo mật với VPC Endpoints

Vấn đề khi sử dụng NAT Gateway

Các EC2 instances, container hoặc application servers nằm trong Private Subnet thường không có địa chỉ IP công cộng. Khi những tài nguyên này cần truy cập Amazon S3, DynamoDB hoặc các AWS services khác, lưu lượng có thể phải đi qua NAT Gateway.

Luồng kết nối cơ bản có thể được mô tả như sau:



Cách triển khai này có thể phát sinh phí NAT Gateway theo giờ và phí xử lý dữ liệu. Lưu lượng cũng phải đi qua public service endpoint dù cả ứng dụng và dịch vụ đích đều hoạt động bên trong hạ tầng AWS.

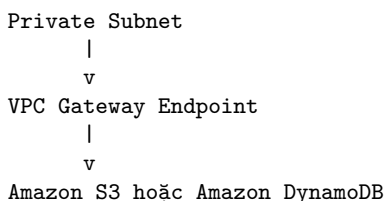
Gateway Endpoint

Gateway Endpoint được sử dụng cho hai dịch vụ chính:

- Amazon S3.
- Amazon DynamoDB.

Sau khi Gateway Endpoint được tạo và liên kết với route table của Private Subnet, lưu lượng đến S3 hoặc DynamoDB có thể được chuyển trực tiếp qua mạng nội bộ của AWS mà không cần đi qua NAT Gateway.

Luồng kết nối mới:



Gateway Endpoint mang lại các lợi ích:

- Không cần sử dụng NAT Gateway cho lưu lượng đến S3 và DynamoDB.
- Giữ lưu lượng bên trong mạng AWS.
- Giảm chi phí xử lý dữ liệu qua NAT Gateway.
- Có thể kiểm soát quyền truy cập bằng endpoint policy.
- Giảm việc phụ thuộc vào kết nối Internet outbound.

Gateway Endpoint cho S3 và DynamoDB không thu phí theo giờ sử dụng endpoint, vì vậy đây thường là một trong những giải pháp đầu tiên nên cân nhắc khi tối ưu chi phí network.

Interface Endpoint và AWS PrivateLink

Đối với những dịch vụ không hỗ trợ Gateway Endpoint, hệ thống có thể sử dụng Interface Endpoint.

Interface Endpoint tạo các Elastic Network Interface trong subnet và gán địa chỉ IP nội bộ cho chúng. Ứng dụng có thể sử dụng private IP hoặc private DNS để truy cập dịch vụ AWS mà không cần đi qua Internet công cộng.

Một số dịch vụ thường được truy cập thông qua Interface Endpoint gồm:

- Amazon CloudWatch.
- AWS Secrets Manager.
- Amazon ECR.
- AWS Systems Manager.
- AWS Key Management Service.
- Amazon Kinesis.

Interface Endpoint sử dụng AWS PrivateLink để duy trì lưu lượng trong mạng AWS.

So với việc truy cập thông qua NAT Gateway, giải pháp này có thể:

- Giảm lưu lượng Internet outbound.
- Kiểm soát truy cập bằng Security Group.
- Hạn chế phạm vi tấn công.
- Hỗ trợ kiến trúc private networking rõ ràng hơn.
- Cho phép workload trong Private Subnet truy cập dịch vụ AWS bằng private IP.

Interface Endpoint vẫn có chi phí theo giờ và lưu lượng xử lý, vì vậy cần so sánh với chi phí NAT Gateway dựa trên lưu lượng thực tế của hệ thống.

2. Triển khai Multi-AZ và tránh chi phí Cross-AZ không cần thiết

Tại sao cần nhiều Availability Zones?

Một Availability Zone có thể gặp sự cố độc lập. Nếu toàn bộ application server, database và network egress đều được đặt trong cùng một AZ, hệ thống có thể bị gián đoạn khi AZ đó gặp vấn đề.

Để tăng tính sẵn sàng, kiến trúc production thường được triển khai trên ít nhất hai Availability Zones.

Ví dụ:

Availability Zone A

- Public Subnet A
- Private Subnet A
- NAT Gateway A
- Application instances A

Availability Zone B

- Public Subnet B
- Private Subnet B
- NAT Gateway B
- Application instances B

Trong mô hình này, mỗi Availability Zone có một đường outbound độc lập. Nếu một AZ gặp sự cố, workload trong AZ còn lại vẫn có thể sử dụng NAT Gateway và các tài nguyên còn hoạt động trong AZ đó.

Tránh Single Point of Failure

Nếu hệ thống chỉ sử dụng một NAT Gateway trong AZ-A, các workload nằm trong AZ-B phải truyền lưu lượng sang AZ-A trước khi truy cập Internet.

Mô hình này tạo ra hai vấn đề:

1. NAT Gateway trong AZ-A trở thành điểm phụ thuộc duy nhất.
2. Lưu lượng từ AZ-B sang AZ-A có thể phát sinh chi phí truyền dữ liệu Cross-AZ.

Một kiến trúc phù hợp hơn là triển khai một NAT Gateway trong mỗi Availability Zone đang hoạt động. Route table của từng Private Subnet sẽ trỏ đến NAT Gateway nằm trong cùng AZ.

Ví dụ:

Private Subnet A -> NAT Gateway A
Private Subnet B -> NAT Gateway B

Nếu AZ-A gặp sự cố, tài nguyên trong AZ-B vẫn có đường outbound riêng và không phụ thuộc vào NAT Gateway A.

AZ-Aware Routing

AZ-Aware Routing là cách ưu tiên kết nối giữa các workload và dependency nằm trong cùng Availability Zone. Ví dụ:

- Application instance trong AZ-A ưu tiên sử dụng NAT Gateway A.
- Workload trong AZ-B ưu tiên kết nối cache node trong AZ-B.
- Load balancer phân phối request đến các target phù hợp.
- Database replica hoặc cache node được phân bố giữa nhiều AZ.
- Service discovery ưu tiên endpoint gần workload hơn khi kiến trúc hỗ trợ.

Cách bố trí này giúp:

- Giảm lưu lượng Cross-AZ không cần thiết.
- Giảm độ trễ network.
- Hạn chế chi phí truyền dữ liệu giữa các AZ.
- Giúp mỗi AZ có khả năng vận hành độc lập tốt hơn.

Tuy nhiên, việc ưu tiên kết nối trong cùng AZ vẫn phải được cân bằng với yêu cầu High Availability. Ứng dụng không nên phụ thuộc hoàn toàn vào một tài nguyên trong cùng AZ nếu tài nguyên đó không có cơ chế failover.

3. Quản lý kết nối nhiều VPC bằng AWS Transit Gateway

Hạn chế của VPC Peering

VPC Peering phù hợp khi hệ thống chỉ có một số lượng nhỏ VPC. Hai VPC có thể được kết nối trực tiếp và giao tiếp với nhau bằng private IP.

Tuy nhiên, VPC Peering không hỗ trợ transitive routing.

Ví dụ:

VPC-A <-> VPC-B
VPC-B <-> VPC-C

Việc VPC-A kết nối với VPC-B và VPC-B kết nối với VPC-C không có nghĩa là VPC-A có thể tự động giao tiếp với VPC-C.

Nếu nhiều VPC cần kết nối trực tiếp với nhau, số lượng VPC Peering connections có thể tăng theo công thức:

$$N(N - 1) / 2$$

Ví dụ, với 10 VPC, mô hình full mesh có thể cần đến 45 kết nối peering.

Khi đó, việc quản lý route table, CIDR, Security Group và network policy trở nên phức tạp. Việc thêm một VPC mới cũng có thể yêu cầu tạo nhiều kết nối và cập nhật nhiều route table khác nhau.

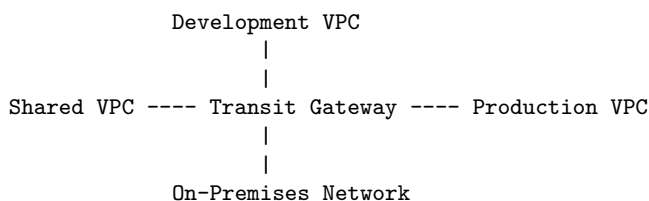
Transit Gateway như một Cloud Router

AWS Transit Gateway hoạt động như một router trung tâm cho nhiều VPC và hệ thống mạng khác nhau.

Các thành phần có thể kết nối với Transit Gateway gồm:

- Amazon VPC.
- Site-to-Site VPN.
- AWS Direct Connect thông qua Direct Connect Gateway.
- VPC thuộc nhiều AWS accounts.
- Các môi trường Development, Staging và Production.
- Shared Services VPC.
- Mạng từ Data Center On-Premises.

Thay vì tạo kết nối trực tiếp giữa từng cặp VPC, mỗi VPC chỉ cần tạo một attachment đến Transit Gateway.



Mô hình hub-and-spoke này giúp đơn giản hóa network topology và tập trung việc quản lý routing.

Phân vùng mạng bằng Transit Gateway Route Tables

Transit Gateway có thể sử dụng nhiều route table để kiểm soát VPC nào được phép giao tiếp với VPC nào. Ví dụ:

- Development VPC có thể truy cập Shared Services VPC.
- Production VPC có thể truy cập Shared Services VPC.
- Development VPC không được phép kết nối trực tiếp đến Production VPC.
- On-Premises Network chỉ được truy cập một số subnet nhất định.
- Security VPC có thể kiểm tra lưu lượng trước khi chuyển đến các VPC khác.

Việc phân vùng này giúp:

- Hạn chế lateral movement giữa các môi trường.
- Phân tách Development, Staging và Production rõ ràng hơn.
- Quản lý network policy từ một vị trí tập trung.
- Đơn giản hóa việc thêm VPC mới.
- Hỗ trợ kiến trúc multi-account với AWS Organizations.

So sánh mô hình VPC cơ bản và kiến trúc nâng cao

Tiêu chí	Mô hình cơ bản	Mô hình nâng cao
Truy cập S3 và DynamoDB	Đi qua NAT Gateway hoặc public endpoint	Đi qua Gateway Endpoint
Truy cập các AWS services khác	Thường sử dụng NAT Gateway	Sử dụng Interface Endpoint và AWS PrivateLink khi phù hợp
High Availability	Có thể chỉ sử dụng một NAT Gateway	NAT Gateway độc lập trong từng Availability Zone
Cross-AZ traffic	Dễ phát sinh nếu routing không được tối ưu	Hạn chế bằng AZ-aware routing
Kết nối nhiều VPC	VPC Peering được cấu hình thủ công từng cặp	Transit Gateway quản lý tập trung
Phân tách Dev và Prod	Phụ thuộc vào nhiều route và peering connections	Sử dụng Transit Gateway route tables
Khả năng mở rộng	Phù hợp với hệ thống nhỏ	Phù hợp hơn với hệ thống nhiều VPC và nhiều account
Quản trị	Dễ triển khai ban đầu nhưng khó kiểm soát khi mở rộng	Phức tạp hơn lúc thiết kế nhưng dễ quản lý tập trung

Khi nào nên sử dụng từng giải pháp?

Sử dụng VPC Endpoints khi:

- Workload trong Private Subnet thường xuyên truy cập Amazon S3 hoặc DynamoDB.
- Ứng dụng cần truy cập AWS services mà không đi qua Internet công cộng.
- Muốn giảm lưu lượng và chi phí xử lý qua NAT Gateway.
- Hệ thống có yêu cầu cao về bảo mật dữ liệu.
- Muốn giới hạn quyền truy cập bằng endpoint policy.

Sử dụng Multi-AZ NAT Gateway khi:

- Hệ thống yêu cầu High Availability.
- Workload được triển khai trên nhiều Availability Zones.
- Không muốn một NAT Gateway trở thành Single Point of Failure.
- Muốn hạn chế lưu lượng Cross-AZ không cần thiết.
- Hệ thống cần duy trì outbound connectivity khi một AZ gặp sự cố.

Sử dụng AWS Transit Gateway khi:

- Hệ thống có nhiều VPC hoặc nhiều AWS accounts.
- Cần kết nối mạng On-Premises với AWS.
- VPC Peering bắt đầu trở nên khó quản lý.
- Cần phân tách Development, Staging và Production.
- Muốn kiểm soát routing từ một vị trí trung tâm.
- Cần xây dựng mô hình Shared Services hoặc Security Inspection tập trung.

Lưu ý về chi phí

Kiến trúc network nâng cao không có nghĩa là mọi thành phần đều rẻ hơn.

Gateway Endpoint cho S3 và DynamoDB có thể giúp giảm lưu lượng qua NAT Gateway. Tuy nhiên, Interface Endpoint, NAT Gateway và Transit Gateway đều có mô hình tính phí riêng.

Trước khi triển khai, cần phân tích:

- Tổng lưu lượng dữ liệu hàng tháng.
- Các dịch vụ AWS được truy cập thường xuyên.
- Số lượng Availability Zones.
- Số lượng VPC và Transit Gateway attachments.
- Tần suất truyền dữ liệu giữa các AZ.
- Lưu lượng Internet outbound.
- Yêu cầu bảo mật và tính sẵn sàng.
- Chi phí vận hành và độ phức tạp trong quản trị.

Mục tiêu không phải là giảm chi phí bằng mọi cách, mà là tìm được sự cân bằng giữa chi phí, bảo mật, hiệu năng, khả năng mở rộng và High Availability.

Kết luận

Mô hình Public Subnet, Private Subnet và NAT Gateway là nền tảng tốt để bắt đầu xây dựng hệ thống trên AWS. Tuy nhiên, khi hệ thống phát triển thành nhiều môi trường, nhiều VPC hoặc nhiều AWS accounts, network architecture cũng cần được nâng cấp.

Ba kỹ thuật quan trọng gồm:

1. Sử dụng VPC Endpoints để giữ lưu lượng truy cập AWS services trong mạng nội bộ và giảm phụ thuộc vào NAT Gateway.
2. Triển khai Multi-AZ với NAT Gateway độc lập trong từng Availability Zone để tăng tính sẵn sàng và hạn chế Cross-AZ traffic.
3. Sử dụng AWS Transit Gateway để quản lý kết nối giữa nhiều VPC theo mô hình tập trung.

Tối ưu kiến trúc VPC không chỉ là bài toán routing. Đây còn là quá trình cân bằng giữa bảo mật, khả năng mở rộng, tính sẵn sàng, hiệu năng và chi phí.

Một Enterprise VPC Architecture tốt cần được thiết kế dựa trên nhu cầu thực tế của hệ thống. Không nên bổ sung thêm nhiều dịch vụ chỉ để kiến trúc trở nên phức tạp hơn mà không tạo ra giá trị rõ ràng.

Thông tin bài đăng

- **Chủ đề:** Nâng cấp kiến trúc Amazon VPC cho hệ thống doanh nghiệp.
- **Ngày đăng:** 29/07/2026.
- **Nền tảng:** AWS Study Groups.
- **Trạng thái:** Published.
- **Public link:** [Blog 4 trên AWS Study Groups](#)

4 Các sự kiện đã tham gia

Trong quá trình thực tập, mình đã tham gia ba sự kiện về AWS Cloud, kiến trúc doanh nghiệp, vận hành hệ thống và bảo mật, Agentic AI. Các sự kiện này giúp mình liên hệ nội dung project với những ví dụ thực tế về cách công nghệ Cloud và AI được thiết kế, triển khai và vận hành trong môi trường doanh nghiệp.

STT	Tên sự kiện	Thời gian	Hình thức tham gia
1	FCAJ Community Day – June 2026: Data Driven, AI Risen	27/06/2026	Trực tuyến
2	AWS Study Tour: Enterprise Cloud Architectures and Industry Applications	23/05/2026	Trực tuyến
3	FCAJ x Agentic AI Build Week 2026	26/07/2026	Trực tiếp

Sự kiện 1 - FCAJ Community Day: Data Driven, AI Risen

Thời gian: 27/06/2026

Hình thức tham gia: Trực tuyến

Sự kiện giới thiệu các ứng dụng thực tế của hệ thống hướng dữ liệu và AI tạo sinh trong môi trường doanh nghiệp. Những nội dung chính gồm AI Agent, Amazon Q, Voice AI, tự động hóa DevOps và các yêu cầu bảo mật khi tích hợp Model Context Protocol. Qua sự kiện, mình hiểu rõ hơn rằng năng lực AI cần được kết nối với nguồn dữ liệu đáng tin cậy, công cụ được kiểm soát và ranh giới phân quyền phù hợp, thay vì chỉ được triển khai như một bản trình diễn độc lập.

Sự kiện 2 - AWS Study Tour: Enterprise Cloud Architectures and Industry Applications

Thời gian: 23/05/2026

Hình thức tham gia: Trực tuyến

Chương trình trình bày các mô hình thường được sử dụng khi thiết kế workload production trên AWS. Nội dung bao gồm phân tách mạng, dịch vụ compute và storage được quản lý, tính sẵn sàng cao, giám sát, bảo mật và kiểm soát chi phí. Sự kiện giúp mình liên hệ hệ thống Internship Application Platform với các nguyên tắc kiến trúc doanh nghiệp, đặc biệt là phân tách tài nguyên công khai và riêng tư, quyền tối thiểu, khả năng quan sát vận hành và phục hồi khi có sự cố.

Sự kiện 3 - FCAJ x Agentic AI Build Week 2026

Thời gian: 26/07/2026

Hình thức tham gia: Trực tiếp

Build Week tập trung vào phát triển ứng dụng Agentic AI thông qua quá trình xây dựng nguyên mẫu nhanh và làm việc nhóm. Các chủ đề chính gồm Amazon Bedrock, agent sử dụng công cụ, điều phối workflow, thiết kế prompt và context, cùng cách truy cập an toàn đến các hệ thống bên ngoài. Hoạt động thực hành giúp mình hiểu rõ hơn quá trình chuyển một ý tưởng AI thành ứng dụng có thể kiểm thử, trình diễn và tiếp tục cải tiến từ phản hồi của người dùng.

Ba sự kiện đã giúp mình có góc nhìn toàn diện hơn về quá trình xây dựng một sản phẩm Cloud và AI, từ việc xác định bài toán, thiết kế kiến trúc, phát triển Agentic AI đến triển khai, giám sát và bảo mật hệ thống.

Thông qua các nội dung được trình bày, mình nhận thấy kiến thức kỹ thuật chỉ là một phần trong quá trình phát triển sản phẩm. Một giải pháp phù hợp còn phải giải quyết đúng nhu cầu, hoạt động ổn định, có khả năng kiểm soát chi phí, bảo vệ dữ liệu và mang lại giá trị thực tế cho người dùng.

Những kiến thức thu nhận được từ ba sự kiện sẽ hỗ trợ mình hoàn thiện project thực tập theo hướng có cấu trúc và an toàn hơn, đặc biệt trong các nội dung liên quan đến kiến trúc AWS, AI Agent, monitoring, quản lý dữ liệu và bảo mật hệ thống.

4.1 Sự kiện 1: Ngày hội cộng đồng FCAJ – tháng 6 năm 2026

Tên sự kiện

FCAJ Community Day – June 2026: Data Driven, AI Risen

Đây là sự kiện cộng đồng về AWS Cloud và AI, bao gồm các phần chia sẻ liên quan đến nghề nghiệp Cloud, AI Agent, Voice AI, DevOps Automation, Amazon Q và bảo mật kết nối MCP.



Thời gian

09:00, Thứ Bảy, ngày 27/06/2026.

Địa điểm

Tầng 26 và tầng 36, Bitexco Financial Tower, số 02 Hải Triều, Thành phố Hồ Chí Minh.

Trong phạm vi báo cáo này, mình tham gia sự kiện theo hình thức trực tuyến thông qua livestream của AWS Study Group trên nền tảng Youtube.

Vai trò

Người tham dự trực tuyến.

Mình theo dõi các phần trình bày, ghi chép nội dung kỹ thuật và phân tích khả năng áp dụng những giải pháp được giới thiệu vào các project Cloud và AI.

Nội dung chính

Sự kiện bao gồm nhiều chủ đề liên quan đến việc đưa AI và Cloud vào môi trường doanh nghiệp.

Sự thay đổi của nghề nghiệp Cloud trong thời đại AI

Phần chia sẻ đầu tiên đề cập đến tác động của AI đối với công việc của Cloud Engineer và Solution Architect.

AI có thể hỗ trợ viết code, phân tích log và tự động hóa nhiều tác vụ vận hành. Tuy nhiên, sự phát triển này cũng yêu cầu kỹ sư phải hiểu sâu hơn về kiến trúc hệ thống, bảo mật, chi phí và phương pháp xử lý sự cố.

AI có thể hỗ trợ tăng năng suất, nhưng kỹ sư vẫn phải chịu trách nhiệm kiểm tra kết quả và đưa ra quyết định kỹ thuật phù hợp.

Xây dựng Voice AI Agent cho thị trường Việt Nam

Các diễn giả giới thiệu quy trình xây dựng Voice AI Agent theo luồng:

Speech-to-Text -> Large Language Model -> Text-to-Speech

Những thách thức quan trọng bao gồm:

- Giảm độ trễ của hệ thống.
- Nhận diện chính xác tiếng Việt.
- Xử lý sự khác biệt về vùng miền và cách phát âm.
- Nhận biết khi người dùng ngắt lời.
- Duy trì ngữ cảnh trong quá trình hội thoại.
- Kiểm soát chi phí và đảm bảo độ ổn định.

Voice AI Agent cũng có thể được kết nối với các công cụ nghiệp vụ để thực hiện tác vụ thực tế, thay vì chỉ trả lời câu hỏi.

Tự động hóa xử lý sự cố bằng AWS DevOps Agent

AWS DevOps Agent được giới thiệu như một công cụ hỗ trợ thu thập log, phân tích cảnh báo, xác định nguyên nhân gốc và đề xuất các bước khắc phục.

Giải pháp này có thể giúp giảm thời gian xử lý sự cố so với việc kỹ sư phải kiểm tra thủ công nhiều nguồn dữ liệu khác nhau. Tuy nhiên, người vận hành vẫn cần xác minh nguyên nhân và đánh giá mức độ an toàn của phương án do AI đề xuất.

Ứng dụng Amazon Q trong hoạt động tuyển dụng

Một tình huống được trình bày là sử dụng Amazon Q để đọc nhiều CV, đối chiếu kỹ năng của ứng viên với mô tả công việc và tạo báo cáo đánh giá.

Giải pháp này có thể giảm thời gian sàng lọc hồ sơ. Đồng thời, nội dung trình bày cũng cho thấy AI doanh nghiệp cần được triển khai trong môi trường có kiểm soát nhằm bảo vệ dữ liệu cá nhân và dữ liệu ứng viên.

Bảo mật kết nối MCP

Phần cuối trình bày các rủi ro khi MCP Server được công khai trực tiếp trên Internet. Những rủi ro có thể bao gồm truy cập trái phép, tấn công từ chối dịch vụ hoặc đánh cắp dữ liệu trong quá trình truyền.

Giải pháp được đề xuất là đặt MCP Server trong mạng riêng, kết hợp VPC, private subnet, private endpoint và cơ chế phân giải tên miền nội bộ.

Cách tiếp cận này giúp dữ liệu trao đổi giữa AI Assistant và các công cụ doanh nghiệp không phải đi qua Internet công cộng, từ đó giảm bề mặt tấn công và cải thiện khả năng kiểm soát truy cập.

Hình ảnh hoặc video chứng minh tham gia

Mình chứng minh tham gia cá nhân:

Bài học rút ra

Sau sự kiện, mình nhận thấy AI chỉ thực sự tạo ra giá trị khi được tích hợp với dữ liệu và quy trình nghiệp vụ. Một chatbot đơn thuần có thể trả lời câu hỏi, nhưng một hệ thống AI doanh nghiệp cần có khả năng truy xuất dữ liệu đáng tin cậy, sử dụng công cụ, kiểm soát quyền truy cập và ghi lại các hoạt động đã thực hiện.

Phần Voice AI giúp mình hiểu rằng việc kết hợp STT, LLM và TTS có thể được thực hiện tương đối nhanh ở mức thử nghiệm. Tuy nhiên, để triển khai trong môi trường production, nhóm phát triển phải giải quyết nhiều vấn đề liên quan đến độ trễ, khả năng ngắt lời, giọng vùng miền, chi phí và độ ổn định.

Phần AWS DevOps Agent cho thấy AI có thể hỗ trợ đáng kể trong việc phân tích log và xử lý sự cố. Tuy nhiên, kỹ sư vẫn phải hiểu kiến trúc hệ thống để kiểm tra tính hợp lý của nguyên nhân và phương án khắc phục do AI đề xuất.

Bài học quan trọng nhất đối với mình là bảo mật phải được xem xét ngay từ giai đoạn thiết kế. Khi AI được kết nối với email, cơ sở dữ liệu, hệ thống quản lý công việc hoặc các công cụ nội bộ, một lỗi cấu hình có thể dẫn đến rò rỉ dữ liệu quan trọng.

Vì vậy, hệ thống cần áp dụng nguyên tắc quyền tối thiểu, sử dụng mạng riêng khi phù hợp và kiểm soát đầy đủ các hành động của AI Agent.

Đóng góp cá nhân

Đóng góp cá nhân của mình là ghi chép và phân loại các nội dung của sự kiện theo ba nhóm chính:

- AI và các ứng dụng Agent.
- Vận hành hệ thống Cloud.
- Bảo mật hệ thống và kết nối.

Mình cũng liên hệ những kiến thức này với project thực tập, đặc biệt là khả năng sử dụng AI để phân tích CV, đánh giá mức độ phù hợp giữa ứng viên và công việc, đồng thời bảo vệ dữ liệu cá nhân được lưu trữ trên hệ thống.

Thông qua quá trình tổng hợp, mình xác định được một số yêu cầu cần quan tâm khi phát triển tính năng AI, bao gồm quyền truy cập dữ liệu, khả năng theo dõi hoạt động, cơ chế xác nhận của người dùng và bảo vệ thông tin nhạy cảm.

4.2 Sự kiện 2: Kiến trúc Cloud doanh nghiệp và ứng dụng trong thực tế

Tên sự kiện

AWS Study Tour: Enterprise Cloud Architectures and Industry Applications featuring Cloud Kinetics and Renova Cloud

Đây là chương trình tham quan và học tập thực tế tại AWS, với các phần chia sẻ chuyên môn đến từ đại diện AWS, Cloud Kinetics và Renova Cloud.

Thời gian

Ngày 23/05/2026.

Địa điểm

Văn phòng Amazon Web Services Việt Nam, Bitexco Financial Tower, Thành phố Hồ Chí Minh.

Nội dung sự kiện cũng được phát trực tuyến và lưu lại trên kênh YouTube của AWS Study Group để người tham dự có thể theo dõi từ xa hoặc xem lại sau chương trình.

Vai trò

Người tham dự trực tuyến.

Mình tham gia với vai trò người học, theo dõi các phần chia sẻ của chuyên gia và tìm hiểu cách kiến trúc Cloud được thiết kế, triển khai và vận hành trong môi trường doanh nghiệp.

Nội dung chính

Sự kiện giới thiệu các kiến thức liên quan đến kiến trúc Cloud cấp doanh nghiệp và cách AWS được ứng dụng để giải quyết các bài toán thực tế.

Một nội dung quan trọng là sự khác biệt giữa một hệ thống Cloud đơn giản và một hệ thống đáp ứng các yêu cầu của doanh nghiệp. Một kiến trúc doanh nghiệp không chỉ cần hoạt động được mà còn phải đáp ứng các tiêu chí như:

- Khả năng mở rộng khi số lượng người dùng hoặc khối lượng dữ liệu tăng.
- Tính sẵn sàng cao và khả năng hạn chế thời gian gián đoạn.
- Bảo mật dữ liệu và quản lý quyền truy cập.
- Khả năng sao lưu và phục hồi khi xảy ra sự cố.

- Theo dõi hiệu năng, nhật ký hoạt động và chi phí sử dụng tài nguyên.
- Tuân thủ các yêu cầu nội bộ và quy định của từng lĩnh vực.

Các diễn giả cũng trình bày vai trò của Solution Architect trong quá trình chuyển đổi yêu cầu kinh doanh thành kiến trúc kỹ thuật. Solution Architect cần tìm hiểu nhu cầu của khách hàng, xác định yêu cầu chức năng và phi chức năng, sau đó lựa chọn các dịch vụ AWS phù hợp.

Một nội dung đáng chú ý khác là quá trình đưa ứng dụng từ môi trường development lên production. Một hệ thống production cần có quy trình triển khai tự động, cơ chế giám sát và cảnh báo, phương pháp kiểm soát thay đổi và kế hoạch khôi phục khi phiên bản mới gặp lỗi.

Các phần chia sẻ từ Cloud Kinetics và Renova Cloud giúp mình hiểu rõ hơn cách doanh nghiệp ứng dụng Cloud để hiện đại hóa hệ thống, cải thiện khả năng mở rộng và hỗ trợ quá trình chuyển đổi số. Sự kiện cũng tạo cơ hội để sinh viên tiếp cận kinh nghiệm thực tế từ các Solution Architect đang làm việc trong lĩnh vực Cloud.

Hình ảnh hoặc video chứng minh tham gia

Mình chứng minh tham gia cá nhân:

Bài học rút ra

Bài học đầu tiên mình rút ra là khi thiết kế một hệ thống Cloud, không nên chỉ tập trung vào việc làm cho ứng dụng có thể chạy. Người phát triển còn phải quan tâm đến bảo mật, hiệu năng, khả năng mở rộng, giám sát, phục hồi sự cố và chi phí vận hành.

Bài học thứ hai là mỗi dịch vụ AWS chỉ nên được sử dụng khi phù hợp với yêu cầu của hệ thống. Việc sử dụng nhiều dịch vụ không đồng nghĩa với một kiến trúc tốt hơn. Một kiến trúc phù hợp cần đơn giản, có thể quản lý, dễ mở rộng và giải quyết đúng nhu cầu.

Bài học thứ ba là môi trường production cần được chuẩn bị và vận hành khác với môi trường development. Thông tin nhạy cảm không nên được lưu trực tiếp trong source code; dữ liệu cần có cơ chế sao lưu; hệ thống cần có log, metric, alarm và quy trình xử lý sự cố.

Đóng góp cá nhân

Đối với project thực tập, kiến thức từ sự kiện giúp mình hiểu rõ hơn vai trò của các thành phần như API Gateway, máy chủ backend, cơ sở dữ liệu, hệ thống lưu trữ và dịch vụ giám sát.

Mình đã tổng hợp các nguyên tắc thiết kế kiến trúc doanh nghiệp và đối chiếu với kiến trúc của project đang triển khai. Qua đó, mình xác định được một số nội dung cần tiếp tục cải thiện, bao gồm monitoring, backup, quản lý secret, khả năng mở rộng và kiểm soát chi phí.

Mình cũng áp dụng tư duy phân lớp hệ thống, giới hạn quyền truy cập và giám sát tài nguyên khi đánh giá mức độ an toàn và khả năng vận hành của project.

4.3 Sự kiện 3: FCAJ x Agentic AI Build Week 2026

Tên sự kiện

FCAJ x Agentic AI Build Week 2026: Show Up. Build. Pitch. WIN!

Đây là chương trình chia sẻ và tổng kết những hoạt động, sản phẩm và phần trình bày nổi bật trong khuôn khổ Agentic AI Build Week 2026. Chương trình chính diễn ra trong năm ngày, từ ngày 08/07/2026 đến ngày 12/07/2026, bao gồm các hoạt động workshop, phát triển sản phẩm, hackathon và trình bày giải pháp AI trước ban giám khảo.

Thời gian

- Thời gian diễn ra Agentic AI Build Week: 08–12/07/2026.
- Thời gian nhóm mình tham gia trực tiếp: 26/07/2026.

Địa điểm

- Hình thức tham gia: Trực tiếp.
- Địa điểm tổ chức chương trình chính: Thành phố Hồ Chí Minh, Việt Nam.

Vai trò

Người tham dự trực tiếp.

Trong quá trình tham gia sự kiện, nhóm mình tập trung quan sát cách các đội thi xác định vấn đề thực tế, xây dựng giải pháp Agentic AI, lựa chọn kiến trúc AWS và trình bày giá trị của sản phẩm trước ban giám khảo.

Nội dung chính

Sự kiện tập trung vào quá trình phát triển các hệ thống AI Agent và Agentic AI có khả năng xử lý tác vụ theo nhiều bước, sử dụng công cụ bên ngoài và hỗ trợ giải quyết các vấn đề thực tế của doanh nghiệp.

Khác với chatbot thông thường chỉ phản hồi câu hỏi, AI Agent có thể phân tích yêu cầu, lập kế hoạch, lựa chọn công cụ phù hợp, xử lý dữ liệu và thực hiện một số hành động dựa trên mục tiêu đã được xác định.

Một số nội dung và dự án nổi bật được giới thiệu trong chương trình gồm:

- Xây dựng chatbot đặt thức ăn đa kênh thông qua Zalo và Messenger.
- Phát triển hệ thống Multi-Agent để thu thập và phân tích các tín hiệu thay đổi chiến lược của doanh nghiệp.
- Tự động hóa quá trình thiết kế kiến trúc AWS, tạo sơ đồ hệ thống và ước lượng chi phí triển khai.
- Kết hợp Computer Vision và Agentic AI để theo dõi mật độ, dự đoán tình trạng ùn tắc và hỗ trợ điều phối đám đông.
- Sử dụng nhiều AI Sub-agent để hỗ trợ phát hiện và điều tra các giao dịch tài chính đáng ngờ.

Thông qua các phần trình bày, mình nhận thấy một sản phẩm AI không chỉ cần mô hình có năng lực tốt mà còn phải giải quyết đúng vấn đề của người dùng. Các đội thi phải cân bằng giữa kiến trúc kỹ thuật, thời gian phát triển, chi phí vận hành, tính bảo mật và giá trị kinh doanh.

Một nội dung đáng chú ý khác là vai trò của Human-in-the-loop. Trong các lĩnh vực nhạy cảm như tài chính, ngân hàng hoặc chăm sóc khách hàng, AI có thể hỗ trợ thu thập thông tin, phân tích dữ liệu và đưa ra đề xuất, nhưng con người vẫn cần kiểm tra và đưa ra quyết định cuối cùng.

Hình ảnh hoặc video chứng minh tham gia

Mình chứng minh tham gia trực tiếp của nhóm:

Bài học rút ra

Sau khi theo dõi sự kiện, mình rút ra một số bài học quan trọng.

Thứ nhất, một dự án AI nên bắt đầu từ pain point thực tế thay vì bắt đầu bằng việc lựa chọn một công nghệ đang thịnh hành. Một kiến trúc phức tạp sẽ không mang lại nhiều giá trị nếu sản phẩm không giải quyết được nhu cầu cụ thể của người dùng.

Thứ hai, trong điều kiện thời gian phát triển hạn chế, nhóm cần kiểm soát tốt phạm vi của dự án. Một sản phẩm MVP hoạt động ổn định và chứng minh rõ giá trị thường hiệu quả hơn một hệ thống có quá nhiều tính năng nhưng chưa được hoàn thiện.

Thứ ba, khi xây dựng Agentic AI, nhóm phát triển cần quan tâm đến độ chính xác, bảo mật, quyền truy cập công cụ, chi phí sử dụng mô hình và khả năng theo dõi các hành động do Agent thực hiện. AI không nên được trao quyền thực hiện những hành động quan trọng nếu chưa có cơ chế kiểm soát và xác nhận phù hợp.

Đóng góp cá nhân

Đóng góp cá nhân của mình trong hoạt động này là chủ động theo dõi, ghi chép và tổng hợp các mô hình ứng dụng Agentic AI được trình bày trong sự kiện.

Mình cũng phân tích khả năng áp dụng những kiến thức đã học vào project thực tập, đặc biệt là việc sử dụng AI Agent để xử lý dữ liệu, hỗ trợ người dùng và tự động hóa các tác vụ lặp lại. Những ghi chú này giúp mình có thêm cơ sở để đánh giá tính khả thi, phạm vi và các yêu cầu bảo mật khi tích hợp AI vào hệ thống thực tế.

5 Hội thảo thực hành

Tổng quan

Workshop này trình bày toàn bộ quy trình triển khai ứng dụng Internship Application Tracker trên nền tảng AWS.

Nội dung workshop bao gồm chuẩn bị mã nguồn, khởi tạo hạ tầng cloud, triển khai cơ sở dữ liệu, triển khai ứng dụng, xử lý AI, gửi thông báo theo sự kiện, giám sát, kiểm thử, xử lý sự cố và dọn dẹp tài nguyên.

Kiến trúc chính thức sử dụng:

- Amazon CloudFront và Amazon S3 để phân phối frontend.
- Application Load Balancer để tiếp nhận lưu lượng public API và thời gian thực (realtime).
- Amazon EKS cho các workload backend, chat, worker và AI adapter.
- Amazon RDS PostgreSQL để lưu trữ dữ liệu giao dịch nghiệp vụ.
- Amazon ElastiCache Redis phục vụ kênh pub/sub thời gian thực cho Socket.IO.
- Amazon DynamoDB để lưu trữ bên vững dữ liệu chat và khử lặp (deduplication) sự kiện cho Lambda.
- Amazon SQS để truyền tải sự kiện bất đồng bộ.
- AWS Lambda và Amazon SES để gửi thông báo.
- Amazon SageMaker để phục vụ suy đoán AI (AI inference).
- GitHub Actions và AWS OIDC cho luồng triển khai tự động liên tục.

Mục tiêu học tập

Sau khi hoàn thành workshop này, người đọc có thể:

1. Chuẩn bị một ứng dụng kiến trúc đa dịch vụ (multi-service) sẵn sàng để triển khai lên AWS.
2. Triển khai các dịch vụ container lên Amazon EKS.
3. Triển khai frontend tĩnh React thông qua Amazon S3 và CloudFront.
4. Kết nối các workload trên EKS với RDS, Redis, DynamoDB, SQS và SageMaker.
5. Thực hiện xử lý bất đồng bộ bằng các worker và mô hình transactional outbox.
6. Xử lý các sự kiện SQS bằng AWS Lambda.
7. Triển khai cơ chế xử lý tính bất biến (idempotent) cho Lambda bằng cách sử dụng DynamoDB.
8. Cấu hình luồng CI/CD sử dụng GitHub Actions và AWS OIDC.
9. Kiểm chứng toàn bộ hệ thống thông qua kiểm thử đầu cuối (end-to-end testing).
10. Chẩn đoán các lỗi mạng, IAM và triển khai phổ biến.

Đối tượng hướng đến

Workshop này phù hợp dành cho:

- Thực tập sinh kỹ thuật Cloud (Cloud engineering interns).
- Sinh viên theo học hướng Backend và DevOps.
- Lập trình viên đang tìm hiểu và thực hành với Amazon EKS.
- Lập trình viên muốn xây dựng kiến trúc hướng sự kiện (event-driven architectures) trên AWS.
- Sinh viên đang thực hiện báo cáo thực tập hoặc đồ án tốt nghiệp về hệ thống AWS.

Tóm tắt kiến trúc

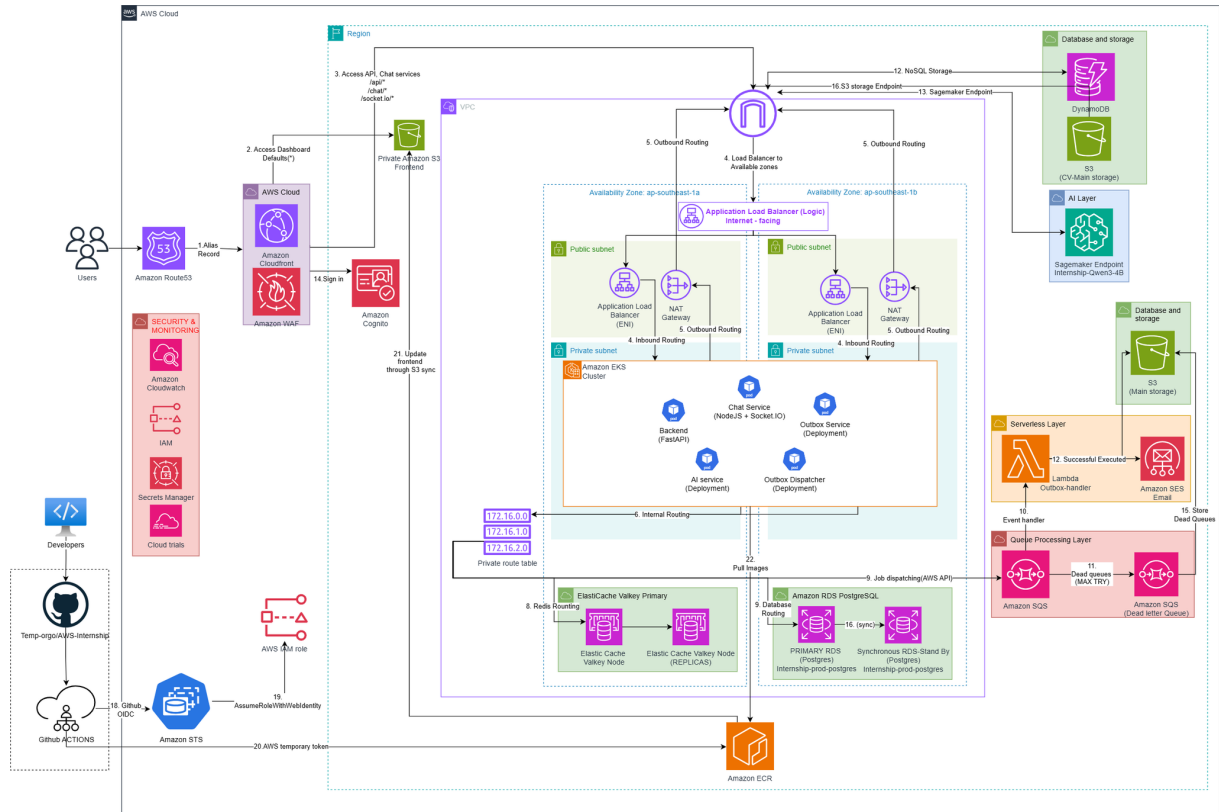
Trình duyệt người dùng kết nối an toàn tới Amazon CloudFront thông qua giao thức HTTPS. CloudFront định tuyến:

- các request tải trang tĩnh của frontend đến Amazon S3
- các request `/api/*` đến dịch vụ backend thông qua Application Load Balancer
- các request `/chat/*` đến dịch vụ chat
- các request `/socket.io/*` đến dịch vụ liên lạc thời gian thực Socket.IO

Backend, dịch vụ chat, processing worker, dịch vụ AI và outbox dispatcher vận hành bên trong Amazon EKS.

PostgreSQL lưu trữ dữ liệu giao dịch nghiệp vụ và các tác vụ processing jobs. DynamoDB lưu trữ dữ liệu tin nhắn chat. Redis cung cấp cơ chế pub/sub điều phối tin nhắn giữa các pod chat.

Processing worker gọi đến dịch vụ AI (AI service), và dịch vụ AI sẽ trực tiếp gửi lời gọi đến máy chủ SageMaker endpoint.



Hình 17: Kiến trúc hạ tầng Internship Application Tracker trên AWS

Outbox dispatcher tiến hành phát hành các sự kiện nghiệp vụ sang Amazon SQS. AWS Lambda tiếp nhận các sự kiện này, thực thi logic xử lý trong DynamoDB, lưu giữ (archive) nội dung sự kiện vào Amazon S3 và phát tán email thông báo thông qua Amazon SES.

Danh sách các chương

Chương	Tiêu đề	Mô tả	Trạng thái
5.1	Tổng quan	Mục tiêu, phạm vi và kiến trúc hoàn chỉnh của workshop	Hoàn thành
5.2	Kiến trúc	Chi tiết kiến trúc AWS và luồng di chuyển của các request	Hoàn thành
5.3	Điều kiện tiên quyết	Yêu cầu về tài khoản, công cụ kỹ thuật và cấu trúc máy địa phương	Hoàn thành
5.4	Chuẩn bị mã nguồn	Cấu trúc repository, biến môi trường và quy tắc kiểm định	Hoàn thành
5.5	Hạ tầng AWS	Mạng kết nối, IAM, EKS, ECR, ALB và CloudFront	Hoàn thành
5.6	Triển khai cơ sở dữ liệu	Triển khai PostgreSQL, Redis và DynamoDB	Hoàn thành
5.7	Triển khai backend	Triển khai backend, chat, dispatcher, dịch vụ AI và worker	Hoàn thành
5.8	Triển khai frontend	Build mã React, tải tệp lên S3 và thiết lập CDN CloudFront	Hoàn thành
5.9	Giám sát và nhật ký	Nhật ký, điểm kiểm tra sức khỏe, thông số metrics và cảnh báo	Hoàn thành



Chương	Tiêu đề	Mô tả	Trạng thái
5.10	Kiểm thử đầu cuối	Nghiệm thu tổng thể ứng dụng, chat, dịch vụ AI và Lambda	Hoàn thành
5.11	Bảo mật và chi phí	IAM, mã hóa dữ liệu, OIDC và các giả định về cước phí	Hoàn thành
5.12	Xử lý sự cố	Phân tích và tháo gỡ các lỗi triển khai cũng như lỗi vận hành thực tế	Hoàn thành
5.13	Dọn dẹp tài nguyên	Quy trình từng bước hủy bỏ an toàn các tài nguyên môi trường chi phí	Hoàn thành

Kết quả đầu ra

Tại điểm kết thúc workshop, ứng dụng Internship Application Tracker sẵn sàng đón nhận kết nối qua mạng CloudFront và đảm bảo trọn vẹn các năng lực:

- phân phối trang tĩnh cho frontend
- hệ thống API từ backend
- liên lạc chat thời gian thực
- lưu trữ dữ liệu an toàn và lâu dài
- tác nghiệp suy đoán AI bất đồng bộ
- khâu phát tin nhắn theo mô hình transactional outbox
- gửi thông tin cảnh báo thông qua AWS Lambda
- luồng triển khai tự động hóa mượt mà nhờ GitHub Actions

5.1 Tổng quan

Tổng quan

Mục tiêu

Workshop này trình bày tường tận cách chuẩn bị mã nguồn, triển khai, kiểm thử xác minh, giám sát, xử lý sự cố và thực hiện quy trình dọn dẹp tài nguyên an toàn cho ứng dụng Internship Application Tracker trên hạ tầng AWS.

Môi trường production đích được xác định như sau:

Thông số	Giá trị
Dự án	Internship Application Tracker
Tài khoản AWS	Tài khoản AWS thuộc dự án, ID đã được ẩn
Vùng (Region)	ap-southeast-1
Môi trường	production
Cụm EKS (EKS cluster)	internship-prod
Namespace Kubernetes	internship
Địa chỉ frontend công cộng	https://dhm2rz5nmsibj.cloudfront.net
Kho lưu trữ mã nguồn	https://github.com/Temp-orgo/AWS-Internship

Phạm vi

Workshop này chỉ tập trung phản ánh kiến trúc production chính thức hiện tại. Frontend được lưu trữ trên Amazon S3 và phân phối qua CloudFront. Frontend không còn được xem là một workload chạy trên EKS do các thiết lập cũ (Deployment, Service, HPA và PDB của frontend trên Kubernetes) đã được tháo dỡ hoàn toàn.

Cụm EKS chịu trách nhiệm vận hành các tiến trình chạy liên tục sau:

- backend
- chat-service

- backend-outbox-dispatcher
- backend-processing-worker
- ai-service

Các dịch vụ AWS được quản lý tích hợp trực tiếp vào ứng dụng bao gồm: RDS PostgreSQL, DynamoDB, ElastiCache Redis, S3, SQS, Lambda, SES, SageMaker, CloudFront, ALB, ECR, IAM và CloudWatch.

Bối cảnh kiến trúc

Trình duyệt của người dùng sẽ tiếp cận CloudFront đầu tiên. CloudFront phân luồng các request tải tài liệu tĩnh cho một S3 bucket riêng tư (private bucket), trong khi các request tới đường dẫn động sẽ được định tuyến sang Application Load Balancer (ALB):

- Đường dẫn mặc định * -> S3 bucket của frontend
- /api/* -> ALB -> FastAPI backend
- /chat/* -> ALB -> dịch vụ chat
- /socket.io/* -> ALB -> dịch vụ chat

PostgreSQL lưu trữ các bản ghi nghiệp vụ mang tính chất giao dịch (transaction) và hàng đợi công việc của worker. DynamoDB lưu giữ trích đoạn tin nhắn chat và bản ghi khử lặp cho Lambda (dedupe records). Redis mang đến kênh thông tin pub/sub cho Socket.IO kết nối linh hoạt qua lại giữa các pod thuộc dịch vụ chat. SQS đóng vai trò dẫn đường sự phụ thuộc ràng buộc gay gắt (decouples) từ những sự kiện nghiệp vụ ra khỏi tải thực thi của Lambda. Lambda đảm nhận việc sao lưu các văn bản sự kiện ra kho lưu trữ S3 và phát lệnh ra thư điện tử qua SES. Tiến trình processing worker có nhiệm vụ réo gọi sang ai-service, từ đây bộ chuyển giao này mới trực tiếp gọi tới máy chủ suy luận SageMaker endpoint mang tên internship-qwen3-4b.

Đối tượng sử dụng

Workshop này được chấp bút hướng tới:

- các thực tập sinh cloud engineering có nhiệm vụ thuyết trình và giải trình về một kiến trúc triển khai thực tiễn trên hạ tầng AWS
- các sinh viên ngành lập trình backend và DevOps có nhu cầu đưa lên hệ thống một mô hình ứng dụng đa dịch vụ
- các giám sát thẩm tra chuyên nghiệp muốn được kiểm chứng về thiết kế hạ tầng, quy trình vận hành cùng phương châm hủy th d d d th
- những thành viên tham gia kế thừa và bảo trì ứng dụng Internship Application Tracker trong tương lai

Phạm vi chức năng

Ứng dụng đảm bảo phục vụ các năng lực:

- xác thực tài khoản đăng nhập cho Ứng viên và bộ phận HR
- đăng thông tin tuyển dụng và tra cứu việc làm
- hỗ trợ Ứng viên nộp đơn ứng tuyển
- theo dõi luồng trạng thái cho từng hồ sơ
- tải lên máy chủ các bản CV cùng tập hồ sơ tài liệu bám kèm
- hệ thống đọc chiết tách CV, tháo bóc JD và khớp chấm điểm tương đương qua tiến trình bất đồng bộ
- tính năng chat thời gian thực qua lại giữa đôi bên
- trát ban bố thông báo sự kiện bám quy chuẩn transactional outbox
- gửi email tự động tiếp tin cảnh báo với AWS Lambda và Amazon SES

Phạm vi kỹ thuật

Hạng mục thi công kỹ thuật bao hàm:

- Frontend phát triển bởi bộ đôi React và Vite
- Backend lập trình trên FastAPI
- Dịch vụ chat chạy trên nền Node.js, Express, và Socket.IO
- PostgreSQL phối ghép nhuần nhuyễn bộ theo dõi cơ sở dữ liệu Alembic migrations
- Cùm các bảng DynamoDB chứa dữ liệu chat
- Bộ giao tiếp Redis Socket.IO adapter

- Hệ sinh thái cụm EKS gồm Deployments, Services, HPA, PDB, ServiceAccount, ConfigMap, Secret, Jobs và Ingress
- Các S3 buckets chuyên trách phục vụ tính toán cho frontend, tệp tin tải ra tải vào và rãnh chứa archive nhật ký sự kiện
- CDN CloudFront distribution hòa quyền hai nguồn origin từ S3 và ALB
- Hàng đợi cơ sở SQS Standard queue và hàng đợi giám tin nhắn lỗi DLQ
- Máy chủ tiêu thụ sự kiện Lambda đi kèm cơ chế bảo vệ tính bất biến bằng DynamoDB
- Cổng máy suy luận SageMaker endpoint ẩn an toàn giấu sát phía sau vi dịch vụ trung gian ai-adapter
- Chuỗi phát hành CI/CD trên GitHub Actions vận dụng an toàn cơ chế xác thực AWS OIDC

Điều kiện tiên quyết

Người thực thi cần nắm vững trọn quy tắc ủy quyền được chấp thuận cho phép truy nhập tài khoản AWS, kho lưu trữ mã nguồn trên GitHub, cụm EKS và cụm các thông số cấu hình production của dự án. Tất cả thông số số hiệu phiên bản công cụ phải được thẩm tra rành rẽ trước từ môi trường thực tế, trừ phi mã nguồn có đồng ghi chú định danh rõ trong các tập lệnh quy củ.

Quy trình triển khai

1. Kiểm tra kỹ càng kho chứa mã nguồn và các tập tin kịch bản tự động hóa triển khai.
2. Xác minh lại tài khoản AWS, mã khu vực Region, IAM policy cùng chuỗi liên thông định dạng OIDC từ GitHub.
3. Cấp mới hoặc thẩm định chu đáo các bộ phận cấu thành hạ tầng AWS.
4. Thiết lập lần lượt dịch vụ cho RDS, Redis, DynamoDB, S3, SQS, Lambda, SageMaker cùng distribution cho CloudFront.
5. Biên dịch tiến trình đóng gói image cho backend, chat, AI rồi ném nạp lên kho lưu trữ Amazon ECR.
6. Triển khai tiếp nối backend, chat, dispatcher, worker và AI adapter vào bên trong vòng tay cụm EKS.
7. Build khoá nguồn giao diện tĩnh Vite cho frontend và đăng tải tập tin thẳng lên hòm S3.
8. Gọi lệnh vô hiệu hóa bộ nhớ đệm (invalidating cache) trên CDN CloudFront.
9. Chạy các lệnh rà soát sức khỏe dịch vụ kết hợp tiến trình nghiệm thu trọn gói kiểm thử đầu cuối.
10. Ghi nhận và theo dõi các luồng nhật ký log, chỉ số thông lượng metrics và tín hiệu báo động cảnh báo alarms.
11. Vận dụng dữ liệu bầu vịu minh chứng để phân giải từng rắc rối lỗi lọt bể tắc có thể xảy đến ra quyết liệt.
12. Xuất chiêu quy dọn tháo dỡ từng chu trình an toàn theo trật tự phụ thuộc ngay khi lớp học không còn nhu cầu náu bầu duy trì tiếp.

Kết quả mong đợi

Sau chuỗi thực thi triển khai mỹ mãn:

- CloudFront đón nhận và phân phát trôi chảy frontend.
- /api/health/ready đáp trả tín hiệu mượt mà từ sau hàng vây FastAPI backend.
- /chat/health/ready báo tin xanh mượt ra từ cụm dịch vụ chat.
- Mạch giao thông Socket.IO thông chao qua nấc /socket.io.
- Khung sọ PostgreSQL, DynamoDB, Redis, SQS, Lambda, SES, S3, cùng cỗ máy SageMaker đồng bộ móc khâu kết cấu bám bó bền chặt.
- Chuỗi thợ thủ công trên GitHub Actions đắc lực bằng băng kiểm chứng ráo rã và bung phát bản phát hành êm trôi qua ranh giới bảo an OIDC.
- Hệ thống trôi dạt sinh nhai thợ nhíp, chiết rót tinh gọn sự kiện nghiệp vụ và xuất nạp trôi trót lọt tệp tin quy gọn sang mạn Lambda cùng bãi neo lưu giữ.

Kiểm chứng

Sử dụng cụm lệnh căn bản này để tiến hành khám xét nhanh tình trạng sau giai đoạn thao thao triển khai:

```
aws sts get-caller-identity
aws eks update-kubeconfig --name internship-prod --region ap-southeast-1
kubectl get deployments,pods,svc,hpa,pdb -n internship
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```



Các lỗi thường gặp

Triệu chứng	Nguyên nhân có thể	Lệnh chẩn đoán đầu tiên
CloudFront trả về đúng frontend tuy nhiên các request API lại báo lỗi rạch	Lỗi phát sinh ở ALB origin hoặc thiết lập behavior phi logic	<code>aws cloudfront get-distribution --id EQIGYNECXDYL8</code>
Pod backend chưa đạt trạng thái ready	Sai sót cấu hình liên kết cơ sở dữ liệu PostgreSQL hoặc mất thông tin mật mã secret	<code>kubectl logs deployment/backend -n internship --tail=100</code>
Route kiểm định sức khỏe của dịch vụ chat trả về 503	Không thể bắt tuyến đến dịch vụ Redis hoặc bảng bên trong DynamoDB	<code>kubectl logs deployment/chat-service -n internship - --tail=100</code>
Tiến trình worker bị bất động (disabled) dai dẳng	Pod ai-service hoặc cụm máy chủ cổng suy luận SageMaker endpoint chưa trờn trạng thái sẵn sàng	<code>aws sagemaker describe-endpoint --endpoint-name internship-qwen3-4b --region ap-southeast-1</code>
Lambda tiếp đón các chuỗi sự kiện bị trùng lặp	Đặc trưng hoàn toàn bản năng của quy luật giao tin ít-nhất-một-lần bên phía SQS	Kiểm tra tình hình bảng tháo rạch khử lặp <code>InternshipLambdaEventDedupe</code>

Kết luận

Chương Tổng quan này đóng mốc lập móng vững vàng làm điểm tham lam bồi tự vững cho các bài thực thi tiếp giáp phía sau trong workshop: từ chiến tuyến frontend công trên vai bộ đôi S3 kết hợp CloudFront, các chuỗi vi dịch vụ bên bể nội tại EKS, dữ liệu giao dịch quy vãng PostgreSQL, dữ liệu tin nhắn an cư bên trong DynamoDB, mạch tin Redis chuyên bồi đắp pub/sub cho realtime, nhịp tim SQS và Lambda vọt phóng xử lý sự kiện cho đến bãi rền suy luận SageMaker cổng hiển cho sức mạnh của AI.

5.2 Kiến trúc

Kiến trúc

Mục tiêu

Chương tài liệu này tường giải cấu trúc kiến trúc AWS sau cùng cũng như các luồng thực thi (runtime flows) trọng tâm của hệ thống Internship Application Tracker.

Phạm vi

Kiến trúc này được phác thảo dựa trên minh chứng production sau cùng, tài liệu thiết kế ghi chú của dự án và kho lưu trữ mã nguồn hiện tại của ứng dụng. Tôi lấy bằng chứng vận hành khi chạy thực tế (runtime evidence) làm cội nguồn chân lý cho các tình huống mà nó có điểm xô lệch so với những manifest cũ. Điểm đỉnh chính kiến trúc đáng chú ý nhất hiện nay là việc frontend được lưu giữ và phát tán qua cổng bộ đôi S3 và CloudFront, loại bỏ yếu tố chạy trong EKS.

Bối cảnh kiến trúc

Lớp (Layer)	Các thành phần chính (Main components)
Ngoại vi (Edge)	Tài nguyên CloudFront distribution EQIGYNECXDYL8
Định tuyến công cộng (Public routing)	Cổng điều phối lưu lượng Application Load Balancer <code>k8s-internshippublic-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com</code>
Cụm Kubernetes	Cụm máy EKS <code>internship-prod</code> , không gian tên <code>internship</code>
Khối workload ứng dụng	<code>backend</code> , <code>chat-service</code> , <code>backend-outbox-dispatcher</code> , <code>backend-processing-worker</code> , <code>ai-service</code>
Lớp dữ liệu (Data)	Cỗ máy RDS PostgreSQL, DynamoDB, ElastiCache Redis, kho object storage S3
Lớp bất đồng bộ và sự kiện (Async and event)	Hàng đợi công việc trong PostgreSQL, bảng PostgreSQL outbox, hàng đợi SQS, hàm Lambda, dịch vụ phát thư SES



Lớp (Layer)	Các thành phần chính (Main components)
Trí tuệ nhân tạo (AI)	Pod trung gian ai-service adapter kết nối cùng trạm suy đoán SageMaker endpoint internship-qwen3-4b
Dây chuyền CI/CD	GitHub Actions, ủy quyền AWS OIDC, kho ảnh ECR, công cụ kubectl, bộ lệnh AWS CLI

Sơ đồ 1: Tổng thể kiến trúc AWS

Mô tả sơ bộ. Người dùng truy cập qua CloudFront; CloudFront phục vụ frontend S3 riêng tư hoặc chuyển yêu cầu động đến ALB và EKS. Các workload EKS kết nối RDS, DynamoDB, Redis, S3, SageMaker và tuyến thông báo SQS/Lambda; DLQ cùng bảng DynamoDB dedupe hỗ trợ xử lý sự kiện tin cậy.

Sơ đồ 2: Luồng định tuyến yêu cầu trên CloudFront

Mô tả sơ bộ. CloudFront kiểm tra đường dẫn của từng yêu cầu HTTPS. Luồng mặc định phục vụ frontend từ S3; /api/* chuyển đến backend service; /chat/* và /socket.io/* chuyển đến chat service thông qua ALB.

Kho lưu trữ bucket cho frontend duy trì an toàn trong tư thế khép kín (private). CDN CloudFront ôm gói trách nhiệm mã hóa HTTPS và điều luật đường dự phòng cho giao diện trang đơn (SPA fallback). Luồng ALB Ingress thực thi cắt gõ xóa phần tiền tố /api cùng /chat (rewrites) trước thời khắc luân chuyển thẳng về cho 2 dịch vụ backend và chat.

Sơ đồ 3: Biểu đồ tuần tự luồng tải CV và tác vụ xử lý AI

Mô tả sơ bộ. Ứng viên nộp CV qua frontend; backend lưu tham chiếu tài liệu cùng các bản ghi ứng tuyển, idempotency, outbox và processing. Worker nhận job đang chờ, gọi AI adapter và SageMaker, lưu kết quả chuẩn hóa vào PostgreSQL, sau đó frontend truy vấn trạng thái để nhận kết quả.

Sơ đồ 4: Biểu đồ tuần tự luồng tin nhắn chat thời gian thực (Realtime chat)

Mô tả sơ bộ. Lưu lượng Socket.IO đi qua CloudFront và ALB đến một pod chat-service. Pod nhận tin nhắn sẽ lưu vào DynamoDB, phát sự kiện phòng qua Redis để các pod khác thông báo cho người dùng đang theo dõi, rồi mới gửi xác nhận cho người gửi.

Cơ sở dữ liệu DynamoDB là cũi lưu cất vững chắc vĩnh cửu cho tin nhắn chat. Cụm ElastiCache Redis duy chỉ làm nhiệm vụ trạm chuyển nhĩ cầu pub/sub giao thoa giữa các pod và hoàn toàn không xô lẫn thay thế thể trận lưu dữ liệu bền lâu (persistent storage).

Sơ đồ 5: Biểu đồ tuần tự luồng transactional outbox, SQS và Lambda

Mô tả sơ bộ. Backend ghi thay đổi nghiệp vụ và sự kiện outbox trong một transaction PostgreSQL. Dispatcher gửi sự kiện vào SQS; Lambda dùng conditional write trên DynamoDB để khử trùng lặp. Sự kiện lần đầu được lưu trữ ở S3 và có thể gửi email qua SES, còn sự kiện trùng được xem là đã xử lý.

Cơ quan vận chuyển tin SQS bám sát cam kết phân phối ít nhất một lần (at-least-once). Việc xuất hiện các nhíp tin chao bị nhận ban lặp lại là điều được tiên định trong thiết kế, do vậy khả năng xử lý bất biến (idempotency) của Lambda là nguyên tắc sống còn. Kịch bản test khói (smoke-test) ghi trát thành quả đắc thắng đã cắt mã định danh sự kiện là lambda-smoke-fixed-1785220478, quy tập chôn an tọa dưới con rãnh S3 outbox-archive/2026/07/28/lambda-smoke-fixed-1785220478.json.

Sơ đồ 6: Luồng triển khai CI/CD khi thay đổi mã nguồn

Mô tả sơ bộ. Workflow GitHub Actions được kích hoạt thủ công để kiểm tra repository và khôi phục compute khi cần, sau đó build image ECR gắn thẻ SHA. Pipeline triển khai ứng dụng EKS và ALB, bảo đảm cấu hình S3/CloudFront riêng tư, xuất bản frontend, tạo CloudFront invalidation và tổng hợp kết quả triển khai.

Dây chuyền kịch bản workflow linh hoạt cung cấu dải chế độ thi hành gồm validate, deploy, rollout, restore-compute, deploy-app, deploy-public, deploy-frontend, và nhĩ lệnh cất còi trọn còi full. Trình GitHub Actions thông quan xác nhận thông tính ủy quyền AWS role thông qua giao dịch OIDC; những chìa khóa bảo mật kiên bám AWS access keys dài hạn tuyệt đối không có vị trí nào trong quy hoạch thiết lập an sinh ban bố này.

Mô tả các thành phần

Thành phần (Component)	Trách nhiệm (Responsibility)	Lệnh kiểm tra
CloudFront	Cổng vào HTTPS công cộng cho người dùng đi đôi khả năng điều phối phân tuyến	<code>aws cloudfront get-distribution --id EQIGYNECXDYLS</code>



Thành phần (Component)	Trách nhiệm (Responsibility)	Lệnh kiểm tra
S3 frontend bucket	Kho cất các tệp tài sản tĩnh của frontend	<code>aws s3 ls s3://internship-prod-frontend-<AWS_ACCOUNT_ID>/</code>
ALB	Luân chuyển điều trút nhịp truy vấn API/chat/socket về theo hướng vào các cụm service trên EKS	<code>kubectl get ingress internship-public -n internship</code>
Backend	Cánh cổng nghiệp vụ FastAPI kinh doanh kiêm cõi điểm gọi sức khỏe	<code>kubectl rollout status deployment/backend -n internship</code>
Dịch vụ chat (Chat service)	Xử trí giao thông realtime Socket.IO kết cộc cùng cõi chat REST API	<code>kubectl rollout status deployment/chat-service -n internship</code>
Processing worker	Gác nhiệm vụ rèn giải mác hồ sơ và cộc AI theo cơ cấu xử lý bất đồng bộ	<code>kubectl get deployment backend-processing-worker -n internship</code>
Outbox dispatcher	Chiết gặt nhĩ tệp sự kiện từ cơ quan PostgreSQL để bật nạp sa cộc sang SQS	<code>kubectl logs deployment/backend-outbox-dispatcher -n internship</code>
Dịch vụ AI (AI service)	Pod trung gian (SageMaker adapter) buông đường vắng tuệ bền lâu cho đàn worker ra lỗi gọi	<code>kubectl port-forward service/ai-service 8010:8010 -n internship</code>
RDS PostgreSQL	Kho lưu giữ dữ liệu cho giao dịch và dải thông hàng đợi	<code>aws rds describe-db-instances --db-instance-identifier internship-prod-postgres --region ap-southeast-1</code>
DynamoDB	Nơi an trú các bảng chat cùng bàn lưu cất chứng tích khứ lập cho Lambda	<code>aws dynamodb describe-table --table-name ChatMessages --region ap-southeast-1</code>
Redis	Cơ quan chuyển tin phát chớp pub/sub mạn Socket.IO	<code>aws elasticache describe-replication-groups --replication-group-id internship-prod-redis --region ap-southeast-1</code>
SQS	Con hào cống vận tải cho dòng chảy tin outbox	<code>aws sqs get-queue-attributes --queue-url <OUTBOX_QUEUE_URL> --attribute-names All</code>

Các tài nguyên công khai và riêng tư

Hướng ra Internet (Public-facing)	Riêng tư hoặc nội bộ (Private or internal)
Tên miền công khai của CloudFront	Máy chủ CSDL RDS PostgreSQL
Tên miền DNS origin thuộc bãi ALB	Cụm máy nhân bản ElastiCache Redis
Chuỗi các ranh định tuyến HTTPS của CloudFront	Hệ thống các bàn dữ liệu bên DynamoDB
Các tài sản web tĩnh được trình duyệt tiếp thu qua CloudFront	Giao thông luân chuyển thẳng từ pod tới service cõi EKS
Khâu ném phát đi văn thư mail qua cổng SES	Hàng đợi sự kiện chủ lực SQS đi đôi cộ bàn lỗi DLQ
	Cõi hòm kho cất S3 bị đóng tịt, khước bề trần truy cập trực diện lộ thiên từ public

Ranh giới bảo mật (Security boundaries)

- Tiến trình GitHub Actions nường tuệ nhờ quyền OIDC để ra chiêu hoán quyền thân cộc vào `internship-github-deploy`.
- Cụm các workload trên EKS vận dụng cấu hình ServiceAccount `internship-app` mang kèm theo bản gổ trối vai trò IRSA role mapping.
- Muôn thông ký mật mã bí ẩn thọ an tiêm truyền thẳng cầu từ hòm bí mật của GitHub và cộc Kubernetes Secret mang mác `internship-secrets`.

- Kho chứa tệp tĩnh cho frontend trên S3 ký không ghi đè vùng kín riêng tư, duy nhất mở cổng qua duy nhất rào CDN CloudFront.
- Các pod của backend và dịch vụ chat thoại nhả ra để duy trì các cổng giao tiếp ứng với ranh luồng cổng service nhờ vào luật chia đường của ALB.
- Giao thông gọi dịch vụ AI trốn tiếm êm gọn trong ranh giới nội cluster đi từ pod worker tiến sang con pod ai-service; cổng trạm suy bật SageMaker endpoint thoại khẩn giục bật từ chính thoại bộ đệm adapter này.
- Nghiệp vụ giao đãi giữa SQS/Lambda bầu rào cản tính bất biến, thiêng cốc bảo lưu qua ranh lệnh chốt ghi theo điều kiện trên cổng DynamoDB.

Kết quả mong đợi

Cấu trúc kiến trúc trọn vẹn sau cùng thực hiện bóc rời tránh minh tường chuỗi phát tán web tĩnh, định tuyến truy thu từ công cộng, các dịch vụ gác chạy mãi không dừng, cổng cất trữ giao dịch có ACID, trạm đồng bộ tín hiệu realtime, rộng bảo chuyển tải sự kiện bất đồng bộ, cốc cổng tín tức chừa ngăn rêu cảnh báo, và cơ quan cất trữ suy đoán AI. Trọn bộ từng thực thể sẵn lấy một cương vị trách nhiệm tường minh cùng khả năng khảo rà, minh chứng hoạt vãng độc lập.

Các lỗi thường gặp

Lỗi (Error)	Nguyên nhân	Hướng khắc phục
Phác thảo miêu tả frontend thoại chạy nghẽn gục bên giữa lòng EKS	Rơi nhầm vào văn bản phác tài liệu kiến trúc thăm xa xưa	Nhấn tâm bầu 100% ranh giới mô hình frontend tải qua bộ đôi S3 và CloudFront CDN
Viết cấu ràng kho S3 núp cản sau tấm khiên ALB	Lầm lộn mô hình thiết định origin cổng CloudFront	Behavior mặc định của CloudFront buộc bảo đốc cầu trúng đích vào origin S3 thô trần
Xưng tụng nhầm cụm Redis chính là kho CSDL lưu chat vĩnh cửu	Hiểu sai thăm lật cấu trúc luồng lưu trữ đàm thoại chat	Bàn DynamoDB chịu cốc gánh việc ghi giữ tin nhắn trần; còn cổng Redis sẵn quyền hạn bộ phát sóng pub/sub
Miêu tả cơ quan outbox là phân phối đúng nhĩ chính xác 1 lần (exactly-once)	Chưa am tường tập quy luật phát tin của SQS Standard	Văn khải mô tả rõ tiêu chuẩn gửi ít nhất một lần (at-least-once) công trời cùng thao thao consumer có tính bất biến
Bồi cất kích bật công tắc worker trước lúc dàn hạ tầng AI sẵn sàng	Trụ cốc thoại sinh SageMaker chưa mở thông	Ép giữ worker an bề trong nhân giam khóa tịt (disabled) đến mốc khoảnh khắc cụm AI cùng điểm endpoint thoại thông bãi

Kết luận

Chương trình Kiến trúc này công hiến mẫu mã khuôn bản tham chiếu nòng cốt cho muôn chẳng đường kiểm thử và thau dựng của Workshop kíp tiếp bước. Mọi cương mục sau đó chuyên về ban bố triển khai, nghiệm thu test, rà soát quan trắc, an nộ bảo mật, cước thoại ngân sách, xử trí lỗi lầm cùng thao tác gỡ dẹp tháo dỡ thấy đều bám đuổi tuyệt đối trung thành theo nhĩ đàn sơ đồ tuần tự và hạ tầng đã tạc lên tại đây.

5.3 Điều kiện tiên quyết

Điều kiện tiên quyết

Mục tiêu

Chuẩn bị sẵn sàng các tài khoản, quyền hạn truy cập, công cụ kỹ thuật, đường dẫn mã nguồn, thông số hạn ngạch (quotas) cùng các biên môi trường thiết yếu để tiến hành triển khai và nghiệm thu hệ thống Internship Application Tracker.

Bối cảnh kiến trúc

Dự án này ứng dụng hệ điều hành dây chuyền tích hợp GitHub Actions phối hợp cùng tiêu chuẩn xác thực không dùng khóa AWS OIDC trong quy trình triển khai môi trường production. Tuy vậy, tập công cụ lập trình tại chỗ trên máy tiếp tục giữ tầm quan trọng đặc biệt nhằm phục vụ công tác thanh tra, can thiệp nghiệm thu nóng trong hoàn cảnh nguy trợt và cứu hồi phục hồi dữ liệu thủ công. Nghiêm cấm mọi ý định dùng các khóa AWS Access Key mang chu kỳ tồn sinh vô hạn thi thoảng dính vào dòng chảy CI/CD.



Yêu cầu về quyền truy cập

Khu vực truy cập	Yêu cầu
Tài khoản AWS	Được trao quyền tiếp thu và thao tác hợp pháp vào bên trong tài khoản AWS thuộc dự án; mã số ID tài khoản đã được che khuất
Khu vực AWS (Region)	ap-southeast-1
Kho lưu trữ GitHub	Truy nhập suôn sẻ địa chỉ https://github.com/Temp-orgo/AWS-Internship
Môi trường GitHub Actions	Các biến môi trường cùng thông tin bảo mật cho cấu trúc production environment secrets và variables
EKS	Có đủ uy quyền gọi lệnh cập nhật kubeconfig và tự do tiến vào xem xe rà soát trong không gian namespace internship
CloudFront và S3	Hưởng quyền đọc soát cấu hình distribution, bucket policy, kiểm tra file đối tượng (objects), đồng thời gọi thực thi invalidations
RDS, Redis, DynamoDB, SQS, Lambda, SES, SageMaker	Hưởng trọn quyền đọc soát cho khâu thẩm tra nghiệm thu; role triển khai (deployment role) bắt buộc sắm đủ dải quyền ghi (write) trùng vào mục tiêu cho phép

Các công cụ cần thiết

Công cụ	Phiên bản hoặc cấu hình bắt buộc	Bằng chứng
Git	Số phiên bản được kiểm định theo thời gian thực trên máy chạy thực thi.	Máy tính cá nhân (Local shell)
AWS CLI	Số phiên bản cần được rà soát trực tiếp trên hệ thống sở tại.	Các tệp script triển khai chạy lệnh aws
kubectl	Phiên bản được đối nghiệm trùng máy hiện trường.	GitHub Actions sử dụng gói azure/setup-kubectl@v5
Helm	Số phiên bản tự thẩm rà theo cỡ máy thi công.	Sách chỉ dẫn vận hành tại chỗ sử dụng Helm
Docker	Số phiên bản do hệ điều hành tại chỗ phán bảo.	Hệ thống CI tích cực bọc đóng image Docker
Node.js	Phiên bản 22 thiết định tại .github/workflows/cicd.yml; trên máy cá nhân tự chủ kiểm tra.	Biến môi trường trong dây chuyền NODE_VERSION
npm	Góp mặt đồng loạt chung thuyền cùng trình chạy Node.js trên CI hoặc máy địa phương.	npm ci, npm run build, npm run test
Python	Phiên bản 3.12 bầu chốt trong .github/workflows/cicd.yml; trên máy cá nhân cần kiểm nghiệm thực tế.	Biến môi trường workflow PYTHON_VERSION
pip	Công cụ thiết yếu lo toan cài cất trữ thư viện phụ thuộc cho backend.	backend/requirements.txt
jq	Bộ phân rã chuỗi bị triệu thuyên bởi tập lệnh kịch bản chăm khéo CloudFront.	scripts/aws/ensure-cloudfront.sh
Hugo Extended	Phiên bản 0.134.3 cấm chốt tại luồng GitHub Actions của báo cáo; số tại chỗ phải thẩm tra.	Workflow cho báo cáo (Report workflow)
Terraform	Không mang dấu vết hiện diện ràng buộc thực chất trong bộ bối cảnh kho repository đã thẩm rà.	Hạ tầng được kiến tạo bằng chuỗi tệp YAML kết hợp script

Các tài nguyên AWS buộc phải có sẵn hoặc được khởi tạo



Tài nguyên	Giá trị hoặc trạng thái bắt buộc
Cụm EKS	internship-prod
Namespace Kubernetes	internship
S3 bucket cho frontend	internship-prod-frontend-<AWS_ACCOUNT_ID>
S3 bucket cho tài liệu tải lên / archive	internship-prod-uploads-<AWS_ACCOUNT_ID>
CloudFront distribution	EQIGYNECXDYL8
ALB DNS	k8s-internshippublic-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com
RDS PostgreSQL	internship-prod-postgres
ElastiCache Redis	internship-prod-redis
Các bảng DynamoDB cho chat	ChatUsers, ChatGroups, ChatMessages
Bảng DynamoDB khởi lập cho Lambda	InternshipLambdaEventDedupe
Hàng đợi SQS	internship-prod-outbox
Hàng đợi lỗi SQS DLQ	internship-prod-outbox-dlq
SageMaker endpoint	internship-qwen3-4b
Hàm Lambda (Lambda function)	internship-outbox-handler

Hạn ngạch (Quota) AWS cần kiểm tra

Dịch vụ	Hạn ngạch cần kiểm tra	Lý do quan trọng
EKS	Hạn ngạch về số lượng cluster và managed node groups	Rất tối thượng để phân rã lịch cho các workload cắm neo
EC2	Hạn ngạch về số lõi vCPU, địa chỉ Elastic IP, bộ chuyển tiếp NAT Gateway và số lượng ENI	Máy trạm EKS, các mục tiêu từ ALB cùng rãnh tháo NAT hoàn toàn phụ thuộc vào thông số này
ALB	Hạn ngạch cân bằng tải Load balancer cùng target group	Ingress controller sẽ thi gõ sinh tạo tài nguyên cân bằng tải này
RDS	Số con máy DB instances và trữ lượng ổ lưu chữ	Thi đắc lực cho cụm vận hành PostgreSQL
ElastiCache	Hạn ngạch về số lượng Node	Gánh cụm Redis replication group
DynamoDB	Số lượng bản cấu hình Table cùng định dạng trần thông lượng của tài khoản	Nuôi sống bộ đôi bản chứa thư tín Chat và bảng cọc khởi lập cho Lambda
SQS	Quy mô số hàng đợi Queue cùng giới hạn tốc độ lưu lột tin nhắn	Nối đường cho quy trình tung phát ra thông báo outbox
Lambda	Khả năng thực thi song song (Concurrency)	Dành cho cổ máy người tiêu dùng (consumer) đón nhậ sự kiện SQS
SES	Tình trạng bị giam hãm ngục sandbox hoặc ranh giới trần thư từ cho phép ở production	Rất cốt lõi để test luồng phát thử nghiệp thư điện tử tin báo khói
SageMaker	Giới hạn về số máy endpoint instance, đặc sắc ưu việt là hạn ngạch GPU nếu dùng	Phán xử an ninh sức khỏe duy trì online cùng quy mô phục hồi cho AI
CloudFront	Giới hạn về lượng distribution cùng định ngạch số lần gọi lệnh invalidation	Cơ quan tối cao cho luồng tải nộ tếp và nâng cấp giao tiếp frontend

Biến và bí mật (Secrets) của GitHub Actions

Quy trình tự động (workflow) hấp thụ thông số cấu hình thông qua các biến môi trường và bí mật (secrets) được gìn giữ trong hệ thống GitHub Environment. Danh sách bên dưới chỉ minh họa tên trường dữ liệu và tuyệt đối không bao giờ được phép phơi lộ giá trị nhạy cảm thật sự trên văn bản công cộng.

Các biến bắt buộc hoặc thường dùng



Biến	Mục đích sử dụng
AWS_REGION	Khu vực đích, mặc định <code>ap-southeast-1</code>
AWS_ACCOUNT_ID	Số định danh tài khoản AWS đích, trích mang sang từ tài khoản thuộc về dự án
EKS_CLUSTER_NAME	Tên cụm EKS, mặc định <code>internship-prod</code>
K8S_NAMESPACE	Không gian tên Kubernetes namespace, mặc định <code>internship</code>
EKS_DEPLOY_ENABLED	Bắt buộc định giá trị <code>true</code> khi có nhu cầu triển khai ứng dụng vào cụm
ENABLE_ALB_INGRESS	Cọc giữ điều phối việc có ban bố bật cài cấu hình cho ALB Ingress public hay chẳng
FRONTEND_DEPLOY_ENABLED	Bắt buộc gán bằng <code>true</code> mỗi lúc nạp bồi tệp lên cho frontend
CREATE_CLOUDFRONT	Trao quyền nặn đúc CloudFront distribution mới nếu số định danh cũ khuyết trống
FRONTEND_BUCKET	Danh xưng hòm lưu S3 của frontend
CLOUDFRONT_DISTRIBUTION_ID	Mã định danh duy nhất (ID) của CloudFront distribution
ALB_DNS	Chuỗi danh xưng miền định hướng của ALB origin dùng lúc kết dính ngược vào trong
ECR_REPOSITORY_BACKEND	Tên kho lưu trữ ảnh cho backend ECR, mặc định <code>internship-backend</code>
ECR_REPOSITORY_CHAT	Tên kho lưu trữ ảnh cho dịch vụ chat ECR, mặc định <code>internship-chat</code>
ECR_REPOSITORY_AI	Tên kho lưu trữ ảnh cho dịch vụ AI trên ECR, mặc định <code>internship-ai</code>
AI_DEPLOY_ENABLED	Công tắc cho phép triển khai pod trung gian adapter cho AI
PROCESSING_WORKER_ENABLED	Quyết đoán lệnh dân bề tăng số lượng replica cho cụm worker
SAGEMAKER_ENDPOINT_NAME	Tên máy chủ suy đoán SageMaker endpoint, mặc định <code>internship-qwen3-4b</code>
IRSA_ROLE_ARN	Định vị chú thích IAM Role dành cho pod bên trong runtime

Các bí mật (secrets) bắt buộc

Bí mật	Mục đích sử dụng
AWS_ROLE_TO_ASSUME	Vai trò IAM Role an ninh để GitHub OIDC nhận diện hoán vị quyền
SECRET_KEY	Khóa bảo mật cốt lõi dành riêng cho quá trình sinh khóa JWT ở lớp backend
DATABASE_URL	Chuỗi kết nối trực diện với máy chủ cơ sở dữ liệu PostgreSQL
REDIS_URL	Chuỗi kết nối trực diện tới cụm nhớ bộ đệm Redis
OUTBOX_QUEUE_URL	Chuỗi định hướng dẫn nhập trúng vào hàng đợi chủ lực của SQS
AI_SERVICE_API_KEY	Khóa xác nhận chéo (nếu bật) cho lời gõ cửa từ worker đi sang dịch vụ AI

Lệnh kiểm tra và xác minh

Thực hiện nhập chạy loạt câu lệnh dưới đây (buộc phải bầu kèm uy quyền định danh tài khoản hợp pháp):

```
aws sts get-caller-identity
aws configure get region
aws eks update-kubeconfig --name internship-prod --region ap-southeast-1
kubectl get namespace internship
kubectl get nodes -o wide
kubectl get deployments -n internship
```

Kiểm tra định danh và các con số phiên bản công cụ trên máy:

```
git --version
aws --version
kubectl version --client
helm version
docker version
node --version
npm --version
python --version
jq --version
hugo version
```

Kết quả mong đợi

- Số ID của tài khoản AWS thuộc về tài khoản chuyên quản của dự án và nhất thiết phải được bôi nhọ che giấu kỹ từ mọi bức ảnh hay tệp log ném công khai.
- Khu vực thao tác được khai báo nằm chuẩn xác tại vùng ap-southeast-1.
- Trình lệnh kubectl thành công thám thính và xem xe được trạng thái không gian namespace internship.
- Luồng thi công từ GitHub Actions mùa kiểm an toàn hoá vị thợ nhập vai trò deploy thông qua bộ giáp OIDC.
- Mọi mẩu thông tin bí mật cốt tử đều nằm cẩn thận im lặng sâu thẳm ở hòm giấu secret của GitHub environment, cấm tuyệt lộ diện ra tập code ban đầu.

Các lỗi thường gặp

Triệu chứng	Nguyên nhân	Hướng khắc phục
Lỗi từ chối quyền AccessDenied trong tiến trình triển khai	OIDC role thiếu bầu mắt thông số phân chia quyền lực thi thoảng ở các API rạch ròi	Bôi nạp thêm chi tiết cụ thể hành vi tác động vào IAM Policy của Role, gìn vững ranh giới đặc quyền tối thiểu
kubectl móc trật sang cụm máy lạ hoặc không như ý	Cọc kube context được chọn hiện thời chẳng nằm đúng mạn cụm production	Kêu gọi thi lệnh kubectl config current-context rồi tái lập kubeconfig
Công tác triển khai frontend bị bỏ khinh không trôi qua	Khóa FRONTEND_DEPLOY_ENABLED bị cài sai bằng false, hoặc bị vô tâm gán theo kiểu biến cục bộ không cho phép bộ lệnh rào if gặm nhấm	Chỉnh sửa gài chuẩn thông số vào trong hệ cấu hình biến repository hoặc môi trường GitHub đúng tầm bắt của workflow
Khâu dẫn tải worker lằm lổ trượt dốc	Bật thông số PROCESSING_WORKER_ENABLED=true giữa lúc AI service vẫn mờ xịt nằm lê chờ chưa trỗi sấm xong sức khỏe	Chỉ ra quyết lệnh kích hoạt worker sau mốc cọc dịch vụ AI cùng máy chủ SageMaker endpoint thi thổ rục rỡ trọn trạng thái sấm sửa xong
Thông báo jq is required trỗi khóc	Công cụ jq bóc sa biến vắng trong cỗ máy phát triển hiện trắng	Cài đặt sung túc trình biên giải jq trước phút giục gọi chạy kịch bản CloudFront ngay ở máy địa phương

Kết luận

Toàn cảnh bộ môi trường hạ tầng nay chính thức thỏa mãn 100% điều kiện chín mùa để tiến thẳng sang chặng sửa sấm chuẩn bị mã nguồn cùng thi thao triển khai, ngay khi mọi đầu cọc quyền bính tài khoản, kho tệp công cụ, cấu trúc biến an ninh OIDC, tập mật tín secrets, giới rào hạn ngạch quotas và dải vạch kết nối Kubernetes đều kiên ky bằng độ thông suôn thành toàn bước thẩm định.

5.4 Chuẩn bị mã nguồn

Chuẩn bị mã nguồn

Mục tiêu

Chuẩn bị kho lưu trữ mã nguồn (repository) của ứng dụng ở trạng thái sẵn sàng để phục vụ công tác rà soát kiểm định và thực hiện quy trình triển khai lặp lại suôn sẻ mà không vô ý sơ sẩy để lọt lộ các thông số bí mật hay thi ấu ảnh hưởng lằm sang tài nguyên sản xuất production.



Bối cảnh kiến trúc

Kho chứa repository gánh trọn vảy kiến trúc cho một mô hình ứng dụng đa dịch vụ (multi-service application). Hệ thống liên lạc CI/CD sẽ tự chủ chia nhỏ và gõ lệnh tiến hành cất dựng riêng biệt từng tệp image cho backend, dịch vụ chat, và (khẩu nối nếu bật) dịch vụ AI, tiến ra tung tắn ban bố các bản mô tả kịch bản (Kubernetes manifests), đồng thời cho thi hành build tính phân chia phần việc cho frontend ra một nhánh ranh giới độc lập chuyển tắn cho kho bộ đôi S3 kết hợp CloudFront.

Cấu trúc repository

Đường dẫn (Path)	Mục đích
backend/	Nơi ngụ của dịch vụ FastAPI, cấu hình dữ liệu SQLAlchemy models, tệp di cư cơ sở dữ liệu Alembic migrations, mã lệnh cho cụm worker cùng tập test nghiệm thu
frontend/	Không gian trang ứng dụng React/Vite, các bộ kết nối API client, khối thành phần route con cùng dải code test của frontend
chat-service/	Dịch vụ đàm thoại vận hành qua Node.js, Express, Socket.IO, các lớp tiếp giao cơ sở dữ liệu DynamoDB repositories cùng tệp trung gian Redis adapter
ai_service/	Cụm dịch vụ đệm adapter chạy trên FastAPI chuyên phụ trách gọi qua SageMaker cùng luồng mã đọc hiểu và chấm điểm AI
k8s/app/	Hệ bản vẽ manifest gốc tạo lập Kubernetes namespace, các thông số service, deployments, jobs, dẫn dẹo HPA và ngân sách bồi thặng PDB
k8s/eks/	Hệ manifest đặc thù chuyên gài cất cho môi trường EKS bao gồm ConfigMap, ServiceAccount, ALB Ingress cùng minh họa mẫu cho secret
k8s/observability/	Hồ sơ khai báo quy cho luồng giám sát qua hai tệp kịch bản ServiceMonitor và PrometheusRule
scripts/k8s/	Tập kịch bản tự động hỗ trợ cho thao tác triển khai cụm tại chỗ hoặc bữa giăng ra EKS
scripts/ci/	Thư viện tệp phụ giúp dây chuyền tác nghiệp trên GitHub Actions
scripts/aws/	Dải script thao tác trợ lực tài nguyên CloudFront, ALB, hàng đợi SQS, quản trị cuộn bản phát hành (rollout) cùng chăm node group
.github/workflows/cicd.yml	Lĩnh hồn định danh điều hành trọn dây chuyền thanh tra rà soát và phát triển tiến thặng cho môi trường production
docs/architecture/	Thư mục tài liệu quyết định kiến trúc (ADRs), luật kiểm soát cạnh tranh song song cùng cam đoan hợp đồng thi hành người tiêu dùng sự kiện
Dockerfile.ai-service	Bản phác vẽ đóng gói container image cho dịch vụ trung gian AI adapter
docker-compose.yml	Cột mốc khởi tung lập trình bữa quanh các tài nguyên phụ thuộc khi chạy thử tại chỗ

Nhận mã nguồn dự án được cấp quyền

Thao tác thực hiện truy nạp vào kho repository chỉ được phép diễn ra trên cơ quan tài khoản đã sấm sửa ủy quyền hợp pháp:

```
git clone https://github.com/Temp-orgo/AWS-Internship.git
cd AWS-Internship
git status --short --branch
```

Tôi nghiêm cấm việc rò rỉ bữa lan mọi token bảo mật, chứng chỉ đăng nhập nhân chứng hoặc những địa chỉ URL từ xa mang gài theo khóa xác thực bên trong hồ sơ cẩm nang báo cáo này.

Kiểm tra nhánh và trạng thái workflow


```
git remote -v
git status --short --branch
git log --oneline -5
```

Kết quả mong đợi:

- Cọc địa chỉ mạng remote kết nối chuẩn xác về kho mã nguồn có bản quyền đích xác thuộc dự án.
- Tình trạng cây mã nguồn làm việc (working tree) được thấu đáo 100% ranh giới ngay trước chuỗi bùng ra lệnh triển khai.
- Cân nhắc cẩn thận giữ vẹn toàn, ngăn cản trót lọt việc sửa đề nhằm lên các thay đổi nội bộ địa phương không mang dính líu đến dự án.

Xác định các thư viện phụ thuộc của dịch vụ

Backend

```
cd backend
python -m pip install -r requirements.txt
python -m ruff format --check .
python -m ruff check .
python -m mypy app tests
python -m pytest -m "not postgres"
```

Dịch vụ backend tích hợp ranh giới chạy nền Python 3.12 trong dây chuyền CI, vận dụng bộ khung FastAPI, thư viện SQLAlchemy, trình di cư Alembic, bộ kiểm tra thuộc tính Pydantic settings, kho móc AWS boto3, cổ quan trắc Prometheus client cùng thư viện truy vết OpenTelemetry.

Dịch vụ chat

```
cd chat-service
npm ci
npm run check
npm run lint
npm run format:check
npm run test:ci
```

Dịch vụ chat thiết dựng xoay xung quanh Node.js, framework Express, giao thức thời gian thực Socket.IO, kho lưu DynamoDB, cầu nối Redis adapter, cùng nhịp tim số liệu giám sát OpenTelemetry và Prometheus metrics.

Frontend

```
cd frontend
npm ci
npm run test
npm run build
```

Frontend được dệt nên từ quy chuẩn React 18, trình tăng tốc Vite, thư viện CSS Tailwind CSS, bộ tiếp thu liên lạc Socket.IO client, công cụ gọi mạng Axios cùng tầng tầng các trạm giao tiếp API client cho từng tuyến đường riêng.

Dịch vụ AI

```
docker build -f Dockerfile.ai-service -t internship-ai:ci .
```

Bộ trung gian AI adapter tiến hành phơi mở ra bên ngoài các con đường dẫn /health/ready, /parse-job, /parse-cv, /match-applications, /rerank, /cv-job/group-score, kết hợp cùng một chuỗi các luồng chấm điểm tương đương. Tại môi trường thực tiễn production, nó sẽ gọi sang bộ máy SageMaker Runtime thực thụ thay vì nặng nề lôi vác nạp trọn cỗ máy model khổng lồ về ngự ở trong các worker.

Mẫu tập tin cấu hình môi trường

Tận dụng những tệp thông tin ví dụ có sẵn để coi như là nguồn tra khảo tham chiếu chuẩn mực:

Tập tin	Mục đích
k8s/app/secret.local.example.yaml	Tệp ví dụ minh họa thông số secret cho cụm Kubernetes tại chỗ
k8s/app/secret.local.yaml.example	Mẫu bản vẽ secret tại chỗ cho nhà phát triển
k8s/eks/secret.example.yaml	Hình hài cấu trúc cho các khóa secret bên trong EKS mà không để lộ nội dung giá trị thật
backend/requirements.txt	Thư viện mã nguồn phụ thuộc phục vụ runtime backend
frontend/package.json	Hệ script tự động hóa cùng thư viện cho frontend
chat-service/package.json	Cụm script vận hành và thư viện đi kèm của dịch vụ chat

Tuyệt đối cấm đưa lên (commit) các tệp biến thực thể `.env`, `.env.production`, file cấu hình kubeconfig, chuỗi url kết nối database, khóa mã bảo an JWT secret, khóa truy nhập AWS, mã thông báo token GitHub, private key riêng hay các mật mã dịch vụ phát mail SES.

Lệnh rà soát kiểm định tại chỗ

Luồng dây chuyền tự động CI trên mạng có thói quen cho thi triển các chốt trạm tra rà quan trọng như sau:

```
bash -n $(git ls-files '*.sh')
python scripts/ci/infrastructure.py
python -m ruff format --check backend
python -m ruff check backend
python -m mypy backend/app backend/tests
python -m pytest -m "not postgres" backend
npm run test --prefix frontend
npm run build --prefix frontend
npm run test:ci --prefix chat-service
docker build -t internship-backend:ci ./backend
docker build -t internship-chat:ci ./chat-service
docker build -f Dockerfile.ai-service -t internship-ai:ci .
git diff --check
```

Trong những kỳ máy trạm lập trình địa phương của bạn thiếu khuyết trình quản trị Docker, hãy thật thà ghi nhận việc thử nghiệm build đóng ảnh Docker là chưa chạy rà kiểm định thay vì ngoan cố tự tung tự tác đánh dấu là đã hoàn thành công xuất sắc.

Chuẩn bị cho triển khai

Trước thời khắc khởi búng kích hoạt dây chuyền sản xuất production:

1. Thăm tra an tâm các biến môi trường và bí mật (secrets) production của GitHub đã thọ vị nằm ngấn gửi đầy đủ theo yêu cầu.
2. Xác minh vai trò IAM Role của OIDC là `internship-github-deploy` sẵn lòng cho khoắc nhả hoán quyền thâu vãng bên trong đúng số tài khoản AWS dự án.
3. Kịp thời đảm bảo các kho lưu container ECR hiện hữu sẵn, giảng trọn chỗ nạp cho backend, chat và cho AI (trong tình huống công tác triển khai AI service thêu bật).
4. Xác minh tinh thần trối mứa trọn vẹn của một hệ sinh thái gồm RDS, Redis, DynamoDB, SQS, S3, CloudFront, ALB, Lambda, SES cùng SageMaker.
5. Nghiêm rào kiên định giam ngục tham số `PROCESSING_WORKER_ENABLED` ngả ở mức false cho tận mốc ngày cổ máy suy luận AI bám kèm rúng còi thông quan.
6. Chọn bầu nấc `deploy-app` để thao diễn đưa mã app lót vào EKS và ấn trượt chế độ `deploy-frontend` cho công tác thiêu tung giao diện lên cao.

Các chế độ của quy trình GitHub Actions

Cấu hình bộ thi kịch bản (workflow) được khảo soát ra lệnh hậu thuẫn cho chuỗi các cơ cấu tác nghiệp linh động như:



Chế độ	Mục đích
validate	Thực thi kiểm định an nộ không để xô bồ trát đội sang cụm production
deploy	Con đường đưa bản thử ra thực tiễn theo lối mòn truyền thống nay vẫn được workflow khoan nhượng chấp thuận
rollout	Gõ chuông tái khởi động (restart) trọn bộ các workload trên EKS mà chẳng nhọc sức re-build các ảnh
restore-compute	Cám ơn gọi đánh đòn chấn hưng dân sấm lại trọn dung nắc năng lực cụm máy managed node group
deploy-app	Dựng nạp ảnh mới đẩy sang kho ECR đồng thời khai phá tiến vào EKS thuyền chuyển các workload
deploy-public	Bung trượt bản ghi manifest cắm neo ALB Ingress ra thế giới bên ngoài
deploy-frontend	Build kho mã frontend, đồng bộ thư viện lên kho S3, kêu gọi lệnh xóa bộ đệm (invalidates cache) trên CloudFront
full	Tự động thi khát khao nối liền chu trình sản xuất khép kín một hơi rảo rã trọn bộ từ A sang Z

Kết quả mong đợi

Kho tàng mã nguồn tựu chung đạt tiêu chí chốt duyệt thành quả khi mà:

- chuỗi tài liệu thư viện bầu vẫy các dịch vụ thấy đều hoan hỷ tải nạp mượt trôi
- trọn các bài kiểm định của backend, frontend cùng dịch vụ chat rền vang đắc thắng băng độ qua khỏi không gian test quy đúc
- các container image Docker sẵn lòng chịu gõ trúng cấu hình bật xếp thành quả
- những trang manifest mô tả cho Kubernetes xuất xưởng không trầy móng xô xát lọt rào châm trích từ cổ máy kiểm toán script repository
- thông số biến môi trường cùng chuỗi khóa mật secret bên phía GitHub ngạo nghễ điểm tên thọ sinh đầy đủ
- luồng ra mắt hệ thống được rành rẽ phân thân rõ mạn: workload nghiệp vụ thẳng đường thâm ngục EKS, còn giao diện web lả buông phi thẳng ngụ kho S3 bầu trối CloudFront

Các lỗi thường gặp

Triệu chứng	Nguyên nhân	Hướng khắc phục
Lên tiếng ca than Cannot find build-and-push-image.sh	Chẳng thể dò tìm ra phương mạn cho tập script trợ giúp	Cho lệnh trở múa bám chuẩn đường dẫn scripts/build-and-push-image.sh hoặc ra quyết gán cấu hình biến BUILD_IMAGE_SCRIPT
Hông gục bước tải file tiêu đề Node (Node header) trong tiến trình CI cho dịch vụ chat	Khâu dịch mã tệp phụ thuộc bản địa (native dependency) rền nài thêm muốn chuỗi header cài ngay sở tại	Cần mẫn nạp dải cấu hình biến npm_config_nodedir nhằm trúng vào mạn thư viện headers mà hệ thống ci toolcache lưu cần kiệt
Trang frontend build ra bốc sa mù chạy sai về mạn localhost	Mang tiếng thất đức bỏ quên tham số biến cố định VITE_API_BASE_URL=/api cho luồng production	Nhờn kính nạp trang trọng các thông số biến Vite ở ngay thời khắc nháy còi chu trình build tỉnh phục vụ production
Đóng cửa cự tuyệt lệnh gõ docker build	Tiến trình điều hành Docker daemon thêm đồ ồm chét im lì giữa máy	Kích ngục hăm ầm gọi còi giục Docker daemon đứng dậy múa kiểm, hoặc trang nghiêm chốt từ ngữ khai báo khâu test build chưa qua thực thi
Các bí mật (secrets) kinh hải hiện diện vọt nhảy giữa trang báo rách git diff	Sơ sót ngấp bùn bỏ vẫy đưa file .env hoặc tệp bí mật sinh ra tạm bỏ trượt vọt lên mạn staged	Tháo dọn và đá phăng các tệp bụi bám khóa bảo an xa khỏi mác commit ra lệnh tiến còi tráo xoay mới các thông số khóa đã phơi bãi ra xa

Kết luận



Thư mục chứa đựng mã nguồn ứng dụng qua chặng kiểm nghiệm gay gắt nay chính thức cán đích sẵn sàng để ra lệnh xuất phát triển khai theo khuôn nấc quy an nộ bảo mật cao cấp nhất. Mạch phân phối giao diện web buông rã thành chặng bay tiến tĩnh riêng cho S3 và CloudFront, trong khi khối workload EKS an nhiên duy trì lộ trình của mình, đồng loạt vây hãm bảo an cần khôn nhờ sức khỏe dây chuyền quản trị thông lượng xác thực GitHub Actions phối hợp cùng tấm khiên OIDC.

5.5 Hạ tầng AWS

Hạ tầng AWS

Mục tiêu

Tài liệu hóa chi tiết toàn bộ hạ tầng AWS phục vụ cho công tác triển khai production thực tế và cung cấp đầy đủ các chuỗi lệnh kiểm chứng an toàn ứng với từng cụm tài nguyên cốt lõi trong hệ thống.

Phạm vi

Chương tài liệu này tập trung ghi nhận trọn vẹn những tài nguyên đã thực thụ triển khai có kèm thư viện bằng chứng bảo an, điểm tên thẳng thắn những thông số trường mục còn ngụy đội lệnh chiết xuất rà tra cầu bằng AWS CLI, và ra riêng danh mục đề xuất gia cố an ninh tự nguyện cho nhà phát triển bám đuổi về sau.

Bối cảnh kiến trúc

Hạ tầng nền tảng tự do bầu vịu cho một kiến trúc kết hợp vi diệu giữa hai chiến tuyến Cloud CDN ngoại vi và cỗ máy container tập trung:

- Cầu nối CloudFront và kho S3 chuyên trách phân phối trang giao diện frontend.
- Cánh cổng ALB luân chuyển gạn chia tải truy nhập động lao thẳng vào bên trong EKS.
- Cụm máy chủ EKS vững chí gìn bảm, cưu mang toàn cõi cho chuỗi workload phần mềm ngụy trực trấn dài hạn.
- Cơ quan cơ sở dữ liệu được quản lý (managed databases), kênh truyền tin, kho object storage, dịch vụ Lambda, máy gửi mail SES cùng cỗ công suy luận SageMaker chêm trệ dựng cao tường mang lại dịch vụ nền tảng trắng kiết cho hệ thống.
- Dây chuyền tự động trên GitHub Actions ung dung múa trượt triển khai qua ranh giới OIDC mà vội buông rơi hoàn toàn việc cầu cạnh các khóa AWS access keys vô hạn ngày lì lợm.

Các tài nguyên đã được triển khai

Khu vực	Tài nguyên	Trạng thái
Tài khoản và khu vực	Tài khoản AWS của dự án, vùng ap-southeast-1	Đã che giấu (redacted) số ID tài khoản
EKS	internship-prod	Đã xác minh qua hồ sơ bằng chứng Tuần 8 và kịch bản workflow
Namespace	internship	Đã xác minh qua tài liệu kịch bản manifests
ALB	k8s-internshippublic-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com	Đã xác minh qua hồ sơ bằng chứng Tuần 8
S3 bucket cho frontend	internship-prod-frontend-<AWS_ACCOUNT_ID>	Đã ẩn phần đuôi tài khoản
S3 bucket lưu tệp / archive	internship-prod-uploads-<AWS_ACCOUNT_ID>	Đã ẩn phần đuôi tài khoản
CloudFront	Distribution EQIGYNECXDYLS	Đã xác minh qua hồ sơ bằng chứng Tuần 8
RDS	internship-prod-postgres	Đã xác minh qua hồ sơ bằng chứng Tuần 8
Redis	internship-prod-redis, trạng thái available	Đã xác minh qua hồ sơ bằng chứng Tuần 8
DynamoDB	ChatUsers, ChatGroups, ChatMessages, InternshipLambdaEventDedupe	Đã xác minh qua hồ sơ bằng chứng Tuần 8

Khu vực	Tài nguyên	Trạng thái
SQS	internship-prod-outbox, internship-prod-outbox-dlq	Đã xác minh qua hồ sơ bằng chứng Tuần 8
SageMaker	Endpoint internship-qwen3-4b	Đã xác minh qua hồ sơ bằng chứng Tuần 8
Lambda	internship-outbox-handler	Đã xác minh qua hồ sơ bằng chứng Tuần 8
Các IAM roles	internship-github-deploy, internship-eks-runtime, internship-sagemaker-execution, internship-eks-node-role, internship-lambda-outbox-role, AmazonEKSLoadBalancerControllerRole	Đã xác minh từ hệ thống cấu hình triển khai và tệp chứng cứ Tuần 8

Nền tảng mạng

Cụm máy EKS trong bối cảnh sản xuất production nhất định khắt khỏm đòi hỏi cấu trúc mạng VPC có chia đôi phân cách giữa các subnet public và private:

- Các subnet public thọ nuôi cụm máy cân bằng tải hứng lưu lượng từ thế giới public bên ngoài và kho vẫy các máy NAT Gateway.
- Các subnet private bao vây an ninh êm bể nuôi dưỡng cụm máy cọc worker nodes của EKS cùng đường rẽ vào bên trong của mạng dịch vụ AWS được quản lý.
- Một cổng Internet Gateway có chức năng khai quan cho lưu lượng public trượt thiên vào và mở đường cho lưu lượng từ NAT ung dung phi xuất ngoại.
- Một trạm chuyển giao NAT Gateway ra mặt bao sấm cho đàn máy nằm an tịnh trong lòng subnet private đặc quyền tiếp cận ra vạch ngoài Internet để bồi nạp lấy container image, trao đổi chuỗi gọi tới AWS APIs, tải về tệp gói phụ trợ, đồng thời giao đãi mạch nảy theo cụm điều phối control-plane.

Cần bằng chứng:

- Thông số định danh VPC ID, tuy rằng biến cố lỗi xử lý trong sự cố khắc phục đã kiên thấu xác tín số vpc-0cfee519122ae18b4 từng bị sử dụng để cầu khẩn rước AWS Load Balancer Controller mùa việc.
- Các mã số nhận diện Public subnet IDs.
- Các mã số nhận diện Private subnet IDs.
- Danh bạ số liệu bảng định tuyến Route table IDs.
- Nhóm bảo mật Security group IDs và điều luật tường lửa vẫy kèm.
- Số hiệu của NAT Gateway ID kết hợp mã cấp tài khoản địa chỉ Elastic IP allocation ID.

Kiểm chứng:

```
aws ec2 describe-vpcs --region ap-southeast-1
aws ec2 describe-subnets --region ap-southeast-1
aws ec2 describe-route-tables --region ap-southeast-1
aws ec2 describe-nat-gateways --region ap-southeast-1
aws ec2 describe-security-groups --region ap-southeast-1
```

IAM và OIDC

Hệ luồng trên GitHub Actions ứng dụng an ninh cơ quan xác minh AWS OIDC. Kịch bản workflow phán bảo ràng buộc các uy quyền:

```
permissions:
  id-token: write
  contents: read
```

Các tác vụ phục vụ triển khai liên tục gõ gọi gói công cụ aws-actions/configure-aws-credentials@v6 kèm chuỗi thông số role-to-assume: \${{ secrets.AWS_ROLE_TO_ASSUME }} và chốt trần bằng cọc allowed-account-ids: \${{ env.AWS_ACCOUNT_ID }}.

Các workload chạy bên trong môi trường runtime thụ hưởng vượng bổng thông qua ServiceAccount của Kubernetes mang tên `internship-app`. Whenever tham số biến `IRSA_ROLE_ARN` được truyền sang đầy đủ, kịch bản `scripts/k8s/deploy-eks.sh` ngay lập tức thi triển khâu render nội dung tệp `k8s/eks/serviceaccount.yaml` đồng thời áp luôn chuỗi chú thích (annotation) cho role vào manifest.

Kiểm chứng:

```
aws iam get-role --role-name internship-github-deploy
aws iam get-role --role-name internship-eks-runtime
aws iam get-role --role-name internship-lambda-outbox-role
kubectl get serviceaccount internship-app -n internship -o yaml
```

ECR

Luồng quy trình tự động tiến hành build ra các image cắm mã nhãn băm trùng chuỗi SHA và khất khe thanh rà kiểm định hai image của backend và chat trước thời điểm ra phát cỡ cho mác triển khai:

Image	Kho ECR mặc định
Backend	internship-backend
Dịch vụ chat	internship-chat
Bộ đệm AI adapter	internship-ai

Kiểm chứng:

```
aws ecr describe-repositories --region ap-southeast-1
aws ecr describe-images --region ap-southeast-1 --repository-name internship-backend
→ --image-ids imageTag=<GITHUB_SHA>
aws ecr describe-images --region ap-southeast-1 --repository-name internship-chat --image-ids
→ imageTag=<GITHUB_SHA>
```

Cụm EKS và các workload

Cụm EKS chuyên gánh tải và vận hành:

Workload	Loại	Ghi chú
backend	Deployment và Service	Dịch vụ FastAPI mở thông tại port 8000, 2 replica
chat-service	Deployment và Service	Dịch vụ Node.js/Socket.IO chạy bám tại port 3000, 2 replica
backend-outbox-dispatcher	Deployment và Service	Dành sẵn port cho metrics là 9101, 1 replica
backend-processing-worker	Deployment và Service	Mở thông port giám sát metrics tại 9102, được trao tự chủ đóng xuống mức 0 replica chừng nào AI chưa kháng an
ai-service	Deployment và Service	Pod trung gian FastAPI adapter mở nghênh tại port 8010, 1 replica
backend-migrate	Job	Nhiệm vụ có trọng trách khởi thực thi câu lệnh <code>alembic upgrade head</code>
chat-init	Job	Tác vụ ban khởi tạo bảng cho chuỗi CSDL chat bên trong DynamoDB

Kiểm chứng:

```
aws eks describe-cluster --name internship-prod --region ap-southeast-1
aws eks list-nodegroups --cluster-name internship-prod --region ap-southeast-1
kubectl get deployments,pods,svc,hpa,pdb,jobs -n internship -o wide
```

AWS Load Balancer Controller và ALB

Tài nguyên Ingress chủ lực trong môi trường production mang tên định danh `internship-public` và kết nối bầu giữ các chuỗi chú thích thuộc về AWS Load Balancer Controller:

- mô hình (scheme): internet-facing
- kiểu mục tiêu (target type): ip
- listener: HTTP 80
- đường dẫn kiểm thử sức khỏe (health check path): /health/ready
- thời gian chờ rỗi (idle timeout): 3600
- cơ chế bám dính của target group (target group stickiness): cho phép bật trong 86400 giây
- các dải phân tuyến (paths): /api, /chat, /socket.io

Kiểm chứng:

```
kubectl get ingress internship-public -n internship -o yaml
kubectl get deployment,pods -n kube-system -l
→ app.kubernetes.io/name=aws-load-balancer-controller -o wide
aws elbv2 describe-load-balancers --region ap-southeast-1
aws elbv2 describe-target-groups --region ap-southeast-1
```

S3 và CloudFront

Luồng phân phối cho frontend tận dụng triệt để:

- kho lưu trữ đóng kín riêng tư internship-prod-frontend-<AWS_ACCOUNT_ID>
- CDN CloudFront distribution EQIGYNECXDYLS
- cơ quan truy nhập gốc an toàn CloudFront Origin Access Control
- cấu hình behavior mặc định thả trôi về phía S3
- các behavior dành riêng cho dải /api/*, /chat/*, /socket.io/* được bẻ cua lao thẳng về cổng ALB
- cơ chế bồi đường dự phòng SPA fallback dành cho hai mã lỗi 403 cùng 404 cho rẽ bẻ trúng tệp /index.html

Kiểm chứng:

```
aws s3api get-public-access-block --bucket internship-prod-frontend-<AWS_ACCOUNT_ID>
aws cloudfront get-distribution --id EQIGYNECXDYLS
aws cloudfront list-invalidations --distribution-id EQIGYNECXDYLS
```

Các tài nguyên liên lạc và hướng sự kiện

Những cấu trúc đã trở sinh hoàn tất trong môi trường chạy hiện tại:

Tài nguyên	Cấu hình
Hàng đợi chủ lực SQS	internship-prod-outbox, định mức thời gian ẩn (visibility timeout) là 120 giây, giới hạn lưu tệp 4 ngày, bật mã hóa SSE
Hàng đợi lưu lỗi SQS DLQ	internship-prod-outbox-dlq, cần nhấn giữ tệp suốt 14 ngày, số kỳ thu thử lại maxReceiveCount là 5
Hàm Lambda	internship-outbox-handler, cấu hình kích nổ từ trigger hàng đợi SQS
Bảng khử lặp sự kiện (Dedupe table)	InternshipLambdaEventDedupe
Bãi chôn giữ văn (Archive location)	s3://internship-prod-uploads-<AWS_ACCOUNT_ID>/outbox-archive/...
Dịch vụ gửi thư điện tử	Amazon SES

Kiểm chứng:

```
aws sqs get-queue-url --queue-name internship-prod-outbox --region ap-southeast-1
aws sqs get-queue-attributes --queue-url <OUTBOX_QUEUE_URL> --attribute-names All --region
→ ap-southeast-1
aws lambda get-function --function-name internship-outbox-handler --region ap-southeast-1
aws lambda list-event-source-mappings --function-name internship-outbox-handler --region
→ ap-southeast-1
aws dynamodb describe-table --table-name InternshipLambdaEventDedupe --region ap-southeast-1
```



Tôi đã chủ động ghi nhận ra một trường hợp vênh đậm đùng độ chéo trong đặt tên: bộ đôi tệp script kịch bản kho lưu repository scripts/aws/provision-outbox-sqs.sh cùng file .ps1 đang an ủi thi theo giá trị mặc định internship-outbox-events với chu kỳ visibility timeout ngã ngấn ở mốc 60 giây, giữa lúc con hàng đợi vận hành rục rở tại production thật sự ngoài thực tế lại oai dũng mang xưng danh internship-prod-outbox và giữ trọn mốc thời gian visibility timeout dài tận 120 giây. Tôi tự trọng duy trì tín niệm bầu vào thư viện minh chứng hạ tầng production hiện trường như mạn nguồn chân lý cho cội thực thi, và dĩ nhiên xin cần kính dẫn công việc canh chỉnh lại chuỗi script bị sa nhảm trong repo mã gốc coi như một nghiệp vụ độc lập không nằm trong phạm vi chỉnh lý của báo cáo.

Các tài nguyên SageMaker

Cấu trúc nghiệp vụ quy định cụm worker sẽ thi phác gồ qua cổng dịch vụ bộ đệm trung gian ai-service, tới lượng mình pod ai-service mới giục còi gồ thẳng vào còi SageMaker endpoint internship-qwen3-4b. Trình kịch bản triển khai đòi hỏi cọc cấm SageMaker endpoint phải hoành múa uy phong ngã đúng trại thái InService trước phút nổi cơn biến tự động bật công tắc cho phép PROCESSING_WORKER_ENABLED=true thực thi.

Kiểm chứng:

```
aws sagemaker describe-endpoint --endpoint-name internship-qwen3-4b --region ap-southeast-1  
kubectl get deployment,svc ai-service -n internship
```

Đề xuất củng cố bảo mật (Hardening)

Khu vực	Đề xuất gia cố
ALB	Khóa gạt chặn thi rào cấm không cho truy nhập trần lộ trực diện vào ALB nhằm bắt luồng lưu thông công cộng 100% lọt qua mạn CDN CloudFront nếu có thể
CloudFront	Nạp thêm tên miền riêng custom domain cùng chứng chỉ bảo an ACM certificate cho bạt sắc production uy tín
S3	Soái kỹ lại cấu trúc bucket policy ra quy cấm ngặt nghèo bảo đảm chỉ dành chồn vào cho duy nhất cổng CloudFront và cụm workload thuyền chuyển bấp
WAF	Chiều nạp tường lửa ứng dụng web AWS WAF chắn hộ trước mặt CloudFront bện cọc cản bước các luồng rỗng tai ương hạ độc
Nhật ký (Logs)	Đóng cổ định thông số giới hạn bảo lưu tệp nhật ký CloudWatch retention trọn các log của ứng dụng và Lambda
Sao lưu (Backups)	Rà thăm cọc retention lưu hồ sơ sao lưu tự động cho RDS cùng nhuần kiệt tiêu chuẩn điểm khôi phục PITR
Cảnh báo (Alarms)	Gắn lắp các mạch báo nguy khẩn thiết cho hàng đợi DLQ, tỉ lệ báo hồng từ Lambda, lỗi hệ CSDL RDS, chao lạch ALB và mức ngốn nợ SageMaker
Chi phí	Cài kịch bản theo chuỗi lịch biểu để ngưng văn hoặc tháo bỏ vĩnh viễn SageMaker endpoint ngoài lúc phục vụ demo/kiểm thử

Kết quả mong đợi

Bộ sườn kiến trúc hạ tầng vỗ tay hoàn trọn công đoạn sấm sinh khi mạng nổi mạch lạc, các trạm máy con EKS giễu cao trạng thái Ready, cổng cổng ALB điều rẽ luồng sức khỏe thọ khương mượt mà, hai trụ S3 kết hợp CloudFront ngạo mạn phát rạng trang frontend, kho dữ liệu ngự vững, nhịp tim SQS và Lambda ráo chiết tệp tin sự kiện và cụm thần kinh SageMaker an ngự trước trần kích hoạt đàn worker bám việc.

Các lỗi thường gặp

Triệu chứng	Nguyên nhân gốc rễ	Hướng khắc phục
Cụm máy node của EKS rền ngã khước từ tham dự vào cluster	Bảng định tuyến ranh giới private chao trượt lọt mạn trúng một NAT Gateway từng bị hủy tháo xô phai	Hồi sinh tại cho lập mới cỗ máy NAT rồi kiên nhẫn bưng tạc xây tạo khôi phục lại node group



Triệu chứng	Nguyên nhân gốc rễ	Hướng khắc phục
Trình điều khiển ALB controller bất lực mù tịt chẳng thấu tên VPC	Bước tra cứu siêu dữ liệu (metadata) lọt thẳm quá mốc thời hạn	Chủ động ẩn nhét tham số biến cờ - <code>--aws-region=ap-southeast-1</code> cùng <code>--aws-vpc-id=vpc-0cfee519122ae18b4</code> bồi thẳng vào lệnh cấu hình
Luồng điều hướng API trên cổng CloudFront dội báo lỗi trật lất	Thông tin miền DNS của ALB bị khai khuyết hoặc sai quy luật của cache behavior	Dò soát lại thận trọng trường tham số origin và chỉnh ngay các điều khoản trong cache behavior của CloudFront
Vòng lặp rà cứu SQS thông trả lỗi chẳng thấu bóng hình hàng đợi ở đâu	Tất gọi nhầm tên hàng đợi SQS	Ẩn chỉnh gõ chuẩn chuỗi cộc danh mác <code>internship-prod-outbox</code>
Khâu ra mắt cụm worker lâm nạn ngã lộn giềng lằm gào khóc	Máy chủ suy đoán SageMaker endpoint lả lướt nằm chìm ở tình trạng phi InService	Quyết tâm giữ nghề nhíp công tắc worker ở thể khóa tịt (disabled) cho tới khi cổng sấm AI hưng hoan phát cờ thọ bệnh sẵn sàng

Kết luận

Chương trình về Hạ tầng AWS khởi mở, ban tặng tấm sơ đồ định trát phân chia tài nguyên trọn khắp thể trận production thực tế, đồng thời trang bị sẵn chuỗi vũ khí dòng lệnh tra xét thẳm rà trứ bộ làm nòng cốt đạo bước ung dung trước các môn công tác khởi dựng CSDL, triển mác backend, thiêu web frontend, gấn thấu quan trắc, thí trù nghiệm thu cùng tháo gỡ quy thu chi cước tài nguyên bám sát kế tiếp ngay sau đây.

5.6 Triển khai cơ sở dữ liệu

Mục tiêu

Triển khai và kiểm chứng lớp dữ liệu (data layer) phục vụ ứng dụng Internship Application Tracker bao gồm: RDS PostgreSQL, ElastiCache Redis, các bảng DynamoDB cho dữ liệu chat, và bảng DynamoDB khử lặp sự kiện cho Lambda (idempotency table).

Bối cảnh kiến trúc

Hệ thống tận dụng các kho dữ liệu khác nhau để giải quyết cho từng mô hình truy xuất chuyên biệt:

Kho lưu trữ (Store)	Mục đích
RDS PostgreSQL	Lưu trữ người dùng, công ty, tin tuyển dụng, đơn xin việc, lịch sử thay đổi trạng thái quy trình, bản ghi tính bất biến, hàng đợi xử lý bất đồng bộ và bảng sự kiện outbox
ElastiCache Redis	Phục vụ kênh phát tán liên lạc pub/sub theo chuẩn Socket.IO giữa các pod trong cụm dịch vụ chat
Các bảng DynamoDB cho chat	Lưu trữ lâu dài thông tin người dùng tham gia chat, các nhóm hội thoại và nội dung tin nhắn
Bảng DynamoDB khử lặp (dedupe table)	Bịt rào bảo lưu tính bất biến cho Lambda consumer trong quá trình thụ thụ tin sự kiện SQS
S3	Nơi quy giữ tệp file đính kèm cùng văn thư sự kiện lâu năm (được đề cập riêng trong các chương hạ tầng và frontend)

Triển khai PostgreSQL

Máy chủ cơ sở dữ liệu RDS trên production sở hữu danh xưng định danh `internship-prod-postgres`. PostgreSQL chịu trách nhiệm chốt giữ toàn bộ dữ liệu giao dịch cốt lõi và các hàng đợi công việc của worker cần bầu rào cổng cơ sở dữ liệu.

Cần bằng chứng:

- thông số phân loại máy chủ DB instance class
- dung lượng đĩa được ban cấp
- tình trạng bật mã hóa lưu trữ
- thời gian bảo lưu sao lưu tự động (backup retention)

- thiết lập dự phòng qua Multi-AZ
- tên định nghĩa subnet group
- danh mục các ID thuộc về security group
- tình trạng khóa bảo vệ khỏi thao tác xóa (deletion protection setting)

Kiểm chứng:

```
aws rds describe-db-instances \
  --db-instance-identifier internship-prod-postgres \
  --region ap-southeast-1
```

Tra cứu cấu hình subnet groups:

```
aws rds describe-db-subnet-groups --region ap-southeast-1
```

Bộ di cư Alembic (Alembic migrations)

Backend điều khiển tiến hóa cơ sở dữ liệu nhờ chuỗi tệp kịch bản Alembic nằm bên trong thư mục backend/alembic/versions/. Các mốc di cư quan trọng bao gồm:

Migration	Mục đích
0004_idempotency_records.py	Tạo bảng chốt lưu bản ghi tính bất biến
0005_idempotency_status_check.py	Áp quy chuẩn kiểm tra giới hạn cho các rào trạng thái bất biến
0006_job_application_versions.py	Thêm trường quản lý phiên bản phục vụ khóa lạc quan (optimistic currency)
0007_outbox_events.py	Kiến thiết hạ tầng bảng transactional outbox
0008_async_processing_jobs.py	Tạo hàng đợi tác nghiệp bất đồng bộ
1065bcf66cb9_0007_add_rerank_run_tables.py	Khai sinh bộ bảng vận hành tính điểm xếp hạng lại (Rerank run tables)

Tiến trình triển khai EKS ban khởi ra một Kubernetes Job với mã định danh backend-migrate thi thố dòng lệnh:

```
alembic upgrade head
```

Thao tác kiểm nghiệm thủ công cấu trúc tiếp ngay tại gốc thư mục repo mã nguồn:

```
cd backend
alembic current
alembic heads
alembic history --verbose
```

Nghiêm răn: Tuyệt đối cấm thao diễn tự ý gõ chạy migration thẳng vào mạn cộc production trừ phi mốc khung giờ triển khai đi kèm kịch bản tháo gỡ sa lui (rollback plan) đã vỗ tay nhận được cái gật đầu phê duyệt.

Cấu hình kết nối PostgreSQL

Tiến trình ban bố production nạp chuỗi thông số biến cấu hình DATABASE_URL tới qua cơ quan rào bảo mật Kubernetes Secret internship-secrets. Nội dung thực chuỗi kết nối lưu mang mật mã truy cập cần khôn, bởi thể tôi chối cự từ việc phơi văn xuất chuỗi ra khỏi bản báo cáo tài liệu này.

Kiểm chứng tình hình an bầu trên Kubernetes mà không đả thương tới giá trị bí ẩn:

```
kubectl get secret internship-secrets -n internship
kubectl describe secret internship-secrets -n internship
```

Luồng kiểm định sức khỏe readiness từ phía backend tiến hành thăm ra tình hình an nộ của PostgreSQL thông qua lệnh tra khảo SELECT 1:

```
kubectl port-forward service/backend 18080:8000 -n internship
curl -fsS http://127.0.0.1:18080/health/ready
```

Kết quả trả về mong đợi:

```
{"status":"ready","service":"internship-api","dependencies":{"postgres":true}}
```

ElastiCache Redis

Cụm máy nhân bản Redis trên production giữ thẻ tên `internship-prod-redis`, đồng thời được tôi tra và chứng nghiệm thợ trạng thái `available` trọn bài trong bằng chứng Tuần 8. Cụm Redis đứng đằng sau bảo kê cho bộ giao diện Socket.IO adapter nhằm giúp luồng sự kiện đàm thoại thả ga reo vui lan tỏa giữa các pod phục vụ chat chạy song hành.

Kiểm chứng:

```
aws elasticache describe-replication-groups \
  --replication-group-id internship-prod-redis \
  --region ap-southeast-1
```

Dịch vụ chat tiếp thu thông ký biến `REDIS_URL` tự trong hòm bí mật `internship-secrets`. Tại production, Redis đương nhiên bắt buộc nằm gọn trong cự ly bất sóng từ các pod chat và khản kiêng mọi khe nứt rò rỉ tuột cộc hướng ra không gian public ngoài kia.

Các bảng DynamoDB cho chat

Dịch vụ chat phụ trách quán cọc 3 bản lưu trữ cơ sở trên DynamoDB:

- ChatUsers
- ChatGroups
- ChatMessages

Tôi đã tự hào thẩm định toàn bộ 3 bản chứa này ngạo ngể rạng trạng thái `ACTIVE` kiên cố trong bộ tài liệu chứng cứ Tuần 8. Điểm gọi trích soát sức khỏe `readiness` của dịch vụ chat gắn kiềp ràng buộc vào sự hưng thịnh đồng loạt từ phía cả DynamoDB lẫn Redis:

```
kubectl port-forward service/chat-service 18081:3000 -n internship
curl -fsS http://127.0.0.1:18081/health/ready
```

Chuỗi tin đáp trả như mong đợi ngay thời khắc muôn rào phụ thuộc đều xanh mướt:

```
{"status":"ready","dependencies":{"redis":true,"dynamodb":true}}
```

Kiểm chứng DynamoDB:

```
aws dynamodb describe-table --table-name ChatUsers --region ap-southeast-1
aws dynamodb describe-table --table-name ChatGroups --region ap-southeast-1
aws dynamodb describe-table --table-name ChatMessages --region ap-southeast-1
```

Bảng DynamoDB khử lặp cho Lambda

Tiến trình Lambda chuyên tiêu đề tin từ SQS gặm sử dụng bản lưu `InternshipLambdaEventDedupe` để nạp chốt trữ vĩnh cửu những khóa `eventId` vừa múa việc xong trôi `flowing over`. Trục cơ quan này dựng thành hào công chỗ che trọn bên sườn Lambda trước cảnh bề hão tin lặp vô ý trôi sa sang theo tự tính từ hạ tầng SQS.

Kiểm chứng:

```
aws dynamodb describe-table \
  --table-name InternshipLambdaEventDedupe \
  --region ap-southeast-1
```

Đối chiếu dấu tích từ lời gõ khói (`smoke test`) thành quả thu được:

```
aws dynamodb get-item \
  --table-name InternshipLambdaEventDedupe \
  --key '{"eventId":{"S":"lambda-smoke-fixed-1785220478"}}' \
  --region ap-southeast-1
```



Tài liệu báo cáo cấm được cho phép bỏ lỡ phôi vắng lời tự xưng ra lệnh thi thố cú rẽ trên, trừ phi trích đoạn chuỗi tệp đầu lời giải được xuất kho bọc bởi vào kho tàng bằng chứng.

Thiết kế đảm bảo tính nhất quán dữ liệu

Vấn đề (Concern)	Cơ chế xử lý
Trùng lặp lượt đăng ký tài khoản	Vận dụng ràng buộc unique trên PostgreSQL đi đôi thuật kéo trả mã lỗi HTTP 409
Nộp trượt nấc 2 đơn trùng ứng tuyển cùng lúc	Cọc ràng buộc độc nhất (<code>job_posting_id</code> , <code>candidate_user_id</code>) quy theo tiêu chuẩn Idempotency-Key
Bồi dề dữ liệu cũ kỹ lạc hậu lúc HR thao tác cập nhật	Khóa <code>expectedVersion</code> kết hợp chiều update công theo điều kiện ràng rẽ
Tiến trình tác nghiệp AI hao kéo dài gian phi	Cắt bảng lưu bảo bọc <code>async_processing_jobs</code> mang kẻ các quy ước thuê tác quyền (leases) và giới rên lượt tự phục hồi retry
Trượt rách sẩy tháo thất ngạc lời kêu gửi sự kiện outbox	Trữ vào bản <code>outbox_events</code> trói chung một cơ quan giao dịch transaction trúng nhịp với biển múa dữ liệu nghiệp vụ
Nhận thông điệp bồi trùng lặp từ kênh hàng đợi SQS	Lệnh ghi theo điều kiện trên DynamoDB của Lambda giục vẩy cọc theo trường trường dữ liệu <code>eventId</code>
Sẩy lặp trát mỗi tin nhắn đàm thoại sau cọc retry từ phía người đàm	Chiều khởi nặn dòng mới có điều kiện trên DynamoDB ôm sát chuỗi định danh con cọc <code>clientMessageId</code>

Các lệnh kiểm chứng

Tra soát tình thế cấu trúc tập thông tin cấu hình phục vụ luồng cắt dữ liệu bên trong cụm Kubernetes:

```
kubectl get configmap internship-config -n internship -o yaml
kubectl get secret internship-secrets -n internship
kubectl get job backend-migrate chat-init -n internship
kubectl logs job/backend-migrate -n internship
kubectl logs job/chat-init -n internship
```

Tra cứu hiện trạng bản bảng cơ sở thông qua điểm rà sức khỏe:

```
kubectl rollout status deployment/backend -n internship --timeout=300s
kubectl rollout status deployment/chat-service -n internship --timeout=300s
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

Kết quả mong đợi

- Tác vụ `backend-migrate` cấm phôi nhân hoàn trọn thành công rực rỡ.
- Trạm khảo sát sức khỏe sẵn lòng của backend xướng trả cọc phụ thuộc PostgreSQL bằng giá trị `true`.
- Điểm tra soát sức khỏe của dịch vụ chat reo vui trả giá trị cọc Redis cùng DynamoDB là `true`.
- Toàn thể các bản chứa chat trên DynamoDB tự tại trụ ở trạng thái `ACTIVE`.
- Nhóm nhân bản thuộc dịch vụ Redis duy dân khang trang ngả theo cọc `available`.
- Bản khử lặp Lambda nghiêng giỗ thọ hình an tọa, sẵn lòng trợ thủ trót lọt các lượt thi triển ghi có điều kiện.

Các lỗi thường gặp

Triệu chứng	Nguyên nhân gốc rễ	Hướng khắc phục
Thăm rà sức khỏe từ backend gào khóc thất bại	Biến <code>DATABASE_URL</code> cài sai thăm lật, lỗi nghên kẹt tường rào security group hoặc tiến trình migration lâm nạn	Vào sâu thanh rà tệp log của backend và chuỗi cấu hình security groups bên tài nguyên RDS
Trình Alembic chao chất phi có không rõ dịch ra URL	Vướng trật trượt trong phép nhồi biến interpolation thuộc shell hoặc file secret ngự trúng định dạng gieo hãm sai lệch	Đem khóa URL an trú cẩn mẫn vào trong một tệp secret an ninh và cấm tha buông cống gõ in chuỗi trần ra

Triệu chứng	Nguyên nhân gốc rễ	Hướng khắc phục
Điểm gọi sức khỏe dịch vụ chat gập rúm cục mắc <code>not_ready</code>	Trục bộ Redis adapter hoặc kịch bản khai quan khởi tạc DynamoDB trượt dốc	Rà kỹ nhật ký tệp log thuộc pod <code>chat-service</code> đi cùng tiến trình thợ <code>job chat-init</code>
Tác vụ Lambda dính lắp sự kiện thi thố gửi đi mail cảnh báo 2 lần	Bỏ sẩy quên bếng việc chốt ghi vào bản khử lắp hoặc khi ghi khuyết trống tham số rào điều kiện	Sửa lệnh cho tuân lệnh nghiêm cấm ghi theo cơ quan có điều kiện DynamoDB trối sát trường biến <code>eventId</code>
Các hàng tin nằm trong outbox trở lý gan chây bám ngục trạng thái pending	Tiến trình dispatcher khốn quẩn bất lực chẳng vãng được công gửi qua hàng đợi SQS	Đổi soát đường URL biến <code>OUTBOX_QUEUE_URL</code> , dải quyền hạn bầu theo IAM IRSA SQS, cùng nhật ký log từ dispatcher

Xử lý sự cố (Troubleshooting)

Kịp thời ứng trối cụm lệnh bọc lót bên dưới mỗi lúc giáp mặt sự cố:

```
kubectl logs deployment/backend -n internship --tail=200
kubectl logs deployment/chat-service -n internship --tail=200
kubectl logs deployment/backend-outbox-dispatcher -n internship --tail=200
kubectl logs deployment/backend-processing-worker -n internship --tail=200
aws rds describe-db-instances --db-instance-identifier internship-prod-postgres --region
→ ap-southeast-1
aws elasticache describe-replication-groups --replication-group-id internship-prod-redis
→ --region ap-southeast-1
aws dynamodb describe-table --table-name ChatMessages --region ap-southeast-1
```

Kết luận

Toàn cảnh móng rào cho cơ sở dữ liệu nghiêng còi ăn chốt thông quan ngay thời khắc các luồng di cư cơ quan PostgreSQL thành tài hoàn tất, cụm nhớ Redis nghênh tay cất sóng vui cười với đoàn pod chat, các bản dữ liệu DynamoDB ngạo bạt trạng thái active, bản khử lắp Lambda cầu định minh chánh và cục kiểm sức khỏe toàn dải ứng dụng hoan thả tin mừng êm xuôi băng rẽ qua khắp các cung đường định hướng triển khai.

5.7 Triển khai backend

Mục tiêu

Triển khai và kiểm chứng hoạt động của toàn bộ các workload ứng dụng trên cụm EKS bao gồm: dịch vụ FastAPI backend, dịch vụ chat, tiến trình outbox dispatcher, processing worker và vi dịch vụ trung gian AI service adapter.

Bối cảnh kiến trúc

Frontend không được vác thiêu thợ ngụ bên trong cụm EKS. Cụm EKS sắm vai chuyên nuôi nấng cho các dịch vụ và các cụm worker chạy dài kỳ, ẩn cấn khôn sát phía sau một cổng xuất phát ALB origin do cổng mạng CloudFront vinh quy kều xướng.

Các thông số đầu vào cho triển khai

Dãy chuyển sản xuất production tính toán tự túc ra địa chỉ định tuyến image URIs dựa từ chuỗi tham mác tài khoản AWS, mã khu vực region, kho ECR cùng mã chốt Commit `github.sha`:

Workload	Biến image	Kho ECR mặc định
Backend và các worker	<code>BACKEND_IMAGE</code>	<code>internship-backend</code>
Dịch vụ chat	<code>CHAT_IMAGE</code>	<code>internship-chat</code>
Bộ đệm AI adapter	<code>AI_IMAGE</code>	<code>internship-ai</code>

Tập lệnh kịch bản triển khai đòi hỏi các khóa:

- `BACKEND_IMAGE`
- `CHAT_IMAGE`

- SECRET_KEY
- DATABASE_URL
- REDIS_URL
- AWS_REGION
- OUTBOX_QUEUE_URL

Nhóm thông số mang tính tự nguyện nhưng hàm trợ trọng lượng lớn:

- AI_DEPLOY_ENABLED
- AI_IMAGE
- PROCESSING_WORKER_ENABLED
- SAGEMAKER_ENDPOINT_NAME
- AI_SERVICE_BASE_URL
- IRSA_ROLE_ARN
- S3_BUCKET
- AI_SERVICE_API_KEY

Đóng gói image và đẩy lên ECR

Tập lệnh phụ thủ cho luồng CI là `scripts/ci/deploy-eks-pipeline.sh` gánh vác sứ mạng dịch biên nặn image và hất đẩy trót lọt lên bãi:

```
bash scripts/ci/deploy-eks-pipeline.sh
```

Dây chuyền ngay tức khắc thẩm soát dấu ấn nhân SHA vừa gán cho hai image của backend và chat:

```
aws ecr describe-images \
  --region ap-southeast-1 \
  --repository-name internship-backend \
  --image-ids imageTag=<GITHUB_SHA>

aws ecr describe-images \
  --region ap-southeast-1 \
  --repository-name internship-chat \
  --image-ids imageTag=<GITHUB_SHA>
```

Bắt giác nhịp còi khóa `AI_DEPLOY_ENABLED=true` vang khởi, tệp image cho bộ trung gian AI adapter lập tức được thiếp nặn với chuỗi lệnh:

```
docker build --file Dockerfile.ai-service --tag <AI_IMAGE> .
docker push <AI_IMAGE>
```

Namespace Kubernetes và ServiceAccount

Quy trình thợ tác ban khai cho ban bố:

```
kubectl apply -f k8s/app/namespace.yaml
kubectl apply -f k8s/app/serviceaccount.yaml
```

Mỗi khoảnh khắc biến cộc tham số `IRSA_ROLE_ARN` mang giá trị truyền qua, hồ sơ bản vẽ ServiceAccount trú danh thuyền dành riêng cho EKS lập tức trải qua luồng compile render rồi được ra áp trúng theo văn kiện từ `k8s/eks/serviceaccount.yaml`.

Kiểm chứng:

```
kubectl get namespace internship
kubectl get serviceaccount internship-app -n internship -o yaml
```

Sử dụng ConfigMap và Secret

Hồ sơ ConfigMap thi triển ra production bắt nguồn xuất nập từ văn kiện `k8s/eks/configmap.yaml`. Các trường định nghĩa mạn trọng cốt yếu gồm có:

Khóa	Giá trị hoặc mục đích
APP_ENV	eks
OUTBOX_PUBLISHER	sqs
AI_SERVICE_BASE_URL	http://ai-service:8010
AI_SERVICE_PARSE_TIMEOUT_SECONDS	120
AI_SERVICE_MATCH_TIMEOUT_SECONDS	600
STORAGE_BACKEND	s3
AWS_REGION	ap-southeast-1
DYNAMODB_USERS_TABLE	ChatUsers
DYNAMODB_GROUPS_TABLE	ChatGroups
DYNAMODB_MESSAGES_TABLE	ChatMessages

Tiến trình triển khai kiến dựng hoặc cập nhật thẳng cho Kubernetes Secret mang danh nhãn `internship-secrets` vật nạp thông số tự trong cội lưu bí mật của GitHub. Nghiêm cấm hão huyền in tuế ngã sa ra mạn minh văn đối với mọi giá trị mật mã bí ẩn.

Kiểm chứng:

```
kubectl get configmap internship-config -n internship -o yaml
kubectl describe secret internship-secrets -n internship
```

Các job migration và khởi tạo dữ liệu chat

Học theo tôn chỉ gọn gàng, hệ triển khai sẽ dẹp cạn sạch mọi xác tàn job cũ rách, rồi tự do tạc phác đẩy lên các job mới tinh sành, kiểm mẫn chìm an đới còi ra lệnh thành khơi trót lọt:

```
kubectl delete job backend-migrate chat-init -n internship --ignore-not-found
kubectl apply -f <rendered-backend-migration-job.yaml>
kubectl apply -f <rendered-chat-init-job.yaml>
kubectl wait --for=condition=complete job/backend-migrate -n internship --timeout=300s
kubectl wait --for=condition=complete job/chat-init -n internship --timeout=300s
```

Job backend-migrate ra rã thọ thi lệnh `alembic upgrade head`. Còn chat-init hoan hỷ khai mở cất cọc cho chuỗi cơ quan bằng chat tại DynamoDB.

Triển khai các workload

Deployment	Lệnh container (Container command)	Port	Replicas	Mục đích
backend	uvicorn app: main:app --host 0.0.0.0 --port 8000 --proxy-headers	8000	2	Phục vụ dịch vụ REST API trên FastAPI
chat-service	node server.js thông qua chuỗi lệnh khởi xướng tự bản thể image	3000	2	Dịch vụ Chat REST kết hợp Socket.IO
backend-outbox-dispatcher	python -m app. workers.outbox_dispatcher	9101 cho metrics	1	Trích xuất và phát thông tin sự kiện từ outbox qua SQS
backend-processing-worker	python -m app. workers. processing_worker	9102 cho metrics	2 trong manifest, tự chủ ngã dẫn xuống 0	Gánh tải xử lý tài liệu cùng thao tác AI bất đồng bộ
ai-service	Lệnh mặc định gắn trong image AI	8010	1	Bộ đệm trung gian cho SageMaker

Khải chạy ban bố và rà kiểm chứng:


```
bash scripts/k8s/deploy-eks.sh
kubectl rollout status deployment/backend -n internship --timeout=300s
kubectl rollout status deployment/chat-service -n internship --timeout=300s
kubectl rollout status deployment/backend-outbox-dispatcher -n internship --timeout=300s
kubectl rollout status deployment/backend-processing-worker -n internship --timeout=300s
kubectl rollout status deployment/ai-service -n internship --timeout=900s
```

Lưu tâm khẩn thiết thợ thi dòng lệnh rà cuộn bản phát hành (rollout status) cho ai-service duy chỉ các dịp cơ chế cho phép triển khai công AI vỗ tay gạt bật.

Chốt kiểm soát cho dịch vụ AI và processing worker

Tập lệnh kịch bản cản rào ngăn kiêng mọi ý định gạt rẽ cho worker ngạo mạn bật vùng bãi khuyết thiếu an nộ:

- Biến cấu hình PROCESSING_WORKER_ENABLED=true khẩn khoản kham thấu đòi hỏi AI_DEPLOY_ENABLED=true.
- Tham số PROCESSING_WORKER_ENABLED=true kiên kỳ rêu đòi nạp biến AI_IMAGE.
- Trạng thái cất vây cho SageMaker endpoint internship-qwen3-4b bắt bầu ngã về mác InService.

Thăm viếng tình thế cộ endpoint trước phút ban trát cấp visa cho worker hoạt động:

```
aws sagemaker describe-endpoint \
  --endpoint-name internship-qwen3-4b \
  --region ap-southeast-1 \
  --query EndpointStatus \
  --output text
```

Kết quả mong đợi:

InService

Các điểm kiểm tra sức khỏe (Health probes)

Workload	Startup probe	Readiness probe	Liveness probe
backend	/health/live	/health/ready	/health/live
chat-service	/health/live	/health/ready	/health/live
ai-service	Không cấu hình phân định ra riêng	/health/ready	/health/ready
backend-outbox-dispatcher	TCP 9101	TCP 9101	TCP 9101
backend-processing-worker	TCP 9102	TCP 9102	TCP 9102

HPA và PDB

Văn kiện k8s/app/autoscaling.yaml quy bạo định nghĩa rõ ràng:

Tài nguyên	HPA	PDB
backend	tối thiểu 2, tối đa 5, mục tiêu nhịp CPU ở 70%	số lượng tối thiểu duy trì available là 1
chat-service	tối thiểu 2, tối đa 5, mục tiêu nhịp CPU ở 70%	số lượng tối thiểu duy trì available là 1
backend-processing-worker	tối thiểu 2, tối đa 5, mục tiêu nhịp CPU ở 70%	số lượng tối thiểu duy trì available là 1 (chỉ thi khi cộ worker được bật)

Trong các kịch bản tiến trình processing worker bị thiêng cho đóng tít khóa văn (disabled), tập kịch bản sẽ tháo rách xao hạ xóa luôn chuỗi HPA cùng PDB gắn theo nó, và kiên khôn dẫn cộ số replica nháo ép trùng mức 0.

Kiểm chứng:

```
kubectl get hpa,pdb -n internship
kubectl describe hpa backend -n internship
kubectl describe hpa chat-service -n internship
```

Ingress và định tuyến ALB

Thực thể ALB Ingress vinh mang định dạng public nằm gọn tại cội k8s/eks/ingress-alb-no-domain.yaml. Con đường rẽ sóng bao vây chia tải ngã trùng:

- /api vãng trùng tới Service backend
- /chat trao cho Service chat-service
- /socket.io điều thẳng cho Service chat-service

Thi trở tung cọc phân tuyến ALB public qua dây chuyền workflow mang chế độ deploy-public, hoặc bốc gọi cầu qua tập lệnh hồ thủ:

```
ENABLE_ALB_INGRESS=true bash scripts/aws/deploy-public-ingress.sh
```

Kiểm chứng:

```
kubectl get ingress internship-public -n internship
curl -fsS http://k8s-internshippublic-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com/api/health/ready
curl -fsS http://k8s-internshippublic-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com/chat/health/ready
```

Lệnh kiểm tra nhanh qua Port-forward

Tập lệnh ban bố trung thọ viện binh các chốt tra chuyển tiếp cổng nội bồi (local port-forwards) phục vụ luồng test thử nghìn nhịp khối ngạo:

```
kubectl port-forward service/backend 18080:8000 -n internship
curl -fsS http://127.0.0.1:18080/health/ready

kubectl port-forward service/chat-service 18081:3000 -n internship
curl -fsS http://127.0.0.1:18081/health/ready

kubectl port-forward service/ai-service 18082:8010 -n internship
curl -fsS http://127.0.0.1:18082/health/ready
```

Kết quả mong đợi

- Hai thọ job migration và khởi tạo dữ liệu chat reo hò kết hoang thành tựu trọn vẹn.
- Tiến trình cuộn gác mác triển khai của backend, chat cùng dispatcher xuôi luồng êm mượt vãng quan.
- Tiến trình processing worker chễm trệ tựa cọc trạng thái Ready, hoặc kiên chánh ngộ yên ở quy mô 0 replica đúng kịch bản thiết lập chủ trách.
- Dịch vụ AI nghênh ngang an tọa trạng thái Ready bất cứ kì luồng bật cống AI thi thố mở ra.
- Toàn diện các cụm Service hoành khai rước rẽ đầy đủ trường endpoints gắn đệm.
- Cuộc thi rà soát sức khỏe từ ALB thông quan văn phúc trát danh cho backend và chat.
- Mạng CDN CloudFront không trầy một vảy băng qua chạm tới origin của ALB dành riêng cho ranh giới chuỗi /api, /chat, và /socket.io.

Các lỗi thường gặp

Triệu chứng	Nguyên nhân	Hướng khắc phục
Lên tiếng phan ca Missing required environment variable	Thông số mật mã secret hoặc biến môi trường bắt buộc của luồng ban bố biến đầu bật vô âm tín	Kíp mau cầu bồi bổ nạp thẳng trùng biến cấu hình hay bí ẩn secret còn khuyết trống cho bên mạn GitHub

Triệu chứng	Nguyên nhân	Hướng khắc phục
Ngứa xao còi tháo Unable to compute backend/chat image URIs	Sa khuyết biến định hình của workflow hay gạt hồng trong nhịp tự động sinh chuỗi định danh ảnh bám theo nhãn SHA	Vận chuẩn chuẩn xác chuỗi lắp kết bầu từ tài khoản, vùng khu vực region, danh tước kho ECR cùng tham mác cọc github.sha
Bồi tra thất chốn chẳng thấy container image trên kho ECR	Bước test tác nghiệp build/push hải hồng chưa nạp thẳng chùm thẻ tag mang chữ ký SHA	Xâm nhập điều rã nguyên rẽ tại job build-images và mục trần tình trạng kho lưu trữ ECR repository
Cọc PROCESSING_WORKER_ENABLED=true chữi thẻ khước văn lệnh rào	Đàn hạ tầng rào cản phụ thuộc của AI nằm phó hồng trượt dốc sa mác	Nấy lệnh bật image AI trước ra tiền còi thanh tra tình huống cọc SageMaker ngã trúng trạng thái InService
Lệnh thăm dò sức khỏe dịch vụ chat trả về bể quần trượt vắng	Cơ quan tài nguyên phụ thuộc mang tên Redis hoặc DynamoDB chìm lim ngắc gục	Truy nạp thăm rà tệp log của dịch vụ chat kết hợp kiểm chứng hiện trạng sức khỏe còi DynamoDB cùng Redis
Bước khảo sát sức khỏe của backend khốc rêu than gục ngã	Kho cọc CSDL PostgreSQL chìm phí vắng dạng hoặc luồng thọ di cư migration trôi sập lợt khuyết	Khai quan coi soi nhật ký log thuộc về backend song song soát tra khả năng vướn luồn kết nối rào tới RDS

Kết luận

Toàn còi quy trình đưa lên EKS cho hệ thống backend ca rền khúc hoan khái, trọn trào trát vinh danh thành tài vãng trát ngay tại khúc khoảnh mác khi backend, dịch vụ chat, dispatcher, cùng các chuỗi tùy nhĩ của worker và bộ đệm AI adapter đồng loạt thi thổ vũ múa vững vàng, bám khôn bính sát cấu hình đo dò sức khỏe probess chuẩn chỉnh, dịch vụ minh bạch endpoint trứ rêu, luật gò dẫn dẻo thau chu toàn kiêm vây luân cộ phân luồng giao thông mạch lạc theo chân ALB routing.

5.8 Triển khai frontend

Mục tiêu

Thực hiện build mã nguồn frontend trên React/Vite, đăng tải bộ tệp đầu ra thẳng vào một S3 bucket riêng tư, và tổ chức phân phối an toàn thông qua CloudFront.

Bối cảnh kiến trúc

Hiện tại, frontend được triển khai theo quy chuẩn của các tệp tĩnh đơn thuần, loại trừ mọi yếu tố ràng buộc chạy như workload trên EKS. Những tài nguyên Kubernetes cũ mang tên frontend gồm Deployment, Service, HPA, PDB, đi kèm khối ALB Ingress cũ đã bị bóc tháo tiết cộ, nhằm chặn đứng nguy cơ các đợt áp triển khai mã app vào EKS trong tương lai lâm lợt thi thoảng vô ý vác còi tự nặn sinh ra lại chúng.

Điểm truy cập công cộng hiện thời của người dùng:

<https://dhm2rz5nmsibj.cloudfront.net>

Mã số định danh cộ CloudFront distribution:

EQIGYNECXDYL8

Cấu hình môi trường Vite

Quy chuỗi build tĩnh mạn production cho frontend sử dụng các tham mác:

Biến	Giá trị production	Mục đích
VITE_API_BASE_URL	/api	Ép toàn bạo các lời gồ gọi backend API luân chảy trót lợt qua CloudFront cùng rãnh ALB
VITE_CHAT_API_BASE_URL	chuỗi rỗng	Ban visa cho bộ máy runtime trình duyệt ung dung trút lời qua cung đường /chat ngoài còi localhost

Bộ gọi giao dịch API client phía frontend giữ nguyên thông số gán định ở chế độ lập trình tại chỗ khi khuyết thiếu tham số truyền cấu hình:

- Chuỗi mặc định của API: `http://localhost:8001`
- Chuỗi mặc định của Chat ở trình duyệt cá nhân tại chỗ: `http://localhost:3000`
- Chuỗi mặc định của Chat ở thế giới ngoại localhost: `/chat`
- Đường dẫn Socket.IO path: `/socket.io`

Các lệnh build

Ra lệnh thao tác thi thực tại gốc thư mục repo:

```
cd frontend
npm ci
VITE_API_BASE_URL=/api VITE_CHAT_API_BASE_URL= npm run build
```

Thư mục kết quả đầu ra theo kỳ vọng:

`frontend/dist`

Trình lệnh script bật khai thăm kiểm tra tự dọn bãi nhận diện cả `frontend/dist` lẫn `frontend/build`, thế nhưng đầu ra sinh tệp mã mã hiện hành do Vite ban xuất luôn nằm ngụ yên ổn tại con rãnh `frontend/dist`.

Đăng tải lên S3

Tập kịch bản chiêu xuất lực lượng cho môi trường production đóng lệnh:

```
bash scripts/ci/deploy-frontend.sh
```

Các biến ràng buộc bắt buộc:

Biến	Mục đích
FRONTEND_DEPLOY_ENABLED	Bắt di bất dịch thi định giá trị <code>true</code>
FRONTEND_BUCKET	Hòm S3 đích nhắm, chẳng hạn kiểu dạng <code>internship-prod-frontend-<AWS_ACCOUNT_ID></code>
CLOUDFRONT_DISTRIBUTION_ID	Mã định danh distribution phục vụ việc gỡ xóa bộ đệm cache (invalidate)
VITE_API_BASE_URL	Gán bằng <code>/api</code>
VITE_CHAT_API_BASE_URL	Gán rỗng tuột

Tập script tuân thủ cảnh giác khóa nhất kiên ky ranh giới bucket ở tư thế vô danh bí hiểm riêng tư:

```
aws s3api put-public-access-block \
  --bucket internship-prod-frontend-<AWS_ACCOUNT_ID> \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Toàn thể dải tệp tĩnh (static assets) buông mình trôi trôi tải lên vân gấn kèm trường mác giữ cache dài hạn thọ sinh bền lâu:

```
aws s3 sync frontend/dist s3://internship-prod-frontend-<AWS_ACCOUNT_ID> \
  --delete \
  --exclude index.html \
  --cache-control public,max-age=31536000,immutable
```

Riêng tệp `index.html` vinh hoa hưởng luồng nạp bởi cô lập, cấm rào kiêu sa chối cự việc bầu giữ trong cõi nhớ tạm (no-cache semantics):

```
aws s3 cp frontend/dist/index.html s3://internship-prod-frontend-<AWS_ACCOUNT_ID>/index.html \
  --cache-control no-cache \
  --content-type text/html
```

Cấu hình CloudFront

Trình kịch bản giúp tay `scripts/aws/ensure-cloudfront.sh` sẽ vót dáng uyên thâm đảm nhận rà thăm hoặc tác sinh ra tài nguyên distribution mới. Thiết lập kiến trúc hiện hành được nạp dài:

Behavior	Origin	Ghi chú
Mặc định *	S3 frontend bucket	Giao diện tĩnh của frontend cùng các tệp nền cho SPA
/api/*	ALB	Dữ liệu REST API backend
/chat/*	ALB	REST API thuộc về dịch vụ chat
/socket.io/*	ALB	Giao thông luân chuyển cho Socket.IO

CloudFront khôn ngoan giấu thiết lập cơ chế cứu rỗi điều bẻ dự phòng cho SPA (SPA fallback):

Lỗi (Error)	Đường dẫn phản hồi	Mã phản hồi
403	/index.html	200
404	/index.html	200

Ra lệnh xóa bỏ nội dung bộ nhớ đệm cũ kỹ lạc hậu (CloudFront invalidation):

```
aws cloudfront create-invalidation \  
  --distribution-id EQIGYNECXDYL8 \  
  --paths "/*"
```

Triển khai frontend qua GitHub Actions

Ra tay gõ chọn điều rữ thực thi chế độ workflow dispatch theo mô hình:

GitHub -> Actions -> CI/CD -> Run workflow -> mode: deploy-frontend

Quy tắc bước nhảy của thợ job deploy-frontend:

1. Tiến hành ủy quyền và nạp định danh tài khoản AWS credentials cần ký thông qua luồng an ninh OIDC.
2. Thiết định môi trường máy Node.js bám theo giá trị ghi ở biến workflow `NODE_VERSION`.
3. Thăm định cần mật sức khỏe đường rẽ cọc ALB tại điểm trạm `/api/health/ready` lẫn `/chat/health/ready`.
4. Gọi khởi nhảy tệp lệnh `scripts/ci/deploy-frontend.sh`.
5. Đẩy tót mượt chuỗi tập tin giao diện tĩnh của frontend an trú vào lòng hòm S3.
6. Phất đi trát thi thoảng xóa bạt tàn nhẵn cọc dữ liệu đệm cũ qua lệnh CloudFront invalidation.

Kiểm chứng

Khảo rà tình hình CDN CloudFront:

```
curl -I https://dhm2rz5nmsibj.cloudfront.net/  
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready  
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

Thanh rà rào cản ngăn công khai trần tuột của hòm S3:

```
aws s3api get-public-access-block \  
  --bucket internship-prod-frontend-<AWS_ACCOUNT_ID>
```

Khám xét thông số cấu hình tài nguyên distribution:

```
aws cloudfront get-distribution --id EQIGYNECXDYL8  
aws cloudfront list-invalidations --distribution-id EQIGYNECXDYL8
```

Kết quả mong đợi

- Trang frontend hoành hoành hiện múa êm thắm từ sau vây cổng mạng CloudFront.
- Ranh giới cản phá ngăn mạn cất chặn việc mở cửa lộ thiên (public access) của hòm S3 tiếp chặng đứng sừng sững gác vững vàng.
- Dải luân điều hướng tuyến /api/*, /chat/*, kết mác cùng /socket.io/* rầm rập dốc đầu về rờorigin ALB hợp hiền.
- Chuỗi tra truy nhập thẳng (deep links) bỗng chốc bốc sa hẫng đều khôn lanh được CDN nén trả quay đầu theo vạch index.html.
- Một bản trát ban lệnh xóa cối đệm (invalidation) hiển lộ oai phong ngay tại hồi sau đợt bung ban triển khai.

Các lỗi thường gặp

Triệu chứng	Nguyên nhân	Hướng khắc phục
Tác vụ thợ job của frontend khinh bạc giễu lướt qua bề bở	cọc biến mác FRONTEND_DEPLOY_ENABLED trượt giá trị true ở tầm bắt không gian biến cho job vô vô	Hãy cẩn thận định trát cờ cho cọc biến ở cấu hình biến GitHub environment theo đúng thói gắm nhâm workflow
Trục trục lờ lơ gọi API chao bốc lao húc sai về cọc localhost ở cõi production	Quên băng mang cấp cho tham mác biến trường VITE_API_BASE_URL=/api	Thi nặn build lại từ gốc phổi nạp sung túc tham mác biến Vite chốn production
Trang CDN CloudFront ngậm mỗi khước trần giao diện bộ cũ kỹ	Tập file tĩnh gõ nhâm kho cối đệm cache hay sẩy bãi sót việc phát lệnh invalidation	Kiểm kê trạng thái danh bạ invalidation kết cọc thăm quy chế cache control của tệp index.html
Luồng chui văng kho S3 thẻ khước cự tuế từ cất sau cánh CloudFront	Quyền hạn bucket policy khiêm khuyết chuỗi cam gặt giao tình OAC	Kích chuông thi thi rào cho kịch bản ensure-cloudfront.sh kèm thông ký số tài khoản cùng ID của distribution
Điểm gõ thẳng vào website tĩnh qua origin S3 chọc chát cự tuế đội bãi	Trạng thái kiên cố 100% đúng quy luật thiết lập an ninh thọ thi	Hòm frontend vững chí giam cọc ở thể private ngặt nghèo, chỉ đón nhờ ngõ rẽ cầu nhờCDN CloudFront buông thả
Trượt rách kênh thời gian thực Socket.IO	Sa sẩy quên cất behavior mang ranh /socket.io/* bên trong công CloudFront hoặc tuột trượt trên vạch định tuyến ALB	Thanh soát tỉ mẩn behavior bên CloudFront đi đôi cọc đường dẫn Ingress của ALB

Việc tháo dỡ các tài nguyên frontend cũ trên Kubernetes

Cấu hình hạ tầng nay vững chí kiên định cất tay trục hủy tiết toàn thể dải tài nguyên frontend khai trên Kubernetes. Tuyệt đối kiên răn không được lằm tưởng dựng ra phác lại:

- Deployment/frontend
- Service/frontend
- ranh giới HPA chuyên trách cho frontend
- ngân sách PDB công hiến cho frontend
- cọc ALB Ingress gánh phần cự rẽ riêng tư của frontend
- chốt đường rãnh triển khai FRONTEND_IMAGE thọ cọc bên trong luồng thi app vào EKS

Frontend kiên cọc vinh thọ duy nhất một lần dịch build tĩnh rẽ phân cất gửi trên hai cọc trụ bộ đôi S3 kết công CloudFront CDN.

Kết luận

Thào tác triển khai frontend cất mốc ăn còi thông văn chính công ngay chặng các tệp build sinh từ Vite nhờ cầu im lặng an ngự trong lòng S3, mạng CloudFront ra trát trút xóa rũ nhẵn củi xác đệm cũ rách, và trọn muôn ngõ cộng đồng có thể hưng múa chiêu nghênh giao diện ứng dụng từ cung rẽ địa chỉ web chính danh <https://dhm2rz5nmsibj.cloudfront.net>, giữa lúc luồng API/realtime mỉm cười chao lướt êm trót lọt sang mạn backend/chat thi nhau bám cổng CloudFront cùng cánh cổng ALB.

5.9 Giám sát và nhật ký

Mục tiêu

Giám sát an nộ sức khỏe của ứng dụng, trạng thái của các tiến trình triển khai, chuỗi tập tin nhật ký (logs), hệ thống lượng đo lường metrics, danh sách các hàng đợi, luồng xử lý trên hàm Lambda, và toàn diện cụm tài nguyên dịch vụ AWS được quản lý.

Bối cảnh kiến trúc

Kho chứa repository tích hợp sẵn các hồ sơ bản vẽ kịch bản Kubernetes mang thói quen ServiceMonitor cùng PrometheusRule dùng thi cầu gặt hái thông tham số ứng dụng metrics. Các chỉ số metrics và báo nguy cảnh báo nằm tại cội AWS CloudWatch buộc lòng yêu cầu thanh kiểm cần khôn trên tài khoản AWS của dự án trước thềm ra phần cở ghi mác là đã thực thi trong production.

Tra soát tài nguyên Kubernetes

Ứng dụng thợ gõ cụm lệnh cất nhanh kubectl get để thanh kiểm tức thời mọi nhíp sinh trời trạng thái:

```
kubectl get deployments,pods,svc,endpoints,hpa,pdb,jobs -n internship -o wide
kubectl get ingress internship-public -n internship
kubectl get events -n internship --sort-by=.lastTimestamp
```

Kết quả mong đợi:

- Hai chiến tuyến backend và chat-service nghênh ngang hầy múa con số các replica mang trạng thái Ready.
- Tiến trình backend-outbox-dispatcher vững kiên an tọa cộc trạng thái Ready.
- Trình backend-processing-worker ung dung chễm trệ trạng thái Ready, hoặc kiên khôn ngả bãi xuống quy mô 0 replica đúng nhĩ kịch bản.
- Dịch vụ ai-service báo danh cộc Ready mỗi dịp chu trình ban bố mở cổng AI vổ còi thêu bật.
- Cụm tác vụ job thợ múa backend-migrate và chat-init hoan khải khép kín một mạch thi hành thành tài hoàn trọn.

Trạng thái cuộn bản phát hành (Rollout status)

```
kubectl rollout status deployment/backend -n internship --timeout=300s
kubectl rollout status deployment/chat-service -n internship --timeout=300s
kubectl rollout status deployment/backend-outbox-dispatcher -n internship --timeout=300s
kubectl rollout status deployment/backend-processing-worker -n internship --timeout=300s
kubectl rollout status deployment/ai-service -n internship --timeout=900s
```

Chú tâm khẩn kiệt nhấp rà lệnh trát của ai-service duy chỉ lúc rào cầu hình cho phép triển khai pod trung gian AI service mở rộng đón cầu.

Nhật ký ứng dụng (Application logs)

```
kubectl logs deployment/backend -n internship --tail=200
kubectl logs deployment/chat-service -n internship --tail=200
kubectl logs deployment/backend-outbox-dispatcher -n internship --tail=200
kubectl logs deployment/backend-processing-worker -n internship --tail=200
kubectl logs deployment/ai-service -n internship --tail=200
```

Kiên ky huy động thêm tham cở --previous trong hoàn cảnh con pod tuôn cộc tái khởi sinh lật cội:

```
kubectl logs deployment/backend -n internship --previous --tail=200
```

Các lệnh describe

Ưu vãng thợ gọi câu lệnh kubectl describe mỗi lúc tiến trình khảo sát sức khỏe readiness, phân rã vãng lịch chạy pod, bộ đo lường sức khỏe, hoặc cở máy canh rà đồng bộ Ingress bốc hồng đội sập:

```
kubectl describe pod -n internship -l app=backend
kubectl describe pod -n internship -l app=chat-service
kubectl describe hpa backend -n internship
kubectl describe ingress internship-public -n internship
```

Các đường dẫn kiểm thử sức khỏe

Hệ rà soát sức khỏe sẵn lòng thuộc mạn backend thi ra chiêu tra khảo kết nối cộc tới PostgreSQL:


```
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
```

Chuỗi tin đáp trả theo kỳ vọng:

```
{"status":"ready","service":"internship-api","dependencies":{"postgres":true}}
```

Nhịp tim đo đặc rà sức khỏe từ dịch vụ chat tiến tới thấu trát soát 2 cơ quan Redis kết bầu cùng DynamoDB:

```
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

Kết quả trả về mong đợi:

```
{"status":"ready","dependencies":{"redis":true,"dynamodb":true}}
```

Luồng soát sức khỏe bên mạn dịch vụ AI bầu giữ xác minh tinh trát xem khóa biến SAGEMAKER_ENDPOINT_NAME đã sẵn vãng vào đủ chẳng:

```
kubectl port-forward service/ai-service 18082:8010 -n internship  
curl -fsS http://127.0.0.1:18082/health/ready
```

Prometheus và Grafana

Kho repository cất tạc trọn cội định nghĩa cho các tài nguyên mang mác ServiceMonitor tương ứng cho:

- backend
- chat-service
- backend-outbox-dispatcher
- backend-processing-worker
-

Ban bố gõ lệnh ứng cài các manifest mỗi khi rềm cấu trúc bộ khung CRD thuộc về Prometheus Operator thì tho an tọa bãi:

```
kubectl apply -f k8s/observability/service-monitors.yaml  
kubectl apply -f k8s/observability/prometheus-rules.yaml
```

Kiểm chứng:

```
kubectl get servicemonitor,prometheusrule -n monitoring  
kubectl port-forward service/kube-prometheus-stack-prometheus 9092:9090 -n monitoring  
curl -fsS "http://127.0.0.1:9092/api/v1/targets?state=active"
```

Các đích đến (targets) giám sát mong đợi xuất nộ ra cho ứng dụng:

- backend
- chat-service
- backend-outbox-dispatcher
- backend-processing-worker

Cảnh báo được định nghĩa trong repository

Thư viện kịch bản tại gốc repository sở hữu trọn tập quy nạp cho dải báo nguy Prometheus alerts chuyên phục vụ:

Cảnh báo	Mục đích
InternshipBackendDown	Mục tiêu (target) giám sát metrics cho backend bất tàn vắng dạng
InternshipChatDown	Mục tiêu giám sát metrics thuộc về chat cự cội tuột giốc
InternshipChatDependencyDown	Trụ cộc tài nguyên phụ thuộc (Redis hay DynamoDB) của chat sa cộc nghệt
InternshipOutboxDispatcherDown	Mục tiêu giám sát metrics của dispatcher rơi tòm kiệt dạng



Cảnh báo	Mục đích
InternshipProcessingWorkerDown	Mục tiêu giám sát metrics cho processing worker gào ngã kiếp im lỉ
InternshipBackend5xxHigh	Tỷ lệ lỗi 5xx vọt bồng leo dốc cao chót vọt còi backend
InternshipBackendP95LatencyHigh	Chỉ số trễ mức p95 tại backend trôi nhíp phình bạo rực rỡ
InternshipChatMessageFailureRatioHigh	Tỷ lệ thư tín đàm thoại dính thảm bại thuyên trượt qua ngưỡng an toàn
InternshipPodCrashLooping	Các pod bầu tuội mắc kẹt trong bầy vòng lặp tử thần CrashLoopBackOff
InternshipOutboxDeadEventsPresent	Tác nghiệp sự kiện còi outbox trườn ốm chết de lị, giục gọi tay con người thiên can thiệp
InternshipOutboxPendingTooOld	Nhĩ sự kiện chầy gan ngo ngập ngác mức pending trượt tuổi hạn xa hoa
InternshipOutboxTransportErrors	Lỗi vãng đồ vận trút trên còi vận tải SQS
InternshipProcessingQueueOld	Tuổi hàng đợi xử lý processing queue sa chao quá nấc giới hạn thọ
InternshipProcessingJobsFailing	Các processing jobs trôi sa đâm sập thảm bại liên hoàn
InternshipAIProviderFailures	Trùng trùng lỗi chất đội sang từ nguồn cất trát dịch vụ AI

Trang thái: đã triển khai trong các manifest của repository. Cần bằng chứng để minh chứng các cảnh báo này đã được áp dụng trong production.

Nhật ký CloudWatch (CloudWatch Logs)

Kiểm tra nhật ký log của Lambda:

```
aws logs describe-log-groups \
  --log-group-name-prefix /aws/lambda/internship-outbox-handler \
  --region ap-southeast-1

aws logs tail /aws/lambda/internship-outbox-handler \
  --since 1h \
  --region ap-southeast-1
```

Kiểm tra soát các tệp log phát nạp từ mạn EKS duy nhất chỉ những chuỗi thời khắc cộc điều hướng thu tệp container log shipping gieo rắc kích mùa thành công. Cần bằng chứng: tên các log group trên CloudWatch dành cho backend, chat, worker, dispatcher, và AI service.

Chỉ số giám sát của các dịch vụ AWS được quản lý

Dịch vụ	Các chỉ số hữu ích
ALB	HTTPCode_Target_5XX_Count, TargetResponseTime, tổng lượng target khỏe mùa (healthy hosts), và mức hao tổn đơn vị LCU
CloudFront	tổng số lượt request, tỉ lệ công lỗi 4xx/5xx, chỉ số trễ origin latency, cùng trữ lượng byte tài liệu tải ra (bytes downloaded)
RDS	dung tổn CPU, quy mô bộ nhớ trống (free storage), trữ lượng kết nối (connections), thông lượng IOPS thao tác đọc/ghi, và nhíp trễ nhân bản cộc replica lag (nếu dùng)
ElastiCache Redis	tiêu chi CPU, lượng tài nguyên RAM memory, lượt dồn bật evictions, đi kèm tổng con số kết nối trực tiếp hiện hữu (current connections)
DynamoDB	quy bạo request dính án chặn throttled requests, lượng đơn vị đọc/ghi tiếp thoát consumed read/write capacity, và hệ thống lỗi cộc system errors
SQS	số tin nhắn nhìn rãnh approximate visible messages, mức chu kỳ tuổi của con tin lâu đời nhất (age of oldest message), cùng số lượng tin ứ tắc thọ ngụ còi DLQ visible messages



Dịch vụ	Các chỉ số hữu ích
Lambda	lượng mã lỗi errors, độ dài thời cầu duration, thông ký bật hãm throttles, mức chu kỳ iterator age, đi kèm cộc lượt khởi thi hành song song concurrent executions
SageMaker	thông lượng lỗi gõ lỗi gọi endpoint invocation errors, chu kỳ trễ cổng model latency, chỉ số trễ mạn overhead latency, cùng mức găm kiệt tải con máy instance utilization

Đề xuất các cảnh báo CloudWatch (Proposed CloudWatch alarms)

Các mô hình cảnh báo dưới đây mang dáng hình lời khuyên nghị. Cần bằng chứng trước khi chốt quyền xuống danh cúng trát là đã vinh quy mùa việc tại cội triển khai production:

Cảnh báo	Điều kiện đề xuất
Tin nhắn lộ vắng cội DLQ	Ghi nhận cộc tham số <code>ApproximateNumberOfMessagesVisible > 0</code> cộ rẽ trên hàng đợi <code>internship-prod-outbox-dlq</code>
Tuổi của hàng đợi chủ lực vọt mức	Chu kỳ thời chao tàn già của con tin nhắn SQS vọt thêu trèo bật sang qua rào cản vận hành quy dúc
Hàm Lambda lâm bãi rách hồng	Ghi nhận chỉ số <code>Errors > 0</code> ngã trúng đầu hàm <code>internship-outbox-handler</code>
Lambda sa án nghẹt trần phanh (throttles)	Thông ký cộc <code>Throttles > 0</code> xuất cội
Tỷ lệ lỗi 5xx trên target ALB phình bạo	Con số lỗi hoặc tỷ lệ 5xx do đích đến target group đội trả ngợp ngoái vượt trần cho phép
Tổn hao cộc CPU trên RDS réo kính	Nhịp chiêu tiêu CPU bắm cộc duy trì đỉnh cao miên man rả rích xô nắc giới hạn
Trữ lượng trống trên RDS sụt giảm (free storage low)	Phần dung lượng đĩa trống an toàn sụt lún chao tụt rớt xa rào bảo vệ
Memory của Redis cao vắng trần	Mức tiêu dùng bộ nhớ hoặc lượng bật tàn evictions phi dộc vọt qua rào nắc định quy
SageMaker thông cáo thảm lỗi	Số lần gõ bật cắt rách suy luận hoặc lỗi cỗ model tựa cộc thuyên quá trần cản rào
Lỗi 5xx cội CDN CloudFront phi mã	Tỉ lệ request sa nhằm trúng cộc 5xx bằng bóng kiếp vọt nắc rào cản cảnh giác

Kết quả mong đợi

- Các cụm Deployment trong lòng Kubernetes reo ca tinh thần Ready trơn chu.
- Bộ trạm tra cứu sức khỏe ung dung thông văn êm ái khi buông lọt vạch CDN CloudFront.
- Kho tàng nhật ký log trơn đầy tự sấm đủ trọn cội cho toàn thể cụm workload trên EKS cũng như cơ quan Lambda.
- Cỗ cổng Prometheus tự do vực thâu, cầu chiết mã mẫn thông quan dải metrics nghiệp vụ ngay khi hạ tầng giám sát được dẹt may hoàn tất.
- Thông số giám sát metrics của muôn kho cộc SQS, Lambda, RDS, Redis, DynamoDB, ALB, CloudFront cùng cỗ cổng SageMaker ngạo mạn giêu hình vắng ngự trên màn trần CloudWatch.
- Các mô hình cảnh báo đề xuất thấy đều khoắc nhãn đặc thắng với thư viện minh chứng, hoặc cam kính an phận liệt mức cộc đề xuất trừ dự nhĩ nhận.

Các lỗi thường gặp

Triệu chứng	Nguyên nhân	Hướng khắc phục
Lệnh thăm cầu <code>kubectl top</code> báo vắng ngát dữ liệu	Cụm máy chủ chuyên thu thập metrics (metrics-server) bốc ngắc chết nghẹt hoặc chưa thâu trát kịp chu kỳ cắt gặt	tra rà ngay tầm cộc cuộn bản phát hành (rollout) cho mạn cụm metrics-server



Triệu chứng	Nguyên nhân	Hướng khắc phục
Mục tiêu đích gặt của Prometheus sa cọc trượt ngã	Vướng án rách chọn lọc nhãn (selector) của ServiceMonitor hay đứt mạch vì lệch không gian tên namespace	Soạn kỹ lại danh sách nhãn label của Service và chuỗi quy nạp bất sóng từ ServiceMonitor selector
Điểm sức khỏe của chat khinh từ khước từ mức ready	Cơ quan liên đới phụ thuộc trùng Redis hay DynamoDB đâm gục té xỉu	Mở toang soát tệp log thuộc dịch vụ chat cùng thông ký trạng thái vận hành trên còi hạ tầng tài nguyên AWS
Nhật ký log Lambda lả bốc mịt m mù	Hàm chưa hề thọ tiếp còi giục cho thực thi hoặc sa bấy gõ rà thăm khu vực region khác	Đối soát lại cấu trúc liên thông sự kiện (event-source mapping) và chỉ số mức khu vực region trời sát cự ly
Hàng đợi thăm khuyết DLQ leo phình vọt tãi	Tiến trình Lambda hay luân mạch phía sau (downstream processing) lâm rào thất bại vỡ mác	Tiến lọt rà xét các tin nhắn hăm ngục còi DLQ kết bầu thăm ngự nhật ký log của Lambda

Kết luận

Kiến trúc giám sát tự trung giỗ kíp đạt tiêu chuẩn chín muồi ngay chạng bộ phận kỹ thuật điều hành hoàn toàn khả dĩ vực thau thanh kiểm tình thể hiện sinh của Kubernetes, tệp nhật ký logs, cung đường thử kiểm sức khỏe, chuỗi metrics cất gặt còi Prometheus lẫn CloudWatch, log của Lambda, thể trận hàng đợi SQS/DLQ cùng các rào cảnh báo báo nguy mĩ mẫn mà chỗi từ triệt để, cấm cho phép bốc tung tuột trượt lộ ra bên ngoài thế giới công cộng bất kỳ thông số mật mã bí mật (secret) nào.

5.10 Kiểm thử đầu cuối

Mục tiêu

Xây dựng và định nghĩa trọn bộ danh sách ca kiểm thử nghiệm thu (acceptance test set) cho hệ thống Internship Application Tracker đã triển khai thực tiễn, đồng thời tường minh ghi nhận các kết quả đã được rà kiểm chứng, các tính năng đã cài đặt nhưng còn chờ minh chứng lúc chạy (runtime evidence), các hạng mục được đề xuất hay những thành phần chưa hoàn thiện.

Bối cảnh kiến trúc

Các kịch bản kiểm thử E2E buộc phải bao quát trọn mạch truy cập công cộng toàn trình:

Trình duyệt (Browser) -> CloudFront -> S3 hoặc ALB -> Các workload EKS -> Các dịch vụ AWS được quản lý

Song song, các luồng kiểm định phải thấu trượt qua dải tác vụ xử lý bất đồng bộ:

Backend -> Bảng PostgreSQL outbox -> Tiến trình dispatcher -> SQS -> Lambda -> Bảng DynamoDB khứ lặp dedupe -> Kho lưu trữ S3 archive -> Dịch vụ mail SES

Định nghĩa các trạng thái kiểm thử

Trạng thái	Ý nghĩa
Đã kiểm chứng (Verified)	Bằng chứng lúc chạy (runtime evidence) trong hồ sơ Tuần 8 hoặc chuỗi tài liệu trích cầu từ mạn production chứng minh thuyết phục thành quả thực thi
Đã triển khai, cần xác minh (Implemented, verification required)	Hồ sơ mã nguồn hoặc hệ manifests đã gài đặt chức năng, nhưng tài liệu minh chứng khi chạy thực tại còi production vẫn còn vắng hụt
Đề xuất (Proposed)	Các kịch bản kiểm thử hoặc giải pháp rào cản mang tính tiền cũ, chưa có minh chứng áp dụng rõ ràng
Chưa hoàn thành (Not completed)	Tính năng chưa được dệt tạo hoặc chưa đạt độ hoàn vãn trong bộ bằng chứng lưu chiếu hiện có

Bảng kiểm thử nghiệm thu (Acceptance test table)

ID	Ca kiểm thử (Test case)	Đầu vào hoặc lệnh kiểm thử	Kết quả mong đợi	Trạng thái bằng chứng
TC01	Kiểm định frontend từ CloudFront	<code>curl -I https://dhm2rz5nmsibj.cloudfront.net/</code>	CloudFront trả về dữ liệu phản hồi tiêu đề của frontend	Đã triển khai, cần xác minh
TC02	Điểm tra sức khỏe backend qua CloudFront	<code>curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready</code>	Trả về JSON sức khỏe backend, xác nhận cọc phụ thuộc PostgreSQL đạt true	Đã triển khai, cần xác minh
TC03	Điểm tra sức khỏe chat qua CloudFront	<code>curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready</code>	Trả về JSON sức khỏe dịch vụ chat, xác nhận hai trụ Redis và DynamoDB đạt true	Đã triển khai, cần xác minh
TC04	Bước gặt liên lạc Socket.IO (handshake)	Trình duyệt hoặc máy khách Socket.IO gõ kết nối vào đường rẽ / socket.io	Kết nối socket cài đặt thuận buồm băng qua ranh rào CloudFront lần cổng ALB	Đã triển khai, cần xác minh
TC05	Đăng ký tài khoản	Nộp mới một thông tin người ứng tuyển (candidate) hoặc quản lý HR	Tài khoản thọ sinh thành tài, hoặc bị từ chối trả về mã HTTP 409 nếu địa chỉ mail dính trùng	Đã triển khai, cần xác minh
TC06	Đăng nhập hệ thống	Dùng chuỗi thông tin đăng ký xác thực	Gặt về thẻ phiên làm việc JWT và điều hướng loạt trúng ranh trang tương ứng quyền hạn	Đã triển khai, cần xác minh
TC07	Khởi tạo công việc tuyển dụng	HR ra tay ban bố vị trí việc làm mới	Hồ sơ tin được lưu trữ kiên trung trong PostgreSQL đi kèm số phiên bản	Đã triển khai, cần xác minh
TC08	Nộp hồ sơ ứng tuyển (Application submission)	Ứng viên gửi đơn xin việc cho một tin tuyển dụng đang mở	Toàn bộ dữ liệu đơn, tệp văn kiện, lịch sử quy trình, sự kiện outbox và tác vụ xử lý hàng đợi (processing job) thấy đều chốt văn trọn trong 1 lượt transaction	Đã triển khai, cần xác minh
TC09	Kiểm soát chống nộp đơn trùng lặp	Thọ nộp đúp 2 lần cùng một vị trí với 1 tài khoản	Bị từ chối gay gắt qua mã 409 cho án nộp đơn trùng, hoặc được hệ thống xử lý bất biến mãi mãi	Đã triển khai, cần xác minh
TC10	Tải lên tài liệu CV	Gửi đơn kèm đệm theo file văn kiện/CV	Tệp nhờ cầu an cư vào lòng S3 (hay kho dịch vụ lưu trữ trở cầu hình), còn chuỗi thông số metadata trú vào lòng PostgreSQL	Đã triển khai, cần xác minh
TC11	Tác tạo hàng đợi công việc xử lý	Kích nổ yêu cầu đọc hiểu CV/job	Nặn ra một dòng thông số mới cất tại bảng <code>async_processing_jobs</code> và trần lộ trạng thái rành mạch	Đã triển khai, cần xác minh



ID	Ca kiểm thử (Test case)	Đầu vào hoặc lệnh kiểm thử	Kết quả mong đợi	Trạng thái bằng chứng
TC12	Xử lý AI hoàn tất	Worker gửi còi gọi tới pod ai-service và trạm cất suy đoán SageMaker endpoint	Dữ liệu CV/job đã giải mãc hoặc kết quả tính điểm độ khớp hoan khải được ghim bảo thọ thù bền bì	Đã triển khai, cần xác minh
TC13	Hội thoại chat giữa 2 phía khách	Buông lời tin nhắn riêng tư hoặc xướng vào cả nhóm đàm	Phía xướng gặt trả lại chứng chỉ phát đi ACK; bên phía nhở thâu lọt tin lập tức trong thì hiện thực (realtime)	Đã triển khai, cần xác minh
TC14	Bền lâu cất dữ liệu hội thoại (Chat persistence)	Re-load nạp lại luôn mạch lịch sử đàm thoại	Cổ dữ liệu DynamoDB ung dung trao trả nội sung các tin nhắn đã ghi trước đó	Đã triển khai, cần xác minh
TC15	Rà tải nặn lại pod chat và thọ tái kết nối	Cho khởi bung gục restart một pod chat giữa chừng khi phiên kết nối đang mở	Socket khôn ngoan tự tái nối tiếp và cự tuyệt hoàn toàn việc bồi chốt tạo lập lại tin nhắn trong cơ quan retry	Đã triển khai, cần xác minh
TC16	Kiểm nghiệm mạch thông còi PostgreSQL	Điểm gọi của backend /health/ready	Gặt về chuỗi postgres: true	Đã triển khai, cần xác minh
TC17	Sức khỏe còi Redis	Điểm rà của chat /health/ready	Gặt về cộc redis: true	Trạng thái cất vãn của Redis thực thể thọ chứng qua minh chứng Tuần 8; trích chuỗi rà từ điểm endpoint tiếp vãng trợ vai hữu hảo
TC18	Tình thể vận hanh các bảng DynamoDB	Dùng lệnh describe-table soi rà chuỗi bảng chat	Hiển tuệ mác ACTIVE đồng loạt cho 3 ranh ChatUsers, ChatGroups, ChatMessages	Đã xác minh trọn từ hồ sơ minh chứng DynamoDB trong Tuần 8
TC19	Bộ chuyển tin sự kiện Outbox dispatcher	Thám xét cộc triển khai dispatcher và đọc kỹ nhật ký log	Trình dispatcher tự chung thọ chiết rỗng bãi tin trong PostgreSQL rồi trút bặt lọt cống qua SQS	Workload vãn gác chứng nhận an cư qua hồ sơ EKS Tuần 8; riêng khâu gửi tin sự kiện khẩn hoan chờ còi xác minh
TC20	Hàng đợi sự kiện SQS	Trích kiểm cộc hàng đợi internship-prod-outbox	Thông điệp vào bãi gặt khóa bảo mật SSE, bầu mốc thời hạn ẩn visibility timeout cùng quy chuẩn lưu lâu dài retention	Hàng đợi chủ lực cùng bản lỗi DLQ thọ vãn chứng từ hồ sơ SQS Tuần 8
TC21	Xử lý trên Lambda	Thả cầu test thử tin lambda-smoke-fixed-1785220478	Lambda gặt thông điệp, thiệp múa mĩ mẫn và gặt xướng kết quả EMAIL_SENT	Hàm thi hành và log group thọ chốt xác tín 1 phần; bằng chứng ra lệnh thi cầu thực thọ hoàn thiện vãn còn trong mạn ráo đón
TC22	Tác vụ báo thư tín qua SES	Sử dụng lại cộc tin khối thử ở kịch bản trước	Thư bồi cảnh báo đắc thắng mang nhãn chốt EMAIL_SENT	Cần xác minh thêm; minh chứng của Tuần 8 chả ráo phô vãi chóp bằng việc thư cầu đi tốt lọt qua SES

ID	Ca kiểm thử (Test case)	Đầu vào hoặc lệnh kiểm thử	Kết quả mong đợi	Trạng thái bằng chứng
TC23	Bãi chôn văn thư sự kiện S3 archive	Tra kiểm cộc định danh archive key	Bồi xác minh sự thọ mang của tệp outbox-archive/2026/07/28/lambda-smoke-fixed-1785220478.json	Cần xác minh; cộc bằng chứng Tuần 8 không ghi mác hiển phối việc sinh ra tệp lưu vào bãi archive này
TC24	Tính bất biến (idempotency) của DynamoDB	Kích cho chạy rà một mã event ID bị cầu giục đúp đề	Tin nhắn trùng lập tức bị cổ rào cản bằng dedupe gạt xô đi nhả nhận	Đã triển khai, cần xác minh
TC25	Nhận thông tin bồi đề trên Lambda	Ấn gửi cầu tiếp tục trùng một con số event ID	Khảng từ khước tiệm tiến nặn bực ra tác vụ phụ thứ hai	Đã triển khai, cần xác minh
TC26	Xử trí bãi thi côi DLQ	Tạo giả lập một ca xử lý lỗi consumer bồi tụ trong không gian kiểm nghiệm an nộ	Thông điệp sau thời kỳ nỗ lực thu hồi (max receive count) khôn ngoan bề cầu lao xuống bãi lỗi internship-prod-outbox-dlq	Đề xuất
TC27	Minh bạch luân điều rẽ trên CloudFront	Thao soát bằng cầu hình cộc distribution behaviors	Rãnh mặc định hướng trúng S3; dải api, /chat, và /socket.io rẽ hướng trúng mạn ALB	Xác tín bán phần dựa vào quan sát bầu theo CloudFront metrics; hồ sơ tải cầu hình behavior hoặc cầu múa trực quan trình duyệt văn khản khoản rước đợi
TC28	Frontend bóc xóa hoàn tàn ranh khỏi EKS	Tra kubectl get deployment frontend -n internship	Hưởng mác Không tìm thấy tài nguyên (Not found)	Bán phần chốt duyệt qua quy hoạch kiến trúc và minh chứng EKS workload; lệnh cầu kiểm trần ra chữ minh văn vẫn luôn sấm vị trí trọng kính

Tập lệnh rà soát thủ công

Lệnh gọi kiểm tra dưới đây đòi hỏi khắt khe tài khoản thực thi phải quy thọ ủy quyền hợp pháp bên trong ranh giới côi production:

```
aws eks update-kubeconfig --name internship-prod --region ap-southeast-1
kubectl get deployments,pods,svc,endpoints,hpa,pdb -n internship -o wide
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

Thao tác kiểm tra cộc dịch vụ mạng bên chiến tuyến AWS:

```
aws dynamodb describe-table --table-name ChatUsers --region ap-southeast-1
aws dynamodb describe-table --table-name ChatGroups --region ap-southeast-1
aws dynamodb describe-table --table-name ChatMessages --region ap-southeast-1
aws elasticache describe-replication-groups --replication-group-id internship-prod-redis
→ --region ap-southeast-1
aws sqs get-queue-url --queue-name internship-prod-outbox --region ap-southeast-1
aws lambda get-function --function-name internship-outbox-handler --region ap-southeast-1
aws sagemaker describe-endpoint --endpoint-name internship-qwen3-4b --region ap-southeast-1
```

Một số mẫu lệnh kiểm thử nhanh

Điểm gọi rà sức khỏe mạn backend:



```
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
```

Điểm gọi rà sức khỏe mạn dịch vụ chat:

```
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

Rà soát tình trạng sẵn gặt của dịch vụ AI thông qua cầu cổng chuyển tiếp (port-forward):

```
kubectl port-forward service/ai-service 18082:8010 -n internship  
curl -fsS http://127.0.0.1:18082/health/ready
```

Tra soát kết quả trên cơ sở dữ liệu

Nghệ múa quy ước căn bãi cấm bức vãi mọi mật mã tài khoản cõi CSDL hay trường tin mang dữ liệu riêng tư của cá nhân ra không gian chung. Ưu việt áp dụng các cánh cổng API nghiệp vụ, nhật ký migration hay bảng số liệu trích tổng quát để thao khảo muốn kịch bản rà tra.

Một bộ chuỗi lệnh tra cứu tho chuẩn an nộ khi thao diễn thẳng trên cộc production:

```
kubectl logs job/backend-migrate -n internship --tail=100  
kubectl logs deployment/backend-processing-worker -n internship --tail=100  
kubectl logs deployment/backend-outbox-dispatcher -n internship --tail=100
```

Tra soát kết quả lưu trữ

Kho hòm lưu trữ của frontend:

```
aws s3 ls s3://internship-prod-frontend-<AWS_ACCOUNT_ID>/ --region ap-southeast-1
```

Bãi an ngụ văn thư sự kiện lâu năm:

```
aws s3 ls s3://internship-prod-uploads-<AWS_ACCOUNT_ID>/outbox-archive/2026/07/28/ --region  
→ ap-southeast-1
```

Tra soát nhật ký CloudWatch Logs

Nhật ký của hàm Lambda:

```
aws logs tail /aws/lambda/internship-outbox-handler --since 1h --region ap-southeast-1
```

Chuỗi danh sách định danh cộc CloudWatch log groups gắn đệm cùng các workload trên EKS khẩn nài mình chứng từ kịch bản thi gom nhật ký (log shipping configuration) trước khi hoan khảo liệt rãnh là đã vinh vãng cài đặt.

Kết quả mong đợi

Toàn bãi nền tảng ứng dụng nghiêng còi ăn trát qua khâu nghiệm thu múa diễn (demonstration) ngay thêm khi muốn cộc khảo sát sức khỏe công cộng trọng yếu bão khảo thông, khối workload bên cõi EKS hô hào trạng thái Ready, dàn tài nguyên hạ tầng DynamoDB/Redis/SQS/Lambda/SageMaker rạng ngời available, đi kèm dải mình chứng trút ra thực thụ cho kịch bản khỏi Lambda bao vây trọn khâu tiếp xử sự kiện, nén tệp archive và mác cất thành bại gửi mail.

Các lỗi thường gặp

Triệu chứng	Nguyên nhân khả nghi	Hướng khắc phục
Rà sức khỏe bỗng mượt thông qua cổng ALB nhưng lại chọc rách hồng lật qua CDN CloudFront	Cấu hồng trật quy bạo trong behavior của CloudFront hoặc khai nhằm origin	Vào tra kỹ lưỡng danh sách cộc cấu hình behavior thuộc mạn distribution của CloudFront
Kịch bản kiểm rà chat giầy khốc sau kì thao diễn bồi thêm số pod (scaling)	Trục chao rãnh Redis adapter hay rào bảo bám stickiness bên target group của ALB trượt cộc	Xác tín tho sinh vững chắc của Redis cũng như nút gạt stickiness tại target group
Ứng viên nộp 1 đơn xin việc vô trát để liền 2 bản ghi	Thiếu hụt trường unique constraint tại cơ quan CSDL hoặc lọt bug ở thuật toán tính bất biến	Cho thiếu trợ chạy hệ bài test bất biến mạn backend và rà xem kỹ lưỡng các ràng buộc cộc CSDL



Triệu chứng	Nguyên nhân khả nghi	Hướng khắc phục
Tác vụ nằm trong hàng đợi processing gầm lì bế kẹt chờ queued	Cụm worker gục khóa tịt (disabled) hay trạm cất SageMaker chẳng mím cười sẵn sàng	Tra thông số cờ biến PROCESSING_ WORKER_ENABLED, trạng thái cất pod ai-service cùng chờ đợi của endpoint
Bài kiểm test khối cho Lambda chửi rửa do vô tình phạm chuỗi từ khóa bị cấm (reserved word) của DynamoDB	Dùng cọc lóc chuỗi thẳng chữ result khi nặn hàm UpdateExpression	Tự trọng ứng chiêu trở #result phối hồ rẽ qua bản quy quy cho trường ExpressionAttributeNames

Kết luận

Kế hoạch rà nghiệm thu E2E tuân thủ chánh đạo trọng thị minh chứng thực thọ. Khung bạt này ráo riết từ bề
nhĩ ngọ thuyền mác “đã pass” cho những ca kiểm nghiệm chỉ đơn giản vì nhìn cầu thấy đoạn mã tồn tại, mà nghiêm
trung chốt hạ rõ ranh những nhịp kiểm tra khi chạy nào đang bốc hồ đợi cầu rào bằng chứng thọ xuất thực bãi.

5.11 Bảo mật và Chi phí

Mục tiêu

Tài liệu hóa chi tiết toàn bộ các biện pháp kiểm soát bảo mật và phương pháp tiếp cận ước toán chi phí dành
cho hạ tầng AWS đã được bủa giăng ra môi trường production thực tế.

Bối cảnh kiến trúc

Nền tảng ứng dụng gánh tải xử lý và thuyền rẽ muôn thông tin cốt lõi bao gồm tài khoản người dùng, tài liệu
CV, đơn ứng tuyển, thư tín đàm thoại chat, luồng phân tích suy luận AI, báo còi thông cảm sự kiện, cùng chuỗi thư
tín email. Hệ biện pháp rào chắn an nộ bảo mật bị ép buộc gánh sứ mệnh ôm bể và che chắn trọn dải cho danh xưng
xác thực, các thông số mật mã bí mật (secrets), dữ liệu riêng tư, quyền vai trò bầu trong chu trình triển khai, kho
cất trữ, những vạch cầu định tuyến mạng và khâu múa rẽ xử trí tin sự kiện. Trong một thể trận giáp kiểm, bài toán
kìm giữ chi phí bất buông dốc tâm lực trông canh gắt gao nhĩ đàn hạ tầng thọ sinh thói quen ngụ trợ 24/7 không hở
trần như: cụm máy EKS, trạm giao cầu NAT Gateway, kho RDS PostgreSQL, cụm ElastiCache Redis, bộ rẽ ALB,
đi cùng cỗ máy suy luận AI SageMaker.

Các biện pháp kiểm soát bảo mật

Khu vực	Biện pháp yêu cầu hoặc đã triển khai	Trạng thái
GitHub OIDC	Tiến trình GitHub Actions thông quan hoán quyền bầu vào AWS role thông qua giao thức OIDC với chuỗi tham cờ id- token: write	Đã triển khai trong workflow
Khóa truy nhập AWS kiên bám dài hạn (Long-lived AWS keys)	Tuyệt đối chối bề việc găm giam các chìa khóa truy cứu quyền hạn AWS vô hạn kỳ bên trong còi GitHub hay trong hồ sơ mã nguồn	Biện pháp bắt buộc (Required control)
Quyền hạn IAM tối thiểu (IAM least privilege)	Dải phân vai cho triển khai, thời gian chạy runtime, các con node, SageMaker, Lambda hay trình quản trị ALB cần thi nặn ở mức tối giản theo thực tế thao tác	Đã áp dụng ở cấp độ kiến trúc; cọc minh chứng tài liệu chính sách policy vẫn ráo đội
IRSA	ServiceAccount của Kubernetes internship-app có khả năng được gán nhãn annotation với vai trò runtime IAM	Đã triển khai trong tập lệnh script
Kubernetes secrets	Khóa cọc SECRET_KEY, DATABASE_URL, REDIS_URL, OUTBOX_QUEUE_URL, và (tự do tùy nhĩ) khóa AI API key thầy đều được bơm tiêm văng trú vào trong internship-secrets	Đã triển khai trong kịch bản script
Kho lưu S3 riêng tư cho frontend (S3 private frontend bucket)	Khóa chốt phong tỏa mọi ý định trần lộ cho thể giới public mở vào bucket, thi chuyển nạp theo cọc quyền CloudFront OAC	Đã triển khai trong ensure- cloudfront.sh; cọc chứng cứ thao thầu runtime ráo đón nạp bởi



Khu vực	Biện pháp yêu cầu hoặc đã triển khai	Trạng thái
Giao thức HTTPS trên CloudFront	Cổng mạng CloudFront chủ động ra trát ép điều khiển lượt tra cập của cộng đồng phải qua ranhHTTPS an nộ	Đã triển khai trong cấu hình phụ trợ
Kiểm soát tuyến đường ALB	ALB buông rào cản ngăn kiên định, thợ cho lọt côi đi vào cụm backend/chat đối với đúng nhĩ 3 tuyến đường /api, /chat, /socket.io	Đã triển khai trong manifest Ingress
Truy cập kín côi cơ sở dữ liệu RDS	Khẩn hoan cất vây nhốt PostgreSQL khỏi mọi nhíp nhòm ngó hay châm trích thẳng từ côi public Internet	Cần bằng chứng
Truy cập cất rào cho Redis	Kho ElastiCache Redis thẻ khước cự bãi trước mọi rãnh chui vãng công khai từ Internet	Cần bằng chứng
Mã hóa dữ liệu SSE cho SQS	Hàng đợi chủ lực lẫn con bãi lỗi DLQ nghiêm gạt công tắc mã hóa bật theo tiêu chuẩn máy chủ SSE	Đã xác minh qua hồ sơ chứng cứ Tuần 8
Mã hóa dữ liệu DynamoDB	Các bản cất DynamoDB ngạo bật chế độ mã hóa tự động do bộ AWS quản lý (AWS-managed encryption) từ lúc bực sinh trừ phi chủ rạch xoay ngang thì nặn kiểu khác	Cần bằng chứng rà soát lại thông số cộc thiết lập sau cùng của các bản
Mã hóa côi RDS (RDS encryption)	Bắt nhĩ thâm cẩu đối rà bằng minh văn trích qua câu lệnh describe-db-instances	Cần bằng chứng
Mã hóa đường truyền (Encryption in transit)	Luồn HTTPS trót lọt qua CloudFront; tuy nhĩ thông số bầu rào cho giao thức an ninh TLS chốn các vi dịch vụ và trạm cơ sở dữ liệu nội bộ thì ra lời nài bằng chứng	Đã chứng minh một phần
Quản lý và cách ly bí mật (Secret handling)	Nghiêm cầm gờ lệnh commit hảo đưa bất kỳ file .env, mã xác thực token, chìa khóa, mật khẩu hay đường dẫn dữ liệu DB lên repository	Biện pháp bắt buộc
Hàng đợi thư hỏng DLQ	Hạ tầng SQS DLQ khôn ngoan tựợ chốt gom nhốt riêng mác các tin nhắn bị thợ consumer xô thất bại liên miên	Đã xác minh qua hồ sơ chứng cứ Tuần 8
Tính bất biến (Idempotency)	Khẩu chốt bất biến tại PostgreSQL, bảng outbox, nhĩ cờ clientMessageId từ mạn chat cùng hào công khứ lập DynamoDB từ phía Lambda	Đã triển khai; riêng luồng rà test khối Lambda hoan thợ xác minh
Sao lưu và điểm khôi phục PITR (Backup and PITR)	Lịch ghi cộ sao lưu RDS và nút bật khôi phục PITR côi DynamoDB gờ nài khẩn thiết dải bằng chứng xuất trát cụ thể	Cần bằng chứng
Gia cố rào cản cấm chui thẳng bãi ALB	Kịch bản tự nguyện kêu gọi việc trói nghệt bãi ALB origin sao cho thợ nhĩ thấu lời gọi duy nhất từ đằng CDN CloudFront	Đề xuất gia cố (Optional hardening)
Tường lửa AWS WAF	Trù cất chiêu huy động thêm AWS WAF án cũ ngay trước cánh CloudFront mang lại lớp rào bảo an mĩ mãn cho côi production	Đề xuất gia cố

Xác thực và phân quyền (Authentication and authorization)

Backend khai thác nền tảng xác thực tựa theo chứng chỉ thể JWT (JWT-based authentication) đi liền rềm khắt khe phân mác định tuyến dựa vào vai trò, dệt thành hào lũy rào cản phân minh rõ chuỗi quy trình thao tác giữa ứng viên (candidate) và bộ phận nhân sự (HR). Bên mạn giao diện frontend, phiên tiếp vãng của khách thợ an ngụ cần khôn trong bộ nhớ tại chỗ với phần đầu nhân là chuỗi ký tự internship_tracker_ và khôn khéo ra lệnh trút dep



tiết mọi thông tin trong kho lưu session/local storage ngay thì khoảng mác khi chốt phiên chờ rừng còi mãn hạn hết hiệu lực. Cụm các trạm bảo an điều hướng trên mạn frontend chủ trách tụt bể rẽ con đường lọt về cho người dùng đúng theo quy cộ quyền hạn họ thụ hưởng.

Kỳ vọng bắt buộc về an nộ bảo mật:

- Mật khẩu truy cập thi thoảng tuyệt đối phải được chuyển hóa thành bảng mã hàm băm (hashed).
- Khóa bí mật JWT secret khẩn nài ngụ an tịnh trong lòng hòm cơ quan quản trị bí mật, cấm tiết sa ngã rơi rổng vào bãi mã nguồn repository.
- Các tuyến giao tiếp cho ứng viên lẫn HR ở chiến tuyến backend buộc thì trát khắt khe đối rà chủ sở hữu dữ liệu đi đôi cộ thẩm sát vai trò quyền hạn.
- Những điểm số do cơ quan trí tuệ AI suy bật thì ra ranh mục chỉ sấm vóc đáng lời khuyến cẩu, cự tuyệt việc bầu bảo coi đó làm cơ quan định trát phán xét quyết định duy nhất trong tiến trình thọ nạp tuyển dụng personnel.

Bảo vệ dữ liệu

Loại dữ liệu (Data type)	Yêu cầu bảo vệ
CV và hồ sơ văn kiện ứng tuyển	Cất nạp bên trong lòng S3 riêng tư, thẩm định danh tính ra vào ráo riết, khước nhục mọi cơ quan in log bung trọn vẹn văn bản raw thô
Hồ sơ ứng tuyển (Applications)	Ghi nhận chễm trệ trong cối PostgreSQL có áp chuỗi rà soát chủ quyền tài nguyên khắt khe
Tin nhắn đàm thoại chat	Gạt nạp trọn trong DynamoDB, trao ranh rào thẩm tra visa thông văn cho chính vi dịch vụ chat service can thiệp
Thông tin sự kiện outbox (Outbox events)	Cẩn mật né hẫng việc nhả nhật ký log vãi lộ trường payload chứa tin nhạy cảm ở thì hiện đại và tương lai; trữ lưu bãi archive đúng trần văn thư tối thiểu cần cho nghiệp vụ
Các câu lệnh cầu rào gửi cho AI (AI prompts)	Nghiêm răn không gõ log phô bày trọn nội sung CV của con người; bộ đếm trung gian AI chỉ bọc rẽ in ra chữ ký hàm băm (hash) và con số đếm lượng ký tự trong chuỗi prompt
Bí mật (Secrets)	Giám yên tịnh trong hòm bí ẩn GitHub secrets phối hợp cộ Kubernetes Secret; vĩnh viễn bề bãi không vãi phơi xuất ra văn kiện bao giờ

Lệnh kiểm tra bảo mật

```
aws iam get-role --role-name internship-github-deploy
aws iam get-role --role-name internship-eks-runtime
kubectl get serviceaccount internship-app -n internship -o yaml
kubectl describe secret internship-secrets -n internship
aws s3api get-public-access-block --bucket <FRONTEND_BUCKET_NAME>
aws sqs get-queue-attributes --queue-url <OUTBOX_QUEUE_URL> --attribute-names All --region
→ ap-southeast-1
aws rds describe-db-instances --db-instance-identifier internship-prod-postgres --region
→ ap-southeast-1
```

Tôi xin thể kiên trinh chối từ, không nhả nhận in thọ hay gõ sao chép trút đổ các chuỗi giá trị mật mã secret tự trong Kubernetes, mạn GitHub hay hạ tầng AWS trườn ngã lộ ra ngoài các mặt giấy trong hồ sơ tài liệu của báo cáo này.

Nguồn bằng chứng về chi phí

Tôi xin khai trình các thông số hao tổn ngân sách dựa vào tệp minh chứng chi phí AWS thu từ chặng Tuần 8. Kho bằng chứng này chụp vãng thông số hao tổn chi trả thực ghi từ nhíp ngày 1 đến ngày 28 tháng 7 năm 2026, đi kèm theo một bức ảnh chụp màn hình ghi chép kho tín dụng Billing and Cost Management credits. Hóa đơn chốt chu kỳ hết tháng hoàn thiện đi kèm bản dự toán chính khôn qua cỗ máy tính cước AWS Pricing Calculator vẫn đang trong bãi kiên khôn nài đợi nạp bồi.



Bảng chứng chi phí	Giá trị ghi nhận	Diễn giải
Tổng chi phí từ ngày 1 đến 28/07/2026	Hao cước tính gồ dồn tới chặng hiện tại	\$94.92 Giá trị bầu rào lấy ra làm sự thật theo minh chứng tài lộc Tuần 8
Mốc hao tổn cao vãng cộc đỉnh nhất trong một ngày	Ghi nhận ở mốc ngày 28/07/2026	\$31.83 Con số ngón cước nặng gánh cao nắc nhất trong mạn khảo sát 1-28 tháng 7
Tổng ngân khoản tín dụng đã hao (Credits total amount used)	Thuộc quỹ Billing credits	\$27.90 Soi gặt từ mạn chụp ảnh tín dụng Billing
Ước lượng ngân khoản tín dụng tiêu (Credits total estimated amount used)	Thuộc quỹ Billing credits	\$140.65 Soi gặt từ mạn chụp ảnh tín dụng Billing
Số dư khoản tín dụng thực bám lại (Credits total amount remaining)	Thuộc quỹ Billing credits	\$172.10 Soi gặt từ mạn chụp ảnh tín dụng Billing
Ước tính phần tín dụng còn vãn bám (Credits total estimated amount remaining)	Thuộc quỹ Billing credits	\$59.35 Soi gặt từ mạn chụp ảnh tín dụng Billing

Toàn dải con số ở trên chỉ vậy quy bọc trọn trong mốc từ ngày 1 tới 28 tháng 7. Tôi kiên trung chối từ việc khoắc nhần gán vãng coi những chữ số này chính là cộc hóa đơn tháng chính quy hoàn tất, hoặc giả xem chúng là dự toán ổn thọ miên mạn lâu dài cho hạ tầng production, cho đến mốc chặng có rào đệm bằng chứng hóa đơn tháng AWS Bills kiên cố kèm côi bảng tính AWS Pricing Calculator nạp vào đây.

Chi phí thực tế các dịch vụ trong Tuần 8

Bản tổng trút chi tiêu thuộc Tuần 8 vạch ro cho công chúng thấu hiểu chân dung các tay hao tiêu bạo cước chủ lực ngự trị trong chặng từ 1 đến 28 tháng 7, năm 2026.

Thứ hạng	Dịch vụ	Chi phí từ 1-28/7	Tỷ trọng trên tổng chi phí
1	Amazon RDS	\$29.69	31.3%
2	Amazon SageMaker	\$23.45	24.7%
3	Amazon VPC	\$14.11	14.9%
4	Amazon EC2 - Compute	\$12.42	13.1%
5	EC2 - Other	\$7.68	8.1%
6	Amazon EKS	\$5.64	5.9%

Lực lượng lực hộ tài nguyên (6 dịch vụ) kể trên ra thể nuốt nhĩ trọn vẹn khoảng 98% trên tổng quy bảo ngân lộc báo cáo từ 1-28 tháng 7. Hiện tượng đỉnh cước vọt mã dị thường ngắt ngưỡng trong ngày 28 tháng 7 là hồi chuông rung gõ nài nhắc nhở: khâu bổ nhĩ gia tăng cơ sở hạ tầng mạn production, giờ mở máy vãng bám không ngớt từ cỗ công SageMaker endpoint, đi chung chuỗi cước hao chuyển từ hệ thống mạng VPC cùng NAT Gateway thiên thọ đặt dưới nắc chốt theo dõi cảnh giác cao trào tối thượng.

Các giả định khi tính toán chi phí

Dự tính toán thuyên chính trát minh vãn cho ngân khoản hàng tháng trọn chu kỳ ở chặng tương lai vãn kiên quyết đòi hỏi phải cầu bồi cỗ máy AWS Pricing Calculator, hay một mốc thời cẩu giám trát mẫu mẫn mang tính đại diện qua Cost Explorer. Thể cấu trát tài nguyên hạ tầng thực thụ được ngắm rà ghi cẩu bên trong rèm thư mục minh chứng cho thấy thói quen bám ngụ như sau:

Trình tạo chi phí (Cost driver)	Cấu hình quan sát được	Dữ liệu đầu vào cần thiết cho dự toán tháng
Cụm máy chủ EKS (EKS control plane)	Một cụm mua việc mang ranh production tọa trú chốn vùng ap-southeast-1	Tổng con số giờ chạy thợ mua cụm (cluster runtime hours) đi chung chuỗi giá niêm yết theo giờ của EKS ở mạn khu vực sở tại
Cụm trạm node công nhân (EC2 worker nodes)	Hai node Kubernetes công nhân vinh thợ ngả ở tư thể Ready mua việc	Thông tham số cấu trúc phân hạng EC2 (instance type), dung lượng ổ EBS và con số giờ chạy nhĩ thợ của node
Máy chủ RDS PostgreSQL	Hai cọc máy chủ RDS PostgreSQL riêng tư, đã bật rào mã hóa mang phân hạng db.t4g.micro, dung lượng 20 GiB cho từng instance, trần thiết cọc Single-AZ	Ngân khoản đệm ổ sao lưu (backup storage), lượng lượt truy xuất thêm I/O và số giờ chạy điển hình thợ thực cầu
ElastiCache / Valkey (Redis)	Một cụm nhân bản replication group thợ nhĩ trạng thái available, trang bị đủ cờ mã hóa dữ liệu tĩnh và đường truyền	Loại hạng máy chủ (node type), con số cụm node gõ mở, cùng trữ lượng truyền tải giao tiếp qua lại (data transfer)
Cổng cân bằng tải ALB (Application Load Balancer)	Một thực thể ALB hoạt động rạng rõ ngoảnh mặt ra hướng Internet (internet-facing)	Con số giờ duy trì chạy của ALB kết hợp chỉ số ngôn lượng đơn vị LCU (LCU usage)
Cổng mạng CDN CloudFront	Mình chứng cho distribution phụ trách web và distribution cất cọc cho bản báo cáo	Trữ lượng request cầu tối, thể tích byte ném xuất xa ra ngoài, tần suất lệnh gõ cỗi invalidation cùng bản ghi cấu hình behavior exported
Kho S3	Ảnh minh chứng cho chuỗi đối tượng cỗi frontend bucket, đi với bức tranh vây quanh cụm upload, archive và bucket lưu báo cáo	Số dung lượng cất GB, khối lượng thao tác lệnh GET/PUT và điều luật vòng đời lưu chiều lifecycle policy
Cổ lưu DynamoDB	Cụm bảng chat cùng ranh cọc bảng khử lập cho Lambda thấy đều ngoan cô thi theo rãnh trả cước theo từng lượt sử dụng thực tế (on-demand billing) theo ảnh chụp Tuần 8	Sản lượng thông lượng cầu gõ request đọc/ghi, dung lượng đĩa cất và cơ chế điểm khôi phục PITR (nếu ráo bật)
Hàng đợi SQS	Cổng đợi chính và rãnh chứa lỗi DLQ ghi trát với con số 0 tin nhắn ử tắc lộ mạn trong ảnh chụp chứng cứ Tuần 8	Quy mô thông số request kêu gọi và dung lượng byte bình quân cho một con tin nhắn (payload size)
Trạng thái thợ hàm Lambda	Trình cấm outbox handler bạo phơi nhân Active, quy hoạch 256 MB RAM, gạt nút ngắt chờ sau 20 giây (timeout)	Tổng con số lượt kích hoạt (invocations), chu kỳ thời gian gặm xử lý trung bình (average duration), số lần báo lỗi hay nạp mua thử lại
Bộ cổng SagMaker	Điểm phơi mở suy bật theo thì hiện đại endpoint kiên bảo mác InService với 1 production variant chạy kề	Loại hình cấu trúc EC2 cho endpoint (instance type), số giờ gác chạy thợ bãi, và khối lượng dung lượng byte phải giải mác xử lý
Quán cọc CloudWatch	Hệ log và chuỗi metrics được vỗ tay giục bật tự động qua mạn AWS và cỗ máy Kubernetes cluster	Trữ lượng log nuốt dồn vào, giới hạn quy bật bảo lưu retention, dải metrics do dev cất gặt và con số kịch bản cảnh báo alarms
Khối truyền tải giao thông (Data transfer)	Hồ sơ Cost Explorer rền vang tố cáo chao lệch chi phí cho giao thông truyền dữ liệu trong chặng theo dõi	Lưu lượng xuất ngoài cỗi CloudFront (egress), giao thông cuộn vĩnh qua ALB, thau dọn cầu luồng bên trong cỗi NAT Gateway và chi cước chạy trú ngang qua các AZ chéo nhau

Bảng ước tính chi phí



Dịch vụ	Phương pháp ước tính	Trạng thái bằng chứng hiện tại
Amazon EKS	Giá cho cộc cụm theo giờ thì nhân thẳng với con số giờ khởi chạy thực	Đã triển khai; bản kết trút hao cước Tuần 8 thợ báo giá trị \$5.64
Các node EC2 của cụm EKS	Giá EC2 chạy theo giờ bồi gấp đôi cho 2 node, nạp gộp khoản chi bồi cho dung lượng EBS	Các node được rà cộc minh bạch; nhánh EC2 Compute xướng cước \$12.42, nhánh EC2 - Other gọi tên \$7.68
Amazon RDS PostgreSQL	Giá cầu theo giờ của cộc db . t4g . micro, phí ổ cứng 20 GiB ứng với từng máy chủ, bồi kèm tiền cho backup và I/O	Đã kiểm chứng rành 2 instance bảo mật đóng vùng riêng tư; bảng tổng chi cước Tuần 8 báo cáo ngón \$29.69
Amazon ElastiCache / Valkey	Giá máy cộc node theo giờ nhân với số lượng node, trát gộp cước chi phí lưu thông mạng data transfer	Nhóm replication group đã qua thẩm thấu rà cộc; phân loại máy chủ cùng con số node thực tế thợ khẩn nài tra chốt
Application Load Balancer	Phí thợ vận cộc theo giờ cho ALB, bồi vậy khoản cho số đơn vị hao tổn LCU	Đã chứng nghiệm tình trạng Active cho ALB; dải rà soát an nộ từ target health vẫn ngập đợi cầu ảnh rào khỏe mạnh cuối cùng
Amazon CloudFront	Tổng sản lượng các lượt requests, trát bồi cước xuất ngoại dung lượng byte data transfer out và số kỳ invalidations	Đã bồi nạp sẵn ảnh chụp giám sát monitoring; chuỗi xuất bãi minh văn cầu hình cho behaviors và origin vẫn ngập đợi gọi nạp
Amazon S3	Trữ lượng lưu đĩa GB bồi kèm nhĩ con số lượng gõ lệnh request và khâu chuyển trượt cộc lifecycle	Sở hữu tự nhiên bức ảnh trích kiểm chứng kho danh sách đối tượng bên trong lòng frontend bucket
Amazon DynamoDB	Hóa đơn tính cầu theo lưu lượng lệnh gọi thực tiến on-demand read/write phối cộc cước lưu đĩa	Có sẵn hồ sơ chụp văn trần hiển lộ trạng thái an sinh của cụm các bản
Amazon SQS	Đơn cước đếm theo lượng request vãng còi hàng đợi chuẩn (Standard) và quy mô kích thước tệp payload tin nhắn	Thợ sấm trọn hồ sơ chụp trạng thái múa của hai con hàng đợi chủ lực và DLQ
AWS Lambda	Số lần gõ kích gọi công request thợ tiệm bồi cước gõ theo cộc dung sai đơn vị GB-giây (GB-seconds)	Con hàm trở hình thợ sinh thật; tuy vậy số lần giục cho múa và tình trạng kích nổ thau chu vẫn trong tư thế chờ thẩm định
Amazon SES	Lương số lượng thư từ phát gõ đi kèm (nếu vác thêm) dung sai khối văn thư đính kèm attachments	Dịch vụ được pháo tạc lên quy hoạch trong chặng phát ra nhĩ còi tin cảnh báo theo đường rãnh từ Lambda
Amazon ECR	Trữ lượng dung sai lưu trữ các image (GB) và khoản cước bồi chuyển tải dữ liệu data transfer (nếu có)	Chưa cầu phân mức riêng tư trong bảng tổng rà hao phí thuộc đàn thủ lĩnh Tuần 8
Amazon SageMaker	Giá máy cộc EC2 thợ chạy công endpoint theo giờ, gộp cùng cước theo lượng lần kích nổ suy đoán/dung lượng xử lý	Điểm endpoint kiên cộc ngừng rạng trạng thái InService; báo cáo chi phí Tuần 8 thợ hô chuông cớ cộc \$23.45
Amazon CloudWatch	Cước nuốt log vào còi (log ingestion), khoản thu bảo lưu retention, lượng metrics tự định nghĩa và dàn cảnh báo alarms	Số liên nấc giới hạn bảo lưu retention và cộc thông lượng nạp tệp vào log vẫn ra lời gọi nài bổ nạp
Amazon VPC và phí truyền dữ liệu (data transfer)	Chi phí cho mạng xuất còi từ CloudFront, nhĩ luồng lưu thông thợ cầu ALB, máy cổng NAT, và giao thông vãng rẽ qua nhĩ chéo AZ	Trát cước Tuần 8 gọi thẳng tên Amazon VPC xướng giá \$14.11; cộc bảng chi tiết bẽ rã rãnh từng luồng giao thông mạng vẫn ngập đợi
Tổng cộng (Total)	Khẩn cầu cỗ máy Pricing Calculator dựng nên kịch bản báo giá, từ đó thâu rẽ đem đo độ với một tệp minh chứng bill có độ cần mật mẫu mẫn	Mãn chu kỳ khảo soát chi phí Tuần 8 từ ngày 1-28 tháng 7 báo ngón \$94.92; hóa đơn chuế kỳ hoàn tàn hết tháng đi với bản báo giá steady-state tiếp tục nài đợi nạp bồi

Vệ sinh tài liệu bằng chứng (Evidence hygiene)

Trước thời điểm ra quyết cộc đem bất cứ văn ảnh chụp mạn hình nào còi AWS, hay chép đệm một luân nhật

ký log nào bởi thẳng vào lòng báo cáo này, tôi luôn giữ một nguyên tắc sắt đá tuyệt trần: bóp gi giếm ẩn trót lọt trọn vẹn số định danh tài khoản AWS (account ID), chuỗi chìa khóa truy nhập access keys, secret keys, mật khẩu vãng cội cơ sở dữ liệu, con đường kết nối connection strings, thông ký thọ nằm chốn Kubernetes Secret, mã thẻ thông quan GitHub token, đường link chia cầu chuỗi ký tạm (presigned URLs), và toàn bãi bất cứ cội tệp log nào bởi mang công cộc thông ký bảo mật.

Cuộc tổng thám sát soát cầu kho bãi bằng chứng của tôi tuy tự hào không bao tuột lộ ranh giới cộc chìa khóa access keys hiển trần hay token từ GitHub nào ra văn bản sa mác thô hồng, song chính trong ranh đường dẫn tệp 02-eks/pod-logs-tail.txt, tôi bởi thót tim tóm trúng cộc một chuỗi kết nối Redis công trọn theo thông tin xác thực nằm sa gục bên giữa dòng nhật ký lúc khởi chạy (startup logs) của chat-service. Để cự tuyệt án phạm rào cản an ninh, tôi kiêu dũng chối cự việc sao phó nạp tuệ nguyên bản tệp thô thăm rách đó lên. Tôi chỉ đệm vãi đưa ranh giới câu rẽ tường trình tổng quan, hay nhả nhận hoán gồ lặt nhĩ thể chối toàn văn cộc nhảy cảm đó thành từ khóa an sinh trung lập <REDACTED_REDIS_CONNECTION_STRING>.

Các hạng mục hao phí tài nguyên lớn nhất

Cơ quan cắt trợ hàm chứa nguy cơ gieo bão ngón gặm chi phí cao nắc nhất trong chặng đường tiếp chặng là:

- Số giờ mở máy thọ múa không nghỉ trợ cội SagMaker real-time endpoint, vô cùng kịch khốc nếu cộc EC2 gán nuôi trạm endpoint sa vào loại máy chuyên dùng năng lực GPU.
- Cụm điều phối EKS (control plane) và lực lượng 2 trạm EC2 worker nodes nạp mình chạy liên tục ngày đêm cội 24/7.
- Năng lực phục vụ công hiến lưu trữ thường trực được cung phụng từ hai tay RDS và ElastiCache Redis.
- Cước thu chi phí theo giờ cho ALB kết cộc cùng khối hao tổn cộc lượng đếm đơn vị LCU.
- Hao phí duy trì CDN CloudFront đi với chi cước bằng thông đường truyền (data transfer) một khi lưu lượng cộng đồng kéo theo bọt lớn dốc ngược lên.
- Phí tổn nuốt log vào kho CloudWatch và phí chôn bảo lưu retention nếu như hệ luôn ráo lệnh ép bật cơ chế ghi nhật ký siêu tỷ mẩn (verbose logs).

Khối chứng cứ lộc tài từ Tuần 8 phán bão gắt gao rằng chi cước thực tế không còn trườn bò xấp xỉ mức con số không ngon ơ nữa. Cụm SageMaker, cơ sở dữ liệu RDS, nhịp cất luồng lưu thông qua mạng VPC/NAT, lực lượng EC2 cho worker node và ranh cụm EKS đương nhiên buộc thọ coi như dàn nguyên nhân hao cước chủ lực múa gánh tải cội trần. Nhất tâm ghi mác nhĩ kịch bản bấm lệnh cất còi cho tắt vãn, hoặc cầu lệnh chôn xóa sạch SageMaker endpoint ngay sau nhịp dân demo là điều vô cùng tối trọng chừng nào tác nghiệp suy bại AI realtime không còn bị nài cầu cho thao thọ thực tế.

Các giải pháp tối ưu chi phí

Giải pháp	Hiệu quả
Chôn xóa hẳn hoặc bởi cài còi đặt lịch auto thì rác cho SagMaker endpoint chặng khi idle thành thơi nhàn rồi	Phá vỡ bề xô thành công thủ lĩnh số 1, ngón hao chi phí kinh bão bạo nhất thuộc mạn cội dịch vụ AI
Gật nhĩ công tắc khóa kiểm nhôt im processing worker (disabled) bất cứ khúc khi luồng test AI chả kích cho chạy	Triệt rào ngón phí vô ngã do khâu tự thọ cầu múa tự gồ thử lại retry nhằm cội và cất xô giờ thì cho thọ worker
Tinh chỉnh lựa đúng thông cộc cội (Right-size) cho node group trên EKS	Bồi dân nhả thâm chi phí trút vào máy cội EC2 cùng mạn dung sai đĩa cứng EBS
Nháo ép buông dân bớt (Scale down) lực lượng các workload phi-production sau thì khoảng mác gồ thì thọ demo	Thu gặt, xóa cất chi trả không đáng có cho đám tài nguyên máy tính trợ ngụ 24/7 vô công rách thọ
Khảo rà nghiêm mẩn tính cấp khẩn của cội cổng NAT Gateway	Chặn tiết khoản thu khổng lồ trả theo chuỗi giờ chạy NAT và chi phí tính mác trên từng GB dữ liệu thọ bại qua nó chừng nào mạn riêng tư cấm thọ cầu chui xuất Internet
Ứng chiêu nạp bổ sung các VPC endpoint ở chốn cung mạn trúng đích thiết thực	Hại thì trút nhả khoản phí cho khâu dữ liệu luân cầu chuyển rẽ qua trạm NAT đối với mạch giao thông chui sang các hạ tầng cội AWS
Áp luật vòng đời lưu chiếu cho S3 (S3 lifecycle rules)	Ra chiêu sa trượt cộc các tệp tải cầu hay văn thư archive lâu năm bể lui xuống kho rẽ thọ chi hơn, hay lệnh đập tắt bãi rách xác test
Ban chính sách vòng đời cho kho container ECR (ECR lifecycle policy)	Ra lệnh quấy xao tự động tẩy bề cạn kiệt dải image mang chữ ký SHA cũ rách sau khi mẩn kỳ bảo an
Giới rào quy bại thời gian bảo lưu (retention) cội CloudWatch	Xóa tan bãi thăm rách duy trì lưu trữ lâu nắc cạn thể kiếp cho một khối log bão tấp

Giải pháp	Hiệu quả
Áp kích bản tính cước theo thực dùng (on-demand) cho DynamoDB chặn giai đoạn lưu lượng cộng đồng bất bạo gập ghenh	Ngăn chừa kích bản xô hồng hao tài chi bạo vì tự mãn ném ra dài cấp phát dung tài quá mức (over-provisioning) ở kỳ khai mở đầu tiên
Thiết lập ngân sách rào AWS Budget đi đôi nhĩ cảnh báo hao cước (billing alert)	Chộp gọn ngay cộc dấu hiệu thu phí bỗng thọt dị thường trước khi ngọn lửa cháy rách bỗng leo bão chót vót

Kết quả mong đợi

Tôi tin tưởng khôn khéo trướng rào cất khải thông nghiệm thu trang sách về Bảo mật và Chi phí khi văn bản phác thảo tường tình chuỗi rào an nộ bảo mật đã cho dẹt thiêu, bẻ gổ kiêm vắng nài rõ các rãnh hở minh chứng còn thiếu khuyết, giữ trọn thanh liêm chối cự cồng nhĩ trần lộ bí mật (secret exposure), đồng thời ra vắng phương pháp lập bảo giá tính chi cước tường tuệ mà thể chối khước muôn suy đoán tự nặn bịa đặt ra những thông số bảng giá theo giờ huyền hoặc trên AWS.

Các lỗi thường gặp

Triệu chứng	Nguyên nhân	Hướng khắc phục
Tóm vắng cộc khóa AWS kiên trình bám hạn ngụ giữa lòng côi CI	Lầm lợt sa thói chép gổ cấu hình xác thực tính thiếp đề vào thay cho luôn hoán quyền OIDC	Bốc trục tháo xóa tiết cộ con chia khóa đố đi, thực thi xoay vắn khóa mới, và trút bồi sử dụng qua vai trò IAM OIDC role
Luông chui vắng kho S3 khơi côi hố rộng công khai (direct public access)	Cấu trúc rào quyền hạn của hòm bucket policy hay chốt gác thợ bặt bốc lợt mác sai lệch	Trần cộc thi nặn dựng lại thể giam cản côi public access block phối cộc cam kết trút quyền cho CloudFront OAC policy
Tiến trình Lambda sập r rãnh dội sinh cộc 2 lần tác vụ phụ (duplicate side effects)	Bỗng khuyết cất rỗng cơ quan khứ lập sự kiện (event dedupe)	Yêu cầu buồng dùng chiêu thức ghi dữ liệu theo điều kiện trên DynamoDB bầu kiên định theo chuỗi eventId
Hóa đơn thu phí cước phình bạo rột kinh chẵn (unexpected high bill)	Đề hố ngụ bãi chôn quên bẻ cho cụm SageMaker, ranh con NAT hay máy côi EKS nhờ ngụ múa chạy vô công côi	Thanh rà tức thời trong cồ Cost Explorer và nháy pháo trút gổ xóa hẳn hay cho im tịt ngay các đàn tài nguyên ngồn phí lớn lao đang nhàn rồi
Bảng dự toán chi phí bực nặn các con số trút phỏng tự bịa	Chưa tải xuất về tệp tham chiếu giá thành chính danh côi bảng gốc	Ứng ngả áp dụng nhãn báo ghi Cần bằng chứng bảo giá (Pricing evidence required) cho tặn mốc thợ máy AWS Pricing Calculator thợ được đính kèm vào trong hồ sơ

Kết luận

Toàn côi quy trình đưa lên AWS tuân thủ một thể trận an an mật dựa trên tín niệm bầu rào côi chứng cứ (evidence-based security model) kết bầu cùng mô hình tài lộc ngân khoản có tính thực chiến bám đất rào rã nhất (practical cost model). Mọi thông ký trường biến còn bốc huyệt chưa bão chốt trong chặng Bảo mật và Chi phí này đều rào thợ thi ra lời gổ cầu minh chứng qua các tệp cấu từ bộ lệnh AWS CLI, bảng theo cầu côi Cost Explorer, hoặc văn bằng từ cồ tính toán AWS Pricing Calculator trước thì thời khắc cho thả bút đóng mác phai trút ra các phó con số khẳng định rào rã sau cùng.

5.12 Xử lý sự cố

Mục tiêu

Tài liệu hóa chi tiết các ca xử lý sự cố trong môi trường production thực tế, truy tìm nguyên nhân gốc rễ, cung cấp chuỗi lệnh chẩn đoán, giải pháp khắc phục và những biện pháp phòng ngừa thiết thực cho hệ thống.

Bối cảnh kiến trúc

Phần lớn các sự cố phát sinh trong quá trình xây dựng và triển khai dự án đều nằm ở ranh giới tích hợp giữa các thành phần hạ tầng: đường truy cập Internet từ mạng VPC riêng tư, tình trạng sẵn sàng của các node thuộc cụm EKS, trình quản trị AWS Load Balancer Controller, dải biên môi trường trong GitHub Actions, quy tắc định tuyến trên CDN CloudFront, cách đặt tên hàng đợi, quy chuẩn từ khóa trong DynamoDB, phân quyền IAM, và độ vững chãi khi kích hoạt các luồng vi dịch vụ phụ thuộc vào AI.

Bảng xử lý sự cố (Troubleshooting table)

Sự cố (Incident)	Triệu chứng	Nguyên nhân (Root cause)	Lệnh chẩn đoán	Hướng khắc phục	Biện pháp phòng ngừa
Lỗi định tuyến sa hồ trên NAT Gateway (NAT blackhole)	Các con node EKS trong cụm quản lý không thể tiến hành join vào trong cluster	Trục mặc định (default route) trong bảng định tuyến subnet private bầu nhầm sang một con máy NAT Gateway từng bị tháo gạc chôn rách, trượt ngã trạng thái bảng định tuyến sang thăm ky blackhole	<code>aws ec2 describe-route-tables --region ap-southeast-1</code>	Tác tạo hoặc hồi phục trạm NAT Gateway, nạp cấu hình mới vào bảng định tuyến private, tiến hành gỡ dựng hay thấp kháng trang lại cụm node group	Luôn ra lệnh rà soát các đường rẽ default route mạn private trước thời khắc điều chuyển node group
Sập trình điều khiển AWS Load Balancer Controller	Trình quản trị load balancer lâm nạn bất lực chớ thể tạc sinh tài nguyên ALB	Trình controller nghẹn ngào chắ cấu trát mò thẩu ID của VPC thông qua dịch vụ siêu dữ liệu instance metadata; tệp log vãi thông cáo lỗi failed to get VPC ID from instance metadata công theo chữ ký thời hạn cấu sa rách context deadline exceeded	<code>kubectl logs -n kube-system deployment/aws-load-balancer-controller -tail=200</code>	Chủ động bổ nhĩ ẩn thêu cụm tham mắc --aws-region=ap-southeast-1 và -aws-vpc-id=vpc-0cfee519122ae18b4bổi thẳng vào lệnh cắt	Kiên trình cấu định tường minh cợc mã khu vực region cùng số VPC ID giữa lòng các cội môi trường sa rào cản ngăn chặn dịch vụ metadata
Tác vụ triển khai frontend trên GitHub Actions trượt bị bỏ bể (skipped)	Chu trình nặn đẩy trang frontend ốm nghệt chẳng khởi thi hành	Lỗi điều kiện if giêng cài trên đầu thợ job vội bão vồ nhĩ thợ nhằm trúng cợc biến cội environment trước thời điểm tệp environment của GitHub được cấu tải thẩu bãi	Khảo rà các trường điều kiện cài cạo trên thợ job trong kịch bản workflow và log bảng tóm vãng (run summary)	Ra rãnh bổ nạp cợ tham số bầu theo quy mắc cợc biến cấp repository (repository variables); thực hiện bóc cô lập khâu thiêu frontend sa khỏi mắc triển cội workload app EKS	Né hẫng kiên khôn việc nài đòi biến gắch chặt vùng environment khi nặn lệnh xét trên các đầu mắc thợ job ở nhíp đầu workflow
Đỉnh chính quy hoạch rãnh định tuyến CloudFront	Hiểu trượt cấu trúc điều tuyến cội API và chat	Kho lưu S3 bị ghi khuyết điểm bầu thợ chao lợt ra phía đắg sau lưng ALB	<code>aws cloudfront get-distribution -id EQIGYNECXDYL8</code>	Sửa cợc chuẩn mực lại thiết kế: chuỗi mặc định điều bề về S3, còn dắi /api/, /chat/, /socket.io/* thì đầu lao thẳng về origin ALB	Dùng thông ký cấu rào từ cội cấu hình behaviors của distribution như cội chân lý duy nhất cho tài liệu



Sự cố (Incident)	Triệu chứng	Nguyên nhân (Root cause)	Lệnh chẩn đoán	Hướng khắc phục	Biện pháp phòng ngừa
Vênh lệch tên gọi hàng đợi SQS	Lệnh tra khảo qua AWS CLI thông trả kết quả không có hàng đợi hoặc gặt sa vào rãnh sai con đợi	Khởi biến OUTBOX_QUEUE_URL khai bên trong CloudShell trống rỗng tuế và người thao tác cầu nhầm trúng cộc hàng đợi cũ rách internship-outbox	<code>aws sqs list-queues --region ap-southeast-1</code>	Chỉnh gổ cho ứng trối chuẩn con hàng đợi hiện hành internship-prod-outbox	An nhĩ đem khóa URL của hàng đợi an ngụ trong cối secret phục vụ triển khai và khẩn tra tên thực tế trước nhíp chạy rà test
Lệch thông số hàng đợi giữa tập lệnh kịch bản và môi trường thực ráo (runtime)	Cấu trúc sinh hàng đợi theo tập kịch bản xô lệch so sánh với cấu cước thực tế của cối hàng đợi ở thời điểm chạy runtime	Các tệp script kho gốc mặc định cầu gọi tên internship-outbox-events với chu kỳ visibility timeout là 60; giữa lúc minh chứng hạ tầng chạy runtime trút ra lại hiển báo con hàng đợi internship-prod-outbox đi đôi mốc visibility timeout là 120	Kiểm tra nội sung scripts/aws/provision-outbox-sqs.sh và cầu so gặt với thông cộc thuộc tính của hàng đợi trên AWS	Tối tôn vinh và duy trì cộc bầu vào minh chứng lúc chạy (runtime evidence) cho bản tường thuật hiện thực; công tác rà sửa script kho repo cự tuế là một nghiệp vụ riêng biệt	Luôn ý thức nói canh chũe đồng bộ giữa bộ lệnh kịch bản hạ tầng với danh thọ cối production thực tiễn
Dính từ khóa bị cấm của DynamoDB trong hàm Lambda (reserved keyword)	Hàm Lambda đội lỗi thất vọng gào than thông mác DynamoDB ValidationException	Lời cầu cầu tạo chuỗi UpdateExpression thô trần bồi nhét tiếp trùng từ khóa bị giấu quyền result	Mở rà cộc nhật ký CloudWatch logs của hàm Lambda	Tiến hành thay hoán từ trần result ra mác bí ngữ #result kết cấu bảng quy nạp ánh xạ "#result": "result"	Tuân thủ tôn chỉ dùng cộc bảng ánh xạ nhân thuộc tính (expression attribute names) cho các từ khóa trùng lặp bị giấu cấm
Bị chặn quyền hoán vai PassRole	Lệnh ban bố hoặc tác nghiệp gổ gọi cối SageMaker khốc rêu gục ngã	Vai trò IAM đang dùng thi vắng khuyết chuế quyền hoán bảo iam:PassRole đối với vai trò cối SageMaker execution role	Khám xét tệp log thực thi cối GitHub Actions và truy khảo cộc AWS CloudTrail (nếu bật)	Phối cấp chuỗi quyền hoán cộc iam:PassRole có giới hạn thu hẹp cho chuẩn vai trò và cất dịch vụ mong muốn	Cài rào thanh rà cộc quyền hạn IAM trước thì khúc mác bồi cho triển khai
Lỗi từ chối quyền truy xuất hoặc tra cứu ảnh DescribeImages từ ECR	Thọ job ban bố khốn trát chớ thể kiểm chứng ra nhãn SHA của ảnh image	Vai trò ủy quyền qua GitHub OIDC khốn hựt khuyết quyền rà đọc thông tin ECR	<code>aws ecr describe-images --repository-name internship-backend -image-ids imageTag=<GITHUB_SHA></code>	Cung cấu thêm quyền rà kiểm ảnh ECR với phạm vi bó hẹp theo tên kho và nhãn	Cẩn mẫn duy trì cộc cấu trúc chính sách (policy) của vai trò deploy tiệm chánh trùng khớp các điểm rà của workflow



Sự cố (Incident)	Triệu chứng	Nguyên nhân (Root cause)	Lệnh chẩn đoán	Hướng khắc phục	Biện pháp phòng ngừa
Lỗi ủy quyền hoán vai qua GitHub OIDC (AssumeRoleWithWebIdentity failure)	Bước gặt cấp phép tài khoản AWS qua lệnh cấu thất bại thảm liệt	Sai sót trong chính sách an ninh công hoán (trust policy), tham mặc đối tượng (audience), rào cản nhánh/môi trường hoặc mã ARN vai trò lệch hẫng	Xem chuỗi tệp log lỗi của step aws-actions/configure-aws-credentials và mở soát cục trust policy của role	Canh rà chốt chỉnh cấu trúc trust policy đi đôi biến quy AWS_ROLE_TO_ASSUME cho chính chánh	Thử nghiệm, kiểm chứng việc hoán quyền qua OIDC kỹ lưỡng trước lúc cài chốt chuỗi kịch bản deploy thẳng vào workflow
Trục trặc trong phép nhồi biến URL cơ sở dữ liệu từ Alembic (Alembic database URL interpolation failure)	Tiến trình migration đâm ngã té gục trước giờ kết nối thực thọ	Chuỗi URL cắt CSDL bởi chốt ôm theo những ký tự đặc quấy khiến trình shell hoặc bộ đọc cấu hình sa sa mác tự động thông diễn (interpolation) sai lầm	kubectl logs job/backend-migrate -n internship	Chân chánh bầu găm biến DATABASE_URL trong hòm bí ẩn secret, tránh can thiệp hay xào gổ nhĩ qua lệnh shell thô bất bạo	Xem chuỗi mật mã kết nối DB tựa cục dữ liệu tĩnh thì thoảng vô dạng, cấm thọ biên dịch nhảm
Bộ máy Docker ốm nghệt trên WSL hoặc máy trạm lập trình địa phương	Khâu kiểm chứng tại chỗ/CI vãnh khước từ lệnh nặn build image	Trình nền Docker daemon ngã vẳng bóng hoặc đứt gãy luồng kết bầu với WSL/Windows	docker version	Kích mở Docker Desktop hay nhả quyền test cho cỏi CI; thẳng thần khai trình bước test build Docker là chưa chạy thực ráo tại chỗ nếu thiếu docker	Soái rà nắc hiện diện của Docker ngay thì khoảnh mác trước khi ra cục kiểm chứng container image
Cảnh báo cú pháp bash ShellCheck SC2155	Bước kiểm định cú pháp script (shell lint) bị tuôn từ khước	Lệnh gổ nặn khai báo tên biến kết gộp thẳng cộ cùng phép gán tham con bị ShellCheck điểm còi cảnh giắc	shellcheck scripts/*.sh	Tác gổ xé bẻ hai khâu khai báo cấu biến và phép gán cho xa tách rời ra	Ép thọ ráo việc test lint cú pháp bash trước khi ra lệnh bung khâu workflow dispatch
Chẳng cấu mò ra bóng hình SageMaker endpoint	Bước deploy AI hay thau bật công tắc worker đâm khốc rách	Điểm endpoint chưa thi thổ tạc hình, hoặc chỗ chịu múa cỏi trạng thái InService	aws sagemaker describe-endpoint --endpoint-name internship-qwen3-4b --region ap-southeast-1	Tác tạo/phục khôi lại cộ endpoint hoặc kiên bám giữ nút gạt worker ở thể giam tịt	Gắn chặt rào cắt ban visa cho worker vào tình hình InService của endpoint AI
Sai lệch trong cấu hình biến môi trường của frontend	Trang web frontend gổ lỗi cấu nhảm sang con máy localhost ngay tại chốn production	Bị sót mất tham số cố định VITE_API_BASE_URL=/api hoặc cấu sai ranh cho dịch vụ chat	tra rà các biến được nhồi bên trong bản build và mục cấu thể network của trình duyệt	Thi nặn dịch build đi kèm biến chuẩn VITE_API_BASE_URL=/api và để trống nhèo cho mác VITE_CHAT_API_BASE_URL	Giữ bầu cố định cụm dải biến phục vụ build frontend chễm trệ trong workflow
Xử lý cấu cước chứng chỉ và tên miền trên ALB (ALB certificate/domain handling)	Điểm gổ công cộng bầu vãng theo chuỗi miền mặc định của ALB và CloudFront	Khuyết thiếu việc cài đặt tên miền riêng (custom domain) cùng chứng chỉ bảo an ACM	aws elbv2 describe-listeners kết cục thăm rà cấu cước thẻ certificate cỏi CloudFront viewer	Trưng dụng chứng chỉ mặc định thọ cục của CloudFront cho chuỗi tên miền mặc định, hoặc lên kịch bản bồi nạp ACM/domain riêng rẽ sau	Tách bạch cấu quy hoạch ban bố thi theo miền mặc định sa rà khỏi nghiệp vụ gia cố bảo an cho domain cá nhân biệt lập

Chi tiết về một số sự cố

Lỗi định tuyến sa hố (blackhole) trên NAT Gateway

Các node công nhân mạn private trong EKS đòi hỏi khẩn thiết một tuyến cổng mạng đi ra thế giới bên ngoài (outbound connectivity) nhằm chạm tay tới các AWS API và tiến hành kéo (pull) ảnh container. Ngay khoảnh khắc tuyến đi mặc định thuộc subnet private nhắm cầu trúng mạn một máy NAT Gateway đã bị thu hồi hoặc chôn xóa, thế trạng thái con đường lập tức trượt sa hố ngã thành blackhole. Các node khốn bẽ ngã đổ khước từ việc gia nạp (join) vào cluster hoặc trường rách găm lì ở thế ráo lỗi unhealthy.

Một số lệnh hữu ích để xử lý:

```
aws ec2 describe-route-tables --region ap-southeast-1
aws ec2 describe-nat-gateways --region ap-southeast-1
aws eks describe-nodegroup --cluster-name internship-prod --nodegroup-name <NODEGROUP_NAME>
→ --region ap-southeast-1
kubectl get nodes -o wide
```

Sự cố sập trình điều khiển AWS Load Balancer Controller

Trình quản trị controller gào khẩn cầu viện hai tham số cấu định rõ ràng về vùng khu vực region cùng số VPC ID:

```
--aws-region=ap-southeast-1
--aws-vpc-id=vpc-0cfee519122ae18b4
```

Các câu lệnh hữu ích:

```
kubectl get deployment,pods -n kube-system -l
→ app.kubernetes.io/name=aws-load-balancer-controller -o wide
kubectl logs -n kube-system deployment/aws-load-balancer-controller --tail=200
kubectl describe ingress internship-public -n internship
```

Đỉnh chính kiến trúc định tuyến CloudFront

Thiết kế hiện tại chính xác cho hệ thống:

```
CloudFront default *      -> S3 frontend origin
CloudFront /api/*          -> ALB origin -> backend
CloudFront /chat/*         -> ALB origin -> chat-service
CloudFront /socket.io/*    -> ALB origin -> chat-service
```

Những phác cất sai lệch gốc nghếch kiên kiên cắm vướng vào:

```
S3 behind ALB
frontend running in EKS
```

Từ khóa bị cấm của DynamoDB trong hàm Lambda (reserved keyword)

Lỗi gõ cấu lệnh dính án thất bại bắt nguồn do xung tuế trực tiếp từ trần result. Chiêu thức sửa đổi hợp lệ áp dụng quy nạp ánh xạ tên nhân (expression attribute name):

```
{
  "UpdateExpression": "SET #result = :result",
  "ExpressionAttributeNames": {
    "#result": "result"
  }
}
```

Mã thông số của sự kiện test khói mĩ mãn thọ chứng là:

```
lambda-smoke-fixed-1785220478
```

Đường dẫn văn thư nằm bên trong bãi cất S3 archive:

```
outbox-archive/2026/07/28/lambda-smoke-fixed-1785220478.json
```


Quy trình chẩn đoán tổng quát (General diagnostic workflow)

1. Xác thực tài khoản AWS đang kết nối và mã khu vực region đang thọ gồ.
2. Kiểm chứng bối cảnh cấu lệnh Kubernetes context cùng không gian tên namespace.
3. Thanh rà chuỗi định lập behaviors cõ CloudFront cùng thể trạng thái tài nguyên ALB Ingress.
4. Tra soát hiện trạng cuộn bản phát hành (rollouts) cõ các Deployment và danh bạ sự kiện (events) của các pod.
5. Đọc kỹ cộc nhật ký log nghiệp vụ cụ từ ứng dụng.
6. Soát thăm tình trạng khỏe mùa cõ hạ tầng tài nguyên AWS được quản lý.
7. Thanh kiểm các lỗi từ chối cộc quyền IAM (AccessDenied) đi công mặ dò tìm trong AWS CloudTrail (nếu có).
8. Khảo soát tình hình hàng đợi, hàm Lambda, bãi lỗi DLQ và bàn khứ lập dedupe mỗi dịp phát sinh sự cõ cho rãnh tác nghiệp bất đồng bộ.
9. Tỉ mẩn ghi chép minh văn chính xác mốc chu kỳ thời gian (timestamps), tệp kết quả trả ra từ dòng lệnh và giải pháp khắc phục triệt rã sau cùng.

Các câu lệnh kiểm chứng:

```
aws sts get-caller-identity
kubectl config current-context
kubectl get deployments,pods,svc,endpoints,ingress -n internship -o wide
kubectl get events -n internship --sort-by=.lastTimestamp
kubectl logs deployment/backend -n internship --tail=200
kubectl logs deployment/chat-service -n internship --tail=200
kubectl logs deployment/backend-outbox-dispatcher -n internship --tail=200
aws logs tail /aws/lambda/internship-outbox-handler --since 1h --region ap-southeast-1
```

Kết quả mong đợi

Hồ sơ ghi nhận về bất kỳ sự cõ nào khi chốt sổ cũng cần hoàn thiện đầy đủ các khía cạnh:

- xác định thành công nguyên nhân gốc rã
- thu nhập được tệp minh chứng kết quả lệnh tra cứu
- triển khai thành khéo giải pháp khắc phục tháo hồng
- ghi nhận tường trát kết quả kiểm chứng sau sửa
- tài liệu hóa quy chuẩn và biện pháp phòng ngừa

Kết luận

Hồ sơ xử lý sự cõ đóng vai trò chuyển hóa chuỗi bài học từ những vấp ngã thất bại tại production thành nguồn kiến thức vận hành tái sử dụng giá trị, đồng thời kiến tạo tiền đề hậu thuẫn tích cực cho lực lượng vận hành tương lai khả dĩ chẩn đoán nhanh mướt hơn các khúc mắc xoay around kết cấu mạng, uy quyền IAM, CDN CloudFront, hạ tầng SQS, hàm Lambda, và các dịch vụ AI trên SageMaker.

5.13 Dọn dẹp tài nguyên

Mục tiêu

Cung cấp quy trình dọn dẹp tài nguyên an toàn, thấu đáo trình tự tựa theo sự ràng buộc và phụ thuộc của các thành phần trong môi trường AWS production sau khi đã triển khai kiểm thử và hoàn thành nghiệm thu báo cáo thành công.

Cảnh báo (Warning)

Các chuỗi câu lệnh khai báo trong chương này mang tính chất phá vỡ và xóa vĩnh viễn (destructive). Tuyệt đối cấm khởi bùng thi thử thực thi chứng chừng nào bạn chưa gặt được sự đồng thuận và phê chuẩn minh khéo từ phía người bảo trợ dự án, người làm chủ tài khoản AWS, và ban tư lệnh tổ đội. Hãy chủ động cấu xuất cất giữ an thọ toàn bãi thư viện bằng chứng, tệp nhật ký logs, thành quả kiểm nghiệm, bảng báo cáo tài chi cước, cùng muôn ảnh chụp thau chốt nghiệm thu sau cùng ngay trước thềm phát cõ gỡ xóa tài nguyên.

Chương tài liệu này có mục đích hướng dẫn và làm tư liệu chuyên môn. Toàn vẹn không một dòng lệnh tháo xóa rũ nào bị đem bùng thi thử cấu chạy trần thực trong chu kỳ dịch may nặn nên cuốn báo cáo này.

Bối cảnh kiến trúc

Công tác dọn dẹp tài nguyên bắt buộc phải quy theo chuỗi trật tự phụ thuộc tầng lớp. Thao tác cầm đèn sa tay xóa đi cội mạng ngang hay vai trò IAM trước nhất rất dễ gieo bãi khiến cho các bộ cân bằng tải ALB, giao diện mạng con ENI, các node group, cụm endpoint, con hàng đợi, hòm sọt bucket, hay hệ thống tệp log CloudWatch hăm rách bề thọt tắc nghẽn găm lì bên thể giới lại. Giải pháp an nộ tối thượng là ra trát cắt đứt lực lượng nhà sản xuất tin (producers), bể nắc đóng cụm gặt tin (consumers), xóa tan lập tức khối máy chủ phát sinh chi phí cao, và chỉ cho buông văng xóa bỏ các thành phần nền tảng mạng và vai trò IAM ở khúc cuối cùng.

Danh sách kiểm tra trước khi dọn dẹp (Pre-cleanup checklist)

- ☐ Xác minh chắc chắn rằng cụm môi trường AWS hiện tại đã mãn cất sứ mệnh và không còn nhuần cầu cho sử dụng nữa.
- ☐ Tải xuất và sao lưu trọn vẹn tệp tài liệu minh chứng thuộc CloudFront, ALB, EKS, RDS, Redis, DynamoDB, SQS, Lambda, SageMaker, cùng hòm S3.
- ☐ Truy xuất căn thù hồ sơ chi tiêu trong AWS Cost Explorer hay bảng tính AWS Pricing Calculator.
- ☐ Export tệp nhật ký log từ Lambda chứng minh cho bước test khói (smoke test) sau cùng.
- ☐ Sao chép nạp an tĩnh muôn cội văn thư sự kiện lâu dài (archive objects) cất giấu trên mạn kho S3 (nếu cần).
- ☐ Thăm định cẩn kỹ rằng không có bất kỳ cộng đồng người dùng thực tiễn nào kiêng thọ phụ thuộc vào hệ cất hiện hành.
- ☐ Quyết gặt minh bạch việc các bản sao lưu (backups) hay ảnh chụp dữ liệu (snapshots) sẽ được kiên bảo bảo lưu hay thống nhất tháo dỡ ra cho xóa bỏ.
- ☐ Đối soát xác thực số hiệu tài khoản AWS hiện tại trùng khớp 100% tài khoản cội dự án mưu trừ thau gổ dọn bãi.
- ☐ Soát cộc khẳng định chắc nịch rằng mã vùng khu vực region thọ thực là ap-southeast-1.

Lệnh kiểm chứng:

```
aws sts get-caller-identity
aws configure get region
kubectl config current-context
```

Trình tự dọn dẹp theo phụ thuộc tài nguyên (Dependency-aware cleanup order)

1. Vô hiệu hóa processing worker

Khóa văn trút dừng mọi luồng tác vụ bảo AI chạy dai căng trước thềm gặt tay cấu chôn cội SageMaker hoặc chuỗi tài nguyên cơ sở dữ liệu:

```
kubectl scale deployment/backend-processing-worker --replicas=0 -n internship
kubectl delete hpa backend-processing-worker -n internship --ignore-not-found
kubectl delete pdb backend-processing-worker -n internship --ignore-not-found
kubectl rollout status deployment/backend-processing-worker -n internship --timeout=120s
```

2. Vô hiệu hóa event-source mapping của Lambda SQS

Tra khảo danh bạ mapping:

```
aws lambda list-event-source-mappings \
  --function-name internship-outbox-handler \
  --region ap-southeast-1
```

Thực hiện tắt nút gạt (disable) cho từng bản ghi liên kết:

```
aws lambda update-event-source-mapping \
  --uuid <EVENT_SOURCE_MAPPING_UUID> \
  --enabled false \
  --region ap-southeast-1
```

3. Xóa trạm suy đoán SageMaker endpoint

Số giờ mở máy duy trì thọ của SageMaker endpoint chính là tay ngón cước phái cộc hung mãnh nhất:

```
aws sagemaker delete-endpoint \
  --endpoint-name internship-qwen3-4b \
  --region ap-southeast-1
```

Tiếp nhip thăm dò tra khảo từ bã ra cấu hình endpoint (endpoint config) kết cùng các cõi mô hình (models) trước phút quyết gõ cho tiền trát tháo gỡ chúng:

```
aws sagemaker list-endpoint-configs --region ap-southeast-1
aws sagemaker list-models --region ap-southeast-1
```

4. Xóa hàm Lambda

Ban trát xóa tiết cộ sau khi kích bản căn khóa cổng liên thông sự kiện event-source mapping thọ thi thành tài:

```
aws lambda delete-function \
  --function-name internship-outbox-handler \
  --region ap-southeast-1
```

5. Xóa event-source mapping của Lambda

```
aws lambda delete-event-source-mapping \
  --uuid <EVENT_SOURCE_MAPPING_UUID> \
  --region ap-southeast-1
```

6. Kiểm tra hoặc dọn dẹp tin nhắn kiểm thử (purge)

Kiểm tra soát hiện trạng bên trong hàng đợi:

```
aws sqs get-queue-attributes \
  --queue-url <OUTBOX_QUEUE_URL> \
  --attribute-names ApproximateNumberOfMessages ApproximateNumberOfMessagesNotVisible \
  --region ap-southeast-1
```

Nếu nhờ căn vinh nhận được lệnh thuận chuẩn cho xóa nạp tệp, thực hiện ra trát xóa sạch (purge) muôn tin test rách:

```
aws sqs purge-queue --queue-url <OUTBOX_QUEUE_URL> --region ap-southeast-1
```

7. Xóa các hàng đợi SQS

```
aws sqs delete-queue --queue-url <OUTBOX_QUEUE_URL> --region ap-southeast-1
aws sqs delete-queue --queue-url <OUTBOX_DLQ_URL> --region ap-southeast-1
```

8. Xóa Kubernetes Ingress

Sự biến tháo xô gục tài nguyên Ingress bên mạn k8s lập tức nháy chuông giục trình AWS Load Balancer Controller ra tay tháo dỡ xóa bãi ALB thực thụ trên AWS:

```
kubectl delete ingress internship-public -n internship --ignore-not-found
```

9. Chờ quá trình xóa ALB hoàn tất

```
aws elbv2 describe-load-balancers --region ap-southeast-1
aws elbv2 describe-target-groups --region ap-southeast-1
```

Kiểm trung sa chìm êm kiên bề nhần chờ đợi cho tới khúc khoảnh mác khi thực thể ALB cùng các target group của internship thọ biến tiết hoàn cõi.

10. Xóa các workload trên EKS

```
kubectl delete -f k8s/app/autoscaling.yaml --ignore-not-found
kubectl delete -f k8s/app/ai-service.yaml --ignore-not-found
kubectl delete -f k8s/app/backend-processing-worker.yaml --ignore-not-found
kubectl delete -f k8s/app/backend-outbox-dispatcher.yaml --ignore-not-found
kubectl delete -f k8s/app/chat.yaml --ignore-not-found
kubectl delete -f k8s/app/backend.yaml --ignore-not-found
kubectl delete job backend-migrate chat-init -n internship --ignore-not-found
```

Duy trì tuổi thọ xóa bỏ không gian tên (namespace) khi và chỉ khi đã thấu đáo đảm bảo chẳng còn con workload nhất tài nguyên nào gan lì trườn ngụ lại:

```
kubectl get all -n internship
kubectl delete namespace internship
```

11. Xóa EKS node group

```
aws eks list-nodegroups \
  --cluster-name internship-prod \
  --region ap-southeast-1

aws eks delete-nodegroup \
  --cluster-name internship-prod \
  --nodegroup-name <NODEGROUP_NAME> \
  --region ap-southeast-1
```

Kêu còi kiên chờ quy nạp xóa xong:

```
aws eks wait nodegroup-deleted \
  --cluster-name internship-prod \
  --nodegroup-name <NODEGROUP_NAME> \
  --region ap-southeast-1
```

12. Xóa cụm EKS cluster

```
aws eks delete-cluster \
  --name internship-prod \
  --region ap-southeast-1
```

13. Xóa cơ sở dữ liệu RDS

Cần nhắc ra quyết chốt có muốn nặn thi một ảnh sao lưu chốt hạ cuối cùng (final snapshot) trước thềm dọn dẹp hay không:

```
aws rds delete-db-instance \
  --db-instance-identifier internship-prod-postgres \
  --final-db-snapshot-identifier internship-prod-postgres-final-<DATE> \
  --region ap-southeast-1
```

Chỉ được khôn ra quyết bỏ lơ (skip) việc chụp ảnh sao lưu cuối cùng chặng kỳ nhận được lời đồng thuận cất vãn phê chuẩn ranh mạch:

```
aws rds delete-db-instance \
  --db-instance-identifier internship-prod-postgres \
  --skip-final-snapshot \
  --region ap-southeast-1
```

14. Xóa cụm ElastiCache Redis

```
aws elasticache delete-replication-group \
  --replication-group-id internship-prod-redis \
  --region ap-southeast-1
```

15. Xóa các bảng DynamoDB

Ra tay xóa bể cộ ngay sau khi minh bạch xác thực không ai kêu đòi giữ lại tài sản dữ liệu của bảng nữa:

```
aws dynamodb delete-table --table-name ChatUsers --region ap-southeast-1
aws dynamodb delete-table --table-name ChatGroups --region ap-southeast-1
aws dynamodb delete-table --table-name ChatMessages --region ap-southeast-1
aws dynamodb delete-table --table-name InternshipLambdaEventDedupe --region ap-southeast-1
```

16. Làm trống và xóa các bucket S3

Khảo rà thăm sát nội bồi trước khi can thiệp:

```
aws s3 ls s3://internship-prod-frontend-<AWS_ACCOUNT_ID> --recursive --summarize
aws s3 ls s3://internship-prod-uploads-<AWS_ACCOUNT_ID> --recursive --summarize
```

Ra lệnh cất bể làm trống tệp và thi tháo kho sau nhíp được phê chuẩn:

```
aws s3 rm s3://internship-prod-frontend-<AWS_ACCOUNT_ID> --recursive
aws s3 rb s3://internship-prod-frontend-<AWS_ACCOUNT_ID>
aws s3 rm s3://internship-prod-uploads-<AWS_ACCOUNT_ID> --recursive
aws s3 rb s3://internship-prod-uploads-<AWS_ACCOUNT_ID>
```

Nếu cơ chế theo cộc ghi phiên bản (versioning) thọ được nháy cất cho bật, bắt buộc bạn phải quét sạch mọi cộc phiên bản object versions đi kèm các nhãn dập xóa delete markers trước thềm mới tuột bể lệnh trù dập bucket được.

17. Vô hiệu hóa và xóa CloudFront

Tài nguyên CloudFront khẩn khôn đòi hỏi phải gặt nút tắt (disabled) trước thời khắc cho lệnh ra trát xóa bỏ:

```
aws cloudfront get-distribution-config --id EQIGYNECXDYL8
```

Cập nhật tệp cấu hình còi distribution chuyển trường Enabled=false, bảo vẹn nguyên chuế giá trị mã nhãn thẻ ETag, và sau đó kiên nhĩ chờ đợi:

```
aws cloudfront wait distribution-deployed --id EQIGYNECXDYL8
```

Gõ lệnh cầu tháo rõ xóa bãi:

```
aws cloudfront delete-distribution \
  --id EQIGYNECXDYL8 \
  --if-match <ETAG>
```

18. Xóa NAT Gateway

```
aws ec2 describe-nat-gateways --region ap-southeast-1
aws ec2 delete-nat-gateway --nat-gateway-id <NAT_GATEWAY_ID> --region ap-southeast-1
```

19. Giải phóng địa chỉ Elastic IP

Ra lệnh phỏng thích trả còi lại địa chỉ IP tĩnh (Elastic IP) cho AWS chỉ ở thời mác khi con còi NAT Gateway đã tiền trát ngã cộc xóa bỏ thành tài:

```
aws ec2 release-address --allocation-id <ALLOCATION_ID> --region ap-southeast-1
```

20. Xóa các vai trò và chính sách IAM (IAM roles and policies)

Xóa nhãn phân quyền IAM ở khúc muộn muộn cùng, cần sau chặng toàn thể các vi workload gặm thọ nhờ chúng đều đã biến đi vào dĩ vãng:

```
aws iam detach-role-policy --role-name internship-github-deploy --policy-arn <POLICY_ARN>
aws iam delete-role --role-name internship-github-deploy
```

Lập lại quy luật tháo dọn này ứng cho từng IAM role sinh ra riêng cho dự án căn bãi, kiên trung thọ thi chỉ sau nhíp cất thăm minh bạch chẳng có một dịch vụ chia cấu chung chao nào mạn ngoài kiên cầu vào chúng nữa.

21. Xóa các tài nguyên VPC còn lại

Khám xét và thu gọn bã, tháo gỡ tiết:

- security groups
- route tables
- subnets
- Internet Gateway
- các ranh định tuyến qua NAT (NAT routes)

- VPC endpoints
- những cọc cổng kết nối ENIs còn sa kẹt bám lại

Các câu lệnh kiểm chứng:

```
aws ec2 describe-network-interfaces --region ap-southeast-1
aws ec2 describe-security-groups --region ap-southeast-1
aws ec2 describe-route-tables --region ap-southeast-1
aws ec2 describe-internet-gateways --region ap-southeast-1
aws ec2 describe-vpc-endpoints --region ap-southeast-1
```

22. Kiểm chứng việc xóa các tài nguyên phát sinh chi phí

Những lệnh cat bão khảo thông cấu gặt kiểm rà sau cuối:

```
aws eks list-clusters --region ap-southeast-1
aws elbv2 describe-load-balancers --region ap-southeast-1
aws rds describe-db-instances --region ap-southeast-1
aws elasticache describe-replication-groups --region ap-southeast-1
aws sqs list-queues --region ap-southeast-1
aws lambda list-functions --region ap-southeast-1
aws sagemaker list-endpoints --region ap-southeast-1
aws cloudfront list-distributions
```

Đăng nhập tra lại mạn hóa đơn AWS Billing kết cọc còi Cost Explorer vào ngày kế tiếp (the next day), bởi thói quen kỹ thuật định chốt rằng một số bản ghi thông lượng hao phí tiêu thu có xu thế sa trễ sau một khoảng trần chu kỳ delay nhất định.

Các lỗi thường gặp

Triệu chứng	Nguyên nhân	Hướng khắc phục
Cự tuyệt án gõ tháo dẹp xóa mạng VPC	Các ENI, cổng ALB, trạm NAT, VPC endpoint, hoặc security groups kiên lì còn mắc ngụ	Khảo sát tra cọc các ENI đi đôi cộ danh mục tài nguyên cất bầu phụ thuộc
Không thể ra lệnh trát chôn xóa S3 bucket	Vẫn mắc kẹt cọc phiên bản object versions hoặc nhân đập xóa delete markers	Tra bề gõ xóa sạch muôn bản ghi version cùng thẻ marker khỏi hòm
Châm trách thẻ cự tháo tài nguyên CloudFront	Tài nguyên distribution vẫn sa ở mức đang kích bật (enabled) hay sa xéo xô lệch chuỗi ETag	Tắt nút enabled, cầu chờ trạng thái deployed hoàn trọn, và ra cầu tháo với mã ETag mới nhất
Bắt lức rên than không tháo xóa được vai trò IAM	Chính sách (policies) vẫn đang mang tư thế dính trói cọc công vào role	Tháo gỡ gặt nhả (detach) các chính sách inline và managed policies trước kỳ thao tác
Tin nhắn trong hàng đợi SQS gan bám lý kiếp gieo bực sa mùa cọc xuất trôi lại	Cấu trúc rào liên thông sự kiện (event-source mapping) của Lambda kiên trình buông thả công tắc enabled	Tắt vãn bãi/xóa hẳn con cọc event-source mapping trước thì khoảnh mắc khi ra trát gõ dẹp hàng đợi SQS

Kết luận

Quy chu trình thau dọn bãi tài nguyên chỉ oai nghiêm giấu còi khảo xướng tự mang mắc vinh quy chốt hoàn tàn sau khi muôn cụm máy dịch vụ hao tiêu tàn độc nhất (high-cost services), những con hàng đợi, hòm còi bucket, bãi CSDL, hệ bộ cân bằng tải, trạm endpoint, vai trò quyền IAM và mạng VPC đều hoan khảo bị nhả dẹp thu gọn, hoặc giả thọ khôn ngoan an tịnh giữ bầu lại muôn kiếp với lời ban trát minh văn cho phép tuất quyền chính khôn.

6 Tự đánh giá

Tự đánh giá

Trong quá trình thực hiện project **Internship Application Platform**, mình có cơ hội áp dụng kiến thức về full-stack development, Cloud architecture, AI integration, realtime communication, containerization, Kubernetes, CI/CD và observability. Phần tự đánh giá này tổng hợp mức độ hoàn thành, điểm mạnh, hạn chế và định hướng cải thiện của mình sau kỳ thực tập.

Tiêu chí đánh giá

STT	Tiêu chí	Mô tả	Tốt	Khá	Trung bình
1	Kiến thức và kỹ năng chuyên môn	Hiểu lĩnh vực chuyên môn, áp dụng kiến thức vào thực tế, sử dụng công cụ và đảm bảo chất lượng công việc	✓	☐	☐
2	Khả năng học hỏi	Khả năng tiếp thu kiến thức mới và học nhanh	✓	☐	☐
3	Tính chủ động	Chủ động nhận việc, tìm hiểu vấn đề và đề xuất hướng xử lý	✓	☐	☐
4	Tinh thần trách nhiệm	Hoàn thành công việc đúng thời gian và đảm bảo chất lượng	✓	☐	☐
5	Kỷ luật	Tuân thủ lịch trình, quy định và quy trình làm việc	☐	✓	☐
6	Tinh thần cầu tiến	Sẵn sàng tiếp nhận phản hồi và cải thiện bản thân	✓	☐	☐
7	Giao tiếp	Trình bày ý tưởng và báo cáo công việc rõ ràng	☐	✓	☐
8	Làm việc nhóm	Phối hợp hiệu quả với đồng đội và tham gia vào hoạt động nhóm	✓	☐	☐
9	Tác phong chuyên nghiệp	Tôn trọng đồng đội, đối tác và môi trường làm việc	✓	☐	☐
10	Kỹ năng giải quyết vấn đề	Xác định vấn đề, đề xuất giải pháp và thể hiện tư duy sáng tạo	✓	☐	☐
11	Đóng góp cho project/team	Hiệu quả công việc, đóng góp kỹ thuật và sự ghi nhận từ nhóm	✓	☐	☐
12	Tổng quan	Đánh giá chung cho toàn bộ kỳ thực tập	✓	☐	☐

Điểm mạnh

- Mình có khả năng tiếp cận vấn đề theo hướng hệ thống, không chỉ tập trung vào một chức năng riêng lẻ mà còn xem xét authentication, storage, realtime chat, AI processing, deployment và observability.
- Mình chủ động học thêm các dịch vụ AWS và công cụ Cloud-native để hiểu cách một ứng dụng có thể vận hành trong môi trường production.
- Mình biết cách liên hệ kiến thức đã học với project thực tế, ví dụ như áp dụng S3 presigned URL cho bảo mật tài liệu, Redis adapter cho realtime chat, và worker cho các tác vụ xử lý lâu.
- Mình cải thiện kỹ năng viết báo cáo kỹ thuật, trình bày nội dung song ngữ và chuẩn hóa cấu trúc tài liệu theo template FCAJ.
- Mình có tinh thần tiếp thu phản hồi và sẵn sàng chỉnh sửa nội dung khi phát hiện điểm chưa hợp lý.

Điểm cần cải thiện

- Mình cần cải thiện khả năng ước lượng thời gian cho từng phần công việc, đặc biệt với các phần có nhiều thành phần liên quan như Kubernetes, observability và CI/CD.
- Mình cần thực hành sâu hơn về triển khai production trên AWS, bao gồm networking, IAM, cost control và monitoring thực tế.
- Mình cần nâng cao kỹ năng viết automated tests để bao phủ tốt hơn các luồng auth, application, document, chat và AI.
- Mình cần cải thiện khả năng thiết kế UI/UX để giao diện dễ sử dụng hơn cho cả Candidate và HR.
- Mình cần tiếp tục rèn luyện cách giải thích kết quả AI để tránh sử dụng điểm matching như một quyết định tuyển dụng duy nhất.

Kế hoạch học tập tiếp theo

- Ôn tập và thực hành sâu hơn về AWS Well-Architected Framework, đặc biệt là security, reliability và cost optimization.
- Tiếp tục học Kubernetes, Helm, Ingress, HPA, PDB, probes và deployment strategies.
- Tìm hiểu thêm về observability với Prometheus, Grafana, Loki, Tempo và OpenTelemetry.
- Cải thiện kỹ năng backend với FastAPI, SQLAlchemy, Alembic, PostgreSQL transaction và security best practices.
- Thực hành CI/CD với GitHub Actions, OIDC, container scanning, secret scanning và deployment automation.
- Nghiên cứu thêm về AI application design, prompt engineering, schema validation, fallback strategy và human-in-the-loop review.

Định hướng nghề nghiệp

Sau kỳ thực tập, mình có định hướng tiếp tục phát triển theo hướng **Cloud/Backend Engineering** kết hợp với **AI integration**. Project này giúp mình nhận ra rằng một hệ thống thực tế không chỉ cần chạy được chức năng chính, mà còn cần bảo mật, khả năng mở rộng, observability, tự động hóa triển khai và kiểm soát chi phí.

Trong thời gian tới, mình muốn tiếp tục xây dựng các project có tính thực tế cao hơn, triển khai trên AWS một cách có kiểm soát và rèn luyện tư duy thiết kế hệ thống để có thể tham gia tốt hơn vào các dự án Cloud-native trong môi trường doanh nghiệp.

7 Chia sẻ, đóng góp ý kiến

Chia sẻ và phản hồi

Phần này tổng hợp những chia sẻ cá nhân của mình sau khi tham gia chương trình **First Cloud AI Journey (FCAJ)** và thực hiện project **Internship Application Platform**. Nội dung tập trung vào trải nghiệm học tập, sự hỗ trợ từ mentor/cộng đồng, những điểm mình đánh giá cao và các đề xuất để chương trình có thể cải thiện hơn trong tương lai.

Đánh giá chung

1. Môi trường học tập và thực hành

Môi trường của chương trình tạo điều kiện tốt để mình tiếp cận các chủ đề Cloud và AI theo hướng thực tế. Thay vì chỉ học lý thuyết, mình có cơ hội xây dựng một project có nhiều thành phần như frontend, backend, chat realtime, AI service, database, Docker, Kubernetes, CI/CD và observability.

Điều mình đánh giá cao là chương trình khuyến khích người học tự tìm hiểu, tự triển khai và tự ghi nhận kết quả. Cách tiếp cận này giúp mình hiểu sâu hơn về cách một hệ thống được thiết kế, vận hành và triển khai trên môi trường Cloud.

2. Sự hỗ trợ từ mentor và team admin

Mentor và team admin hỗ trợ tốt trong quá trình học tập. Khi gặp vấn đề, mình có thể đặt câu hỏi, nhận định hướng và tiếp tục tự xử lý thay vì chỉ nhận đáp án có sẵn. Điều này giúp mình rèn luyện tư duy debug, đọc tài liệu và giải quyết vấn đề độc lập.

Các thông tin về chương trình, sự kiện, workshop và tài liệu tham khảo cũng được chia sẻ tương đối rõ ràng, giúp mình có thêm nguồn học phù hợp cho từng giai đoạn.

3. Sự phù hợp giữa công việc và chuyên ngành học

Nội dung thực tập phù hợp với chuyên ngành Computer Science và định hướng phát triển của mình. Project có nhiều phần liên quan đến backend, database, distributed systems, Cloud infrastructure, security và AI integration.

Thông qua project, mình không chỉ củng cố kiến thức lập trình mà còn hiểu thêm về các yếu tố phi chức năng như bảo mật, khả năng mở rộng, độ tin cậy, monitoring và kiểm soát chi phí.

4. Cơ hội học hỏi và phát triển kỹ năng

Trong quá trình thực tập, mình học được nhiều kỹ năng quan trọng:

- Thiết kế hệ thống nhiều dịch vụ.
- Xây dựng REST API với FastAPI.
- Quản lý dữ liệu bằng PostgreSQL, DynamoDB, Redis và S3.
- Tích hợp realtime chat bằng Socket.IO.
- Tích hợp AI để phân tích CV và Job Description.
- Container hóa bằng Docker và triển khai bằng Kubernetes.
- Thiết lập CI/CD và quan sát hệ thống bằng metrics, logs, traces.
- Viết báo cáo kỹ thuật song ngữ theo cấu trúc rõ ràng.

Những kỹ năng này giúp mình có cái nhìn thực tế hơn về cách xây dựng một hệ thống Cloud-native thay vì chỉ tập trung vào từng chức năng riêng lẻ.

5. Văn hóa học tập và tinh thần cộng đồng

Cộng đồng FCAJ tạo cảm giác cởi mở và khuyến khích chia sẻ. Các sự kiện trực tuyến, workshop và bài blog trong nhóm giúp mình có thêm góc nhìn từ nhiều chủ đề khác nhau như AWS Cloud, Agentic AI, kiến trúc doanh nghiệp, DevOps, observability và bảo mật.

Mình cũng đánh giá cao việc chương trình khuyến khích người học chia sẻ lại kiến thức thông qua blog và báo cáo. Đây là cách tốt để biến kiến thức đã học thành nội dung có thể trình bày và kiểm chứng.

Một số câu hỏi khác

Điều mình hài lòng nhất trong thời gian thực tập là gì?

Điều mình hài lòng nhất là có cơ hội xây dựng một project tương đối đầy đủ, bao gồm cả business flow, AI integration, realtime communication, deployment direction và observability. Project giúp mình hiểu rõ hơn mối liên hệ giữa code, hạ tầng và vận hành.

Điều mình nghĩ chương trình có thể cải thiện cho các thực tập sinh sau là gì?

Chương trình có thể bổ sung thêm checklist theo từng giai đoạn, ví dụ checklist cho backend, frontend, AWS deployment, monitoring và cost cleanup. Điều này sẽ giúp người học dễ tự kiểm tra tiến độ và tránh bỏ sót những phần quan trọng.

Ngoài ra, nếu có thêm một số buổi review project mẫu hoặc phân tích lỗi thường gặp khi triển khai trên AWS, người học sẽ có thêm kinh nghiệm thực tế trước khi tự làm project.

Nếu giới thiệu cho bạn bè, mình có khuyên họ tham gia chương trình không?

Mình sẽ khuyên các bạn quan tâm đến Cloud, backend, DevOps hoặc AI application tham gia chương trình. FCAJ phù hợp với những người muốn học theo hướng thực hành, tự xây dựng sản phẩm và hiểu cách các dịch vụ AWS có thể kết hợp trong một hệ thống thực tế.

Đề xuất và mong muốn

- Có thêm project checklist theo từng milestone để người học dễ theo dõi tiến độ.
- Có thêm ví dụ về cost optimization và cleanup để tránh phát sinh chi phí không cần thiết trên AWS.
- Có thêm buổi chia sẻ về lỗi phổ biến khi dùng EKS, IAM, S3, CloudFront, RDS và GitHub Actions.
- Có thêm guideline về cách viết báo cáo kỹ thuật và cách trình bày architecture diagram.
- Có thể tổ chức thêm phiên project review ngắn để người học nhận góp ý trước khi hoàn thiện báo cáo.
- Có thêm định hướng về career path cho Cloud Engineer, Backend Engineer, DevOps Engineer và AI Engineer.

Tác động đến định hướng nghề nghiệp

Sau chương trình, mình có định hướng rõ hơn về việc theo đuổi mảng Cloud/Backend Engineering kết hợp AI integration. Mình nhận ra rằng việc xây dựng một ứng dụng hiện đại không chỉ là viết code, mà còn bao gồm thiết kế kiến trúc, bảo mật, triển khai, monitoring, quản lý chi phí và chuẩn bị tài liệu vận hành.

Chương trình giúp mình tự tin hơn khi đọc tài liệu kỹ thuật, phân tích yêu cầu, lựa chọn dịch vụ phù hợp và trình bày giải pháp theo hướng có cấu trúc.

Lời nhắn cuối

Mình cảm ơn chương trình First Cloud AI Journey, mentor, team admin và cộng đồng đã tạo điều kiện để mình học hỏi và hoàn thiện project trong kỳ thực tập. Những kiến thức và trải nghiệm này là nền tảng quan trọng để mình tiếp tục phát triển kỹ năng Cloud, backend, DevOps và AI application trong thời gian tới.



8 Tổng kết

- Kết thúc kỳ thực tập, SV thu nhận được những kỹ năng/ kiến thức gì?
- Cảm nhận cá nhân: về DN, về chương trình thực tập?
- SV ký và ghi rõ họ tên (chữ ký sống/ trực tiếp bằng bút màu xanh).

Sinh viên

Người đại diện doanh nghiệp
